

How the Engine Works

The internal mechanisms of the Lynx Harness agent engine — what happens inside, step by step, and why.

Every request this engine sends to a model is recomputed from an append-only event log immediately before dispatch — and that single decision is what makes resume, fork, compaction, and replay variations of one operation.

Contents

Chapter 1 · The core idea in one page
Chapter 2 · Just enough architecture
Chapter 3 · The core data structures
Chapter 4 · The engine in 100 lines
Chapter 5 · The append-only log
Chapter 6 · The surface
Chapter 7 · From log to request: <code>`deriveMessages()`</code>
Chapter 8 · Projections
Chapter 9 · The turn and step loops
Chapter 10 · Building the request
Chapter 11 · The reconstruction invariant
Chapter 12 · The inbox
Chapter 13 · Phases, cancellation, and quiescence
Chapter 14 · Agent lifecycle
Chapter 15 · The tool registry
Chapter 16 · The execution pipeline
Chapter 17 · Scheduling a step's tool calls
Chapter 18 · Approval and sandbox escalation
Chapter 19 · Prompt assembly
Chapter 20 · Variable interpolation
Chapter 21 · Runtime context injection
Chapter 22 · Extension points
Chapter 23 · Adapters and <code>`prepareCall`</code>
Chapter 24 · Streaming and block assembly
Chapter 25 · Failures and retry
Chapter 26 · Measuring and spilling
Chapter 27 · Pruning and compaction
Chapter 28 · Subagents
Chapter 29 · Persistence
Chapter 30 · Crash repair and chunk packing
Chapter 31 · One real turn, annotated
Chapter 32 · When things go wrong
Chapter 33 · Adding a tool

Chapter 34 · Composition in full

Chapter 35 · Limits and fragile areas

Chapter 36 · Design decisions

Chapter 37 · Rebuild it

Appendix A · Glossary

Appendix B · Data-structure reference

Appendix C · Configuration reference

Appendix D · Mechanism index

Chapter 1 • The core idea in one page

What you'll learn: the single design decision that explains almost everything else in this engine, and what it buys.

Prerequisites: none. No code in this chapter.

The engine, in one sentence

An **agent** here is a loop that talks to a language model, runs the tools the model asks for, feeds the results back, and repeats until the model stops asking. That much is ordinary. What makes this particular engine worth studying is *where it keeps its state*.

Most programs of this shape hold the conversation in memory — an array of messages that grows as the turn proceeds. This engine does not. It keeps an **append-only log of events**, and every request it sends to the model is *recomputed from that log* immediately before dispatch.

Nothing is ever overwritten. Nothing is deleted. The log only grows.

Why that is not a detail

If the conversation lives in an in-memory array, then a handful of things you would like to do are all awkward in the same way. Resuming a crashed session means reconstructing that array from something. Forking a conversation means deep-copying it and hoping nothing shared leaks. Trimming an over-long history means mutating it in place, which destroys the thing you might need to show a user later. Replaying a session for a test means faking the array.

If the conversation is instead *derived* from a log, all four become the same operation: read the log, fold it forward. Resume reads the log from disk. Fork copies a prefix. Trimming appends a marker saying "these old entries are superseded by this summary" — and the raw entries stay exactly where they were. Replay re-folds a recorded log.

The engine takes this seriously enough that the rule has a name in the codebase and a mechanical check behind it. The check asserts that the messages in an outgoing request are byte-identical to what re-deriving the log produces at that moment. When it fails, the error says **"log-reconstruction desync"** — the request drifted from its source of truth.

(That check ships as a diagnostic companion. It is real, runnable code, and it is the clearest statement of the contract — but it is not switched on in the shipped configuration. Chapter 11 is precise about this.)

The shape that follows

Three consequences run through the whole book, and it is worth having them in mind from the start.

Writing to the log is how you change anything. A user's message, the model's reply, a tool call, a tool's result, even the boundaries of a turn — all of them are events appended to the log. If something is going to be visible to the model, it is logged first. There is no side channel.

History is rewritten by addition, not subtraction. When the conversation grows too long for the model's context window, the engine does not delete old messages. It appends a summary plus a marker saying which range that summary stands in for. A view over the log — the **surface** — honors the marker and hides the superseded range. The raw events remain on disk forever. This one indirection is what makes compaction safe, and it is the subject of Chapter 6.

The loop itself is small and does almost nothing clever. The core is under 1,800 lines. It does not know how to retry a failed request, or how to shrink an oversized conversation, or what a subagent is. All of that lives in plugins that hook well-defined points in the loop. When you read the loop expecting to find the retry logic and it is not there, that is not an omission — it is the design. Chapter 25 shows who actually retries.

What this book does

It explains the machinery, mechanism by mechanism: what happens inside, step by step, and why. Every claim is tied to real code at a real line. Where the code and the documentation disagree, the code wins and the disagreement is noted. Where something could not be confirmed, it is marked **?** Unknown rather than smoothed over.

By the end you should be able to rebuild the core from an empty folder. Part VI lays out that path explicitly.

Key takeaways

- Every model request is recomputed from an append-only event log rather than read from an in-memory transcript.
- That single decision is what makes resume, fork, compaction, and replay into variations of one operation.
- History is rewritten by appending a superseding marker, never by deleting — the raw log is permanent.
- The loop is deliberately small; retry, compaction, delegation, and tooling are plugins hanging off documented extension points.

Exercises

1. Before reading on, write down how *you* would implement "the conversation got too long, shorten it" if the conversation were a plain array of messages. Keep your answer; compare it to Chapter 27.
2. Name three things that become difficult if a request is allowed to contain a message that was never written to the log. (Chapter 11 gives one answer you probably will not have listed.)

Next: [Chapter 2 · Just enough architecture](#)

Chapter 2 · Just enough architecture

What you'll learn: where the engine sits in the system, the ~10% of the plugin framework you need to read the rest of the book, and a map of every mechanism the book covers.

Prerequisites: [Chapter 1](#).

One application, one profile

There is exactly one application entry: `apps/server`. Its `src/index.ts:16` hardcodes the string `'web'` as the profile it boots — the profile name is a source literal, not an argument. Passing `--profile` is rejected outright:

```
if (argv.some(arg => arg === '--profile' || arg.startsWith('--profile='))) {
  program.error('error: this application serves the Web GUI only; --profile is not supported')
}
```

— `apps/server/src/args.ts:33-35`

So there is one shape of running system to understand, which is a considerable simplification. Everything in this book describes that system.

New term — profile. Not a file. A profile is an *empty* root config plus a stack of patch layers applied over it. At boot, `prepareProfile()` writes the literal string `[]` to the profile's `cordis.yml` (`apps/server/src/profile-boot.ts:80-84`), and the ~90 plugin rows that make up the running system are injected as patches on top. No checked-in file anywhere in the repo lists the web profile's composition. Chapter 34 covers this in full; for now, just know that the plugin list is assembled at startup rather than read from one place.

Everything is a plugin

New term — Cordis. The dependency-injection and plugin framework this harness is built on, vendored into `vendor/cordis/`. You need four of its concepts.

Context — the object every plugin receives, conventionally `ctx`. Reading an unknown property off it is a *service lookup*: `ctx.tools` walks up the fiber chain looking for whoever registered `tools`, and throws if nobody did (`vendor/cordis/src/reflect.ts:152-166`). Contexts nest, and a child sees everything its ancestors registered.

Service — a class that registers itself under a name. Construction *is* registration: `Service`'s constructor calls `provide(name, self)` before your subclass constructor body finishes (`vendor/cordis/src/service.ts:42-59`). The engine is one of these — `class AgentLoop extends Service` (`packages/core/agent-loop/src/index.ts:352`), published as `ctx.agentLoop`.

Fiber — one running instance of one plugin, with a lifecycle. A fiber declares what it needs:

```
static inject = ['agents', 'sessions', 'llm', 'tools', 'systemPrompt', 'sessionProjections']
```

— `packages/core/agent-loop/src/index.ts:353`

A fiber whose dependencies are not all satisfied sits in `PENDING` and **its plugin body never runs**. When a dependency later disappears, every dependent fiber is unloaded automatically, cascading up the graph (`vendor/cordis/src/reflect.ts:314–336`). Nothing is topologically pre-sorted; startup order emerges from dependencies resolving.

Effect — `ctx.effect(fn, label)` runs `fn` immediately and collects whatever it returns as a teardown handle. This is how *every* registration in the codebase works — registering a tool, a prompt section, or a projection all return the disposer that undoes them. The repo states the rule as "registrations are effects." One nuance that matters later: disposers nested inside a single effect run strictly in reverse order, but distinct top-level effects on the same fiber tear down **concurrently** when the fiber itself unloads (`vendor/cordis/src/fiber.ts:675–696`).

That is enough. Chapter 34 returns to realms, patch semantics, and lazy config evaluation once you have something concrete to apply them to.

Where the engine sits

The engine is `packages/core/agent-loop` — six files, 1,756 lines. It is mounted as one ordinary plugin row among ninety:

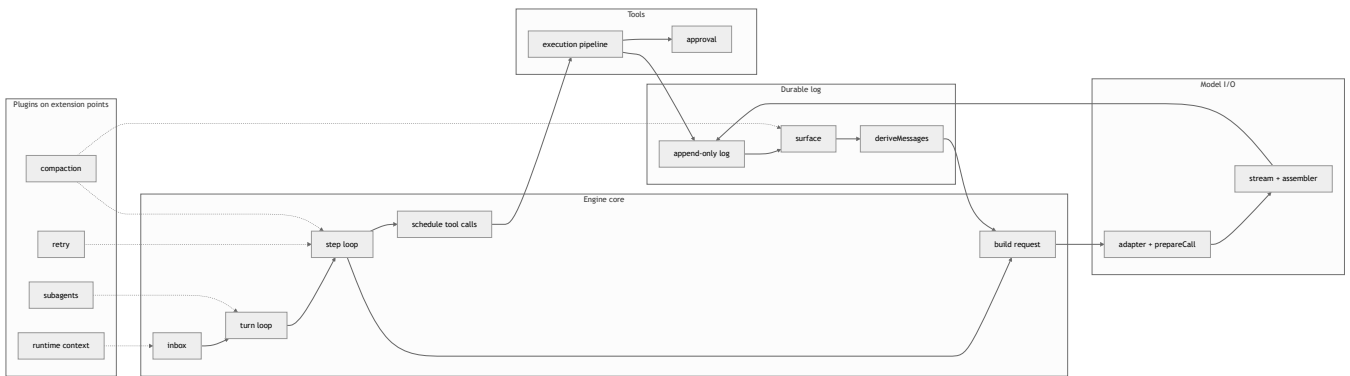
```
- id: agent-loop
  name: '@deepseek-ai/dsh-agent-loop'
  config:
    agents: []
```

— `packages/bundle/base/cordis.patch.yml:486–489`

`agents: []` means the engine creates nothing at startup; the browser asks for sessions as users open them.

Its six injected services are the entire surface it depends on. Everything else in this book — compaction, retry, subagents, approval, skills — reaches the engine through *extension points* rather than through imports. The engine does not know those plugins exist.

The mechanism map



Solid arrows are the main path of a turn. **Dotted arrows are plugins reaching in through extension points** — and they are the interesting part of the architecture: compaction rewrites the surface and can force a request to be re-issued; retry decides whether a failed request runs again; runtime context injects messages through the inbox so they become ordinary logged history rather than a side channel.

The full map, with all 37 mechanisms and their dependencies, is in `_notes/mechanism-map.md`.

What is *not* running

A recurring hazard when reading this codebase: many capable-looking packages exist but are never mounted. The book flags these as it goes, but the headline cases:

Not running in the web profile	Why
Hook bridges (<code>hooks-claude-code</code> , <code>hooks-codex</code>)	Absent from every composition layer
<code>dsh-schedule</code>	Opt-in overlay only
<code>packages/experimental/**</code>	Referenced by no composition file
Runtime invariants (<code>dsh-invariants</code>)	Mounted nowhere — diagnostics are deliberately omitted
SQLite session search	Mounted with <code>openAt: never</code> ; the database is never opened
The native DeepSeek adapter	<code>disabled: true</code> in the web layer

That last one has a startling consequence covered in Chapter 32: a fresh install has **no registered model provider at all** until one is configured through the UI.

Key takeaways

- One application, one hardcoded profile; `--profile` is refused in argument parsing.
- A profile is an empty root config plus patch layers, assembled at boot — no file lists the composition.
- Four Cordis concepts carry the book: context (service lookup), service (construction is registration), fiber (dependency-gated lifecycle), effect (registration returns its own undo).

- The engine depends on exactly six services; everything else reaches it through extension points.
- Several substantial packages exist but never load — always check before believing a mechanism runs.

Exercises

1. `AgentLoop` injects six services. Predict what happens to a running agent if one of them — say `tools` — is unloaded while a turn is in flight. Check your answer against `vendor/cordis/src/reflect.ts:314-336`.
2. The engine's config is `agents: []`. Find the other place in the codebase where an agent could be created instead, and say who calls it. (Hint: Chapter 14.)
3. Why might a project deliberately *omit* runtime invariant checking from its shipped configuration, given the checks exist and pass? List two reasons; Chapter 11 gives the design's own.

Next: [Chapter 3 · The core data structures](#)

Chapter 3 • The core data structures

What you'll learn: every type that flows through the engine, with its real definition and a real example. This chapter is the vocabulary for the rest of the book.

Prerequisites: [Chapter 2](#).

Examples in this chapter come from `snapshots/session/text-turn/session.jsonl`, a recorded session held in the repository as a replay fixture. **It is normalized:** session ids, paths, message ids, and the two largest payloads appear as placeholders (`{{session:1}}`, `{{cwd}}`, `{{system}}`, `{{tools}}`). Everything else — event order, chunk shapes, token counts — is as captured.

1. The event envelope

Everything durable is a `SessionEvent` :

```
seq: number    // monotonic sequence number within the session
time: number   // epoch ms
type: T        // key of SessionEventMap
data: SessionEventMap[T]
```

— `packages/core/session/src/types.ts:391-398`

Two optional fields appear on surface-eligible events only (§3), and one more on any event:

- `ignorable?: true` — "marks an event a reader may safely skip when it does not recognize `type`" (`types.ts:399-409`). Absent means required; an unrecognized required event makes a reader refuse the whole log rather than misinterpret it.

seq is always the event's zero-based index in the log. It is assigned at exactly one place — `seq: this.log.length` (`packages/core/session/src/index.ts:627`) — and contiguity is enforced at four independent boundaries (Chapter 5).

2. The event vocabulary

The core map:

```
export interface SessionEventMap {
  'turn/start':      { turn: number }
  'turn/end':        { turn: number; reason: TurnEndReason }
  'step/start':      { turn: number; step: number }
  'step/end':        { turn: number; step: number }
  'user/message':    UserMessage
  'assistant/chunk': { turn: number; step: number; chunk: StreamChunk }
  'assistant/message': { turn: number; step: number; message: AssistantMessage
                        usage?: TokenUsage; interrupted?: true }
  'tool/call':       { turn: number; step: number; callId: ToolCallId
                      name: string; arguments: string }
  'tool/result':     { turn: number; step: number; message: ToolResultMessage
                      error?: { name: string; code: string }; meta?: JsonValue }
  'request/header':  { header: EpochHeader; reason: RequestHeaderReason; startsSeries?: true }
  'request/context': RequestContext
  'session/end-seed': Record<string, never>
}
```

— packages/core/session/src/types.ts:216–320

Note 'user/message': UserMessage — the event's data is the message, passed through verbatim when derived. There is no wrapper.

This map is open. Other packages extend it by TypeScript declaration merging, and the build generates the complete vocabulary — currently ~50 types including compaction/start, approval/asked, todo/write, llm/retry, agent/inbox/spliced (packages/core/session/src/known-event-types.ts:22–74). That generated set is a read-side gate: a stored log containing a type outside it is **refused** unless the event is marked ignorable (session-persistence/src/coordinator.ts:1143–1148).

This is why adding an event type does not require a format-version bump. SESSION_FORMAT_VERSION = 0 (types.ts:51) moves only for structural change — the envelope, the header shape, core event semantics, or the surface mechanism.

TurnEndReason

Six variants (types.ts:150–169), merge-extensible:

Kind	Meaning
completed	The model finished without asking for more work
max-tokens	A step hit the output ceiling; sticky once set (Chapter 9)
blocked	A pre-step decision rejected the step
aborted	Carries the AgentCancelCause
error	Carries an LlmFailure
interrupted	The loop never writes this. A persistence backend closed a crash-orphaned turn on reload (Chapter 30)

3. The surface

The model does not see the log. It sees an ordered view over it called the **surface**, and only three event types may join:

```
type SurfaceEventType = 'user/message' | 'assistant/message' | 'tool/result'
```

— `types.ts:330–334`

Those three **must** carry a surface intent; every other type must not (enforced by a conditional parameter type on `append()`, Chapter 5). The intent is:

```
type SurfaceOp = 'append' | { op: 'replace'; start: number; end: number }
```

— `types.ts:359–361`

plus optional `sourceEventSeqs: number[]` — "seq numbers of earlier events that this event cites as sources" (`types.ts:410–422`). For an assistant message, those are the chunk events it was assembled from. For a compaction replacement, they must cover every node being superseded.

The surface itself exposes just two things:

```
interface SessionSurface {  
  readonly nodes: readonly number[] // ordered seqs currently visible  
  readonly replaceGeneration: number // bumped once per committed replace  
}
```

— `packages/core/session/src/surface.ts:137–142`

`replaceGeneration` is small but load-bearing: it is how the message cache knows to rebuild (Chapter 7) and how the engine knows to start a new request series (Chapter 10).

4. Messages and content

```
interface Message {  
  readonly id: MessageId  
  readonly role: 'system' | 'user' | 'assistant'  
  readonly content: ContentBlock[]  
  readonly source: MessageSource  
}
```

— `packages/llm/llm/src/message.ts:131–140`

`source` records provenance, keyed by `kind: user`, `plugin` (carries the plugin name), `model` (carries provider/model), `tool` (carries the call id) — `message.ts:102–107`. Messages are built only through frozen factories (`createMessage`, `createAssistantMessage`, `createToolResultMessage`); there is no mutable

constructor.

Content blocks (`packages/llm/llm/src/types.ts:54-110`), also merge-extensible:

Block	Shape
<code>text</code>	<code>{ type, text }</code>
<code>reasoning</code>	<code>{ type, text }</code> — the model's chain of thought, distinct from visible text
<code>image</code>	<code>{ type, attachment }</code> — in practice user messages only
<code>tool-call</code>	<code>{ type, id, name, arguments }</code> — <code>arguments</code> is the raw JSON string the model produced, unparsed
<code>tool-result</code>	<code>{ type, toolCallId, content, isError? }</code>

A real message

From `session.jsonl:23`, the assistant's reply to "Reply with exactly the word: PONG":

```
{
  "role": "assistant",
  "content": [
    { "type": "reasoning", "text": "The user wants me to reply with exactly the word \"PONG\" and not use any tools." },
    { "type": "text", "text": "PONG" }
  ],
  "source": {
    "kind": "model",
    "provider": "deepseek-official",
    "model": "deepseek-v4-flash"
  },
  "id": "{message:3}"
}
```

Two blocks: the reasoning, then the answer.

5. Streaming chunks

What arrives from a provider, provider-neutral:

```
type StreamChunk =
  | { type: 'block-start';      index: number; blockType: ContentBlockType }
  | { type: 'text-delta';      index: number; text: string }
  | { type: 'reasoning-delta'; index: number; text: string }
  | { type: 'tool-call-delta'; index: number; id: ToolCallId; name?: string; argumentsDelta: string }
  | { type: 'block-end';      index: number; block: ContentBlock }
  | { type: 'usage';          usage: TokenUsage }
  | { type: 'finish';         reason: FinishReason; replayState?: ReplayEnvelope }
```

— `packages/llm/llm/src/types.ts:364-376`

Contract: usage arrives before the terminal finish; nothing follows finish; tool arguments stay raw strings.

`FinishReason` (`types.ts:116-125`): `stop`, `tool-calls`, `max-tokens`, `aborted{failure}`, `error{failure}`.

Real usage from the same turn (`session.jsonl:21`):

```
{"inputTokens":3091,"outputTokens":23,"cacheReadTokens":0,"reasoningTokens":20}
```

3,091 input tokens for a one-line prompt — almost all of it system prompt and tool schemas.

6. The request header

What pins a request's identity:

```
interface EpochHeader {
  config: LlmCallConfig          // provider, model, reasoningEffort?, temperature?, maxTokens?,
  stop?
  adapterDefaults?: LlmCallConfigAdapterDefaults
  system?: string
  tools?: ToolSchema[]
}
```

— `packages/core/session/src/types.ts:179-188` , `packages/llm/llm/src/call-config.ts:23-30`

`adapterDefaults` is `{ reasoningEffort?: true; maxTokens?: true }` — a record of which fields the *caller* omitted and the *adapter* filled in. It exists so the next request can strip them and re-resolve rather than freezing one model's defaults onto another (Chapter 10).

A header event carries a `reason: initial` , `resume` , `change` , or `series` (`types.ts:200-208`). From `session.jsonl:12` :

```
{"header":{"config":{"provider":"deepseek-official","model":"deepseek-v4-flash"},
  "system":"{{system}}","tools":"{{tools}}"},
  "reason":"initial"}
```

7. Tools

```
interface ToolDefinition extends ToolSchema { // name, description, parameters (JSON Schema)
  output: { schema, render(args, value), presentationMeta?(args, value) }
  execute(args: unknown, exec: ToolRunContext): Promise<unknown>
  finalizeContent?(exec, result): ContentBlock[] | undefined
  timeoutMs?: number
  isConcurrencySafe?(args: unknown): boolean
  presentCall?(args) / presentResult?(args, result)
}
```

— `packages/core/tools/src/index.ts:214-280`

`execute` returns a **canonical value**, not model-facing content; `output.render` projects that value into content blocks. The separation matters for replay.

The result:

```
type ToolExecutionResult =
  | { isError: false; value: JsonValue; content: ContentBlock[]
    meta?: JsonValue; additionalContexts?: UserMessage[]; concludesTurn?: true }
  | { isError: true; error: ToolFailure; content: ContentBlock[]
    meta?: JsonValue; additionalContexts?: UserMessage[] }
```

— `packages/core/tools/src/index.ts:549–573`

Three fields deserve flagging now:

- `value` **never reaches the durable log**. The `tool/result` event records only the message, `error.info`, and `meta` (`tool-calls.ts:269–289`).
- `additionalContexts` are user messages staged for the *next* step, not part of the result the model reads.
- `concludesTurn` lets a single tool result end the turn — and is typed `never` on failures, so a failed call can never stop a turn.

Key takeaways

- `seq` is the log index, assigned at one place; contiguity is a hard invariant.
- The event vocabulary is open and generated; `SESSION_FORMAT_VERSION` moves only for structural change.
- Only three event types touch the surface, and they must declare `append` or `replace` .
- `replaceGeneration` is the signal that history was rewritten — it drives both cache invalidation and request-series boundaries.
- Tool results separate the canonical `value` (never logged) from rendered `content` (logged and shown to the model).

Exercises

1. `'user/message': UserMessage` means the event data is the message itself. What would break if a producer wrapped its text in `<context>` tags at derive time instead of at append time? (`surface.ts:90–94` answers it.)
2. Given the surface may only contain three event types, explain why `turn/start` and `tool/call` still have to be in the log at all.
3. The recorded turn reports 3,091 input tokens to send the word "PONG". Where did they go, and which chapter would you look in to reduce them?

Next: [Chapter 4 · The engine in 100 lines](#)

Chapter 4 · The engine in 100 lines

What you'll learn: the whole engine, simplified until it fits on two screens — then an honest inventory of everything the real one adds.

Prerequisites: [Chapter 3](#) for the type names.

The minimal engine

This is not pseudocode for a different system; it is the real control flow with the hard parts removed. Every name here is a real name from `packages/core/agent-loop/src/agent.ts`.

```

class MinimalAgent {
  constructor(
    private ctx: Context,
    private session: Session,
    private options: { provider: string; model: string },
  ) {}

  /** Entry point: run turns until nothing is pending. */
  private async kick(): Promise<void> {
    while (await this.turn()) {}
  }

  /** One turn: open a boundary, run steps until the turn has an ending. */
  private async turn(): Promise<boolean> {
    const turn = ++this.turnNumber
    this.session.append('turn/start', { turn })
    let turnEnds: TurnEndReason | null = null
    let step = 0

    try {
      while (true) {
        // 1. Claim pending input and assemble the prompt.
        const claimed = this.inbox.claim(step === 0 ? 'next-turn' : 'next-step', turn)
        const assembly = await this.ctx.systemPrompt.assemble({ agent: this, scope: this })

        // 2. Open the step and log the claimed input.
        step += 1
        this.session.append('step/start', { turn, step })
        for (const message of claimed) {
          this.session.append('user/message', message, { surfaceOp: 'append' })
        }

        // 3. Run the step: one model call plus its tool calls.
        turnEnds = await this.step(turn, step, assembly)
        this.session.append('step/end', { turn, step })

        // 4. Stop when the turn has an ending and nothing is queued for a next step.
        if (turnEnds && this.inbox.nextStep.length === 0) break
      }
    } finally {
      this.session.append('turn/end', { turn, reason: turnEnds ?? { kind: 'completed' } })
    }

    return this.inbox.hasPending
  }

  /** One step: build a request from the log, stream it, run any tool calls. */
  private async step(turn: number, step: number, assembly: PromptAssembly)
    : Promise<TurnEndReason | null> {

    // 5. THE CENTRAL MOVE: the request is derived from the log, not remembered.
    const request = {
      provider: this.options.provider,
      model: this.options.model,
      system: renderPrompt(assembly),
      tools: assembly.tools,
      messages: this.session.deriveMessages(),
    }
    this.session.append('request/header', { header: canonicalHeader(request), reason: 'initial' })
  }
}

```

```

// 6. Stream, logging every chunk and accumulating blocks.
const assembler = new BlockAssembler()
const chunkSeqs: number[] = []
for await (const chunk of this.ctx.llm.stream(request)) {
  chunkSeqs.push(this.session.append('assistant/chunk', { turn, step, chunk }).seq)
  assembler.push(chunk)
}

// 7. Commit the assistant message, citing the chunks that built it.
const message = createAssistantMessage({
  content: assembler.blocks(),
  source: { provider: request.provider, model: request.model },
})
this.session.append('assistant/message', { turn, step, message },
  { surfaceOp: 'append', sourceEventSeqs: chunkSeqs })

if (assembler.finish.kind === 'max-tokens') return { kind: 'max-tokens' }

// 8. No tool calls means the model is done talking: the turn can end.
const toolCalls = message.content.filter(block => block.type === 'tool-call')
if (toolCalls.length === 0) return { kind: 'completed' }

// 9. Run the tools, logging each call and each result.
for (const call of toolCalls) {
  const callSeq = this.session.append('tool/call', {
    turn, step, callId: call.id, name: call.name, arguments: call.arguments,
  }).seq
  const result = await this.ctx.tools.execute({
    callId: call.id, name: call.name,
    arguments: JSON.parse(call.arguments || '{}'),
    agent: this, signal: this.signal,
  })
  this.session.append('tool/result', {
    turn, step,
    message: createToolResultMessage({
      callId: call.id, content: result.content, isError: result.isError,
    }),
  }, { surfaceOp: 'append', sourceEventSeqs: [callSeq] })
}

// 10. Tools ran, so the model gets another step to react to their results.
return null
}
}

```

Reading it

The three loops. `kick` iterates turns, `turn` iterates steps, and (in the real engine) `step` iterates request retries. Each has exactly one exit condition, and confusing them is the single easiest way to misread this engine.

Step 1 — input arrives at a boundary, never mid-flight. Messages are *claimed* from a queue at the top of a step. Nothing interrupts a step in progress. The first step of a turn claims from `next-turn` ; later steps claim from `next-step` . Chapter 12 explains why that asymmetry exists.

Step 2 — the log records structure, not just content. `turn/start` , `step/start` , `step/end` , `turn/end` carry no model-visible text. They exist so the log can be *parsed* later — to find an unterminated turn after a crash, to count turns for a UI, to know which events belong to which step.

Step 5 — the line the whole design exists for. `messages: this.session.deriveMessages()` . Not `this.messages` . There is no in-memory transcript to drift from. Everything appended in steps 2, 7, and 9 is already in the log, so re-deriving picks it up automatically. Delete this line's discipline and most of the rest of the book stops making sense.

Step 7 — `sourceEventSeqs` links the summary to its raw material. The assistant message cites every chunk event it was assembled from. That provenance is what lets a UI expand a message back into its stream, and what compaction uses to prove it is superseding the right things.

Step 10 — returning `null` is how the loop continues. A step that ran tools returns `null` , meaning "no ending yet"; the turn loop sees `turnEnds` still null and runs another step so the model can react. This is the only path by which a turn contains more than one model call.

What the real implementation adds

Nothing above is wrong, but a great deal is missing. Each item links to the chapter that covers it.

Correctness and safety

- Every `append` is validated against the live surface *before* it lands, and reentrant appends are refused — [Ch 5](#)
- History can be rewritten by superseding ranges rather than deleting — [Ch 6](#)
- The derived-message cache is incremental and invalidated by rewrites — [Ch 7](#)
- A machine-checkable assertion that the request equals the derivation — [Ch 11](#)

Control

- Turn endings are *sticky* for `max-tokens` , and a rejected step blocks the turn entirely — [Ch 9](#)
- Request headers are logged only when they change, with four distinct reasons — [Ch 10](#)
- A durable inbox with two queues, backing three different ways to send input — [Ch 12](#)
- Cancellation, wake latching, and a phase machine so teardown never waits on a model call — [Ch 13](#)
- A lifecycle that rolls back cleanly if setup fails midway — [Ch 14](#)

Tools

- A four-stage pipeline with pre-execute, approval, guards, and post-execute — [Ch 16](#)
- Concurrency: exclusive barriers and a bounded parallel pool, with results committed in model order — [Ch 17](#)
- Two independent approval paths — [Ch 18](#)
- Malformed model JSON does not crash anything; it becomes a retryable tool error — [Ch 17](#)

What the model sees

- Prompt sections, contexts, tools, and variables, layered and shadowed per agent — [Ch 19](#)
- A runtime-context snapshot injected *only when it changes* — [Ch 21](#)
- Extension points where every plugin in the system attaches — [Ch 22](#)

Model I/O

- Adapter resolution, exact-model defaults, one-shot prepared calls — [Ch 23](#)
- Partial output salvaged when a stream is cancelled — [Ch 24](#)
- Retry — which the loop does not implement — [Ch 25](#)

Staying inside the context window

- Token measurement, spilling, pruning, and LLM-summarizing compaction — [Ch 26](#), [Ch 27](#)

Recursion

- Subagents, built from this same engine — [Ch 28](#)

Durability

- Persistence, crash repair, and a lossless chunk-packing codec — [Ch 29](#), [Ch 30](#)
-

Key takeaways

- Three nested loops: turns, steps, retries.
- Input is claimed at step boundaries; nothing interrupts a step in flight.
- `deriveMessages()` at request time is the design's load-bearing line.
- A step returning `null` — "no ending yet" — is what makes a turn multi-step.
- Boundary events carry no model-visible content; they exist to make the log parseable.

Exercises

1. In the minimal `turn()`, a step that produces no tool calls ends the turn. Trace what happens if a tool's result sets `concludesTurn`. Which line would need to change, and what would it look like?
2. The minimal `step()` runs tool calls strictly in sequence. List two things that must be true about a tool before it is safe to run it concurrently with its siblings, then check your list against [Chapter 17](#).
3. Add cancellation to the minimal engine using a single `AbortSignal`. Where does it need to be checked? Count the places, then compare to the real `turn()`, which checks six times.

Next: [Chapter 5 · The append-only log](#)

Chapter 5 · The append-only log

What you'll learn: what `append()` actually does, in order; why `seq` can never have a gap; and how the log stays readable by a runtime that does not recognize every event in it.

Prerequisites: [Chapter 3](#).

1. The problem

A conversation with a model is a sequence of things that happened: a person asked something, the model thought, it called a tool, the tool answered. If you store that as a mutable array of messages, you have quietly thrown away most of what happened — the tool's raw arguments, the order of streamed chunks, the boundary between one turn and the next — and kept only what you currently think you need.

That is fine until you need to resume a crashed session, show a user what the model was doing when it stopped, fork a conversation at a known-good point, or prove that a request was built from what you think it was built from. Each of those wants a different projection of the same history.

So the engine stores the history, not the projection. The log is the record; messages are one view of it.

2. Mental model

Think of a ledger. Entries are written in ink, in order, each numbered. You never erase an entry. If an earlier entry turns out to be wrong or no longer relevant, you write a *new* entry saying so, and readers who care about the current state learn to skip the superseded one.

Two properties fall out, and the code enforces both:

- **The number is the position.** Entry `n` is the `n`th entry. Not a database id, not a timestamp — an index.
- **Writing is the only mutation.** There is no `update`, no `delete`, no in-place edit anywhere in the API.

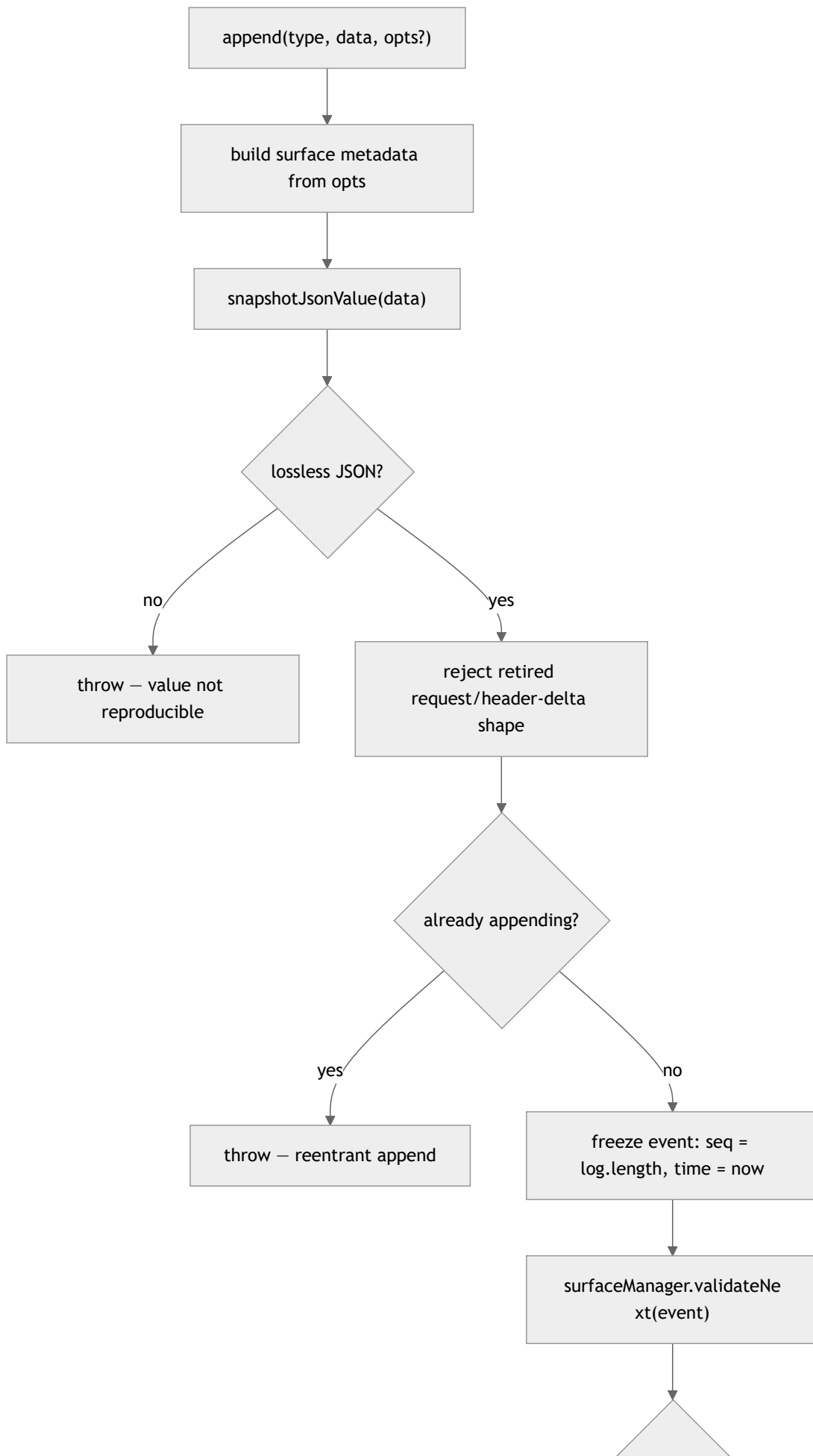
New term — `Session`. The object holding one conversation's log. Notably it is a **plain class, not a Cordis service** (`packages/core/session/src/index.ts:423`), with a private constructor reached through two factories: `Session.create(...)` for fresh or forked sessions (`index.ts:480-482`) and `Session.fromRestore(...)` for the exclusive-ownership path persistence backends use (`index.ts:493-495`).

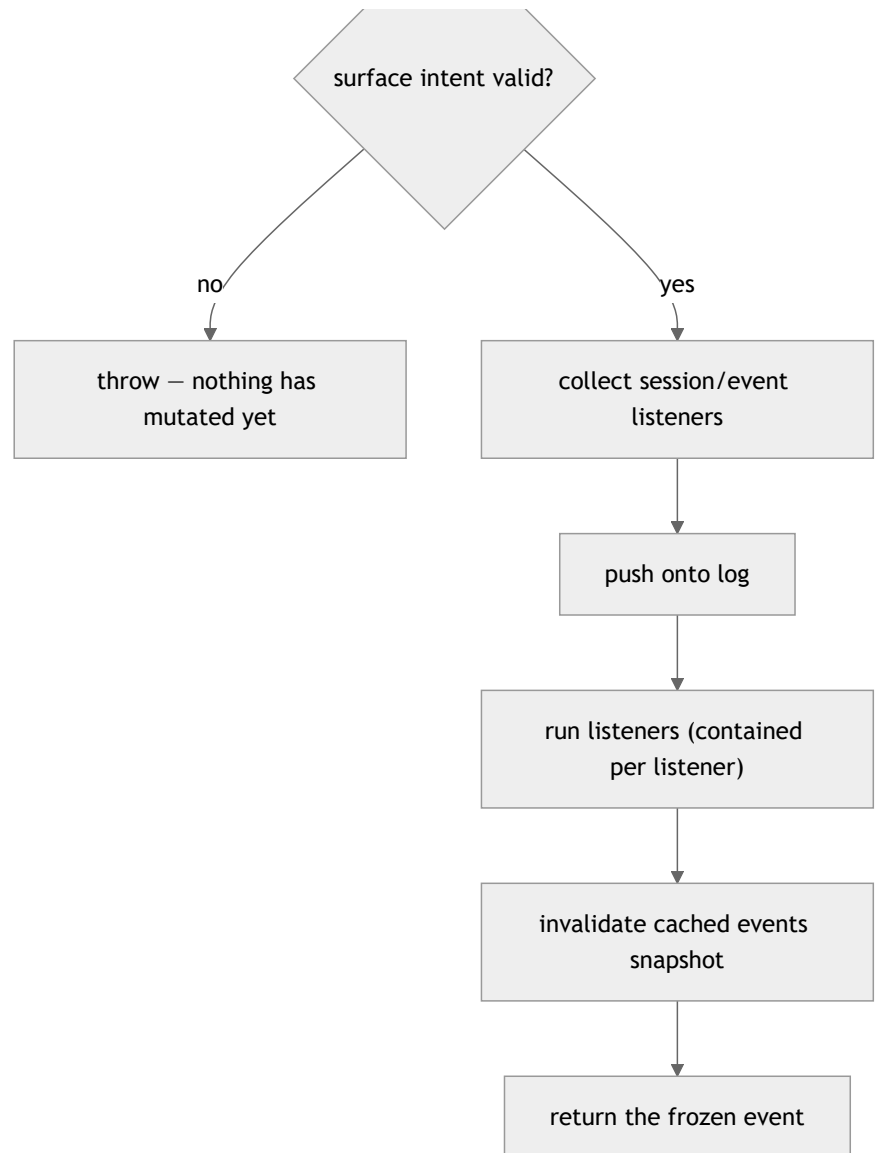
The Cordis service is a separate thing, `SessionStore` (`ctx.sessions`, `index.ts:790-1153`), and it is **in-memory only**. Its own source says so:

"Persistence is intentionally not implemented here — persistence plugins subscribe to `session/event` and flush on `session/flush` / `dispose`." — `packages/core/session/src/index.ts:786-789`

That separation matters: the log's semantics do not depend on where it is stored. Chapter 29 covers the storage.

3. Lifecycle of one append





The ordering of H and K is the subtle part, and §6 returns to it.

4. Step-by-step walkthrough

`Session.append<T>(type, data, ...opts)` — `packages/core/session/src/index.ts:602-653`.

The signature is doing real work:

```

append<T extends SessionEventType>(
  type: T,
  data: SessionEventMap[T],
  ...opts: T extends SurfaceEventType ? [opts: SurfaceIntent] : []
): SessionEvent<T>

```

— `index.ts:602-606`

That conditional rest parameter means the compiler **requires** a surface intent for `user/message`, `assistant/message`, and `tool/result`, and **forbids** one for every other type. You cannot forget to say whether a message appends or replaces, and you cannot accidentally attach surface metadata to a `turn/start`.

Then, in order:

Snapshot the payload. `snapshotJsonValue(data)` (`index.ts:612-615`) takes a deep, lossless-JSON copy. "Lossless" is strict: it rejects `BigInt`, functions, symbols, `undefined`, negative zero, non-finite numbers, circular references, sparse arrays, and class instances including `Map` / `Set` / `Date` (`index.ts:588-591`). Anything that would not survive a JSON round-trip identically is refused at the boundary rather than discovered later as corrupt storage.

Refuse reentrancy.

```
if (entry?.appending) throw ...
```

— `index.ts:621-624`

An append cannot be triggered from inside another append's synchronous `session/event` dispatch. Without this, a listener that logs something in response to what it just observed could interleave two events and break the `seq = log.length` relationship.

Build and freeze the event.

```
{ type, seq: this.log.length, time: Date.now(), data: dataSnapshot, ...surfaceMetadataSnapshot }
```

— `index.ts:625-631`

This is the only place in the codebase where a `seq` is assigned.

Validate against the surface before committing. `this.surfaceManager.validateNext(event)` (`index.ts:632`) checks the candidate's surface intent against the live surface fold — is the replaced range actually present, does the provenance cover what it must — *before* the event is pushed. A bad marker throws with no state mutated.

Dispatch, then push — in that order, sort of. The listener list is snapshotted before the push, but the callbacks run after it (`index.ts:636-641`). The JSDoc states the contract precisely: "The listener snapshot resolves before the log push, but callbacks run after it" (`index.ts:63-65`). So a listener observing event *n* sees a log that already contains it, but a listener registered *during* that dispatch will not be called for this event.

Contain observer failures. `invokeContainedSessionObservers` (`index.ts:379-397`) catches and logs per listener. One listener throwing cannot un-commit the append or block the others. The append has already happened; observers do not get a veto.

5. Data at each stage

Taking `session.jsonl:9` — the user's message entering the log:

Stage	Value
Caller passes	<code>type: 'user/message', data: UserMessage, opts: { surfaceOp: 'append' }</code>
After snapshot	A deep frozen copy; the caller's object is no longer shared
After envelope	<code>{ type: 'user/message', seq: 8, time: <ms>, data: {...}, surfaceOp: 'append' }</code>
After surface validation	Surface nodes gain seq 8 at the tail
Returned to caller	The frozen event — the loop keeps <code>.seq</code> to cite later

The returned event is the *logged* one, "its assigned `seq / time` plus the SNAPSHOT of `data` that entered the log" (`index.ts:585-587`) — not the object you passed in.

6. Control decisions

Decision	Condition	Location
Reject payload	not losslessly JSON-representable	<code>index.ts:612-615</code>
Reject legacy shape	event is the retired <code>request/header-delta</code>	<code>index.ts:616</code> , helper <code>:361-370</code>
Reject reentrancy	an append is already in progress	<code>index.ts:621-624</code>
Reject surface intent	replaced range absent, provenance incomplete, wrong event type	<code>surface.ts:185-243</code> via <code>validateNext</code>
Contain listener failure	any listener throws or rejects	<code>index.ts:379-397</code>

Note what is **not** a decision: there is no branch where an append is silently dropped. Every path either commits or throws.

7. Edge cases and failure modes

A gap in `seq` is impossible by construction, and checked anyway. Three independent boundaries re-verify it:

- Seed validation on construction: `if (snapshot.seq !== index) throw ( index.ts:523-525 )` — a restored log must number `0..N-1` with no holes.
- The surface fold: `if (event.seq !== expectedSeq) throw ( surface.ts:328-330 )`.

- Persistence: every event in a batch is checked against `state.cursor + i` before writing (`session-persistence/src/coordinator.ts:722-726`).

A property enforced in one place is a convention; enforced in four, it is a load-bearing invariant.

An unknown event type is a hard stop, unless marked skippable. The build generates `KNOWN_SESSION_EVENT_TYPES` (`known-event-types.ts:22-74`), the complete vocabulary across every loaded plugin. On load:

```
for (const event of events) {
  if (KNOWN_SESSION_EVENT_TYPES.has(event.type) || event.ignorable === true) continue
  throw this.unsupported(meta, `session "${meta.id}" contains event type "${event.type}" ...`)
}
```

— `session-persistence/src/coordinator.ts:1143-1148`

This is the mechanism that lets the vocabulary grow without a format-version bump. A *required* event the runtime does not understand causes a loud refusal — never a silent misreading. A purely informational one, marked `ignorable: true` by its writer, degrades gracefully.

Version refusal is directional. A header whose `version` is not `SESSION_FORMAT_VERSION` throws `SessionFormatUnsupportedError`, with different text for "written by a newer harness — upgrade" versus older (`coordinator.ts:78-82`, `1128-1131`). The constant is pinned at `0` (`types.ts:51`), and the doc comment is blunt that while unreleased, "no compatibility is implied, incompatible logs are rejected, and no migration is provided."

The header is deliberately not in the log. `SessionHeader` (`types.ts:56-94`) carries `version`, `id`, `createdAt`, `cwd`, `parentSession`, `seedLength`, `origin`, `delegationDepth`, `agentPreset` — validated and deep-frozen at construction, and kept out of the event stream because it is "a storage concern, not replayable conversation state" (`index.ts:441`).

The recorded fixture's header line carries `"version":0` and `"delegationDepth":0`. Both are confirmed:

version is literally `SESSION_FORMAT_VERSION`. The JSONL format module imports the constant from this package (`session-persistence-jsonl/src/format.ts:13`) and refuses a header whose version is a number differing from it, before reading any events (`:267-276`). There is no separately-maintained storage version.

delegationDepth is the persisted floor for subagent recursion. It is written to the header line (`:63`), validated as a non-negative safe integer (`:101-104`), and read back by `delegationDepthOf`, which returns `Math.max(header.delegationDepth ?? 0, runtimeOption ?? 0)` — so a resumed child cannot delegate as if it were top-level ([Ch 28](#)).

8. Configuration knobs

None. The log has no tunables — no size cap, no retention policy, no compaction threshold. Everything that bounds growth lives outside it (Chapters 26–27), which is precisely why the log can promise permanence.

9. Interactions

- **The surface** (Ch 6) validates every surface-eligible append before it lands, and is the only reason `append` can reject an otherwise well-formed event.
- `deriveMessages` (Ch 7) reads the surface, never the log directly.
- **Projections** (Ch 8) fold the log on every committed event.
- **Persistence** (Ch 29) subscribes to `session/event` ; the log itself knows nothing about storage.
- **The engine** writes eleven event types across a turn — see Ch 9.

10. Build it yourself

Minimal version:

```
class MiniSession {
  private log: SessionEvent[] = []
  append(type: string, data: unknown): SessionEvent {
    const event = Object.freeze({ type, seq: this.log.length, time: Date.now(), data })
    this.log.push(event)
    return event
  }
  get events(): readonly SessionEvent[] { return this.log }
}
```

That is genuinely the core. What the real one adds, and why each was necessary:

Addition	Why it exists
Lossless-JSON snapshot	A payload that cannot round-trip becomes corrupt storage discovered much later
Freezing	Callers share references; without freezing, a caller could mutate logged history
Reentrancy refusal	A listener appending during dispatch would break <code>seq = log.length</code>
Pre-commit surface validation	A bad marker must fail with nothing mutated, not leave a half-valid surface
Contained observers	One bad listener must not be able to un-commit history
<code>ignorable</code> + known-type gate	The vocabulary must grow without breaking old readers, and must fail loudly when it cannot
Cached <code>events</code> snapshot	Readers ask constantly; re-copying the array each time is wasteful

Key takeaways

- `append()` is the only mutation, and it either commits or throws — never silently drops.
- `seq = log.length` is assigned in one place and re-verified at three more.
- Surface validation runs *before* the push, so an invalid marker leaves no trace.
- Observers run after the commit and cannot veto it.

- Vocabulary grows freely; `ignorable` plus a generated known-type set turns "I don't recognize this" into either safe skipping or a loud refusal.

Exercises

1. A listener on `session/event` calls `session.append(...)` in response. Exactly which line stops it, and what would break if that line were removed?
2. The surface is validated before the push but listeners run after it. Construct a scenario where the reverse order would produce an observably wrong result.
3. `SESSION_FORMAT_VERSION` is `0` and no migration path exists. Given the vocabulary can grow without a bump, name a change that *would* require one, and say which of the four structural categories it falls into (`types.ts:28-50`).

Next: [Chapter 6 · The surface](#)

Chapter 6 • The surface

What you'll learn: the one indirection that lets this engine rewrite conversation history without deleting anything — and the rules that keep a rewrite honest.

Prerequisites: [Chapter 5](#).

1. The problem

Chapter 5 promised the log is permanent. That promise collides immediately with reality: a model has a finite context window, and a long conversation will exceed it. Something has to shrink.

The obvious move is to delete old messages, or splice a summary over them in the message array. Both destroy the record. A user scrolling back would find their own earlier questions gone. A crash-recovery routine would have nothing to recover. A bug report would be unreproducible.

So the engine needs a way to say *"the model should no longer see events 12 through 40; it should see this summary instead"* while events 12 through 40 remain exactly where they are, forever.

2. Mental model

New term — the surface. An ordered list of the log positions the model currently sees. Not the events themselves — just their `seq` numbers, in order.

```
interface SessionSurface {
  readonly nodes: readonly number[]
  readonly replaceGeneration: number
}
```

— `packages/core/session/src/surface.ts:137-142`

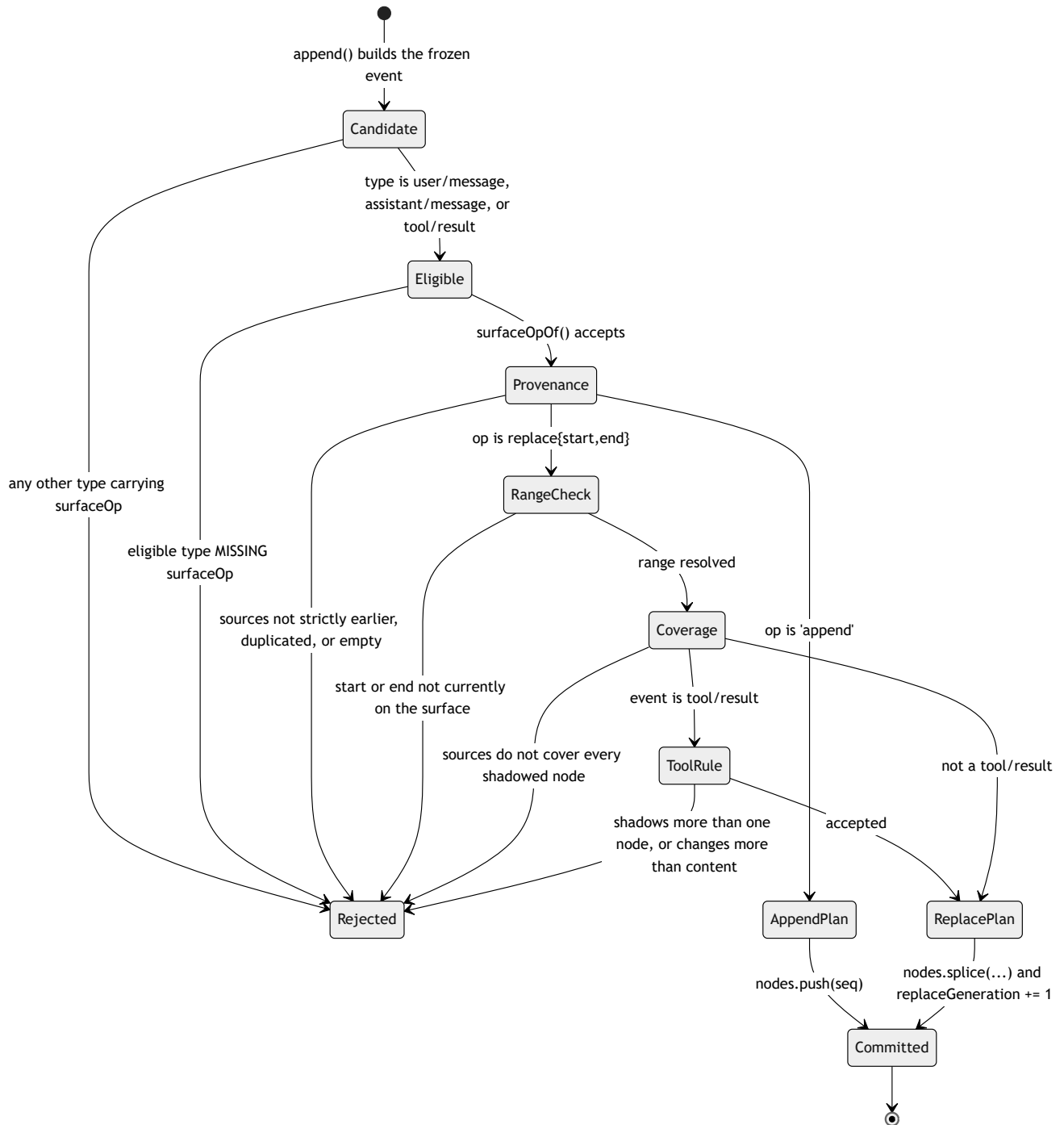
Think of the log as a bookshelf where books are only ever added at the right end, and the surface as a reading list naming which books to read and in what order. To "remove" a run of books you do not burn them; you write a new book summarizing them, add it to the shelf, and rewrite the reading list so those entries are replaced by the new one's position.

The books are still there. Anyone reading the shelf directly — a UI showing the human transcript, a crash-repair routine — sees everything. Only the reading list changed.

Every surface-eligible append must declare which it is doing:

```
type SurfaceOp = 'append' | { op: 'replace'; start: number; end: number }
```


3. Lifecycle of a surface event



4. Step-by-step walkthrough

Eligibility

Only three event types may ever touch the surface:

```
type SurfaceEventType = 'user/message' | 'assistant/message' | 'tool/result'
```

— `types.ts:330-334`

Unlike the event map, this union is **closed** — plugins cannot extend it. `surfaceOpOf` (`surface.ts:185-208`) enforces it in both directions: a non-eligible event carrying `surfaceOp` or `sourceEventSeqs` throws, and an eligible event *missing* `surfaceOp` throws. There is no default.

Provenance

`assertProvenance` (`surface.ts:210-243`) checks `sourceEventSeqs`:

- every entry is a non-negative safe integer **strictly earlier** than the citing event's own `seq` (`:230-236`) — you cannot cite the future or yourself;
- no duplicates (`:232-234`);
- non-empty, **except** that an `assistant/message` may carry a present-but-empty array, for "a known empty provider stream" (`:221-222`).

That exception is small but real: a provider can legitimately return nothing, and the engine still logs an assistant message to carry the usage numbers.

Replace: resolving the range

`start` and `end` name **surface nodes by seq**, not log positions or array indices. Both must currently be on the surface; `replacementRange` (`surface.ts:246-266`) looks them up and throws `"start seq ... not found in surface"` otherwise. `start === end` replaces exactly one node.

Replace: coverage

This is the rule that makes a rewrite honest:

```
shadowedSeqs.filter(seq => !sources.has(seq))
```

— `surface.ts:239-242`

`sourceEventSeqs` must be a **superset of every shadowed node's seq**. You cannot supersede an event without citing it. The type documentation calls this "complete shadowed-node coverage." The practical effect: given a replacement, you can always reconstruct exactly what it stands for, without guessing from the range bounds.

Applying it

```
} else if (plan?.kind === 'replace') {
  state.nodes.splice(plan.startIdx, plan.endIdx - plan.startIdx + 1, plan.seq)
  state.replaceGeneration += 1
}
```

— `surface.ts:362-379`

The shadowed run is spliced out of the node list and the single new event's `seq` takes its place.

`replaceGeneration` increments exactly once per committed replace — no other code path touches it.

An append is simply `state.nodes.push(plan.seq)` (`surface.ts:366-367`).

5. Data at each stage

A real compaction, from `packages/compaction/compaction-basic/src/region.ts:472-475`:

```
session.append('user/message', checkpointMessage, {
  surfaceOp: { op: 'replace', start, end },
  sourceEventSeqs: [startEvent.seq, summaryEvent.seq, ...shadowedSeqs],
})
```

Stage	Surface nodes	replaceGeneration
Before	[3, 5, 7, 9, 11, 13, 15]	4
Event 42 appended, replacing 5–11	— validation only —	4
After splice	[3, 42, 13, 15]	5
Log length	unchanged in meaning: events 5, 7, 9, 11 still exist at their positions	

Note the sources include the `compaction/start` and `compaction/summary` events *as well as* every shadowed node — more than the minimum the rule requires, so the provenance records the whole transaction, not just what was hidden.

6. Control decisions

Decision	Condition	Location
Reject non-eligible	any type outside the three carrying surface fields	<code>surface.ts:185-208</code>
Reject missing intent	eligible type with no <code>surfaceOp</code>	<code>surface.ts:185-208</code>
Reject bad provenance	not earlier / duplicated / empty (non-assistant)	<code>surface.ts:210-243</code>
Reject unknown range	<code>start</code> or <code>end</code> not currently a surface node	<code>surface.ts:246-266</code>
Reject incomplete coverage	a shadowed seq is not cited	<code>surface.ts:239-242</code>
Reject wide tool rewrite	more than one node shadowed, or a field other than content changed	<code>surface.ts:287-318</code>
Bump generation	only on a committed replace	<code>surface.ts:370-371</code>

7. Edge cases and failure modes

`tool/result replacement` is deliberately narrow. `assertToolResultRewrite` (`surface.ts:287-318`) permits shadowing exactly one node, and permits the replacement to differ from the original **only in the tool-result block's `content`**. Everything else — call id, turn, step, error flag, the rest of the message — must be deep-JSON-equal, checked by nulling `content` on both sides and comparing.

This exists so the tool-result pruner ([Ch 27](#)) can shrink an enormous result's text while making it structurally impossible to quietly change what the tool *was* or whether it *failed*. A narrow rewrite primitive is safer than a general one.

The surface is the wrong source for a human transcript. The codebase is explicit:

"The model-visible surface deliberately shadows replaced ranges, so it is the wrong source for a human transcript — a landed replacement would erase conversation the user already saw. Append-origin events are that transcript's durable source material; replacement copies stay model-only." — `surface.ts:44-48`

Hence three exported guards (`surface.ts:35-68`): `isSurfaceEvent`, and the split into `isAppendSurfaceEvent` versus `isReplacementSurfaceEvent`. A UI reads append-origin events from the log; the model reads the surface. Same log, two views, and conflating them would make compaction visibly eat a user's history.

The fold is incremental, with a pure fallback. `SurfaceManager` (`surface.ts:398-460`) keeps `_lastProcessedSeq` and folds forward lazily when `nodes` or `replaceGeneration` is read (`:432-441`). It also caches a `_pendingPlan` so an event validated by `validateNext` immediately before its append is not re-validated. For offline reconstruction there is a pure `foldSurface(events)` that replays a complete log from scratch (`:387-395`).

Nothing browser-specific. `surface.ts` deliberately avoids `node:` imports so web clients can consume it directly (`surface.ts:1-9`) — the browser folds the same surface from the same events rather than trusting a server-computed view.

8. Configuration knobs

None. Like the log, the surface has no tunables. What to compact and when is entirely the caller's decision ([Ch 27](#)); the surface only enforces that whatever they do is well-formed and fully attributed.

9. Interactions

- [Ch 5](#) — `append()` calls `validateNext` before committing, so every rule here fails with nothing mutated.
- [Ch 7](#) — `deriveMessages()` walks `surface.nodes`; `replaceGeneration` invalidates its cache.
- [Ch 10](#) — a bumped generation makes the next request log a new header series.
- [Ch 27](#) — the only two production writers of `replace`: region compaction and the tool-result pruner.
- [Ch 21](#) — watches replacement events to know when its injected snapshot was compacted away.

10. Build it yourself

Minimal version — a node list and two operations:

```
class MiniSurface {
  nodes: number[] = []
  replaceGeneration = 0
  apply(seq: number, op: SurfaceOp): void {
    if (op === 'append') { this.nodes.push(seq); return }
    const startIdx = this.nodes.indexOf(op.start)
    const endIdx = this.nodes.indexOf(op.end)
    this.nodes.splice(startIdx, endIdx - startIdx + 1, seq)
    this.replaceGeneration += 1
  }
}
```

Twelve lines, and it captures the whole idea. What the real one adds:

Addition	Why it exists
Closed eligibility + mandatory intent	An event silently defaulting to "append" would corrupt history in a way nothing detects
Provenance coverage	Without it, a replacement's meaning depends on range bounds that later rewrites can shift
Strictly-earlier source check	Citing a later event would make the log unreplayable in order
Narrow <code>tool/result</code> rewrite	A general rewrite could change whether a tool failed; only its text should shrink
Append-origin guards	The human transcript and the model transcript genuinely differ after a compaction
Incremental fold + pure <code>foldSurface</code>	The hot path reads this constantly; offline readers need to rebuild from nothing

Key takeaways

- The surface is an ordered list of visible log positions — the model's view, not the record.
- Only three event types participate, and each must declare `append` or `replace` explicitly.
- A replacement must cite every node it shadows, so what was superseded is always reconstructable.
- `tool/result` rewrites may change content and nothing else.
- `replaceGeneration` is bumped only by a committed replace, and is the signal the rest of the engine watches.
- Human transcript and model transcript diverge after a rewrite — by design, with separate guards for each.

Exercises

1. A plugin wants to redact a secret that a tool accidentally printed. Which surface operation does it use, what must it cite, and what can it *not* change? Would the same approach work for redacting something in a `user/message`?
2. `replaceGeneration` is a counter, not a boolean. Two readers cache derived state. Explain why a counter works where a "dirty" flag would not.
3. Compaction cites more sources than the coverage rule requires. Name a debugging question that the extra citations answer and the minimum set would not.

Next: [Chapter 7 · From log to request](#)

Chapter 7 • From log to request: `deriveMessages()`

What you'll learn: the function that turns an event log into the message array a model receives — and why it is deliberately dumber than you would expect.

Prerequisites: [Chapter 6](#).

1. The problem

The log holds everything that happened. The model needs a list of messages. Something must project one onto the other.

The temptation is to make that projection clever — reformat tool results, wrap plugin-injected text in tags, collapse consecutive messages, add framing to distinguish sources. Every one of those is a decision about what the model sees, and putting decisions here has a specific cost: the projection runs on *every* request, including replays of old logs. A projection that changes behavior changes the meaning of history that was recorded under the old behavior.

So this projection does as close to nothing as it can.

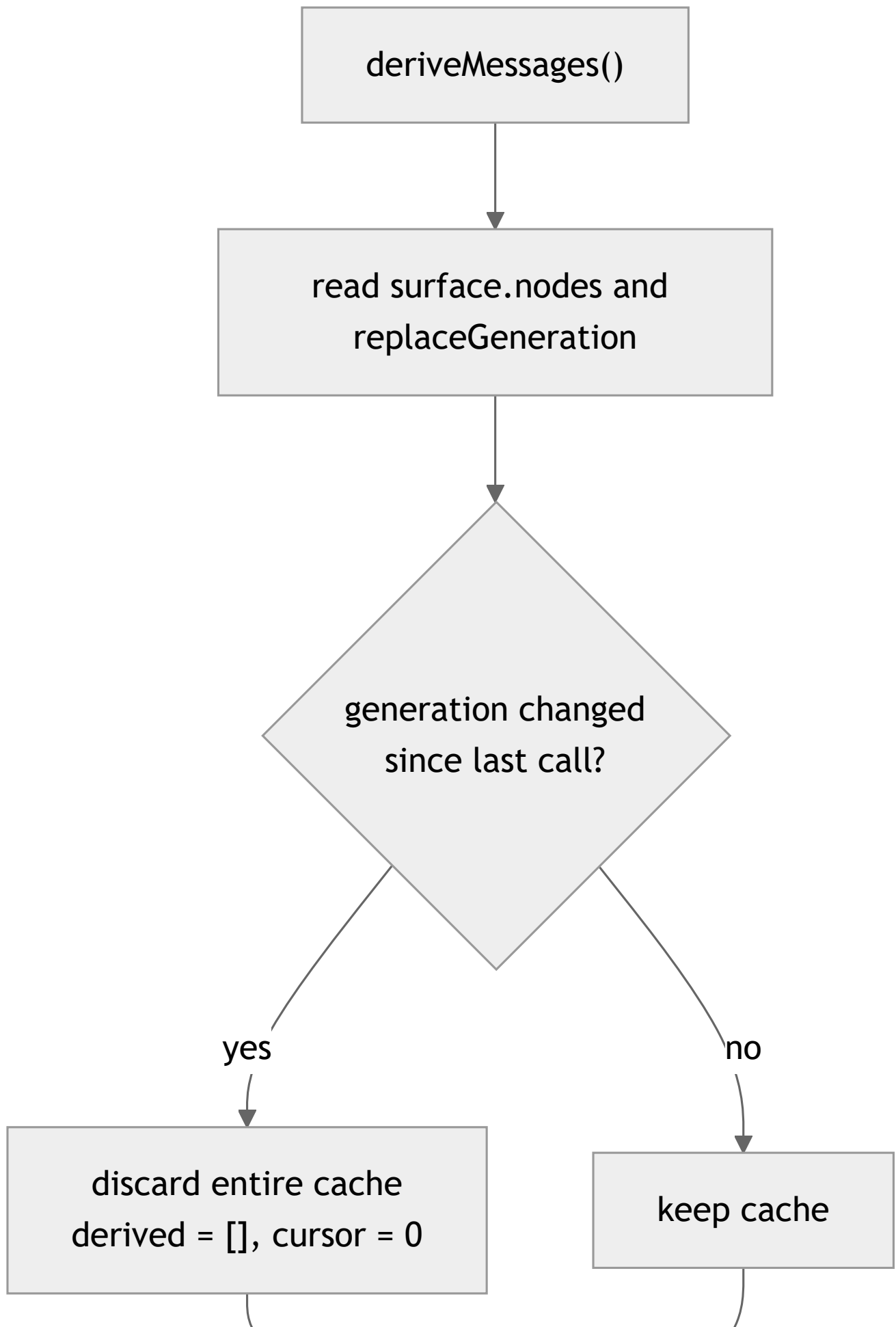
2. Mental model

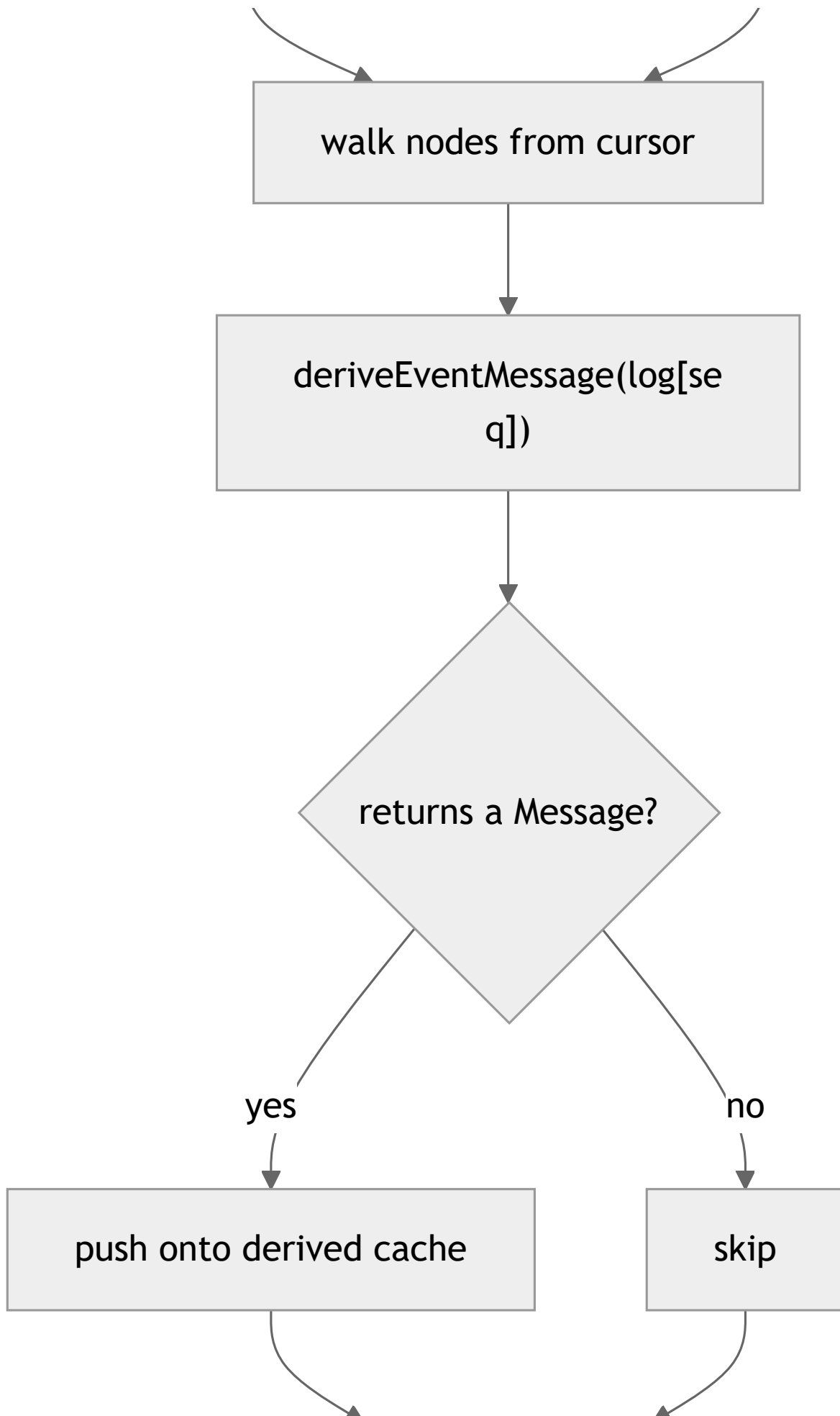
Two functions, cleanly separated.

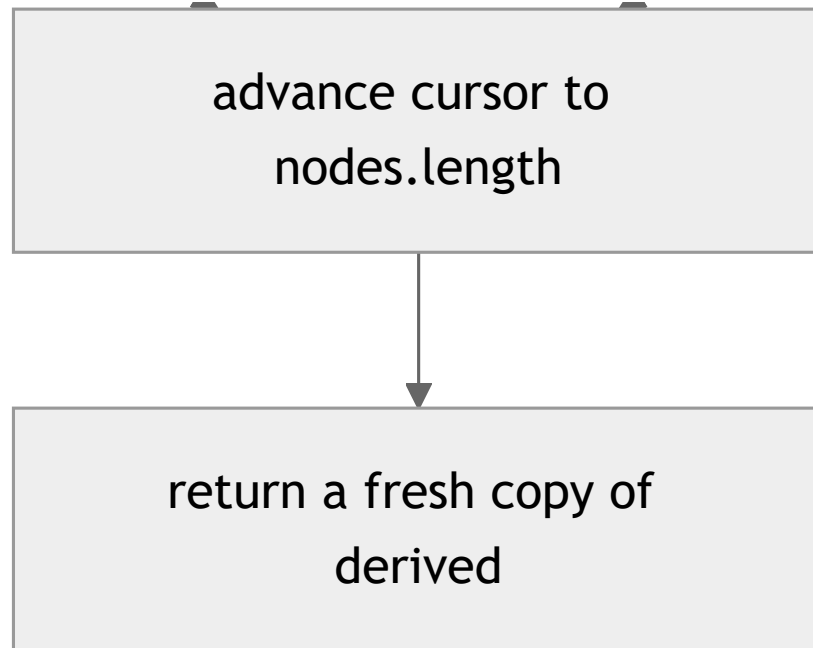
The per-event rule decides what one event contributes: a message, or nothing. It is pure, total, and boring.

The walker applies that rule across the surface's node list, in order, with a cache — because a long session re-derives constantly and re-walking thousands of events per request would be wasteful.

3. Lifecycle







4. Step-by-step walkthrough

The per-event rule

```
export function deriveEventMessage(event: SessionEvent): Message | null {
  switch (event.type) {
    case 'user/message': {
      return event.data
    }
    case 'assistant/message': {
      if (event.data.message.content.length === 0) return null
      return event.data.message
    }
    case 'tool/result': {
      return event.data.message
    }
    default:
      return null
  }
}
```

— `packages/core/session/src/surface.ts:83-114`

Three things to notice.

`user/message` **returns** `event.data` **directly**. Recall from Chapter 3 that the event map declares `'user/message': UserMessage` — the data is the message. This is a verbatim pass-through, and the source is emphatic about keeping it that way:

"Do NOT re-add per-type framing (e.g. `<context>`) here: framing is caller-owned — a producer bakes it into `content` ... keeping this projection a verbatim pass-through." — `surface.ts:90-94`

If a plugin wants its injected message to look distinctive to the model, it builds that into the content at append time. The consequence is that what the model sees is exactly what is in the log — no transformation layer to reason about, and replay is faithful by construction.

An empty-content assistant message derives to `null`. This is not defensive coding against an impossible case. When a step hits the output ceiling, the engine still logs an `assistant/message` to carry the token usage, and that message may have no content blocks. The type documentation states the rule: "an empty-content `assistant/message` ... derives to null and must not enter the transcript" (`types.ts:257`).

The `default` case is intentionally non-exhaustive. No `assertNever`, despite the codebase's general rule that closed unions must end in one. The reason is in the comment: "only message-producing events derive history; turn/step boundaries, chunks, usage, and errors are trace/replay data" (`surface.ts:84-86`). New event types should silently contribute nothing to the transcript, which is the correct default for a vocabulary that grows.

The walker

```
deriveMessages(): Message[] {
  const surface = this.surface
  const nodes = surface.nodes
  const generation = surface.replaceGeneration
  if (generation !== this.derivedGeneration) {
    this.derived = []
    this.derivedNodes = 0
    this.derivedGeneration = generation
  }
  for (const seq of nodes.slice(this.derivedNodes)) {
    const msg = this.deriveEventMessage(this.log[seq]!)
    if (msg) this.derived.push(msg)
  }
  this.derivedNodes = nodes.length
  return [...this.derived]
}
```

— `packages/core/session/src/index.ts:724-745`

The cache is append-only, which is safe *only* while the surface is append-only. A replace splices the middle of `nodes`, so positions the cache already consumed may have changed — hence the generation check, which throws the whole cache away rather than trying to patch it.

Cost: **$O(\text{new nodes})$** amortized, except immediately after a compaction, where it is $O(\text{surface})$.

5. Data at each stage

From the recorded turn, at the moment the request is built (`session.jsonl` events 1–11 are in the log):

Surface node	Event type	Derives to
seq 4	agent/inbox/spliced	<i>not on the surface — never a node</i>
seq 8	user/message (the person's prompt)	the <code>UserMessage</code> , verbatim
seq 9	user/message (runtime-context snapshot)	the <code>UserMessage</code> , verbatim
—	turn/start, step/start, session/title	not surface-eligible, absent from <code>nodes</code>

So the outgoing `messages` array is exactly two messages. The boundary events are in the log but never reach the model — which is why Chapter 3 insisted the surface and the log are different things.

After the model responds, the `assistant/message` at seq 23 joins the surface, and the *next* derivation returns three messages with only one new item walked.

6. Control decisions

Decision	Condition	Location
Rebuild cache	<code>replaceGeneration !== derivedGeneration</code>	<code>index.ts:727-731</code>
Skip event	type not one of the three message-producing types	<code>surface.ts:110-111</code>
Skip assistant message	<code>content.length === 0</code>	<code>surface.ts:100-104</code>
Walk incrementally	otherwise, from <code>derivedNodes</code> to <code>nodes.length</code>	<code>index.ts:732-736</code>

7. Edge cases and failure modes

The returned array is fresh; the messages inside are shared. `return [...this.derived]` gives every caller its own array, so no caller can observe another's later mutation. But the `Message` objects are shared references, reused across calls. That is safe because they are already deep-frozen — they came from frozen durable event data, and the source notes the cache therefore "needs no second deep clone" (`index.ts:718-721`).

There is no validation here. If the log contains an `assistant/message` whose content is nonsense, this function hands it to the model. Validation happens at append time (Chapter 5), which is the right place: an invalid event should never enter the log, and re-checking on every derivation would be both slow and too late.

A compaction makes the next derivation expensive. Worth knowing when reading a profile: the first `deriveMessages()` after a compaction walks the entire post-compaction surface. Since compaction just *shrank* that surface, this is usually cheap — but it is the one non-incremental case.

8. Configuration knobs

None.

9. Interactions

- **Ch 6** — the node list and generation counter this reads.
- **Ch 10** — the sole production caller: `agent.ts:355` passes the result straight into the request as `messages`.
- **Ch 11** — the diagnostic that asserts an outgoing request's messages equal a fresh call to this function.
- **Ch 27** — compaction's `replace` is what forces the cache rebuild.

10. Build it yourself

Minimal version:

```
deriveMessages(): Message[] {  
  return this.surface.nodes  
    .map(seq => deriveEventMessage(this.log[seq]!))  
    .filter((m): m is Message => m !== null)  
}
```

Correct, and fine until a session has a few thousand events. What the real one adds:

Addition	Why it exists
Append-only cache + cursor	Every step re-derives; re-walking the whole surface each time is wasted work
Generation-guarded invalidation	A replace edits the middle of the node list, so an append-only cache is no longer valid
Fresh array, shared frozen messages	Callers must not see each other's mutations, but deep-cloning frozen data every call is pointless

The interesting lesson is what is *absent*: no reformatting, no framing, no merging of adjacent messages. Every one of those would have been a decision baked into replay.

Key takeaways

- Three event types produce messages; everything else contributes nothing, deliberately without an exhaustiveness check.
- `user/message` is a verbatim pass-through — framing is the producer's job, decided at append time.
- An empty-content assistant message is skipped; it exists only to carry usage.
- The cache is append-only and thrown away wholesale when `replaceGeneration` moves.
- The result is a fresh array of shared frozen messages.

Exercises

1. Suppose you wanted tool results shown to the model wrapped in `<tool_result>` tags. Where does that belong, given this chapter, and what goes wrong if you add it to `deriveEventMessage` instead?
2. The cache is invalidated by generation, not by log length. Construct a sequence of appends and one replace where a length-based check would return a wrong result.
3. `deriveEventMessage` is exported and pure. Name two callers outside the `Session` class that would want it, and what each is doing. (One is in Chapter 11.)

Next: [Chapter 8 · Projections](#)

Chapter 8 • Projections

What you'll learn: how anything other than the message list gets computed from the log — turn counts, todo lists, retry counters — without re-reading the whole log every time.

Prerequisites: [Chapter 5](#). Chapter 7 is useful contrast but not required.

1. The problem

`deriveMessages()` ([Ch 7](#)) is one projection of the log, hardcoded into the session. But plenty of other things want their own view of the same events:

- the engine needs the last turn number, so a resumed agent continues at turn 8 rather than turn 1;
- the retry plugin needs a count of retries so far, and it must survive a process restart;
- a UI wants turn/step counts, a turn outline, a todo list, the current plan mode.

Each is a fold over the same event stream. Writing each one as a bespoke loop over `session.events` would mean every consumer re-walks the log independently, and none of them could be cached across a restart.

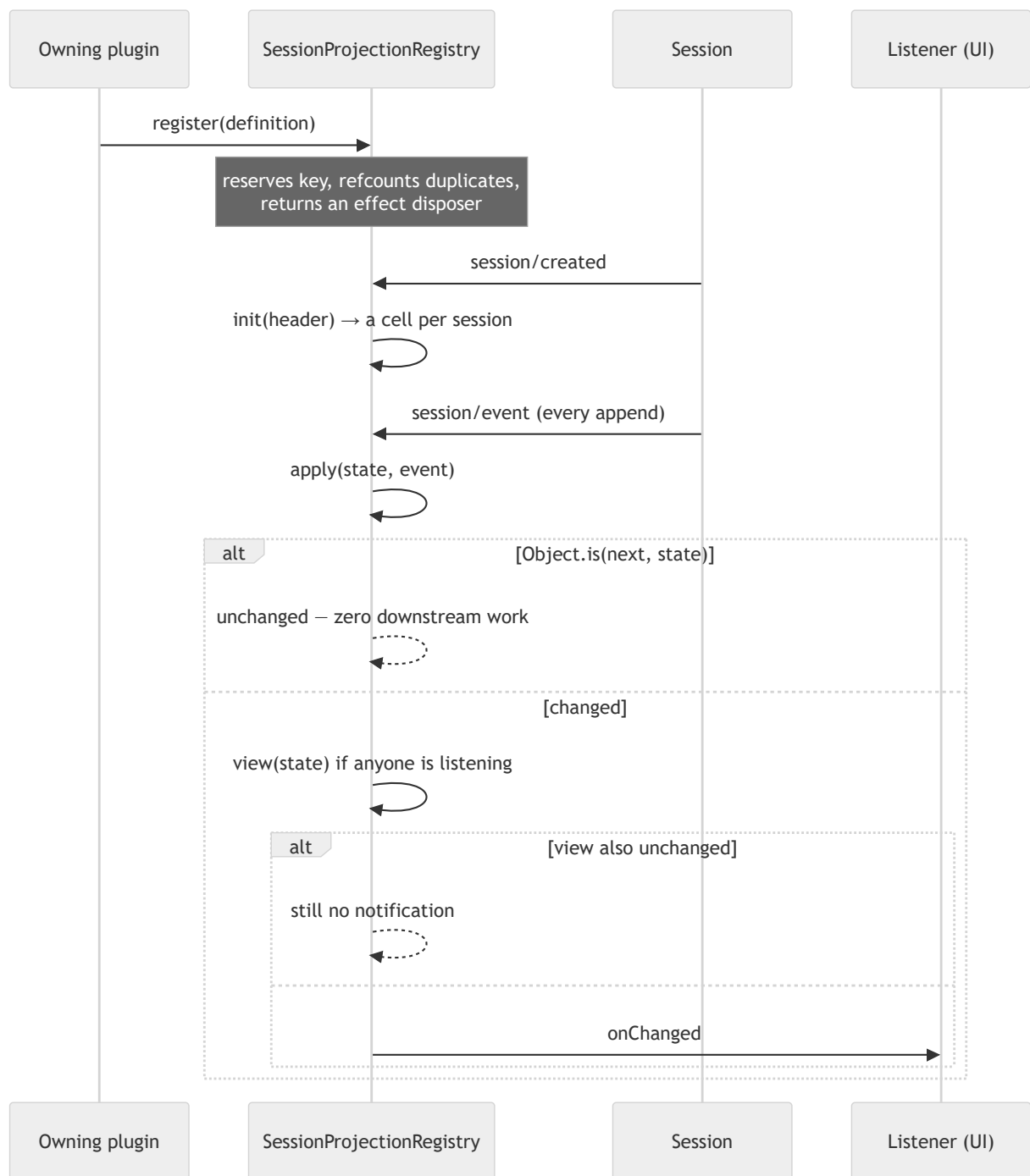
2. Mental model

New term — projection. A named, versioned, pure fold over the session log, registered by the package that owns the concept.

Three things make it more than "a reduce":

- **It is registered, not called.** A package declares its fold once; the registry runs it against every session and every event automatically.
- **It is versioned.** The persisted cache is stamped with the fold's version, so changing the fold's semantics discards stale caches rather than forward-applying them into garbage.
- **It is split in two.** Internal `state` (host-side, may be anything JSON) and an optional `wire` view (what a browser is allowed to see). A projection with no `wire` is host-only.

3. Lifecycle



4. Step-by-step walkthrough

The definition

```
export interface ProjectionDefinition<K, S> {
  key: K
  stateSchema: ZodType<S>
  init(header: SessionHeader): S
  apply(state: S, event: SessionEvent): S
  wire?: { viewSchema: ZodType<...>; view(state: S): ... }
  stateVersion: number
}
```

— `packages/session/session-projection/src/index.ts:42-86`

The constraints on `apply` are strict and stated in the source:

- **Pure and synchronous.** "An async unit would tear the carriers' consistency cut" (`index.ts:39`).
- **Reference-stable when uninterested.** "A unit uninterested in an event **MUST** return the same state reference — an unchanged reference (`Object.is`) produces zero downstream work" (`index.ts:58-59`).
- **Plain JSON state**, because it gets persisted (`index.ts:40`).

`stateVersion` is the cache-invalidation lever: "bump whenever the serialized state fields or the fold semantics change, so persisted (`sessionId, key, ver, seq, val`) rows from an older unit are discarded instead of being forward-applied into garbage" (`index.ts:80-84`).

Registration

`register()` (`index.ts:241-281`) goes through `ctx.effect(...)`, so unloading the owning plugin removes the key — and a client then "reads it as capability absence" (`index.ts:179-180`) rather than getting a stale value.

It also **refcounts**. The same package mounted in several agent presets registers the same key several times; a `refs` counter keeps one registration alive until the last unmounts (`index.ts:264-278`). A `stateVersion` mismatch between two registrants of the same key **throws** (`index.ts:267-269`) — two different folds cannot share a name.

Driving

The registry subscribes exactly once, at construction: `session/created` to `init` a fresh cell per registered unit (`index.ts:197-207`), and `session/event` to fold (`index.ts:208-210` → `drive()`, :625-661).

`drive()` is a **double debounce**:

1. Run `apply`. If the returned reference is `Object.is`-equal to the previous state, stop — nothing changed.

2. Only if it changed, and only if someone is actually listening, compute the `wire` view.
3. Compare *that* by `Object.is` too. Notify only if the view changed.

So a fold whose internal state moved but whose client-visible view did not produces no client traffic at all.

Reading

```
stateOf(session, key) {
  const registration = this.registrations.get(key)
  if (registration === undefined) return undefined
  this.materializeCells(session)
  return this.cellFor(registration, session).state
}
```

— `index.ts:307-315`

First touch lazily folds the whole in-memory log (`buildCell` , `:579-587`). `undefined` means the key is *unregistered* — not that the value is empty.

5. A real projection: the engine's own

The engine registers exactly one, and it is small enough to read whole:

```
export const turnBoundaryProjectionDefinition = {
  key: 'turnBoundary',
  stateVersion: 2,
  stateSchema: turnBoundaryProjectionSchema,
  init: () => ({
    openTurnStartSeq: null, lastStepStartSeq: null,
    lastStepBoundary: null, lastTurn: 0,
  }),
  apply: (state, event) => {
    switch (event.type) {
      case 'turn/start':
        return { ...state, openTurnStartSeq: event.seq, lastTurn: event.data.turn }
      case 'turn/end':
        return { ...state, openTurnStartSeq: null }
      case 'step/start':
        return { ...state, lastStepStartSeq: event.seq,
          lastStepBoundary: { kind: 'start', seq: event.seq } }
      case 'step/end':
        return { ...state, lastStepBoundary: { kind: 'end', seq: event.seq } }
      default:
        return state
    }
  },
} satisfies ProjectionDefinition<'turnBoundary', TurnBoundaryProjection>
```

— `packages/core/agent-loop/src/index.ts:55-93` , registered at `:409`

Note the `default: return state` — the same reference, honoring the zero-work rule. Four event types matter; the other ~46 cost one switch miss each.

What it is for. When an agent is constructed over a resumed session:

```
const lastTurn = this.loopCtx.sessionProjections.stateOf(session, 'turnBoundary')?.lastTurn ?? 0
this.phase = { kind: 'idle', lastTurn }
```

— `packages/core/agent-loop/src/agent.ts:101-102`

That is how a resumed conversation continues at turn 8 instead of restarting at turn 1. There is no counter stored anywhere; the number is recomputed from the log, exactly like the messages are.

It has **no wire** — it is host-only. `openTurnStartSeq` is also how a reader can tell a turn was left open by a crash, which Chapter 30 uses.

6. Control decisions

Decision	Condition	Location
Reject registration	<code>stateVersion</code> not a non-negative safe integer	<code>index.ts:241-281</code>
Reject duplicate	same key registered with a different <code>stateVersion</code>	<code>index.ts:267-269</code>
Share registration	same key, same version	<code>index.ts:264-278</code>
Skip downstream work	<code>apply</code> returned the same reference	<code>index.ts:637-657</code>
Skip notification	view reference unchanged	<code>index.ts:637-657</code>
Return <code>undefined</code>	key not registered	<code>index.ts:307-315</code>
Discard cached row	<code>ver</code> mismatch, or <code>seq</code> outside the usable window	<code>index.ts:482-522</code>

7. Edge cases and failure modes

Cold restore is guarded against stale checkpoints. `restoreFloor(checkpoint)` (`index.ts:413-423`) computes the lowest usable watermark across all units, minus one — deliberately one *behind*, so a suffix read can detect that the log has since shrunk below a stale checkpoint's watermark (crash-repair truncation) instead of trusting the row. If a row is unusable while `baseSeq > 0`, `restore` **throws**, forcing the caller to re-read from seq 0 rather than silently producing a fold over the wrong prefix.

Checkpoints are detached copies. `checkpoint()` writes `structuredClone(state)`, "never the live cell reference: the watermark cache is this registry's authoritative mutable state" (`index.ts:384-395`).

An unloaded plugin's key vanishes. Because registration is an effect, unmounting a domain plugin removes its key from snapshots; clients are expected to read absence as "this deployment does not have that capability."

The persisted cache, verified: `packages/session/session-projection-cache`, mounted in the base composition with `writeEveryEvents: 200` and `writeIntervalMs: 5000` (`packages/bundle/base/cordis.patch.yml:162-166`).

It injects `['storageDomain', 'sessionProjections', 'sessions']` (`src/index.ts:78`) and opens a schema-validated KV domain at `init` (`:91`), keeping **one record per session** on the `session_projcache` domain in `per-record` layout, over the shipped JSON backend. The base bundle supplies that stack — `storage`, `storage-json` rooted at `dshHomePath('storages')`, and `storage-domain` configured `backend: json`.

The two config values are throttle bounds *between mandatory checkpoint points*, not the only times it writes: `writeEveryEvents` is "committed events per session that force a durable checkpoint write between mandatory points," and `writeIntervalMs` "the longest time a dirty checkpoint may stay unwritten" (`:49-52`). Reads are served synchronously from the domain's in-memory tables, so a cache read is never stale-but-wrong — at worst it is older than the last durable write.

8. Configuration knobs

Setting	Default	Effect
<code>writeEveryEvents</code>	<code>200</code>	Throttled write-behind cadence by event count
<code>writeIntervalMs</code>	<code>5000</code>	...and by elapsed time

— `packages/bundle/base/cordis.patch.yml:162-166`

Per-projection, `stateVersion` is the only knob, and it is a code constant rather than configuration.

9. Interactions

- **Ch 5** — every committed append drives every registered fold.
- **Ch 13** — `turnBoundary.lastTurn` seeds a resumed agent's phase.
- **Ch 25** — the retry plugin counts retries in a projection (`llmRetry`), which is why a retry budget survives a restart.
- **Ch 30** — `openTurnStartSeq` marks a turn left open by a crash.

10. Build it yourself

Minimal version:

```
function project<S>(events: readonly SessionEvent[], init: S, apply: (s: S, e: SessionEvent) => S): S {
  return events.reduce(apply, init)
}
```

That is the whole idea. What the real one adds:

Addition	Why it exists
A registry keyed by name	Consumers should not each re-walk the log
<code>stateVersion</code>	A changed fold must discard old caches, not forward-apply them
RefCounted registration	The same package mounts under several presets
<code>Object.is</code> double debounce	Most events are irrelevant to most folds; notifying on every append would flood clients
Checkpoint / <code>restoreFloor</code> / <code>hydrate</code>	A cold start should not re-fold a 10,000-event log
Separate <code>state</code> and <code>wire</code>	Host-side folds often hold more than a browser should see

Key takeaways

- A projection is a registered, versioned, pure fold — the general form of what `deriveMessages` does for messages.
- `apply` must return the *same reference* when uninterested; that identity check is the main performance mechanism.
- Notification is debounced twice: state identity, then view identity.
- `stateVersion` exists so a changed fold invalidates its persisted cache instead of corrupting it.
- The engine's own `turnBoundary` projection is how a resumed session knows its turn number — recomputed, never stored.

Exercises

1. `apply` returns `state` unchanged for ~46 of the ~50 event types. What concretely would degrade if it returned `{ ...state }` instead?
2. `stateOf` returns `undefined` for an unregistered key, and clients treat that as capability absence. Why is that better than returning the `init` value?
3. Design a projection answering "how many tool calls failed in this session." Give its `key`, `init`, `apply`, and decide whether it needs a `wire` view.

Next: [Chapter 9 · The turn and step loops](#)

Chapter 9 · The turn and step loops

What you'll learn: every branch point in the engine's control flow — when a turn continues, when it stops, and which of the six turn endings it records.

Prerequisites: [Chapter 4](#) for the shape, [Chapter 7](#) for `deriveMessages` .

1. The problem

"Call the model, run its tools, repeat" hides a surprising number of decisions. When exactly does a turn end? What if the model asks for tools *and* the user sends a new message while they run? What if one step hits the token ceiling and the next completes normally — did the turn succeed? What if a plugin wants to reject a step before it starts?

Every one of these needs an answer that is consistent, recorded, and replayable. This chapter is the full inventory.

2. Mental model

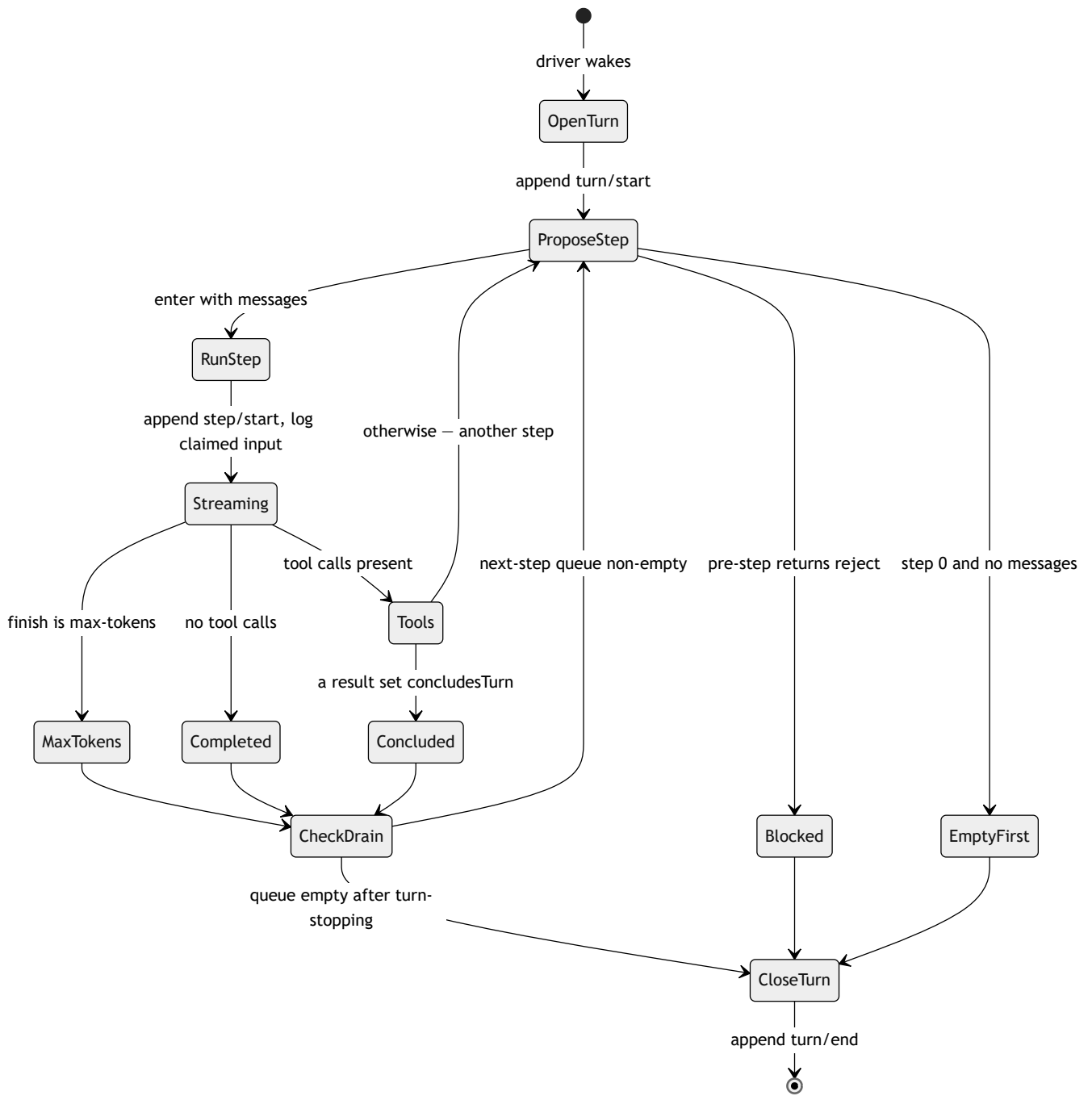
Three nested loops, each with exactly one exit condition:

Loop	Iterates	Exits when
driver	turns	<code>turn()</code> returns <code>false</code>
turn	steps	an ending is set <i>and</i> nothing is queued for a next step
step	request attempts	the request succeeds, or nobody asks for a retry

Confusing the turn loop with the step loop is the single easiest way to misread this engine. A *turn* is one unit of conversation — roughly, one user prompt and everything the model does in response. A *step* is one model request within it. A turn containing three tool round-trips has four steps.

New term — turn ending. A `TurnEndReason` recording *why* a turn stopped. It is set during the loop and written once, in a `finally` , so a turn always gets exactly one ending no matter how it exits.

3. State diagram



4. The driver

```
private async kick(): Promise<void> {
  try {
    while (await this.turn()) {}
  } catch (_error) {
    // Reported failures and cancellation are contained at the driver boundary.
  } finally {
    if (this.phase.kind === 'running') {
      const { turn, wakeRequested } = this.phase
      this.setPhase({ kind: 'idle', lastTurn: turn })
      if (wakeRequested && this.inbox.hasPending) this.wakeDriver()
    }
  }
}
```

— packages/core/agent-loop/src/agent.ts:219-232

The empty `catch` looks alarming and is correct. By the time an error reaches here it has already been reported: `throwError()` (`agent.ts:212-217`) emits `agent/error` with the turn and step position *before* rethrowing. The driver's job is to stop looping, not to report. Swallowing here is what keeps one failed turn from killing the agent.

The `finally` returns to idle and replays a latched wake ([Ch 13](#)).

5. The turn loop, branch by branch

`turn()` — `agent.ts:255-339` . Entry appends `turn/start` , and even that append is guarded: a failure routes through `throwError` (`:263-267`).

Inside `while (true)`:

- ① **Abort check.** `signal.throwIfAborted()` (`:273`). The first of six in this function.
- ② **Propose a step.** `preStep(target, {turn, step})` (`:275`) runs the `agent/pre-step` waterfall ([Ch 22](#)).
- ③ **Rejected.**

```
if (decision.kind === 'reject') {
  turnEnds = { kind: 'blocked' }
  return false
}
```

— `:276-279`

`return false` stops the **driver**, not just the turn. A rejected step is a hard stop: no further turns are attempted.

- ④ **Already ended, nothing new.** `if (turnEnds && decision.messages.length === 0) break` (`:280`).

⑤ First step with nothing to say.

```
if (phase.step === 0 && decision.messages.length === 0) {
  turnEnds = { kind: 'completed' }
  return false
}
```

— :283–286

A waking message that was cleared before the step opened, or a pre-step listener that rewrote the messages to empty. The comment is precise: such a turn "still owns the initial turn boundary, but it spends no model call." **A turn can open and close without ever contacting the model** — it will appear in the log as `turn/start` immediately followed by `turn/end`.

⑥ **Open the step and log input.** Append `step/start`, then one `user/message` per claimed message with `surfaceOp: 'append'` (:288–293). This is where injected context becomes ordinary logged history.

⑦ **Run the step** and fold its ending:

```
const stepEnd = await this.step(decision.assembly, decision.startsRequestSeries === true)
if (turnEnds === null || turnEnds.kind !== 'max-tokens') turnEnds = stepEnd
```

— :296–299

max-tokens is sticky. Once any step hits the ceiling, a later step completing normally cannot downgrade the turn's outcome. Without this, a turn that truncated mid-answer and then ran a cleanup step would be recorded as a clean success.

⑧ **Always close the step.** `step/end` is appended in a `finally` (:301), including on throw.

⑨ **Last chance to continue.**

```
if (turnEnds && this.inbox.nextStep.length === 0) {
  await this.dispatch.serial('agent/turn-stopping', { turn, signal })
  signal.throwIfAborted()
}
if (turnEnds && this.inbox.nextStep.length === 0) break
```

— :304–308

The condition is evaluated, listeners run, then it is **evaluated again**. A listener that wants the turn to continue calls `agent.steer(...)`, which puts a message in `nextStep`, and the re-test sees it. Note this is *data-driven*: listeners do not vote by return value, so their registration order cannot change the outcome.

In the shipped configuration this extension point has **zero live listeners** — its only implementations are the hook bridges, which are not mounted ([Ch 22](#)).

⑩ **Otherwise continue.** `target = 'next-step' ( :309 )` — after the first step, input comes from the step queue rather than the turn queue.

Exits

On throw (:311–324): if the signal aborted, the ending is `{kind: 'aborted', reason}` and the error rethrows. Otherwise the failure is *structured* — an `LlmError` keeps its `failure` ; anything else flattens:

```
turnEnds = {
  kind: 'error',
  error: error instanceof LlmError
    ? error.failure
    : { message: errorChain(error), code: 'UNKNOWN' },
}
```

— :318–323

So the log never holds an unstructured error. Every failure has a code, even if it is `UNKNOWN` .

Always (:325–332): append `turn/end` with the accumulated reason.

Then decide whether to run another turn:

```
if (!this.inbox.hasPending) return false
phase.abort = new AbortController()
phase.wakeRequested = false
phase.step = 0
return true
```

— :333–338

A fresh abort controller per turn. This is why a cancellation aborts one turn rather than the agent, and why a wake latched against the old controller is stale — hence clearing `wakeRequested` on the same lines ([Ch 13](#)).

6. The step loop, branch by branch

`step()` — `agent.ts:341–438` . Returns `StepEndReason | null` , where `null` means "no ending yet — run another step."

Inside `while (true)` (this is the *retry* loop):

① **Capture the surface generation** (:349) before building, for request-series detection ([Ch 10](#)).

② **Build the request** (:350–359), then `startsRequestSeries = false` (:360) so only the first attempt can open a series.

③ **Stream.**

```
const stream = preparedCall?.stream(request) ?? this.loopCtx.llm.stream(request)
```

— :364

The prepared call is preferred; the bare `llm.stream` is the fallback for when no adapter resolved ([Ch 23](#)).

④ **Per chunk** (:366–370): abort check, append `assistant/chunk`, collect its `seq`, push into the assembler. Every chunk is logged individually — which is why Chapter 30's packing codec exists.

⑤ **On throw during streaming** (:372–389): **only if the signal aborted**, salvage `assembler.interruptedBlocks()` and, if non-empty, append an `assistant/message` with `interrupted: true` citing the chunks so far. Then rethrow regardless. A cancelled turn keeps whatever the model managed to say.

⑥ **Request failed.**

```
if (finish.kind === 'error' || finish.kind === 'aborted') {
  const action = await this.dispatch.waterfall('agent/request-error', {...},
    () => Promise.resolve<RequestErrorAction>(undefined))
  signal.throwIfAborted()
  if (action?.kind !== 'retry') {
    throw new LlmError(finish.failure.message, finish.failure.code, finish.failure)
  }
  continue
}
```

— :390–408

The loop itself never decides to retry. Its built-in default is `undefined`, which is terminal. A listener must return `{kind: 'retry'}` for `continue` to be reached — and `continue` rebuilds the request from scratch, picking up any history changes a listener made in the meantime. That is exactly how compaction-on-overflow works ([Ch 27](#)).

⑦ **Commit the assistant message** (:410–427), citing every chunk seq.

⑧ **Ceiling hit.** `if (finish.kind === 'max-tokens') return { kind: 'max-tokens' } ( :428 ).`

⑨ **Nothing more to do.** `if (toolCalls.length === 0) return { kind: 'completed' } ( :430–431 ).`

⑩ **Run the tools.**

```
const { concluded } = await executeToolCalls(
  this.loopCtx, turn, step, toolCalls, signal,
  context => this.inbox.splice('next-step', this.inbox.nextStep.length, 0, [context]),
)
return concluded ? { kind: 'completed' } : null
```

— :432–436

The callback is how a tool's `additionalContexts` reach the next step: appended to the `next-step` queue, so they are claimed at the next boundary like any other input (Ch 17).

The single condition for a turn to continue: the model emitted at least one tool call, **and** no committed result set `concludesTurn`.

7. Edge cases and failure modes

Six abort checks in `turn()`. At `:261`, `:273`, `:287`, `:303`, plus `throwIfAborted` inside `preStep` twice. Cancellation is checked before each boundary append, so an aborted turn cannot leave a `step/start` without a `step/end`.

A failed `turn/end` append is still reported. The `finally` wraps its own append in `try/catch` and routes failure through `throwError` (`:328-331`) — a storage failure while closing a turn does not vanish silently.

Turn endings can be recorded for turns that never ran a model call. Branch ⑤. Worth knowing when reading logs.

`step()` can loop indefinitely in principle. The `retry while (true)` has no iteration cap of its own; bounding is entirely the listener's job. `llm-retry` enforces `maxRetries` for `normal` policies but retries **indefinitely** under an `always` policy (Ch 25). The engine trusts the listener.

8. Configuration knobs

The loops have none. The only engine-level tunable is `maxParallelToolCalls`, which belongs to the scheduler (Ch 17). Every other lever — whether to retry, whether to compact, whether to reject a step — lives in a plugin.

9. Interactions

- **Ch 12** — supplies claimed messages, and `nextStep.length` is the turn-continuation condition.
- **Ch 13** — owns the abort controller these loops check.
- **Ch 10** — called once per step attempt.
- **Ch 17** — returns `concluded`, the only tool-side way to end a turn.
- **Ch 22** — `agent/pre-step`, `agent/request-error`, and `agent/turn-stopping` all fire here.
- **Ch 27** — the sole reason `continue` at ⑥ is useful in the shipped system.

10. Build it yourself

The minimal turn loop from Chapter 4 already has the shape. The additions, ordered by how likely you are to need them:

Addition	Why it exists
<code>null</code> vs an ending as the step's return	Distinguishes "keep going" from "done" without a second flag
Sticky <code>max-tokens</code>	A later clean step must not mask a truncated answer
<code>turn-stopping</code> re-test	Gives plugins a final chance to continue, without letting listener order decide
Fresh abort controller per turn	Cancellation should scope to a turn, not kill the agent
Structured error endings	Replay and UI need a code, never a bare string
Abort check before each boundary append	Prevents unbalanced <code>step/start</code> with no <code>step/end</code>
Interrupted-message salvage	Cancelling mid-answer should not discard what was already said
Retry delegated to a waterfall	Retry policy is deployment-specific; the loop should not own it

Key takeaways

- Three loops: turns, steps, request attempts. Each has one exit condition.
- A turn continues only when the model emitted tool calls and no result concluded the turn.
- `max-tokens` is sticky; a later clean step cannot upgrade the outcome.
- A rejected pre-step stops the driver entirely, not just the turn.
- `agent/turn-stopping` is evaluated, then re-evaluated — listeners act by queueing data, not by returning a vote.
- The engine never retries on its own; the default action is terminal.
- Every failure becomes structured before it reaches the log.

Exercises

1. Trace what is logged when a pre-step listener returns `reject` on the very first step of a session. List every event, in order, and give the turn's ending.
2. A step hits `max-tokens`, and a tool result in the *next* step sets `concludesTurn`. What ending does the turn record, and which line decides it?
3. `step()`'s retry loop has no iteration cap. Write the smallest listener that would hang an agent forever, then name the two mechanisms that would still stop it ([Ch 13](#) has both).

Next: [Chapter 10 · Building the request](#)

Chapter 10 · Building the request

What you'll learn: how one model request is assembled, why the engine logs a header only sometimes, and the small dance that keeps one model's defaults from leaking onto another.

Prerequisites: [Chapter 7](#), [Chapter 9](#).

1. The problem

A request needs a provider, a model, a system prompt, a tool list, some generation parameters, and the message history. The history is solved ([Ch 7](#)). The rest is harder than it looks, for three reasons.

The route can change mid-session. A user switches models. The request must follow, but the *previous* model's resolved defaults must not come along — a reasoning-effort level valid for one model may not exist on another.

Some parameters come from the adapter, not the caller. If nobody specified `maxTokens`, the adapter fills in the model's default. That value is now in the config, indistinguishable from one a user chose — unless something records the difference.

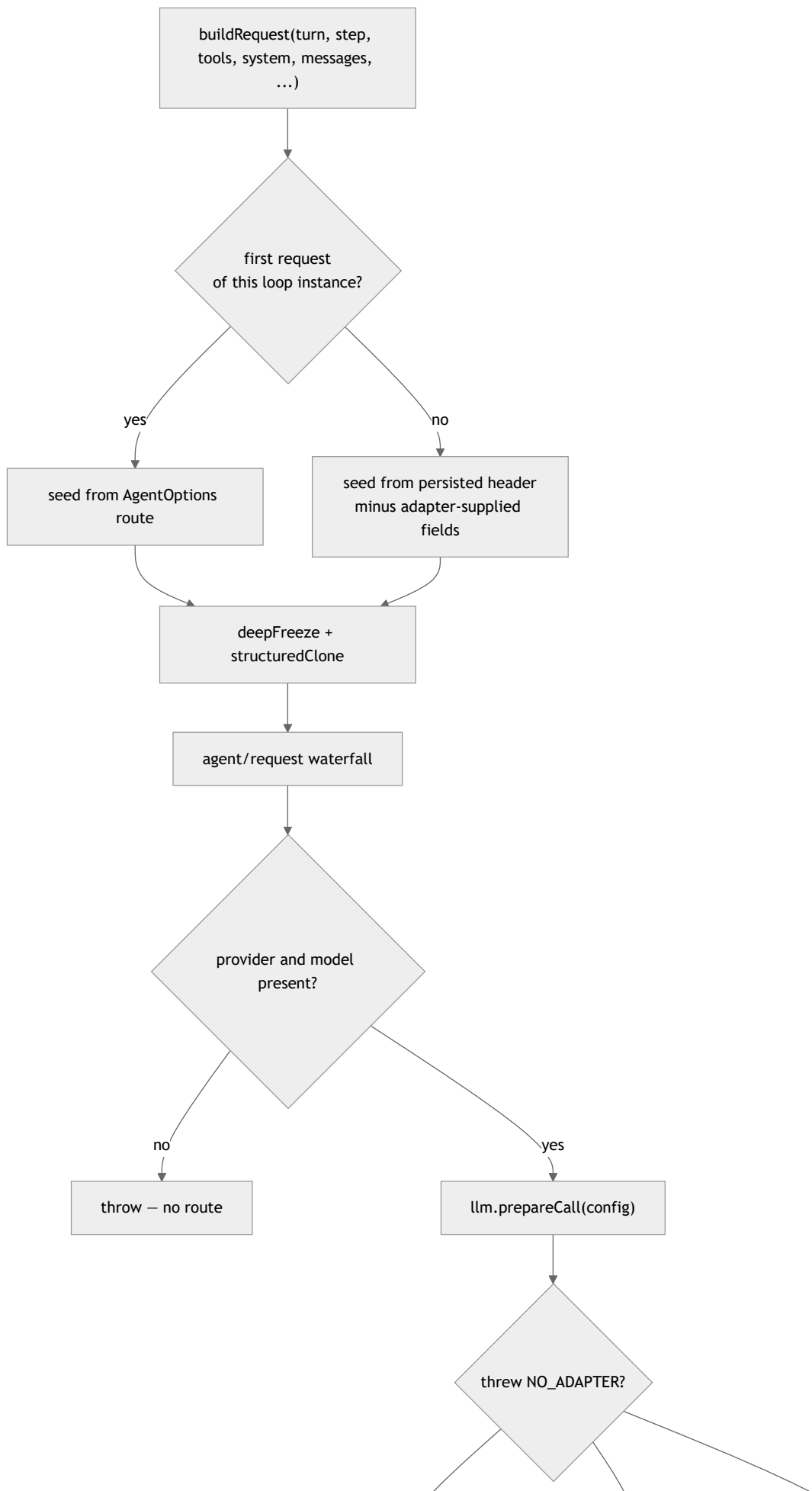
The request must be reconstructable. Chapter 1's thesis requires that a reader of the log can rebuild exactly what was sent. Messages come from the surface, but the system prompt and tool schemas are not messages. They have to be recorded too — without writing a copy of a 35 KB tool schema on every single step.

2. Mental model

New term — request header. The non-message half of a request: config, system prompt, tool schemas, plus a note of which fields the adapter supplied. Logged as a `request/header` event **only when it changes**.

So the log holds a sparse series of headers, and the header in force for any step is the most recent one at or before it. Chapter 11's check calls this a *fold*. It means a thousand-step session with a stable configuration contains one header event, not a thousand.

3. Lifecycle





4. Step-by-step walkthrough

`buildRequest` — `packages/core/agent-loop/src/agent.ts:444-544`.

Seeding the config

On the **first** request of this loop instance, the seed is the declared route plus whatever the caller specified:

```
{ ...route, ...reasoningEffort && { reasoningEffort }, ...maxTokens && { maxTokens } }
```

— `:472-476`

On **later** requests, the seed is the previous header put through `requestProposal`:

```
function requestProposal(header: EpochHeader): LlmCallConfig {
  if (header.adapterDefaults === undefined) return header.config
  const proposal = { ...header.config }
  if (header.adapterDefaults.reasoningEffort === true) delete proposal.reasoningEffort
  if (header.adapterDefaults.maxTokens === true) delete proposal.maxTokens
  return proposal
}
```

— :61–67

This is the answer to the second problem. `adapterDefaults` marks which fields the adapter supplied; `requestProposal` **strips exactly those** so they get re-resolved against whatever model is in force now. A user-chosen `maxTokens` survives; an adapter-chosen one does not.

Restoring a persisted reasoning effort

```
const persistedReasoningEffort = persistedConfig?.provider === route.provider
  && persistedConfig.model === route.model
  && persistedHeader?.adapterDefaults?.reasoningEffort !== true
  ? persistedConfig.reasoningEffort
  : undefined
```

— :461–465

Three conditions, all required: same provider, same model, and the value was not adapter-supplied. A resumed session keeps an explicitly-chosen effort level — but only if it is resuming onto the **exact** model that owned it.

The proposal waterfall

The seed is frozen (`deepFreeze(structuredClone(...))`, :468) and offered to listeners:

```
const proposedConfig = await this.dispatch.waterfall('agent/request', { turn, step, signal },
  () => Promise.resolve(seedConfig))
```

— :478–481

This is where a `/model` switch takes effect — `installModelSelection` overrides provider, model, and reasoning effort here (Ch 22). The waterfall deliberately **cannot touch messages**: "Model-visible content must use logged channels; this waterfall cannot mutate messages" (`packages/core/agent/src/runtime-types.ts:242–243`). Config is not model-visible content; messages are, and they must come from the log.

Then a hard failure if the route is still incomplete (:483–485).

Resolving against the adapter

```
try {
  preparedCall = await this.loopCtx.llm.prepareCall(proposedConfig, signal)
  config = preparedCall.config
} catch (error: unknown) {
  if (!(error instanceof LlmError) || error.code !== 'NO_ADAPTER') throw error
  config = proposedConfig
}
```

— :488–495

Note the narrowness: **only** `NO_ADAPTER` falls through, and only to keep using the unresolved config. Every other failure — an unsupported reasoning effort, invalid model metadata — propagates and fails the step. The comment explains the exception: "Middleware may serve an unregistered route." Something could intercept `llm/stream` and answer without a registered adapter.

In the shipped configuration this fallback matters more than it sounds. With no provider configured, *every* request takes it, and `preparedCall` stays `undefined` — which also means no retry policy ([Ch 32](#)).

The three-way logging decision

```
if (!this.requestHeaderLogged) {
  this.session.append('request/header', { header, reason: baseline === undefined ? 'initial' :
'resume' })
  this.requestHeaderLogged = true
} else if (baseline === undefined || !headerEquals(baseline, header)) {
  this.session.append('request/header', { header, reason: 'change',
...startsSeries ? { startsSeries: true } : {} })
} else if (startsSeries) {
  this.session.append('request/header', { header, reason: 'series' })
}
```

— :507–518

Four reasons, each meaning something distinct:

Reason	When
<code>initial</code>	This loop instance's first request, and the log had no header — a fresh session
<code>resume</code>	This loop instance's first request over a log that already has headers
<code>change</code>	The header differs from the folded baseline
<code>series</code>	The header is unchanged, but a new message series began

And the fourth case: unchanged, not a new series → **nothing is appended**.

What starts a series

```
const startsSeries = startsRequestSeries || this.requestSurfaceGeneration !== surfaceGeneration
```

— :505–506

Either the caller said so, or **the surface was rewritten since the last header this instance logged**. That second clause is compaction's signature: after history is replaced, the next request is not a continuation of the previous message series even though the header is byte-identical. A reader folding the log needs to know that, and `replaceGeneration` ([Ch 6](#)) is what tells it.

Request context

```
if (previousContext?.provider !== requestContext.provider
    || previousContext.model !== requestContext.model
    || previousContext.contextWindow !== requestContext.contextWindow) {
  session.append('request/context', requestContext)
}
```

— :527–532

A separate, smaller event carrying provider, model, and context window — for presentation and telemetry, not reconstruction. Also only on change.

Freezing and marking

```
const request = markAgentLoopRequest(deepFreeze({
  ...header.config,
  messages: boundaryMessages,
  ...header.system !== undefined ? { system: header.system } : {},
  ...header.tools !== undefined ? { tools: header.tools } : {},
  sessionId: this.session.id,
  signal,
}))
```

— :535–542

`markAgentLoopRequest` adds the object to a `WeakSet` (`packages/llm/llm/src/call-config.ts:66–78`). It is a **process-local identity tag**, not a field — it marks *this exact object*, never survives serialization, and never appears in the type. Its purpose: a listener on `llm/stream` can distinguish a loop-built request (frozen, derived from the log, must not be rewritten) from an ad-hoc one-shot call such as compaction's own summarization request.

5. Data at each stage

From the recorded turn (`session.jsonl:12–13`), the first request of a fresh session:

Stage	Value
Seed	<code>{ provider: 'deepseek-official', model: 'deepseek-v4-flash' }</code>
After <code>agent/request</code>	unchanged — no listener modified it
After <code>prepareCall</code>	<code>maxTokens</code> filled from model metadata; <code>adapterDefaults: { maxTokens: true }</code>
Header logged	<code>reason: 'initial'</code> , with <code>system</code> and <code>tools</code> (elided as <code>{{system}}</code> / <code>{{tools}}</code> in the fixture)
Context logged	<code>{ provider: 'deepseek-official', model: 'deepseek-v4-flash' }</code>
Request messages	2, from <code>deriveMessages()</code>

On the next step with the same route, the header comparison matches and **no header event is written**.

6. Control decisions

Decision	Condition	Location
Strip adapter-supplied fields	<code>adapterDefaults</code> marks them	:61–67
Restore persisted effort	same provider and model and not adapter-supplied	:461–465
Fail the step	proposal lacks provider or model	:483–485
Tolerate missing adapter	<code>LlmError</code> with code exactly <code>NO_ADAPTER</code>	:491–495
Log header	never logged / differs / unchanged-but-new-series	:507–518
Log context	provider, model, or context window changed	:527–532

7. Edge cases and failure modes

A resumed session logs `resume`, not `initial`. The distinction is per *loop instance*, not per session: `requestHeaderLogged` is an instance field (`agent.ts:84`). Reopening a session produces a new anchor header even if nothing changed, which is what a folding reader needs to know a new process took over.

`headerEquals` compares the whole header, including the tool schema array. A tool being registered or unregistered mid-session therefore produces a `change` header — correct, because the model's options genuinely changed.

Compaction produces a `series` header with identical bytes. Easy to mistake for a redundant write when reading a log. It is not: the bytes match, the meaning does not.

The signal is in the frozen request. `deepFreeze` covers the object graph, but `signal` is a live `AbortSignal` — frozen as a reference, still able to fire. Chapter 11's check asserts the *messages* array is frozen, not that the whole request is inert.

8. Configuration knobs

Setting	Default	Effect
<code>AgentOptions.provider / .model</code>	from <code>agent-default-model</code> : <code>deepseek-official</code> / <code>deepseek-v4-flash</code>	The declared route this instance starts from
<code>AgentOptions.maxTokens</code>	unset	Validated as a positive safe integer at agent construction (<code>index.ts:198-203</code>)
<code>AgentOptions.reasoningEffort</code>	unset	Falls back to the persisted value under the three-condition rule

— `packages/bundle/base/cordis.patch.yml:73-79`

9. Interactions

- [Ch 7](#) — supplies `messages` .
- [Ch 19](#) — supplies `system` and `tools` .
- [Ch 23](#) — `prepareCall` and `adapterDefaults` .
- [Ch 6](#) — `replaceGeneration` drives series detection.
- [Ch 11](#) — checks this function's output against the log.
- [Ch 22](#) — `agent/request` is where model switching lands.

10. Build it yourself

Minimal version:

```
const request = {
  provider: this.options.provider,
  model: this.options.model,
  system: renderPrompt(assembly),
  tools: assembly.tools,
  messages: this.session.deriveMessages(),
}
```

What the real one adds:

Addition	Why it exists
<code>adapterDefaults</code> + <code>requestProposal</code>	Otherwise one model's defaults silently follow you to another model
The three-condition effort restoration	A persisted effort level is only meaningful on the model that had it
<code>agent/request</code> waterfall	Model switching must not require touching the loop
<code>NO_ADAPTER</code> -only fallback	Middleware may serve unregistered routes; every other failure is real
Change-only header logging	A copy of the tool schemas on every step would dwarf the conversation
<code>series</code> reason	A byte-identical header can still begin a new series after a rewrite
<code>deepFreeze</code> + <code>markAgentLoopRequest</code>	Lets an observer tell "derived from the log" from "hand-built", and prevents rewriting the former

Key takeaways

- The header is logged only on change, with four distinct reasons; a stable session logs one.
- `adapterDefaults` records which fields the adapter filled in so they can be re-resolved, not inherited.
- A persisted reasoning effort survives resume only on the exact same provider *and* model.
- A surface rewrite starts a new request series even when the header bytes are unchanged.
- Only `NO_ADAPTER` is tolerated from `prepareCall`; everything else fails the step.
- `markAgentLoopRequest` is a process-local `WeakSet` tag, not a serialized field.

Exercises

1. A user switches from model A to model B mid-session, having never set `maxTokens`. Walk through `requestProposal` and say exactly what the request to B contains. Now repeat with the user having explicitly set `maxTokens: 4096`.
2. Compaction runs, then the next step builds a request with an identical header. Which reason is logged, and what would a log reader get wrong if that event were skipped?
3. `markAgentLoopRequest` uses a `WeakSet` rather than a field on the request. Give two things that would break if it were a boolean field instead.

Next: [Chapter 11 · The reconstruction invariant](#)

Chapter 11 • The reconstruction invariant

What you'll learn: the code that states this engine's central claim as a runnable assertion — and the more interesting fact that it ships switched off.

Prerequisites: [Chapter 7](#), [Chapter 10](#).

1. The problem

Chapter 1 made a strong claim: every request is a pure function of the session log. Chapters 7 and 10 showed the machinery. But a claim like that decays. Someone adds a field to the request that is not in the header. Someone caches a message array "just for this one path." Someone passes `messages` through a transform before dispatch. Each change is locally reasonable and each quietly breaks resume, fork, and replay — in ways that surface much later, as a session that replays differently than it ran.

A comment saying "must equal the derivation" does not catch that. An assertion does.

2. Mental model

New term — invariant companion. A small plugin a package ships alongside itself, checking a durable relationship *its own package owns*, at runtime, against live data.

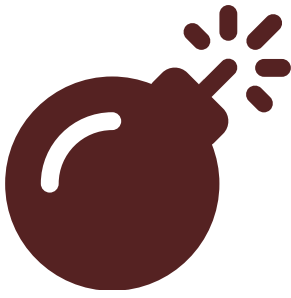
The repository has about thirty of these, one per package, all with the same shape: `src/invariant.ts`, exporting `name`, `inject`, and an `apply` that calls `ctx.invariants.register(PACKAGE_NAME, install)`. The engine's checks one thing: that what goes out matches what the log says should go out.

The design rule for what qualifies is narrow and worth stating, because it explains why there are not more of these:

"A companion checks an event-stream or mutable-data relationship its package owns; confirming a method, plugin name, injection, or fixed pure result is a type, load, or unit-test concern, never a runtime invariant." — `packages/runtime-diagnostics/invariants/README.md`

Checking that a function exists is a compiler's job. Checking that two independently-computed views of live state agree is not.

3. Lifecycle



Syntax error in text

mermaid version 11.16.0

4. Step-by-step walkthrough

The whole check is 35 lines (`packages/core/agent-loop/src/invariant.ts:19-55`).

Where it attaches

```
ctx.on('llm/stream', (options: GenerateOptions, next) => {  
  if (!isAgentLoopRequest(options)) return next()  
  ...  
}, { global:true, prepend: true })
```

— `invariant.ts:21, :54`

Two flags carry weight. `global: true` means it sees every stream regardless of scope. `prepend: true` puts it at the front of the waterfall — and the comment says why: "Prepend prevents a short-circuiting replay listener from silencing the check." A listener that answers a request without delegating ([Ch 22](#)) would otherwise skip everything registered behind it, including this. A check that can be bypassed by registration order is not a check.

`isAgentLoopRequest` is the `WeakSet` membership test from [Chapter 10](#). One-shot calls — compaction's summarization, session titling — pass straight through.

The structural checks

```
if (!Object.isFrozen(options)) fail('a loop-built request must be frozen')  
if (options.sessionId === undefined) fail('a loop-built request must carry a session id')  
const session = ctx.sessions.get(options.sessionId)  
if (!session) fail('a loop-built request must carry a live session id, got "...")  
if (!Object.isFrozen(options.messages)) {  
  fail('a loop-built request must carry a frozen messages array')  
}
```

— `:23-30`

Frozen, identified, and the identity resolves to a session that is still live. The messages array is checked separately from the request itself — freezing the outer object does not freeze the array.

The log must be able to explain the request

```
const events = session.events
if (!events.some(event => event.type === 'step/start')) {
  return fail('a loop-built request with no step/start in its session log')
}
const header = foldRequestHeader(events)
if (header === undefined) {
  return fail('a loop-built request with no request/header event in its session log')
}
```

— :31–38

A request being dispatched means a step opened, which means `step/start` is in the log. And [Chapter 10](#)'s sparse header series must fold to something. Both are preconditions for the real comparison.

The central assertion

```
const expected = session.deriveMessages()
if (JSON.stringify(options.messages) !== JSON.stringify(expected)) {
  fail(`llm request for session "..." diverges from the dispatch-time durable derivation (log-reconstruction desync)`)
}
```

— :39–42

This is Chapter 1's thesis, executable. Not "similar", not "equivalent modulo formatting" — the same JSON. Crucially it re-derives **at dispatch time**, from inside the dispatch path, so it catches drift introduced anywhere between building the request and sending it.

And the non-message half

```
const headerMatches = options.model === header.config.model
  && options.system === header.config.system
  && options.temperature === header.config.temperature
  && options.maxTokens === header.config.maxTokens
  && JSON.stringify(options.stop) === JSON.stringify(header.config.stop)
  && JSON.stringify(options.tools ?? []) === JSON.stringify(header.config.tools ?? [])
```

— :44–49

Messages come from the surface; everything else comes from the folded header. Together they cover the whole request, which is what makes "reconstructable" a complete statement rather than a partial one.

5. Control decisions

Decision	Condition	Location
Skip entirely	not a loop-built request	: 22
Fail	request not frozen	: 23
Fail	no session id, or id does not resolve	: 24–26
Fail	messages array not frozen	: 27–29
Fail	no <code>step/start</code> in the log	: 32–34
Fail	header fold yields nothing	: 35–38
Fail	messages $\neq$ derivation	: 39–42
Fail	any header field diverges	: 44–52
Delegate	all checks pass	: 53

6. It does not run in the shipped system

Here is the part that matters most, and it took direct checking to establish.

The registry that runs these companions is mounted nowhere. `dsh-invariants` lives at `packages/runtime-diagnostics/invariants`. Grepping every `.yaml` and `.yml` in the repository for it returns only `pnpm-lock.yaml` dependency entries — **zero plugin rows**, in the base bundle, the web-app bundle, the standard agent preset, or any example overlay. There is no environment-variable seam either.

The package's own documentation is explicit about the intent:

"`dsh-base` deliberately omits runtime diagnostics. Custom compositions mount the registry and add companions for any other loaded package whose contracts they want checked. Loading the registry alone installs no checks." — `packages/runtime-diagnostics/invariants/README.md`

So in the running product, nothing checks this. The code is real, correct, and dormant.

⚠️ **Docs contradict themselves here.** The same README's summary paragraph claims "the standard agent composition already mounts it with the four core companions." The composition files say otherwise, and per this book's ground rules the composition files win. Flagged for the evidence report.

Why this is still worth a chapter

Three reasons, and the first is the most important.

It is an executable specification. The clearest statement of what this engine guarantees is not a paragraph of prose — it is 35 lines that would fail if the guarantee broke. Read it as the definition of "reconstructable." Every design decision in Chapters 5 through 10 exists to make this assertion true.

It is the test oracle. The check runs wherever a composition mounts it, which includes test compositions — `packages/workflow/workflow-worker-thread/tests/integration.spec.ts:9` imports it directly. So the guarantee is enforced during development, just not in production.

 **Verified (run).** The engine ships two suites dedicated to this contract: `packages/core/agent-loop/tests/request-reconstruction.spec.ts` (24 cases) and `tests/invariant.spec.ts` (8 cases). Running `npx vitest run packages/core/agent-loop` on this checkout gives **342 passed, 0 failed, across 18 files**. So the reconstruction guarantee is not merely stated and not merely implemented — it is exercised by 32 dedicated cases on every test run, and they pass. What ships disabled is the *runtime* enforcement, not the verification.

Switching it off is a defensible choice, and worth understanding. The check calls `deriveMessages()` and two `JSON.stringify` passes over the entire history, on every request. On a long conversation that is real work on the hot path, to verify something that CI already covers. The registry exists precisely so a deployment debugging a suspected desync can turn it on.

7. Edge cases and limitations

One-shot calls are outside the contract. The package states it plainly: "Request reconstruction covers loop-built requests only — the `dsh-agent-loop` companion reconstructs requests explicitly built by the loop; direct one-shot LLM calls remain outside that contract even when callers freeze them or attach a session id." Compaction's summarization request (Ch 27) is exactly such a call: it carries a session id and is frozen, and it is deliberately *not* checked, because it is not meant to equal the derivation — it is a different prompt entirely.

`JSON.stringify` **compares key order.** Two structurally equal objects with different insertion order would fail. In practice both sides come from the same frozen event data, so ordering is stable — but it makes the check stricter than deep equality, which is the safe direction.

A failure is attributed, not anonymous. An `InvariantError` carries the code `INVARIANT` and the owning package's full npm name, with the message prefixed `invariant violated by "<package>": ...` — so a violation names who owns the broken relationship without the registry importing any product code.

8. Configuration knobs

Only meaningful in a composition that mounts the registry:

Setting	Default	Effect
<code>enabled</code>	<code>true</code>	Global switch
<code>package_allowlist</code>	<code>[]</code>	Regex sources; empty admits all
<code>package_blocklist</code>	<code>[]</code>	Applied after the allowlist; a blocklist match wins

Filters are compiled once at startup — changing them needs a plugin reload.

9. Interactions

- **Ch 7** — calls `deriveMessages()` as the expected value.
- **Ch 10** — supplies both the `WeakSet` tag and the header series this folds.
- **Ch 22** — `prepend: true` exists because of waterfall short-circuit semantics.
- **Ch 27** — the main producer of requests this deliberately ignores.

10. Build it yourself

Minimal version — put it right before dispatch:

```
if (JSON.stringify(request.messages) !== JSON.stringify(session.deriveMessages())) {
  throw new Error('log-reconstruction desync')
}
```

Two lines will catch the majority of real regressions. What the real one adds:

Addition	Why it exists
A <code>WeakSet</code> tag to scope the check	One-shot calls are legitimately different and must not fail
<code>prepend: true</code>	A short-circuiting listener could otherwise silence it
Frozen checks on request and messages	Catches mutation-after-build, which equality alone would miss
The header half	Messages alone are not the whole request
Package-attributed errors	A violation should name its owner without the checker importing product code
A registry with filters	On the hot path, this must be switchable per deployment

Key takeaways

- 35 lines assert that an outgoing request's messages equal a fresh derivation from the log, plus that every header field matches the folded header.
- It runs prepended and globally, so no listener can short-circuit past it.

- **It is mounted nowhere in the shipped configuration** — the registry that runs companions is deliberately absent from the base composition.
- It remains the clearest available definition of the engine's central guarantee, and it runs in test compositions.
- Only loop-built requests are in scope; one-shot calls such as compaction's are excluded by design.

Exercises

1. The check re-derives messages *inside* the dispatch path rather than comparing against a value captured at build time. Name a bug the first catches and the second would not.
2. Suppose you removed `prepend: true`. Describe a listener that would then hide a genuine desync, and say whether it would look malicious or ordinary.
3. The engine's guarantee holds for loop-built requests only. Design a second invariant covering compaction's one-shot summarization call — what relationship does *it* own that could be checked at runtime? (Re-read §2's rule for what qualifies before answering.)

Next: [Chapter 12 · The inbox](#)

Chapter 12 • The inbox

What you'll learn: how input reaches a running agent without interrupting it, why there are two queues instead of one, and the real difference between `followup`, `steer`, and `inject`.

Prerequisites: [Chapter 9](#).

1. The problem

An agent is mid-turn: it has sent a request, the model is streaming, three tool calls are queued behind it. The user types something.

You cannot splice that message into the request already in flight. You cannot drop it. You cannot append it to the history mid-step, because the history is what the *next* request will be built from and a half-written turn would produce an incoherent prompt.

And "just queue it" is not one answer but several. Did the user mean *"stop what you're doing and consider this"* — join the current turn at the next opportunity? Or *"when you're finished, here's the next thing"* — a new turn? Those are different, and a plugin injecting background context wants a third thing again: be there when it matters, but do not wake a sleeping agent.

2. Mental model

New term — inbox. Two ordered queues of pending `UserMessage`s, drained at step boundaries.

```
next-turn: [ prompt A ] [ prompt B ]      ← one popped per turn opening
next-step: [ ctx ] [ steer ] [ tool ctx ] ← all drained every step
```

Two rules do most of the work:

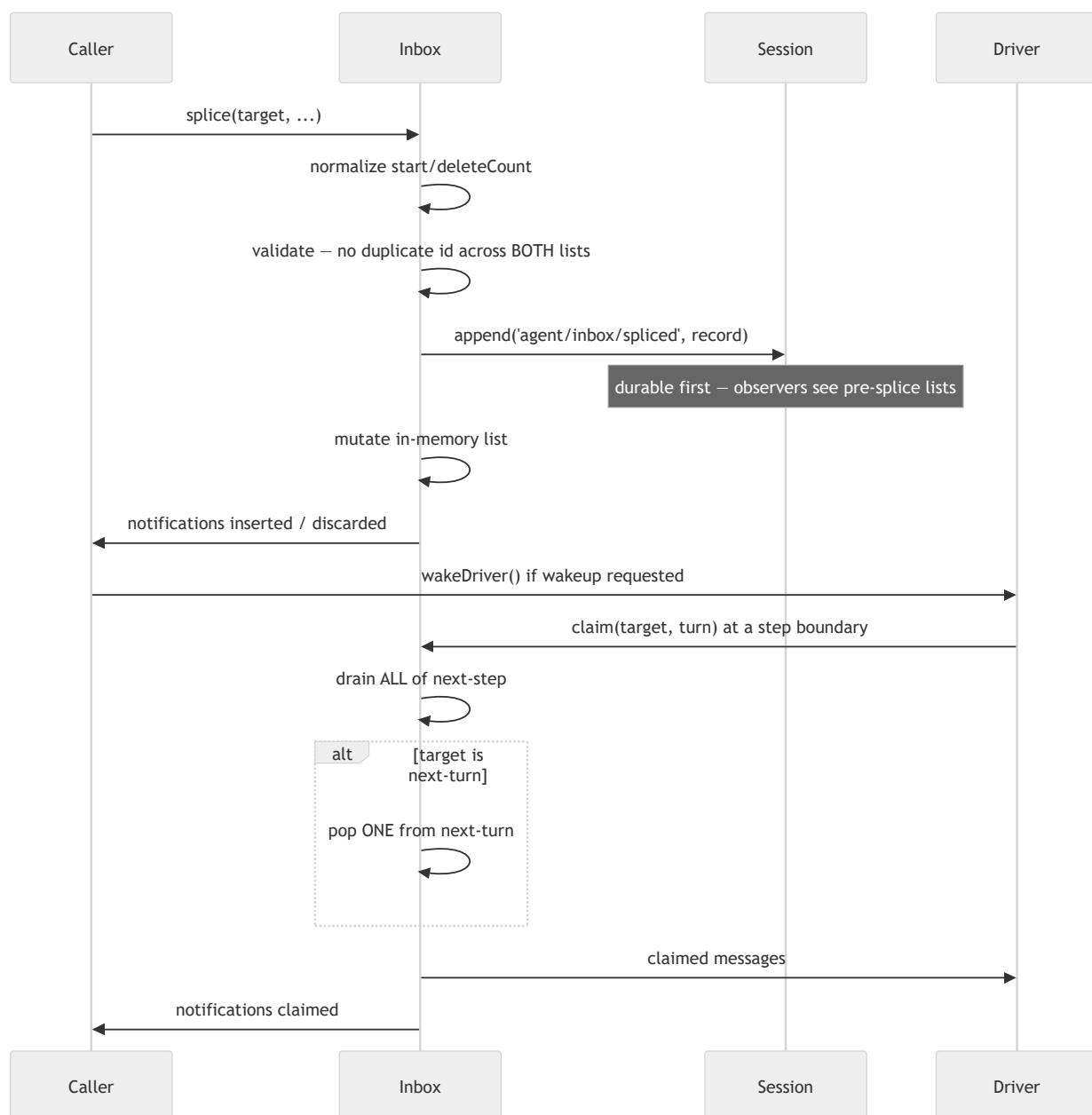
- `next-step` **is drained completely, at every step.**
- `next-turn` **yields exactly one item, and only when a turn is opening.**

So `next-turn` behaves like a FIFO of "one whole turn's worth of prompt," while `next-step` is "everything pending right now, folded into the step about to run."

The third axis is orthogonal to both: whether delivery is allowed to *wake* an idle agent.

New term — durable-first. The inbox commits its change to the session log before touching memory. It is not a side buffer — every splice is a logged event, and the queues are rebuilt by replaying those events.

3. Lifecycle



4. Step-by-step walkthrough

It is a projection, not a buffer

```

constructor(session, notifications) {
  // replay every agent/inbox/spliced event from seedLength onward
}
  
```

— `packages/core/agent/src/inbox.ts:28-40`

Replay starts at `session.header.seedLength ?? 0`, skipping a forked session's inherited prefix — a fork's own log already encodes that history structurally, so replaying the parent's inbox activity would double-count it. A malformed persisted splice **throws** (:34-39) rather than being skipped: a queue that cannot be

reconstructed exactly is not safe to run on.

This is why a crashed agent resumes with its pending input intact. Nobody wrote a queue to disk; the queue is the log.

The splice

`splice(target, start, deleteCount, inserted)` (:139–146) delegates to `mutate` (:157–193), which:

Normalizes like `Array.prototype.splice` (:166–176) — clamping to bounds, truncating non-integers. This is why the engine can call `splice(target, Infinity, 0, [message])` to mean "append" without a special case.

Validates before committing (:202–219): it re-derives the resulting list with `toSpliced` and asserts no message `id` appears twice **across both lists combined**, throwing `"message ... is already pending"`. A message can be pending in at most one target at a time — otherwise a message could be claimed twice and appear in the history twice.

Commits durably, then in memory:

```
this.session.append('agent/inbox/spliced', splice) // :186
this.state[target] = ...                          // :187
```

The ordering is deliberate, and documented: appending first means synchronous `session/event` observers "see the pre-splice lists" (:130–132). An observer watching the log sees the *intent* before the queue reflects it, so it can reason about both states.

The claim

```
claim(target: InboxTarget, turn: number): UserMessage[] {
  const claimed = this.mutate('next-step', 0, this.nextStep.length, [], false)
  if (target === 'next-turn') {
    claimed.push(...this.mutate('next-turn', 0, 1, [], false))
  }
  for (const message of claimed) this.notifications.claimed(message, turn)
  return claimed
}
```

— `inbox.ts:63–78`

Marked `@internal` — "the agent loop's step-boundary operation, not a plugin extension point."

The asymmetry is the whole design. `next-step` is drained with `deleteCount = this.nextStep.length`; `next-turn` with `deleteCount = 1`. And the `false` in both calls is `discardRemoved` — claimed messages are **not** reported as discarded. They get their own notification, `claimed`, because "removed because it is being used" and "removed because it was thrown away" are different events for anything watching.

The three senders

All three route through one private method:

```
send(message: UserMessage, target: InboxTarget, wakeup: boolean): void {
  const wakingAfterAbort = wakeup && this.phase.kind !== 'idle' && this.phase.abort.signal.aborted
  const resolvedTarget = wakingAfterAbort ? 'next-turn' : target
  this.inbox.splice(resolvedTarget, Infinity, 0, [message])
  if (wakeup) this.wakeDriver(wakingAfterAbort)
}

followup(input) { this.send(input, 'next-turn', true) }
steer(input)    { this.send(input, 'next-step', true) }
inject(input)   { this.send(input, 'next-step', false) }
```

— packages/core/agent-loop/src/agent.ts:122–141

Method	Queue	Wakes?	Meaning
followup	next-turn	yes	"Here's the next thing to do." One item becomes one turn.
steer	next-step	yes	"Consider this at your next opportunity." A running agent picks it up at the next step boundary; an idle one starts a turn.
inject	next-step	no	"Have this available when you next think." Never wakes anything.

`inject` on an idle agent produces **no activity at all** — the message sits in the log, the status stays `idle`, and no request is made. The test asserts exactly this: after `agent.inject(...)`, the session contains only the `agent/inbox/spliced` event and the mock adapter records zero requests (`packages/core/agent-loop/tests/agent.spec.ts:31–42`).

The reclassification

```
const wakingAfterAbort = wakeup && this.phase.kind !== 'idle' && this.phase.abort.signal.aborted
```

A waking message arriving *after* the current activity was aborted cannot join it — that turn is unwinding. So it is retargeted to `next-turn`, where it will open a fresh turn instead of being silently swallowed by a dying one.

The flag is computed **before** the splice, deliberately. A `session/event` observer could react to the splice by calling `cancel()`, which would change `phase.abort.signal.aborted` mid-call; capturing first means a reentrant cancel cannot reclassify a message that was correctly classified a microsecond earlier (`agent.ts:124–125`).

5. Data at each stage

From the recorded session (`session.jsonl:5` and `:7`):

```
{"type":"agent/inbox/spliced","data":{"target":"next-turn","start":0,
  "inserted":[{"content":[{"type":"text","text":"Reply with exactly the word: PONG..."}],
  "source":{"kind":"user","role":"user","id":"{{message:1}}"}}]}
```

then, after `turn/start` :

```
{"type":"agent/inbox/spliced","data":{"target":"next-turn","start":0,
  "removedCount":1,"inserted":[]}}
```

Stage	next-turn	next-step
After <code>followup</code>	<code>[msg1]</code>	<code>[]</code>
Turn opens, <code>claim('next-turn', 1)</code>	<code>[]</code>	<code>[]</code>
Claimed	—	<code>msg1</code> returned to the loop
Loop appends	—	<code>user/message</code> at seq 8

Note the second splice event records `removedCount: 1` with an empty `inserted` — the claim is itself a logged mutation. The queue's entire history is in the log.

6. Control decisions

Decision	Condition	Location
Retarget to <code>next-turn</code>	waking while the current activity is already aborted	<code>agent.ts:124-126</code>
Reject splice	resulting lists would contain a duplicate message id	<code>inbox.ts:202-219</code>
Throw on replay	a persisted splice record is malformed	<code>inbox.ts:34-39</code>
Claim one vs all	<code>next-turn</code> yields 1; <code>next-step</code> yields everything	<code>inbox.ts:63-78</code>
Notify <code>claimed</code> not <code>discarded</code>	removal was a claim	<code>inbox.ts:74-77</code>
Skip wake	<code>wakeup</code> is false	<code>agent.ts:128</code>

7. Edge cases and failure modes

`clear()` discards both queues, durably.

```
clear(): void {
  this.splice('next-step', 0, this.nextStep.length, [])
  this.splice('next-turn', 0, this.nextTurn.length, [])
}
```

— `inbox.ts:58-61`

Called from `cancel()` unless `keepInbox` is set ([Ch 13](#)). Each is a logged splice, so a reader can see exactly what was thrown away and when — cancellation does not create a gap in the record.

A claimed message may never become a `user/message`. If the step is subsequently rejected by a pre-step listener, the claimed messages are gone from the inbox and never appended. The event documentation warns about precisely this: "If the step is later rejected, the message ends here — never re-emitted as a `user/message`" (`runtime-types.ts:196-197`). Anything tracking message lifecycle has to treat `claimed` as "consumed", not "delivered".

`nextStep.length` **is a turn-continuation condition.** [Chapter 9's](#) ⑨ tests it twice around `agent/turn-stopping`. So a plugin can extend a turn purely by putting something in this queue — which is exactly how a blocking Stop hook would work, if one were mounted.

Tool results feed this queue. A tool's `additionalContexts` are spliced into `next-step` by the callback the step loop passes to the scheduler (`agent.ts:434`). They are not part of the tool result the model reads; they are separate messages that arrive at the next boundary ([Ch 17](#)).

8. Configuration knobs

None. Queue depth is unbounded, and there is no priority or expiry.

9. Interactions

- [Ch 9](#) — claims at each step boundary; `nextStep.length` gates turn continuation.
- [Ch 13](#) — `cancel()` clears it; wake latching decides when a queued message actually starts a driver.
- [Ch 21](#) — the runtime-context snapshot arrives as a claimed message, not a side channel.
- [Ch 17](#) — `additionalContexts` are spliced into `next-step`.
- [Ch 5](#) — every splice is an `agent/inbox/spliced` event.

10. Build it yourself

Minimal version:

```

class MiniInbox {
  nextTurn: UserMessage[] = []
  nextStep: UserMessage[] = []
  claim(target: 'next-turn' | 'next-step'): UserMessage[] {
    const claimed = this.nextStep.splice(0)
    if (target === 'next-turn') claimed.push(...this.nextTurn.splice(0, 1))
    return claimed
  }
  get hasPending() { return this.nextTurn.length > 0 || this.nextStep.length > 0 }
}

```

The two-queue asymmetry is the essential part and it is already here. What the real one adds:

Addition	Why it exists
Durable <code>agent/inbox/spliced</code> events	A crashed agent must resume with its pending input; the queue is the log
Append-before-mutate ordering	Observers need to see the pre-splice state to reason about the transition
Duplicate-id validation	The same message claimed twice would appear twice in history
<code>claimed</code> distinct from <code>discarded</code>	"Consumed" and "thrown away" are different facts for anything watching
Replay from <code>seedLength</code>	A fork already encodes inherited history; replaying it would double-count
Abort-time retargeting	A message that raced a cancellation must not be swallowed by a dying turn
Throw on malformed replay	A queue that cannot be reconstructed exactly is not safe to run on

Key takeaways

- Two queues: `next-turn` yields one item per turn opening, `next-step` is drained entirely at every step.
- `followup` / `steer` / `inject` differ on exactly two axes — which queue, and whether delivery may wake an idle driver.
- `inject` on an idle agent does nothing observable but write a log event.
- Every mutation is a logged event committed *before* memory changes; the queue is a replay of those events.
- A waking message that arrives after cancellation is retargeted to the next turn, classified before the splice so a reentrant cancel cannot change the answer.
- A claimed message can still be discarded if its step is rejected.

Exercises

1. A plugin calls `inject` three times while the agent is running a step, then the user calls `steer`. Exactly what does the next `claim` return, and in what order?
2. The duplicate-id check spans both lists rather than one. Construct the failure it prevents — trace the message through to where it would appear twice.

3. `claim` pops one item from `next-turn` but drains all of `next-step`. Design a scenario where draining all of `next-turn` too would produce an incoherent prompt.

Next: [Chapter 13 · Phases, cancellation, and quiescence](#)

Chapter 13 · Phases, cancellation, and quiescence

What you'll learn: the three-state machine behind `agent.status`, how a cancellation converges, and the latch that stops a wake being lost in the gap between "aborting" and "idle".

Prerequisites: [Chapter 9](#), [Chapter 12](#).

1. The problem

Stopping an agent is not one operation. A stop request arrives while a model is streaming, three tools are mid-execution, and a user message just landed in the inbox. Several things must all be true afterwards:

- the stream stops and the partial answer is not lost ([Ch 9](#) ⑤);
- every started tool call settles rather than being orphaned;
- the log ends up balanced — no `step/start` without a `step/end`;
- and if the user's message arrived a microsecond before the cancel, it must not be silently eaten.

That last one is the subtle one. There is a window where the agent is neither running usefully nor idle: the abort has fired, the unwinding is in progress. A wake delivered into that window has nowhere to go. Losing it means a user pressing Enter and nothing happening.

2. Mental model

New term — phase. The agent's internal state, richer than the two-value `status` the outside sees.

```
type Phase =  
  | { kind: 'idle'; lastTurn: number }  
  | { kind: 'maintenance'; abort: AbortController; lastTurn: number; wakeRequested: boolean }  
  | { kind: 'running'; abort: AbortController; turn: number; step: number; wakeRequested: boolean }
```

— `packages/core/agent-loop/src/agent.ts:39-47`

Externally, `maintenance` and `idle` both report `'idle'`:

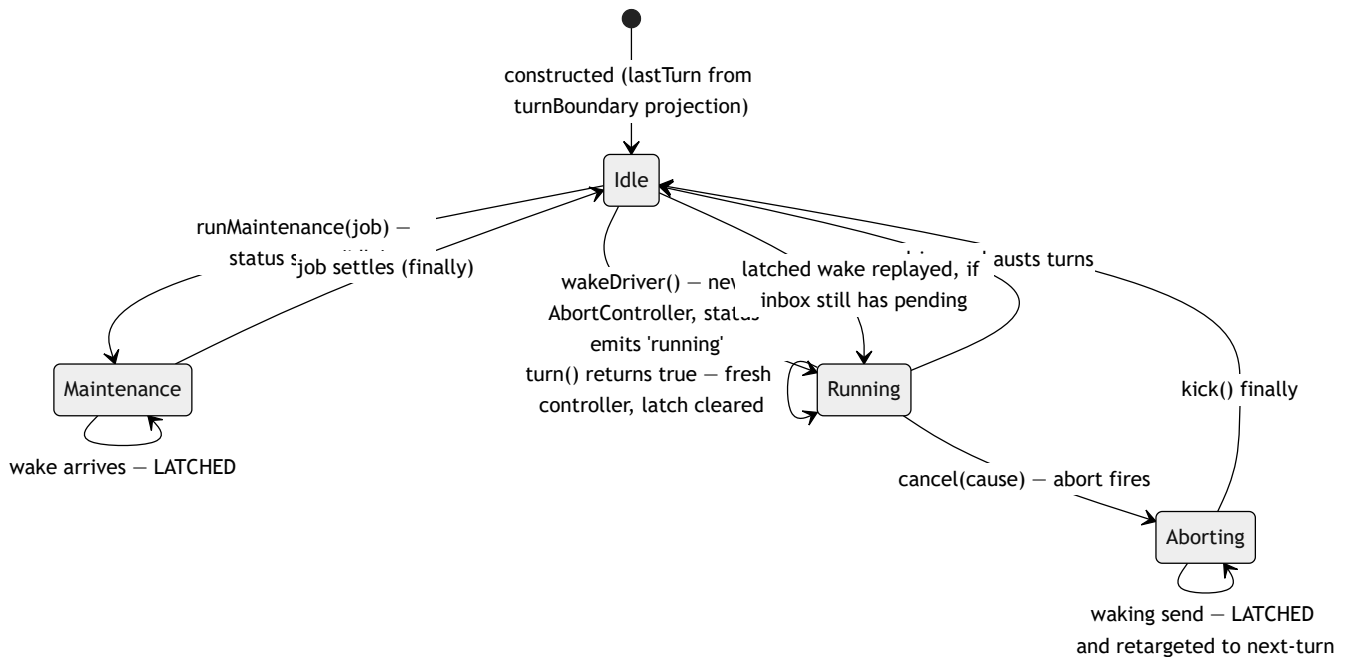
```
get status(): AgentStatus {  
  return this.phase.kind === 'idle' || this.phase.kind === 'maintenance' ? 'idle' : 'running'  
}
```

— `:108-110`

New term — maintenance. A phase for work that needs the agent to hold still — manual compaction is the real user — during which no turn may start, but which the outside world should not see as "the agent is busy thinking."

The two non-idle phases carry their own `AbortController` and a `wakeRequested` latch. Both are per-phase, and the controller is **replaced between turns** (Ch 9), which is why cancelling scopes to a turn rather than killing the agent.

3. State diagram



4. Step-by-step walkthrough

Status is emitted only on a real change

```

private setPhase(next: Phase): void {
  const previousStatus = this.status
  this.phase = next
  const status = this.status
  if (status !== previousStatus) {
    this.dispatch.emit('agent/status', { status })
  }
}

```

Because the collapse happens *before* comparison, an `idle` → `maintenance` → `idle` cycle emits nothing at all. A UI never flickers "busy" for a manual compaction, and listeners that reset state on `idle` (the retry counter, for instance) are not triggered spuriously.

Cancellation

```
cancel(cause: AgentCancelCause, options: CancelOptions = {}): void {
  if (!options.keepInbox) {
    this.inbox.clear()
    if (this.phase.kind !== 'idle') this.phase.wakeRequested = false
  }
  if (this.phase.kind !== 'idle') this.phase.abort.abort(cause)
}
```

— :143–149

Three things happen in a deliberate order. The inbox is cleared *first* — and clearing it also clears the latch, because a latched wake whose message is gone should not start an empty turn. Then the signal fires. Cancelling an idle agent is a no-op on the signal but still clears the inbox.

`keepInbox` exists for cancellations that mean "stop this turn, keep the queue" rather than "stop everything."

The `cause` travels: [Chapter 9](#) records it as `turnEnds = { kind: 'aborted', reason: signal.reason }`, so the log says *why* a turn was cancelled, not merely that it was.

The wake latch

```
private wakeDriver(wakeAfterAbort = false): void {
  if (this.phase.kind !== 'idle') {
    const reason = this.phase.abort.signal.reason as AgentCancelCause | undefined
    if (reason?.kind !== 'disposed' && (this.phase.kind === 'maintenance' || wakeAfterAbort)) {
      this.phase.wakeRequested = true
    }
    return
  }
  // idle: start a driver
  const driver = Promise.withResolvers<void>()
  this.activityDone = driver.promise
  this.setPhase({ kind: 'running', abort: new AbortController(),
    turn: this.phase.lastTurn, step: 0, wakeRequested: false })
  this.loopCtx.agents.withInitiator(this, () => this.kick()).then(driver.resolve, driver.reject)
}
```

— :181–202

This is the answer to the problem in §1. When the agent is not idle, the wake cannot be delivered now, so it is *latched* for replay at convergence — but only under two conditions:

- **the phase is** `maintenance` (the driver is deliberately parked and will look again), **or**
- `wakeAfterAbort` (the wake arrived into the aborting window).

A live, non-aborted running driver does **not** latch. It does not need to: it will claim the queue itself at its next step boundary.

And one exclusion that matters more than it looks:

```
reason?.kind !== 'disposed'
```

Disposal never latches. If it did, teardown would queue a wake that starts a turn while the agent is being destroyed — and `dispose()` awaits `whenIdle()` ([Ch 14](#)), so teardown would wait on a model call it just asked to stop.

Replaying the latch

Two places, both guarded identically:

```
if (wakeRequested && this.inbox.hasPending) this.wakeDriver()
```

— `kick()`'s `finally ( :229 )` and `runMaintenance`'s `finally ( :167 )`

The `hasPending` guard is what makes a latch safe: if the queue was cleared in between, the replay is suppressed. The doc comment states the rule precisely — "A wake sent while idle always opens its turn boundary, even when its message was cleared; only a latched replay is suppressed when the queue no longer holds the wake" (`:173-180`).

Latches go stale between turns

```
phase.abort = new AbortController()
phase.wakeRequested = false
```

— `:334-336`

A fresh controller makes any latch set against the old one meaningless: the driver is alive again and will claim the queue itself.

Maintenance

```
runMaintenance<T>(job: (signal: AbortSignal) => Promise<T>): Promise<T> {
  if (this.phase.kind !== 'idle') throw new Error(`agent "${this.id}" already has active work`)
  ...
  return (async () => {
    try {
      return await job(maintenance.abort.signal)
    } finally {
      this.setPhase({ kind: 'idle', lastTurn: maintenance.lastTurn })
      if (maintenance.wakeRequested && this.inbox.hasPending) this.wakeDriver()
      done.resolve()
    }
  })()
}
```

— :151–171

It refuses to start unless idle, gets its own cancellable signal, and always returns to idle. Manual /compact uses this (Ch 27): the agent holds still, history is rewritten, and any wake that arrived meanwhile fires on the way out.

Quiescence

```
async whenIdle(): Promise<void> {
  let activity: Promise<void>
  do {
    await (activity = this.activityDone)
  } while (activity !== this.activityDone)
}
```

— :204–209

The loop is the point. Awaiting `activityDone` once is not enough — a latched wake replayed in `kick()`'s `finally` installs a *new* `activityDone` before the old one's awaiters run. The re-check catches that, so `whenIdle()` means "no activity, and none started while I was waiting." Chapter 14 depends on this.

5. Control decisions

Decision	Condition	Location
Emit <code>agent/status</code>	collapsed status actually changed	:113–120
Clear inbox and latch	<code>cancel</code> without <code>keepInbox</code>	:144–147
Fire abort	phase is not idle	:148
Latch a wake	not idle, cause $\neq$ <code>disposed</code> , and (maintenance or wake-after-abort)	:186–189
Start a driver	phase is idle	:192–201
Replay a latch	<code>wakeRequested</code> and <code>inbox.hasPending</code>	:167, :229
Clear a stale latch	new turn installs a fresh controller	:334–336
Refuse maintenance	phase is not idle	:152

6. Edge cases and failure modes

A wake while idle always opens a turn, even if its message was cleared. Asymmetric with the latched case on purpose: an explicit wake delivered to an idle agent gets its turn boundary regardless. That turn may immediately close via [Chapter 9](#)'s branch ⑤ — `turn/start` followed by `turn/end`, no model call. The boundary is honored; the work is not invented.

`cancel` during a splice observer. Covered in [Chapter 12](#): `wakingAfterAbort` is captured before the splice so a reentrant cancel cannot reclassify a message.

Errors never escape the driver. `kick()`'s empty `catch` ([Ch 9](#)) contains everything. A failed turn returns the agent to idle rather than leaving it stuck `running` — the `finally` runs regardless of how the loop exited.

The phase is re-derived on construction, not stored.

```
const lastTurn = this.loopCtx.sessionProjections.stateOf(session, 'turnBoundary')?.lastTurn ?? 0
this.phase = { kind: 'idle', lastTurn }
```

— :101–102

A resumed agent starts idle at the right turn number, computed from the log ([Ch 8](#)).

7. Configuration knobs

None. Cancellation has no timeout, no grace period, no forced kill. Convergence is cooperative: the loop checks its signal at six points per turn, tool dispatch drains rather than abandons ([Ch 17](#)), and `whenIdle()` waits for genuine quiescence.

That is a real design position with a real consequence: **a tool that ignores its abort signal can delay teardown indefinitely**. There is no watchdog. The timeout policy plugin exists to bound individual calls, but the engine itself will wait.

8. Interactions

- **Ch 9** — consumes the signal, rotates the controller, returns to idle.
- **Ch 12** — `hasPending` gates latch replay; `cancel` clears the queues.
- **Ch 14** — disposal is a `disposed` -cause cancel followed by `whenIdle()` .
- **Ch 22** — `agent/status` is consumed by compaction (to reset retry counters), the session controller, and the goal driver.
- **Ch 27** — manual compaction runs inside `runMaintenance` .

9. Build it yourself

Minimal version:

```
class MiniAgent {
  private running = false
  private abort?: AbortController
  send(message: UserMessage): void {
    this.inbox.push(message)
    if (!this.running) { this.running = true; void this.kick().finally(() => { this.running = false }) }
  }
  cancel(): void { this.abort?.abort() }
}
```

Fine until the first race. What the real one adds:

Addition	Why it exists
A third phase (<code>maintenance</code>)	Some work needs the agent still without looking busy
Status collapsed before comparison	A maintenance cycle should not flicker listeners
Per-phase, per-turn abort controller	Cancellation should scope to a turn, not the agent
The wake latch	A wake arriving during the aborting window would otherwise be lost
<code>disposed</code> excluded from latching	Otherwise teardown waits on a turn it just cancelled
<code>hasPending</code> guard on replay	A latch whose message was cleared must not start an empty turn
Clearing the latch on a new controller	A latch against a dead controller is meaningless
<code>whenIdle</code> 's re-check loop	A replayed wake installs new activity before old awaiters run

Key takeaways

- Three phases, two external statuses; `maintenance` is deliberately invisible as "idle".
- The abort controller is per-phase and replaced every turn, so cancellation scopes to a turn.
- A wake that cannot be delivered is latched — but only from maintenance or the aborting window, and never during disposal.
- Latch replay is guarded by `hasPending`, so a cleared queue suppresses it.
- `whenIdle()` re-checks, because a replayed wake can install new activity mid-await.
- Convergence is entirely cooperative; a tool ignoring its signal will delay teardown, with no watchdog.

Exercises

1. A user sends a message at the exact moment `cancel()` fires. Trace it through `send`, `wakeDriver`, and the `kick` finally. Now repeat with the cause being `disposed` and say where the paths diverge.
2. `setPhase` compares collapsed status rather than phase kind. Name a listener that would misbehave if it emitted on every phase change, and say how.
3. `whenIdle()`'s loop re-reads `activityDone`. Write the interleaving that makes a single `await` return too early, and say which line installs the new promise.

Next: [Chapter 14 · Agent lifecycle](#)

Chapter 14 • Agent lifecycle

What you'll learn: how an agent is built, published, and torn down — and why the teardown is registered before any of the things it tears down exist.

Prerequisites: [Chapter 13](#), plus the Cordis effect model from [Chapter 2](#).

1. The problem

Creating an agent means acquiring several things that must be released in reverse: a session (possibly loaded from disk), a scope, registry entries in two registries, and a running driver. Any of those steps can fail, and the caller can cancel at any point.

The failure mode to avoid is a *partial* agent: entered in one registry but not the other, or published while its session load is still in flight, or — worst — an agent whose owner unloaded during setup and which nothing is now tracking. That last one is a leak with a running model loop attached to it.

Three independent parties can also decide the agent should stop before it has finished starting: the caller (who passed a signal), the owner fiber (which may unload), and the factory itself (which may be torn down). Each has a different reason, and each must produce a clean rollback.

2. Mental model

The engine's answer has one shape, applied consistently:

Register the undo before you do.

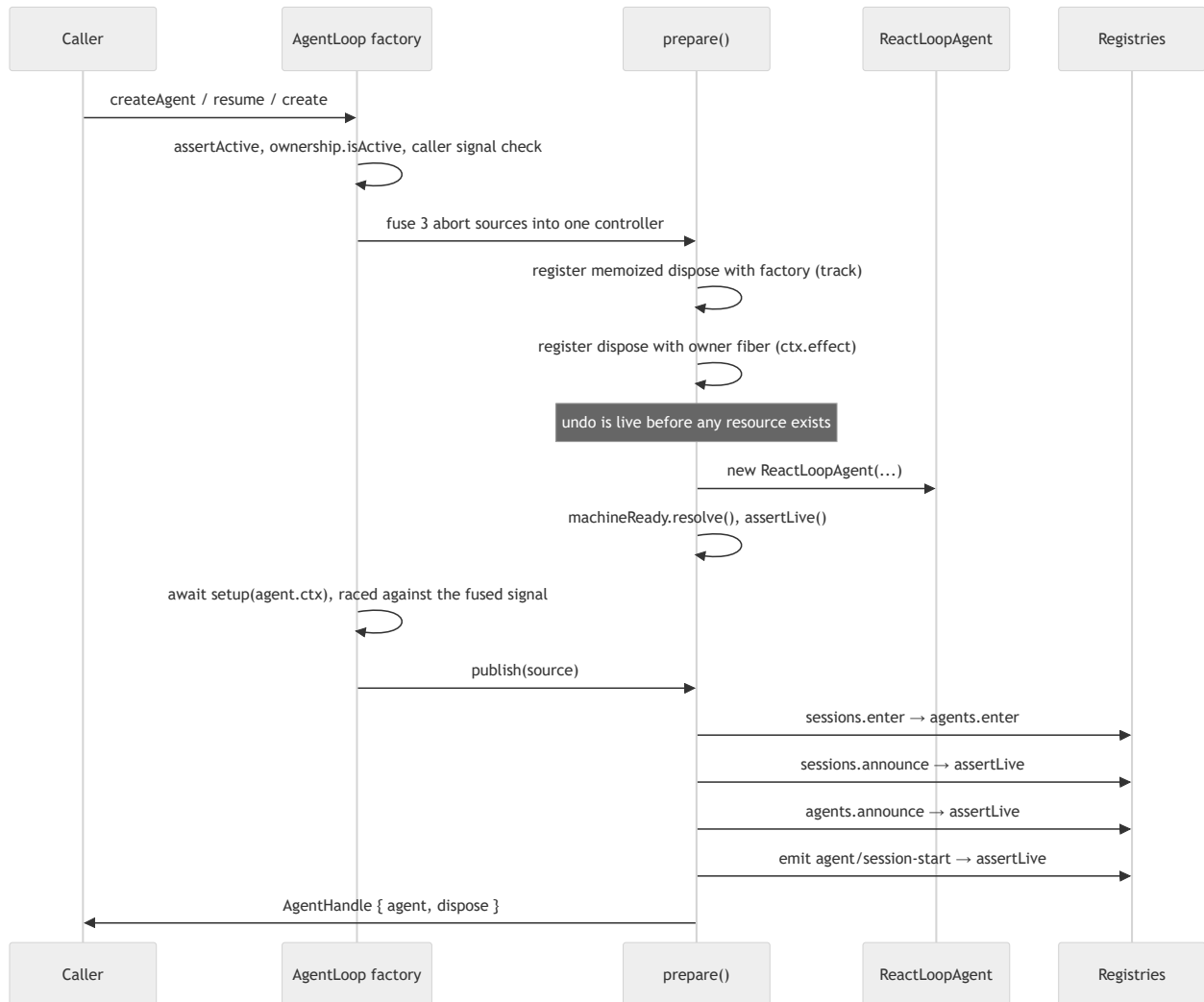
`prepare()` constructs a memoized teardown closure and registers it with both the factory and the owner fiber **before any resource exists**, over mutable slots that are filled in as the resources appear. An unload arriving while the scope is still being minted finds a working disposer that simply has less to do.

New term — publication. The moment an agent becomes visible: entered into the agent and session registries, announced, and its session-start event emitted. Before publication an agent exists but nothing can find it; after it, everything can.

Three roles:

Piece	Owns
FactoryOwnership	Whether the factory is accepting work; every live agent's teardown; outstanding startup jobs
PreparedAgent	One agent's resources, its fused cancellation signal, and its memoized <code>dispose</code>
<code>publish(source)</code>	The ordered registry entry and announcement

3. Lifecycle



4. Step-by-step walkthrough

The gate

```

assertAgentOptions(options)
ownerCtx.fiber.assertActive()
if (!this.ownership.isActive()) throw new Error('agent loop is not active')
if (callerSignal?.aborted) throw ...

```

— `packages/core/agent-loop/src/index.ts:523-534`

Four checks before anything is allocated. `isActive()` (:110-112) is both a flag and a fiber-state test:

```
const INACTIVE_STATES = new Set([FiberState.UNLOADING, FiberState.DISPOSED, FiberState.FAILED])
```

— :38-42

A factory whose fiber is unloading cannot own a new lifecycle even if its own flag has not flipped yet.

Fusing three cancellations

```
const abort = new AbortController()
const onCallerAbort = (): void => { abort.abort(...) }
const onFactoryTeardown = (): void => { abort.abort(this.ownership.signal.reason) }
callerSignal?.addEventListener('abort', onCallerAbort, { once: true })
this.ownership.signal.addEventListener('abort', onFactoryTeardown, { once: true })
```

— :542-550

Three owners, one signal, each contributing its own reason — caller cancellation, owner-fiber unload (wired below), and factory teardown, which aborts with `new Error('agent loop is not active')` (:139).

Downstream code checks one signal and gets an accurate reason whichever party stopped it.

The memoized teardown

```
const dispose = (ownerTriggered = false): Promise<void> => (disposing ??= (async () => {
  abort.abort(new Error(`agent "${id}" lifecycle disposed`))
  callerSignal?.removeEventListener('abort', onCallerAbort)
  this.ownership.signal.removeEventListener('abort', onFactoryTeardown)
  try {
    if (machine === undefined) await machineReady.promise
    if (machine !== undefined) {
      machine.cancel({ kind: 'disposed' })
      await machine.whenIdle()
      await machine.scope.dispose()
    }
  } finally {
    try { detachAgent?(); detachSession?(); }
    finally { untrack(); if (!ownerTriggered) await unfollowOwner() }
  }
})())
```

— :560-583

`disposing ??=` makes it idempotent: every racing owner awaits **one** quiescence rather than starting parallel teardowns.

The order is strict reverse: stop the machine, wait for it to actually stop, unwind the scope, leave the registries, release bookkeeping. And note `machine.cancel({ kind: 'disposed' })` — the cause that [Chapter 13](#) singles out as never latching a wake. That exclusion exists exactly so the `whenIdle()` on the next line terminates.

`if (machine === undefined) await machineReady.promise` handles disposal racing construction: wait until we know whether there is a machine at all.

Registering the undo first

```
const untrack = this.ownership.track(dispose)
let unfollowOwner: () => Promise<void> | void
try {
  unfollowOwner = ownerCtx.effect(() => () => {
    if (disposing !== undefined) return
    abort.abort(new Error(`agent "${id}" setup aborted: owner disposed during setup`))
    return dispose(true)
  }, `agentLoop.lifecycle(${id})`)
} catch (error: unknown) {
  untrack()
  // ...detach both listeners...
  throw error
}
```

— :584–600

This is the pattern from §2 made concrete. `track` and `ctx.effect` both run **before** `new ReactLoopAgent(...)`. The closure they register reads mutable slots (`machine`, `detachAgent`, `detachSession`) that are still `undefined` — and that is fine, because the disposer is written to handle every one of them being absent.

The `ownerTriggered` flag prevents self-unregistration: when the owner's own effect is what is running, it must not try to unregister itself from inside itself.

Publication

```
publish: (source) => {
  assertLive()
  detachSession = agent.ctx.sessions.enter(session)
  detachAgent = loopCtx.agents.enter(agent, ownerCtx.agent)
  agent.ctx.sessions.announce(session)
  assertLive()
  loopCtx.agents.announce(agent)
  assertLive()
  emitAgentEvent(loopCtx, agent, 'agent/session-start', { source })
  assertLive()
  return { agent, dispose }
}
```

— :619–633

Four `assertLive()` calls, one after every step that runs listener code. Each announcement can synchronously run third-party listeners, and any of them can start teardown — `agent/created` is even documented as able to veto publication by throwing. Re-checking after each means a teardown begun mid-publication is noticed immediately rather than after the agent is fully wired.

Enter both registries first, *then* announce. Nothing observing an announcement can find a half-entered agent.

5. The three entry points

Method	Session comes from	Ownership
<code>create(id, options, meta)</code> (:652–661)	<code>sessions.prepare()</code> — fresh	The factory's own context
<code>createAgent(ownerCtx, options)</code> (:669–685)	<code>sessions.prepare()</code> with optional seed/meta	Caller's context
<code>resume(ownerCtx, options)</code> (:716–722)	<code>persistence.prepare()</code> — loaded from disk	Caller's context

All three funnel into `setupAndPublish` (:688–708):

```
using ownedPreparation = preparation
const prepared = this.prepare(ownerCtx, id, agentOptions, signal)
try {
  const setupCommit = await raceAbort(setup?.(prepared.agent.ctx), prepared.signal, id)
  setupCommit?.commit()
  return prepared.publish(source)
} catch (error: unknown) {
  await prepared.dispose()
  throw error
}
```

The `setup` callback is where an agent's preset composition mounts ([Ch 34](#)) — it runs after construction but **before** publication, so an agent is never visible without its tools. It is raced against the fused signal, so a hanging setup cannot pin the lifecycle.

Resume has an extra barrier

```
preparation = await raceAbortCall(
  () => persistence.prepare(id, fused),
  fused, id,
  (abandoned) => { abandoned[Symbol.dispose]() },
)
```

A disk load may outlive its owner, so it is raced against caller cancellation, owner unload, **and** factory teardown. The fourth argument is the interesting one: `releaseAbandoned`. If the race is lost, the load still completes eventually — and when it does, the abandoned preparation is disposed rather than leaked (:179–185). Cancelling a load you cannot stop is not the same as forgetting about it.

Then, after the await, the ownership checks are re-run (:756–757) because everything may have changed while the disk spun.

6. Control decisions

Decision	Condition	Location
Refuse creation	inactive fiber, inactive factory, or pre-aborted caller signal	:523–534
Abort the lifecycle	any of the three fused sources fires	:542–550
Skip re-entrant teardown	<code>disposing</code> already set	:560 , :589
Wait for construction	<code>machine === undefined</code> during dispose	:568
Abort publication	<code>assertLive()</code> after any announcement	:619–632
Roll back setup	<code>setup</code> threw or was cancelled	:704–707
Release an abandoned load	the fused signal won the race	:179–185
Re-check after load	always, before publishing	:756–757

7. Edge cases and failure modes

Factory teardown waits for everything.

```
async dispose(): Promise<void> {
  this.accepting = false
  this.teardown.abort(new Error('agent loop is not active'))
  this.inactive.resolve()
  await Promise.all([
    ...[...this.liveAgents].map(dispose => dispose()),
    ...this.startupTasks,
  ])
}
```

— :137–145

Both live agents and in-flight *startup* jobs. An agent still being created when the factory shuts down is awaited too — `trackWrapper` (:128–130) joins each public create/resume continuation, swallowing its result so a rejected startup does not reject the teardown.

Configured agents wait for a draining twin. When a configured agent has a fixed session id and a previous lifecycle with that id is still unwinding, `waitForDrainingConfiguredIdentity` (:494–514) waits on `agent/disposed` and `session/disposed` until the id leaves both registries. It waits only if the id is *currently occupied* — a live healthy occupant is a genuine collision, surfaced by the create that follows rather than waited on forever.

Restore falls back to create only for genuine absence.

```
try { await this.resumeWith(...); return } catch (error) {
  if (!this.ownership.isActive()) return
  const exists = (await persistence.list()).some(header => header.id === sessionId)
  if (exists) throw error
}
this.create(sessionId, agentOptions, meta)
```

— :479–490

Corruption and backend failures stay loud. Only a session that genuinely is not there falls through to first creation.

Declarative startup failures are reported, not thrown. A config-driven agent that fails to start emits `agent-loop/config-start-failed` with the session id and error (:448–467), iterating listeners with individual try/catch so one bad listener cannot suppress delivery to the others. Consumers buffering work for that identity can reject it instead of waiting forever.

8. Configuration knobs

Setting	Default	Effect
<code>agents</code>	<code>[]</code> in the web profile	Agents created at plugin startup
<code>sessionId</code> / <code>resumeSessionId</code>	unset	Mutually exclusive; a duplicate exact identity across entries is rejected at construction (:334–349)
<code>cwd</code>	unset	Workspace metadata for a fresh session

Launcher-supplied identities (`CONFIGURED_AGENT_IDENTITYES_KEY` , :267) override both config keys for any entry the launcher names, so an overlay repointing a row's model route cannot accidentally drop its identity (:277–290).

9. Interactions

- **Ch 13** — disposal is a `disposed`-cause cancel plus `whenIdle()` ; the latch exclusion exists for this.
- **Ch 2** — `ctx.effect` is the registration primitive throughout.
- **Ch 29** — `persistence.prepare()` and `SessionPreparation` ownership.
- **Ch 34** — the `setup` hook is where a preset mounts.

- **Ch 28** — a subagent is created through `createAgent`, so everything here applies recursively.

10. Build it yourself

Minimal version:

```
async function createAgent(ctx: Context, id: SessionId): Promise<AgentHandle> {
  const session = ctx.sessions.create(id)
  const agent = new ReactLoopAgent(ctx, id, {}, session)
  const detach = ctx.agents.enter(agent)
  return { agent, dispose: async () => { agent.cancel({ kind: 'disposed' }); await
agent.whenIdle(); detach() } }
}
```

Correct on the happy path. What the real one adds, and the failure each prevents:

Addition	Failure it prevents
Undo registered before resources exist	An unload mid-construction leaks a running agent
Memoized <code>dispose</code>	Racing owners start parallel teardowns of the same agent
Three fused abort sources	A caller-cancelled create keeps running because only the factory was checked
<code>assertLive()</code> after each announcement	A listener starts teardown and the agent finishes wiring itself up anyway
Enter both registries before announcing	An observer finds a half-entered agent
<code>releaseAbandoned</code> on a raced load	A cancelled disk load completes and leaks its preparation
Re-check ownership after the load	The factory shut down while the disk spun
Factory awaits startup tasks	Teardown completes while an agent is still being created
<code>exists</code> check before create-fallback	A corrupt session is silently replaced by an empty one

Key takeaways

- The teardown closure is registered with both the factory and the owner fiber before any resource exists, over mutable slots.
- One memoized `dispose` serves every racing owner, running strict reverse order.
- Three cancellation sources are fused into one signal, each with its own reason.
- Publication enters both registries, then announces, re-checking liveness after every listener-running step.
- `setup` runs between construction and publication, so an agent is never visible without its composition.
- Resume races the disk load against all three cancellation sources and disposes the result if it loses.

Exercises

1. The owner fiber unloads while `setup` is awaiting. Trace the path from `ctx.effect`'s disposer to the agent's registries being left. Which flag stops the disposer unregistering itself?
2. `dispose` awaits `machineReady` when `machine` is undefined. Construct the interleaving that makes this necessary, and say what would happen without it.
3. Factory teardown awaits `startupTasks` as well as `liveAgents`. Give an agent state that is in neither set, and say whether that is a leak.

Next: [Chapter 15 · The tool registry](#)

Chapter 15 • The tool registry

What you'll learn: what a tool actually is in this codebase, how one gets registered, and why a tool returns a value rather than the text the model reads.

Prerequisites: [Chapter 2](#) for effects.

1. The problem

A tool has to satisfy two audiences that want different things.

The **model** needs a name, a description, and a JSON Schema for its arguments — and needs them to be stable, because changing the tool list changes the request header and invalidates the provider's prefix cache ([Ch 10](#)).

The **system** needs something else entirely: a typed result it can validate, persist, re-render on replay, and show in a UI as something better than a wall of text. And it needs to know whether two tool calls may run at the same time.

A naive design has `execute` return a string and calls it a day. Then replay cannot re-render, the UI has nothing structured to draw, and concurrency is a guess.

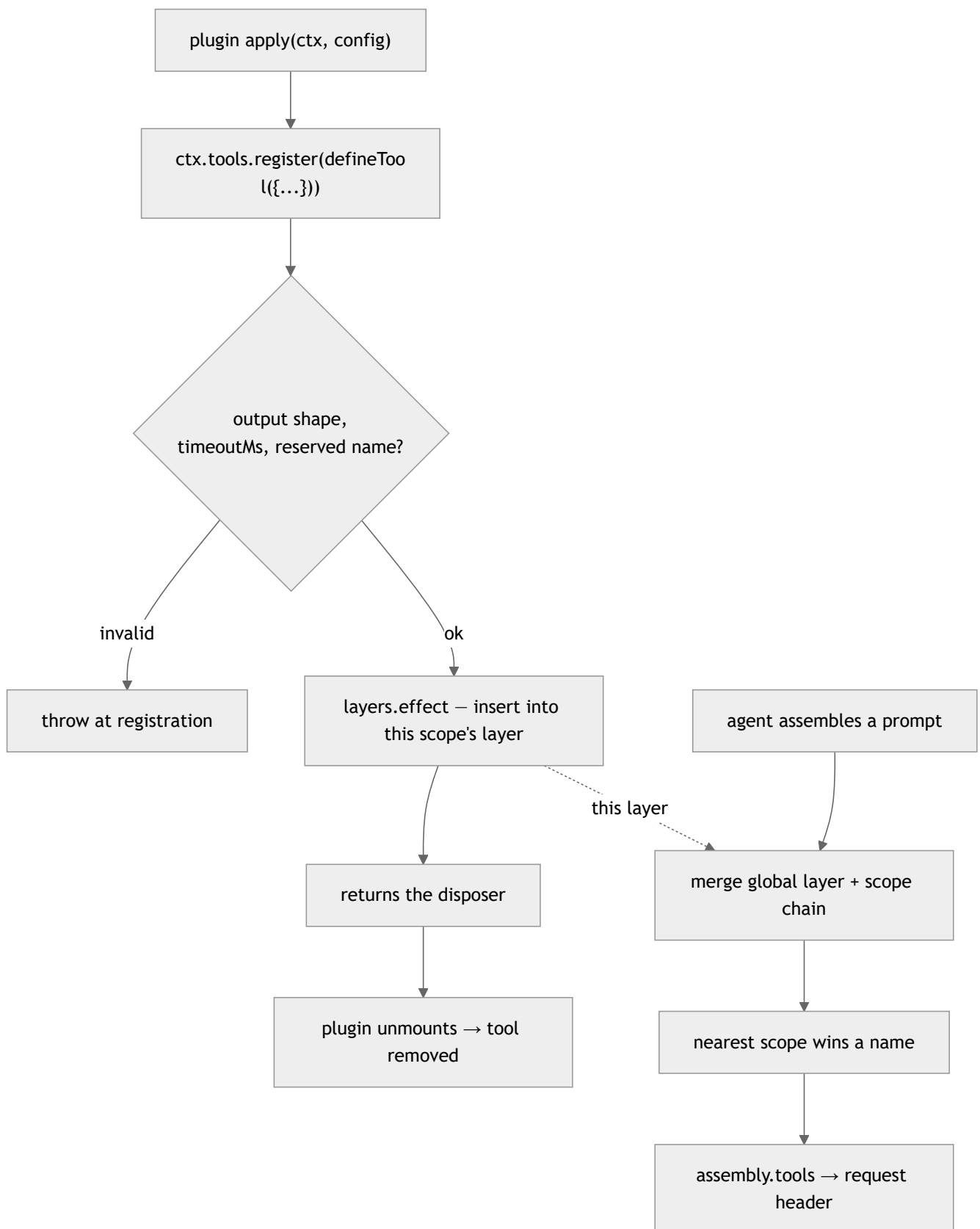
2. Mental model

New term — `ToolRuntime`. The service at `ctx.tools` (`packages/core/tools/src/index.ts:780`), holding a scope-layered registry of tool definitions.

A tool definition separates three things that are usually conflated:

Concern	Field	Audience
What the model may call	<code>name</code> , <code>description</code> , <code>parameters</code>	the model
What actually happens	<code>execute</code> → a canonical value	the system
What the model reads back	<code>output.render(args, value)</code> → content blocks	the model

That split is the chapter's central idea. `execute` does not produce prose. It produces data, which is validated against a schema and then *projected* into content. Replay re-renders from the stored value; a UI presenter draws from the same value; the model reads the rendered blocks.



3. Registration is an effect

```
register(definition: ToolDefinition): () => void {
  const name = definition.name
  // ...output-shape checks, timeoutMs checks, reserved-name check...
  return this.layers.effect(
    this.ctx,
    layer => layer.tools.insert(name, definition),
    { label: 'tools.register()' },
  )
}
```

— `packages/core/tools/src/index.ts:1028-1053`

The returned function unregisters the tool. This is the repo-wide pattern from [Chapter 2](#): a registration is an effect, so unloading the plugin that registered a tool removes it with no bookkeeping.

`this.layers` is a `ScopedLayers` — a global layer plus one per scope. That is what lets the same registry serve every agent while a preset-mounted tool is visible only to agents on that preset ([Ch 34](#)). Chapter 19 covers the same layering for prompt sections.

Two hard refusals: duplicate names within one scope throw (:719-721), and the transport name `run_code` is reserved and can never be registered or shadowed (:1045-1047).

A concrete registration, from the shell tool:

```
ctx.tools.register(defineTool({ ... })))
```

— `packages/shell/tool-bash/src/index.ts:241`, inside the plugin's `apply()`

So the effect nests twice: `register()`'s layer insertion is one, and the plugin's `apply()` already runs inside the plugin's own effect scope. Unmount the plugin and both unwind.

4. The definition

```
interface ToolDefinition extends ToolSchema {
  output: ToolOutputDefinition
  execute(args: unknown, exec: ToolRunContext): Promise<unknown>
  finalizeContent?(exec, result): ContentBlock[] | undefined
  timeoutMs?: number
  isConcurrencySafe?(args: unknown): boolean
  presentCall?(args): ToolCallView | undefined
  presentResult?(args, result): ToolResultView | undefined
}
```

— `packages/core/tools/src/index.ts:214-280`

`ToolSchema` — `name`, `description`, `parameters` — is the part that reaches the model, and comes from the LLM package so the same type flows into the request.

`output` is mandatory: `{ schema, render(args, value), presentationMeta?(args, value) }` (:204–211). The schema validates what `execute` returned; `render` turns it into content blocks.

`timeoutMs` is cooperative and, notably, **never sent to the model** (:247). Enforcement lives in a separate plugin, `dsh-tool-call-timeout-policy`, which wraps `dispatch` (Ch 16).

`isConcurrencySafe` is how a tool opts into parallel execution (Ch 17). Omitting it means exclusive.

`finalizeContent` is snapshotted at call creation and invoked exactly once per outcome, even for pipeline failures that bypass the normal post-execute path.

5. `defineTool`

Most first-party tools do not build a `ToolDefinition` by hand. `defineTool()` (`packages/core/tools/src/schema.ts:545–617`) compiles an author-facing schema DSL to JSON Schema and wraps the callbacks. The important wrap:

```
async execute(args: unknown, exec: ToolRunContext): Promise<JsonValue> {
  const violations = validate(args)
  if (violations.length > 0) throw new ToolArgsError(violations)
  return userExecute(args as InferArgs<S>, exec) as Promise<JsonValue>
}
```

— `schema.ts:585–589`

Arguments are validated against the compiled schema before the author's body runs, and a violation throws `ToolArgsError` carrying **every** violation rather than the first (`schema.ts:461–470`, code `INVALID_ARGS`).

The presenters are wrapped differently — **softly**:

```
// presentCall / presentResult / isConcurrencySafe: swallow invalid args → undefined / false
```

— `schema.ts:594–616`

The asymmetry is deliberate and worth internalizing. `execute` must be strict: running a tool with bad arguments could do damage. Presenters must be lenient: they run against *old logged arguments during session replay*, where a schema may since have changed. A presenter that threw would make historical sessions unopenable.

A raw `ToolDefinition` without `defineTool` is legal and gets **no** automatic validation — whatever Chapter 17's argument parsing produced reaches `execute` untouched, including a bare string.

The raw path is not dead, and what uses it is instructive. Two production sites register a hand-built definition, and both do so for the same reason: **their parameter schema is not known at compile time**, so the typed DSL cannot express it.

MCP tools (`packages/mcp/mcp-client/src/tools.ts:182`) are discovered from a remote server at runtime. Their schemas arrive over the wire, so `ctx.tools.register(definition)` takes a value assembled from the server's response.

The subagent structured-output tool (`packages/subagent/subagent-in-process-driver/src/structured.ts:74`) takes its schema from the *caller's* requested output shape. Notice what it then has to do by hand:

```
execute(args: unknown, exec: ToolRunContext): Promise<{ recorded: true }> {
  const violations = validateJsonSchemaValue(schema, args)
  // ToolArgsError → isError result with INVALID_ARGS: the model retries
  // within the same turn, exactly like a schema-validated defineTool call.
  if (violations.length > 0) throw new ToolArgsError(violations)
  ...
}
```

It re-implements `defineTool` 's validation step explicitly, and the comment names the behavior it is matching. **That is the obligation the raw path carries:** the framework performs no argument validation for you, so a raw definition must do it itself or receive whatever [Chapter 17's](#) `parseArguments` produced — including a bare string.

Every shipped tool with a *statically known* schema — `bash` , `str_replace_editor` , `glob` , `grep` , `skill` , `web_search` — uses `defineTool` .

6. Control decisions

Decision	Condition	Location
Reject registration	malformed output , bad timeoutMs	:1028–1044
Reject reserved name	name is run_code	:1045–1047
Reject duplicate	name already in this layer	:719–721
Throw ToolArgsError	defineTool validation fails	schema.ts:585–589
Return undefined / false	a presenter or classifier gets invalid args	schema.ts:594–616

7. Edge cases

A tool can be unregistered while one of its calls is in flight. `finalizeContent` is captured at call creation (`:1409`), so a tool that unregisters mid-call still gets *its own* finalizer applied to its own outcome — not whatever tool now occupies that name after a hot reload.

The tool list is part of the request header. Registering or unregistering mid-session changes `assembly.tools` , which changes the header, which logs a `change` header ([Ch 10](#)) and invalidates provider prefix caching. That is one reason the plan-mode prompt tells the model the catalog stays the same across

modes: *"The tool catalog stays the same across modes for request-cache stability... those tools remain listed only to keep the request shape stable"* (`packages/bundle/base/cordis.patch.yml:315`). Plan mode restrains the model with instructions rather than by removing tools.

8. Configuration knobs

Setting	Default	Effect
<code>mode</code>	<code>native</code>	<code>native ptc both</code> . In the web profile it reads <code>DSH_TOOLS_MODE</code> , unset keeping the default. Its own comment calls this a temporary seam pending per-session selection (<code>packages/bundle/web-app/cordis.patch.yml:31-37</code>)
<code>maxParallelSubCalls</code>	—	Concurrency for <code>run_code</code> sub-dispatches

9. Interactions

- **Ch 16** — the four stages that run a registered definition.
- **Ch 17** — reads `isConcurrencySafe` live, per call.
- **Ch 19** — the registry registers a tool provider with the prompt service, so `assembly.tools` comes from here.
- **Ch 34** — scope layering is what makes a preset's tools per-agent.

10. Build it yourself

Minimal version:

```
const tools = new Map<string, ToolDefinition>()
function register(def: ToolDefinition): () => void {
  tools.set(def.name, def)
  return () => tools.delete(def.name)
}
```

What the real one adds:

Addition	Why it exists
<code>ScopedLayers</code> instead of a Map	One registry must serve many agents with different tool sets
Effect-based registration	Unmounting a plugin must remove its tools with no bookkeeping
Mandatory <code>output</code> schema + <code>render</code>	Replay and UI need structured data, not prose
Strict <code>execute</code> , lenient presenters	Bad args must not run; old args must still render
Snapshotted <code>finalizeContent</code>	A hot reload mid-call must not apply the wrong tool's finalizer
Reserved <code>run_code</code>	The PTC transport name must never be shadowed

Key takeaways

- `ctx.tools` is a scope-layered registry; registration is an effect that returns its own undo.
- A definition separates the model-facing schema, the canonical value `execute` produces, and the rendered content the model reads.
- `defineTool` validates arguments strictly before `execute` and leniently for presenters, because presenters replay historical arguments.
- A raw definition gets no validation at all.
- The tool list is part of the request header, so registry changes have a cache cost — which is why plan mode restrains by instruction rather than by removing tools.

Exercises

1. Why must `output.render` be a pure function of `(args, value)`? Construct a replay bug that a render reading external state would cause.
2. A tool is unregistered while a call is in flight, and a different tool registers the same name immediately after. Which `finalizeContent` runs, and which line decides?
3. Plan mode keeps mutation tools listed but forbids using them. Give one benefit and one risk of that choice versus removing them from the catalog.

Next: [Chapter 16 · The execution pipeline](#)

Chapter 16 • The execution pipeline

What you'll learn: the four stages every tool call passes through, the exact rule deciding which results skip post-processing, and why a denial is treated more like a result than like an error.

Prerequisites: [Chapter 15](#).

1. The problem

"Call the function" is the easy part. Around it sit a set of concerns that all want to intervene at different moments:

- a policy may want to block a call, or ask a human first;
- a timeout wrapper wants to substitute its own signal for the call's duration;
- a spill policy wants to shrink an enormous result *after* it returns;
- the caller may cancel at any point — before the body runs, or halfway through it;
- and whatever happens, the model must receive a well-formed result, because a turn with a tool call and no matching result is not a valid conversation.

Bundling all of that into one `await tool.execute()` produces a function nobody can reason about. Splitting it into stages lets each concern attach where it belongs — and, crucially, lets the **scheduler** overlap the slow part across many calls while keeping the ordered parts ordered ([Ch 17](#)).

2. Mental model

Four stages, exposed as one interface:

```
export interface ToolRuntimeScheduler {
  prepare(exec: ToolExecutionInput): Promise<ScheduledToolPreparation>
  dispatch(exec: ToolRunContext): Promise<ScheduledToolDispatch>
  finalize(exec: ToolRunContext, result: ToolExecutionResult): Promise<ToolExecutionResult>
  finish(exec: ToolRunContext, result: ToolExecutionResult): ToolExecutionResult
}
```

— `packages/core/tools/src/index.ts:444–453`

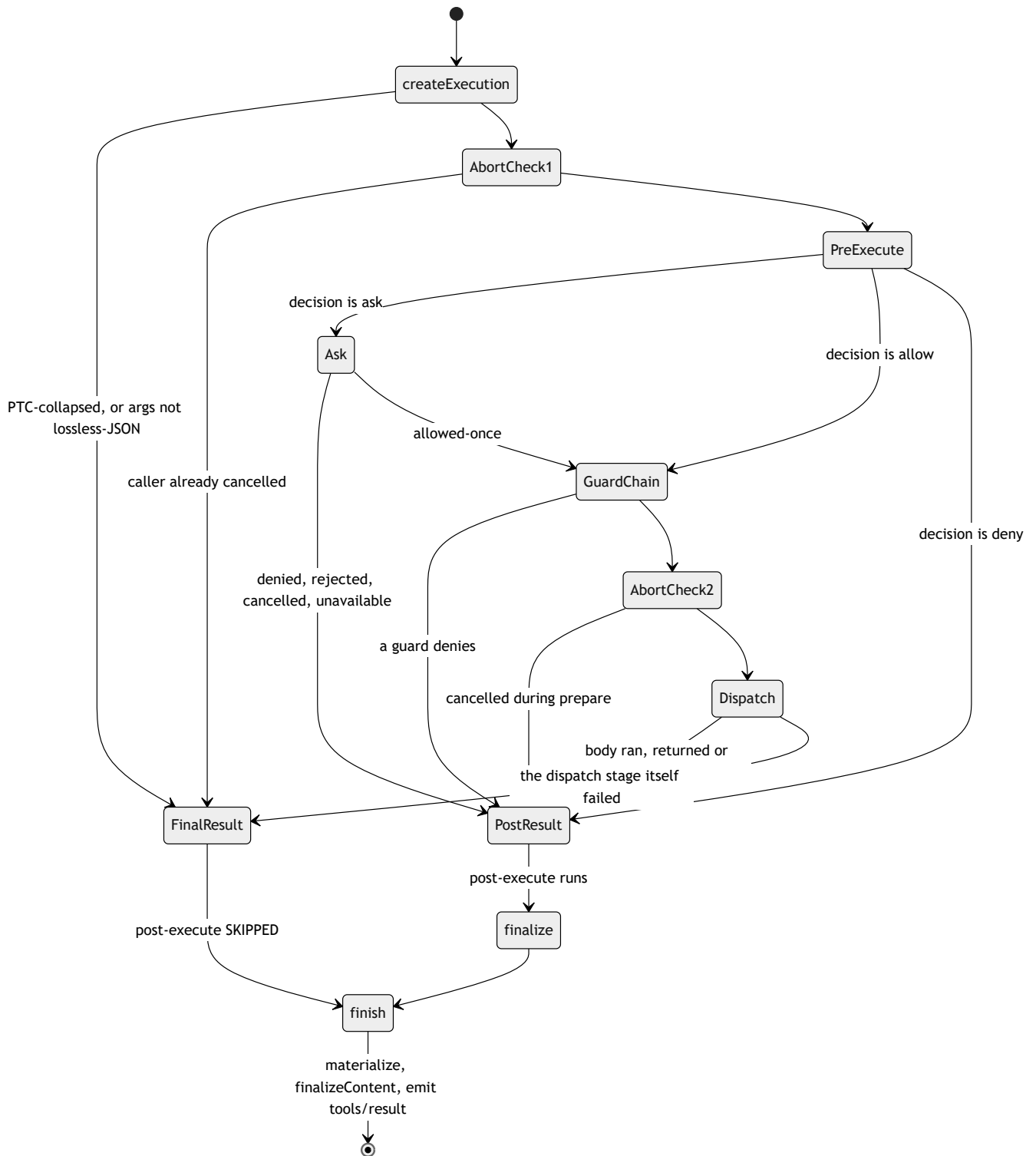
Reached through a symbol key, `TOOL_RUNTIME_SCHEDULER` (:459), wired to four private methods (:789–794).

Stage	Nature	What it decides
prepare	ordered, may await	<i>May</i> this call run?
dispatch	overlappable	What did it return?
finalize	ordered, may await	Does anything want to change that?
finish	ordered, synchronous	Freeze, finalize content, notify

The ordinary public `execute()` is not a second implementation — it chains these same four (:1333–1353). The scheduler just exposes the seams so a caller can interleave.

New term — canonical result. A result already validated and rendered through its tool's output contract. Re-normalizing one is a no-op, tracked in a `WeakMap` (:1775–1781), so a result passing through several wrappers is not re-validated each time.

3. Lifecycle



4. Stage 1 — prepare

`prepareExecution` (:1454–1498), in order:

Create the execution (`createExecution` , :1355–1442). Mints an opaque `ToolExecutionToken` , resolves `rootCallId` , and snapshots and freezes the arguments.

It also checks the **PTC collapse** first, before anything else observes the call. When the calling scope presents `mode: 'ptc'` and the call is model-direct rather than a `run_code` sub-dispatch, the call short-circuits to a `final-result` carrying an actionable "call it from inside `run_code`" message (:1427–1434). The comment says why this is first:

```
"pre-execute listeners, approval ask, and guards must never observe — or worse, approve — a call that can only fail" — :1364–1370
```

Asking a human to approve a call that is structurally incapable of running would be worse than useless.

Caller-cancellation check (:1461–1463) → `final-result` with `toolAbortedBeforeDispatchResult()`.

The tools/pre-execute waterfall (:1466–1469). Default with no listeners: `{ kind: 'allow' }`. A listener returns one of:

```
type PreToolDecision =  
  | { kind: 'allow' }  
  | { kind: 'deny'; reason: string }  
  | { kind: 'ask'; reason?: string }
```

— :576–584

Approval, if the decision was `ask` (:1470–1472) — [Chapter 18](#).

Post-ask cancellation race (:1474–1476). If the caller cancelled *while* approval was pending **and** the approval channel itself reported cancellation, the call becomes a `post-result` — not a `final-result`. It still goes through post-execute.

The guard chain (:1477–1490), consulted only when the pre-execute decision was `allow`. A denial from either source becomes a `post-result` with a synthetic error.

Second cancellation check (:1491–1493), then `{ kind: 'dispatch', exec }` (:1494).

Anything thrown anywhere in the stage becomes a `final-result` (:1495–1497).

5. Stage 2 — `dispatch`

`dispatchScheduledExecution` (:1560–1590) runs the `tools/execute` **around-dispatch waterfall** (:1564–1567), whose innermost `next()` is the tool body.

`dispatchToolBody` (:1523–1551):

1. **Fuses signals.** A wrapper may have replaced `exec.signal` — a timeout policy substituting its own. `fuseToolSignals` (:1880–1907) re-fuses the caller's *original* signal underneath, so a wrapper cannot detach caller cancellation.
2. Re-checks abort, resolves the definition, sets `state.bodyInvoked = true`.

3. Calls `tool.execute(exec.arguments, exec)`.
4. `createSuccessResult` validates the returned value against `output.schema`, calls `output.render`, and for a top-level call also `output.presentationMeta`.

A wrapper that authors its *own* result has it re-validated through the same contract by `normalizeDispatchResult` (:1817–1835) — unless already marked canonical.

Cancellation during dispatch replaces a successful result with the aborted-after-start outcome (:1509–1516) — code `TOOL_ABORTED`, distinct from `ABORTED_BEFORE_DISPATCH`, because the body *did* run and may have had effects.

`dispatch` never returns `'dispatch'`. Its only outcomes are `post-result` (the normal case, body ran or threw) and `final-result` (the dispatch stage itself failed).

6. Stages 3 and 4 — `finalize` and `finish`

`finalize` (:1600–1612) runs the `tools/post-execute` waterfall (`postExecute`, :1733–1772), applies a cancellation override, then calls `finish`. A post-execute decision may:

- `block` → the result becomes `isError: true` with the decision's feedback as content; **tool-deferred context is discarded**, only the block decision's own `additionalContexts` survive (:1729–1730);
- `accept` with `value` → re-validated and re-rendered through `createSuccessResult`, and it **cannot** replace the value of an already-failed result (`TypeError`, :1756–1758);
- `accept` with `content` → content replaced only;
- both `content` and `value` → `TypeError` (:1748–1750).

`finish` (:1622–1637) is synchronous and does three things: materialize (freeze and lossless-JSON-check the presentation fields, :1838–1853), apply the **snapshotted** `finalizeContent` (:1640–1645), and `notifyResult` (:1648–1667) — which freezes `exec` itself (:1651) and emits `tools/result` with listener failures contained.

7. The `needsPost` rule

This is the part most worth getting exactly right, and it is decided entirely by *which stage produced the outcome*:

```
case 'dispatch': {
  ... dispatch(prepared.exec).then(outcome => {
    slots[index] = { ..., needsPost: outcome.kind === 'post-result' }
  })
}
case 'post-result': slots[index] = { ..., needsPost: true }; break
case 'final-result': slots[index] = { ..., needsPost: false }; break
```

— `packages/core/agent-loop/src/tool-calls.ts:172–196`

and then:

```
const result = slot.needsPost
  ? await ctx.tools[TOOL_RUNTIME_SCHEDULER].finalize(slot.exec, slot.result)
  : ctx.tools[TOOL_RUNTIME_SCHEDULER].finish(slot.exec, slot.result)
```

— `tool-calls.ts:152-154`

Outcome	Produced by	Post-execute?
PTC collapse, bad args	<code>createExecution</code>	No
Cancelled before prepare ran	<code>prepare</code> entry	No
Denied by pre-execute or a guard	<code>prepare</code>	Yes
Cancelled while approval pending	<code>prepare</code>	Yes
Body ran, returned or threw	<code>dispatch</code>	Yes
Dispatch stage itself failed	<code>dispatch</code>	No

The interesting line is that **denials get post-execute**. A call the guard refused still passes through `tools/post-execute`, where a listener can inspect or rewrite the synthetic denial. That is deliberate — the JSDoc notes "Thrown tools still reach this waterfall as errors" (:420-422). Post-execute is about *results*, and a denial is a result.

What skips it are the cases where the call never became a real dispatch candidate: it was structurally impossible, or the pipeline itself broke.

8. Edge cases

value never reaches the log. `ToolExecutionSuccess.value` is "deliberately omitted from durable events" (:551). `materializeFinalResult` keeps it on the in-memory object returned to a caller, but the durable `tool/result` event records only the message, `error.info`, and `meta` (Ch 5).

A wrapper cannot escape caller cancellation. `ToolDispatchExecution` deliberately makes `signal` mutable so a wrapper can substitute one (:384-387), but `fuseToolSignals` re-fuses the original underneath every time.

`run_code` is a second consumer of this same interface. PTC mode re-implements the ordered-prepare / overlapping-dispatch / ordered-commit discipline for sub-calls (`ptc.ts:280-587`), using the same `TOOL_RUNTIME_SCHEDULER` symbol. So the scheduler seam has two independent users, which is why it is exported at all.

9. Configuration knobs

Mostly none — the pipeline's behavior comes from which plugins register on its waterfalls. In the shipped web profile:

Plugin	Attaches to	Effect
tool-call-timeout-policy	tools/execute	Substitutes a timeout signal for the call's duration
spill-policy	tools/post-execute	Persists oversized plain-text results out of line (Ch 26)
repeat-tool-reminder	agent/pre-step	Not this pipeline, but observes the same chain

✓

Verified: **no first-party always-on plugin registers a tools/pre-execute listener that returns ask** . Grepping the whole tree for a non-test `{ kind: 'ask' }` finds only `packages/hooks/hooks-claude-code/src/index.ts:243` — and the hook bridges are not mounted (Ch 2). The generic ask seam exists, is fully implemented, and is dormant. What shipped tools actually do is [Chapter 18](#).

10. Build it yourself

Minimal version:

```
async function execute(input: ToolExecutionInput): Promise<ToolExecutionResult> {
  const tool = tools.get(input.name)
  if (!tool) return toolErrorResult(new Error(`no tool "${input.name}"`))
  try {
    const value = await tool.execute(input.arguments, input as ToolRunContext)
    return { isError: false, value, content: tool.output.render(input.arguments, value) }
  } catch (error) {
    return toolErrorResult(error)
  }
}
```

What the real one adds:

Addition	Why it exists
Four separable stages	The scheduler must overlap dispatch while keeping policy and commits ordered
Collapse check before any listener	Never ask a human to approve a call that cannot run
Two cancellation checks in <code>prepare</code>	Cancellation can land before or during the policy await
<code>ABORTED</code> vs <code>ABORTED_BEFORE_DISPATCH</code>	Whether the body ran changes whether a retry is safe
Signal re-fusing	A wrapper must not be able to detach caller cancellation
Canonical-result marking	A result crossing several wrappers must not be re-validated each time
<code>needsPost</code> by producing stage	Denials deserve post-processing; impossible calls do not
Snapshotted <code>finalizeContent</code>	Applies even to outcomes that bypass post-execute
Contained <code>tools/result</code> listeners	A UI listener must not be able to fail a tool call

Key takeaways

- Four stages: `prepare` (may it run), `dispatch` (what happened), `finalize` (does anything want to change it), `finish` (freeze and notify).
- The PTC collapse is checked before any listener or approval sees the call.
- `ABORTED_BEFORE_DISPATCH` and `ABORTED` differ by whether the body ran — which is exactly what a caller needs to decide about retrying.
- `needsPost` is determined by which stage produced the result; denials get post-execute, structurally-impossible calls do not.
- A tool's canonical `value` never reaches the durable log; only the rendered content does.
- The generic ask/deny seam is fully implemented and has no always-on producer in the shipped profile.

Exercises

1. A guard denies a call and a `tools/post-execute` listener `accept`s it with a new `value`. What happens, and which line stops it?
2. Trace the two cancellation checks in `prepare`. Construct a timing where the first passes and the second fails, and say what result the model sees.
3. `run_code` re-implements the ordered/overlapping discipline rather than calling `execute()` per sub-call. Give two things it would lose by looping over `execute()`.

Next: [Chapter 17 · Scheduling tool calls](#)

Chapter 17 • Scheduling a step's tool calls

What you'll learn: how one assistant step's tool calls are run — some in parallel, some as barriers — while the log they produce stays in strict model order.

Prerequisites: [Chapter 16](#), [Chapter 9](#).

1. The problem

A model emits five tool calls in one response: three file reads, a shell command, another read. Running them strictly in sequence wastes most of the wall-clock time — the reads do not conflict. Running them all at once is unsafe: the shell command might modify a file another call is reading.

So concurrency has to be per-call, and decided by the tool. But that creates a second problem. Whatever order they *finish* in, the results must reach the log in **model order** — the order the model asked for them. A model that requested A, B, C and reads back C, A, B in its history is being shown a conversation that did not happen, and on the next turn it will reason about a sequence that never occurred.

And then cancellation. If the user stops the agent after two of five calls have started, the log still needs a result for all five — because a `tool/call` with no matching `tool/result` is an unbalanced conversation that a provider will reject on the next request.

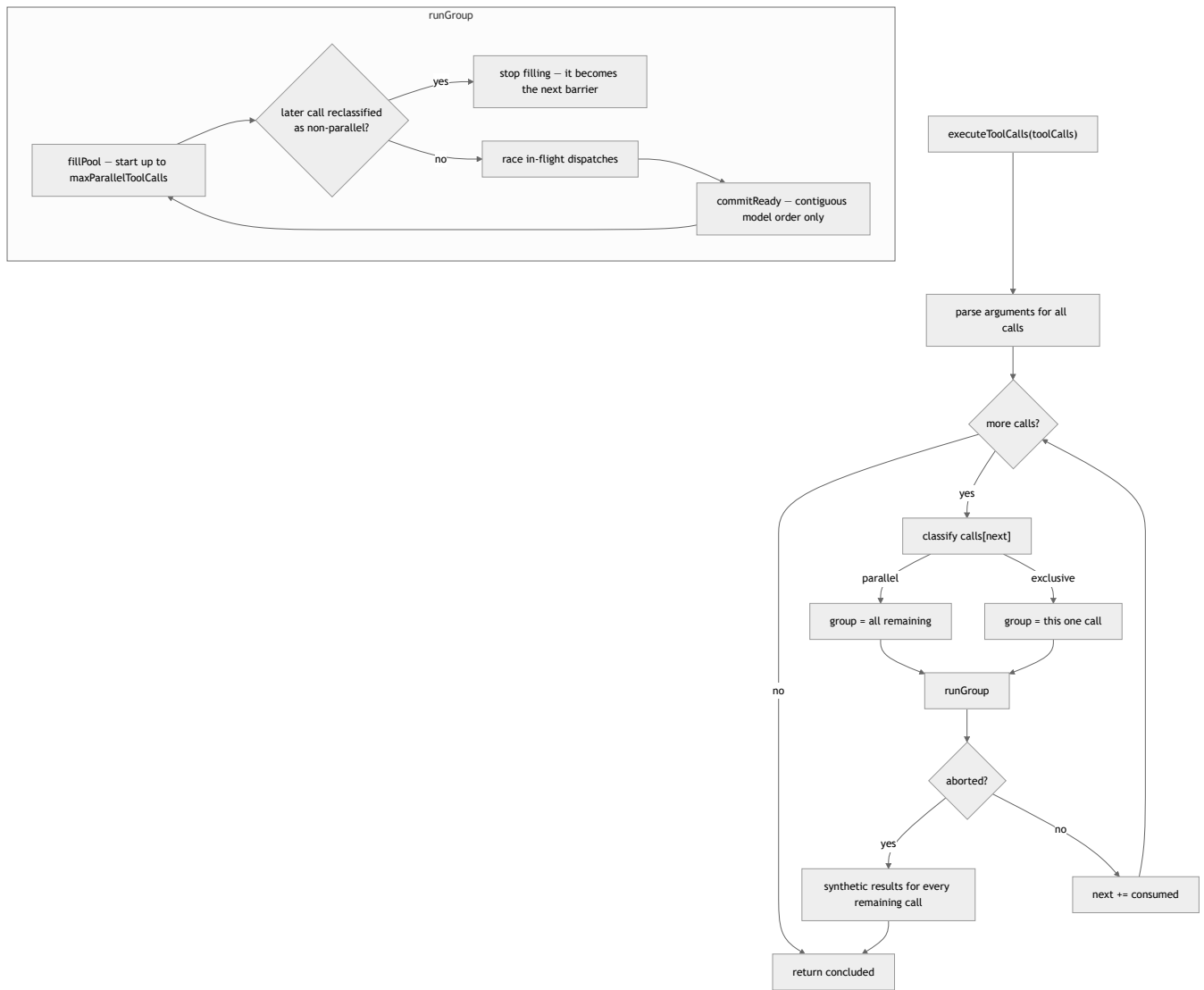
2. Mental model

Three rules, and everything follows:

1. **Classify at the boundary.** Walk the calls in model order. The first call's mode decides the group: if it is parallel-safe, the group is *all remaining calls*; otherwise the group is *just that one call* — an exclusive barrier.
2. **Dispatch may overlap; commits may not.** Bodies run concurrently up to a cap. Results commit only in contiguous model order — a result that finishes early waits for its predecessors.
3. **Every model call gets a result.** Including ones cancellation prevented from ever starting, which receive a synthetic error.

New term — barrier. An exclusive call that runs alone. Everything before it has committed; nothing after it starts until it finishes.

3. Lifecycle



4. Step-by-step walkthrough

Parsing arguments

```
function parseArguments(raw: string): unknown {
  try {
    return raw ? JSON.parse(raw) : {}
  } catch {
    return raw
  }
}
```

— `packages/core/agent-loop/src/tool-calls.ts:105-111`

Empty string becomes `{}`. Valid JSON is parsed — to *any* JSON value, not necessarily an object. **Invalid JSON returns the raw string unchanged.**

That last branch looks like a bug and is not. Malformed model output must not crash the scheduler; it must become a tool result the model can read and correct. The string flows through to `execute`, where `defineTool`'s validation rejects it with a `ToolArgsError` (Ch 15) — which becomes an ordinary `isError` result that still goes through post-`execute`, so the model sees `Error: invalid arguments: ...` and can retry **within the same turn**.

Grouping

```
const first = planned[next]!
const mode = ctx.tools.executionMode(first.exec).kind
const group = mode === 'parallel' ? planned.slice(next) : [first]
```

— `tool-calls.ts:87-90`

`executionMode` is synchronous, side-effect-free, and **fail-closed** (Ch 15): no declaration, a throwing classifier, or any non-`true` return all yield `exclusive` (`packages/core/tools/src/index.ts:1267-1276`).

The comment on the grouping line notes it commits before classifying again, "so registry changes affect unstarted calls."

Filling the pool

```
while (!aborted && nextToStart < group.length && inFlight.size < maxParallelToolCalls) {
  const nextCall = group[nextToStart]!
  if (nextToStart > 0 && mode === 'parallel'
    && ctx.tools.executionMode(nextCall.exec).kind !== 'parallel') break
  await startCall(nextToStart)
  nextToStart++
  throwSchedulerFailure()
  await commitReady()
  throwSchedulerFailure()
  if (signal.aborted) aborted = true
}
```

— `tool-calls.ts:199-214`

Two details carry weight.

Modes are re-read live. A call classified parallel when the group formed may classify differently by the time it is about to start — a tool unregistered, an agent's tool set narrowed. The pool stops filling at that point, and that call becomes the caller's next barrier. A batch that began as all-parallel can split mid-flight.

The cap is read per group.

```
const { maxParallelToolCalls } = ctx.agentLoop.config
```

— `tool-calls.ts:132`

Destructured once at group start, from a config property that is a **read-through getter** (`index.ts:389-391`). A settings change caps the *next* group without disturbing one in flight — deliberate, and the comment says so (Ch 10 shows the same discipline for headers).

Starting one call

```
callSeqs[index] = appendToolCall(session, turn, step, call.block)
started++
const prepared = await ctx.tools[TOOL_RUNTIME_SCHEDULER].prepare(call.exec)
```

— `tool-calls.ts:165-170`

The `tool/call` event is appended **before** `prepare` — so the log records that a call was attempted even if policy denies it. Its `seq` is kept so the eventual result can cite it.

`prepare` is awaited *in the pool-filling loop*, which means the ordered policy stage runs in model order. Only `dispatch` overlaps.

Committing in order

```
const commitReady = async (): Promise<void> => {
  while (committed < group.length) {
    const slot = slots[committed]
    if (slot === undefined) break
    const call = group[committed]
    const result = slot.needsPost
      ? await ctx.tools[TOOL_RUNTIME_SCHEDULER].finalize(slot.exec, slot.result)
      : ctx.tools[TOOL_RUNTIME_SCHEDULER].finish(slot.exec, slot.result)
    appendToolResult(session, turn, step, call!.block, result, callSeqs[committed]!)
    for (const context of result.additionalContexts ?? []) acceptContext(context)
    concluded ||= result.concludesTurn === true
    committed++
  }
}
```

— `tool-calls.ts:147-161`

`if (slot === undefined) break` is the ordering rule in one line: advance only across **contiguous** settled slots. If call 3 finishes before call 2, its slot is filled but `committed` stays at 2 until 2 lands.

Note what else is ordered by consequence: `finalize` / `finish` run here, so post-execute runs in model order too — not in completion order. And `additionalContexts` are accepted in model order, so injected context from parallel tools still reaches the next step deterministically.

Cancellation

```
if (aborted) {
  for (const call of group.slice(started)) appendSkippedToolCall(session, turn, step, call.block)
  return { consumed: group.length, aborted: true, concluded }
}
```

— `tool-calls.ts:238-243`

and at the top level:

```
if (outcome.aborted) {
  for (const call of planned.slice(next)) appendSkippedToolCall(session, turn, step, call.block)
  return { concluded }
}
```

— `tool-calls.ts:96-99`

Every call the scheduler never started gets a synthetic `tool/call` + `tool/result` pair:

```
appendToolResult(session, turn, step, block, {
  content: [{ type: 'text', text: 'Error: tool call aborted before dispatch' }],
  isError: true,
  error: { message: 'tool call aborted before dispatch',
    info: { name: 'AbortError', code: TOOL_ABORTED_BEFORE_DISPATCH } },
}, callSeq)
```

— `tool-calls.ts:250-260`

The module doc states the purpose: "so replay stays valid" (:8). A cancelled turn still produces a conversation a provider will accept.

Started calls are **drained**, not abandoned — the loop awaits everything in flight before recording the skipped ones, so results commit in order and then the tail is filled in.

Scheduler failure

```
} catch (error: unknown) {
  schedulerFailure ??= { error }
  await Promise.allSettled(inFlight.values())
  throw schedulerFailure.error
}
```

— `tool-calls.ts:232-236`

An internal failure stops new dispatches, waits for started ones to settle, and rethrows the **first** failure — **without fabricating results**. The distinction from cancellation is deliberate: cancellation is an expected outcome with a defined meaning per call, so synthetic results are correct. A scheduler bug is not, and inventing results would hide it.

5. Control decisions

Decision	Condition	Location
Group is all remaining	first call classifies <code>parallel</code>	:90
Group is one call	anything else	:90
Stop filling	a later call reclassifies non-parallel	:204–205
Stop filling	pool at <code>maxParallelToolCalls</code>	:200
Advance commit	next slot in model order has settled	:147–150
Choose <code>finalize</code> vs <code>finish</code>	<code>slot.needsPost</code>	:152–154
Mark turn concluded	any committed result had <code>concludesTurn</code>	:158
Synthesize skipped results	aborted	:97 , :241
Rethrow without results	internal scheduler failure	:232–236

6. Edge cases and failure modes

`concludesTurn` is a **single-result veto**. `concluded || result.concludesTurn === true` — one result ends the turn regardless of what its siblings did. It is typed `never` on failures (Ch 3), so a failed call can never stop a turn. The real producer is the subagent driver's structured-output tool, which concludes once it has staged the model's answer.

The unreachable check.

```
if (committed !== started) throw new Error('tool-call scheduler: uncommitted settled calls')
```

— :245

Marked `/* v8 ignore next */` as unreachable — a non-aborted group commits every started call. It is a guard against a future edit breaking the invariant, not a live path.

Order of events in the log. For a parallel group, `tool/call` events appear in *start* order (which is model order, since the pool fills in order) and `tool/result` events in *model* order. They interleave, which can look odd when reading a raw log — but every result cites its call's `seq` via `sourceEventSeqs`, so the pairing is explicit rather than positional.

`additionalContexts` **do not go into the result**. They are handed to the caller's acceptor, which splices them into the `next-step` inbox queue (`agent.ts:434`). So a tool can inject a message the model sees *next*, separate from its own result content (Ch 12).

7. Configuration knobs

Setting	Default	Effect
<code>maxParallelToolCalls</code>	<code>10</code>	Max in-flight parallel-safe calls per group. <code>1</code> makes everything serial. Validated as an integer ≥ 1 ; a rejected value keeps the running scheduler on its last good cap

— `packages/core/agent-loop/src/constants.ts:6`, schema `index.ts:306-308`

It is **deployment-wide**, not per agent or per tool — one of the engine's real limitations ([Ch 35](#)).

Per-tool, the lever is `isConcurrencySafe`. Real examples:

Tool	Declaration	Effect
<code>read (tool-fs)</code>	<code>isConcurrencySafe: () => true</code>	parallel
<code>web_search</code>	<code>isConcurrencySafe: () => true</code>	parallel
<code>bash / pwsh</code>	<code>none</code>	exclusive
<code>str_replace_editor</code>	<code>none</code>	exclusive — even for its read-only view command

That last row is worth noticing: two first-party file-reading tools make opposite choices. `tool-fs`'s dedicated `read` declares itself safe; `str_replace_editor` does not classify per-command, so its `view` is serialized along with its mutations.

8. Interactions

- [Ch 16](#) — the four stages this orchestrates.
- [Ch 9](#) — consumes `{ concluded }` to decide the turn's ending.
- [Ch 12](#) — receives `additionalContexts`.
- [Ch 5](#) — every call and result is appended here, with provenance.
- [Ch 13](#) — supplies the shared step signal.

9. Build it yourself

Minimal version — the sequential loop from [Chapter 4](#):

```
for (const call of toolCalls) {
  const callSeq = session.append('tool/call', {...}).seq
  const result = await ctx.tools.execute({...})
  session.append('tool/result', {...}, { surfaceOp: 'append', sourceEventSeqs: [callSeq] })
}
```

Correct, and often fast enough. What the real one adds:

Addition	Why it exists
Per-call classification	A shell command and a file read have different safety properties
Barrier grouping	An exclusive call must see the effects of everything before it
Bounded pool	Unbounded parallelism exhausts file handles and provider limits
Contiguous ordered commit	Completion order is not model order; history must be model order
Live reclassification	The registry can change mid-group
Per-group cap read	A settings change must not disturb a batch in flight
Synthetic results on abort	An unbalanced call/result pair breaks the next request
Drain-don't-abandon	Started work must settle before the turn unwinds
No synthetic results on scheduler failure	Inventing results would hide a bug

Key takeaways

- The first call in a group decides the grouping: parallel-safe takes all remaining, anything else forms a one-call barrier.
- Dispatch overlaps; `prepare`, commits, post-execute, and context acceptance all stay in model order.
- Modes are re-read before each start, so a batch can split into a barrier mid-flight.
- Malformed model JSON becomes a retryable tool error, never a crash.
- Cancellation synthesizes results for unstarted calls so replay stays valid; a scheduler failure deliberately does not.
- `maxParallelToolCalls` is deployment-wide and read fresh per group.

Exercises

1. A model emits `[read, read, bash, read]`. Walk through the grouping, saying how many groups form and which calls are in each. Now reverse it to `[bash, read, read, read]`.
2. Call 2 of a 4-call parallel group throws inside its body while calls 1, 3, 4 succeed. Give the exact sequence of `tool/result` events appended.
3. `str_replace_editor` does not declare `isConcurrencySafe` even for `view`. Write the classifier that would make `view` parallel, then give one reason the authors might have chosen not to.

Next: [Chapter 18 · Approval and sandbox escalation](#)

Chapter 18 · Approval and sandbox escalation

What you'll learn: the two independent ways a tool call can be gated on a human's decision — one built into the pipeline and dormant, one built into tools and live.

Prerequisites: [Chapter 16](#).

1. The problem

Some tool calls should not happen without a person saying yes. Writing outside the workspace, running a shell command with wider permissions than the session normally allows — these need a human in the loop.

Three things make this harder than a confirm dialog.

The decision must be recorded. "Did the user approve this?" is a question someone will ask later, about a session that has since been resumed on another machine. So the ask and the answer both belong in the log.

The model has to be able to *request* elevation coherently. A tool that simply fails with "permission denied" teaches the model nothing. It needs a way to say "I know this needs more access, here is why" — and that request has to be part of the call, not a side channel.

Refusal must not be bypassable by registration order. If a deployment sets policy to *never ask*, no plugin registered in the right position should be able to turn that into a grant.

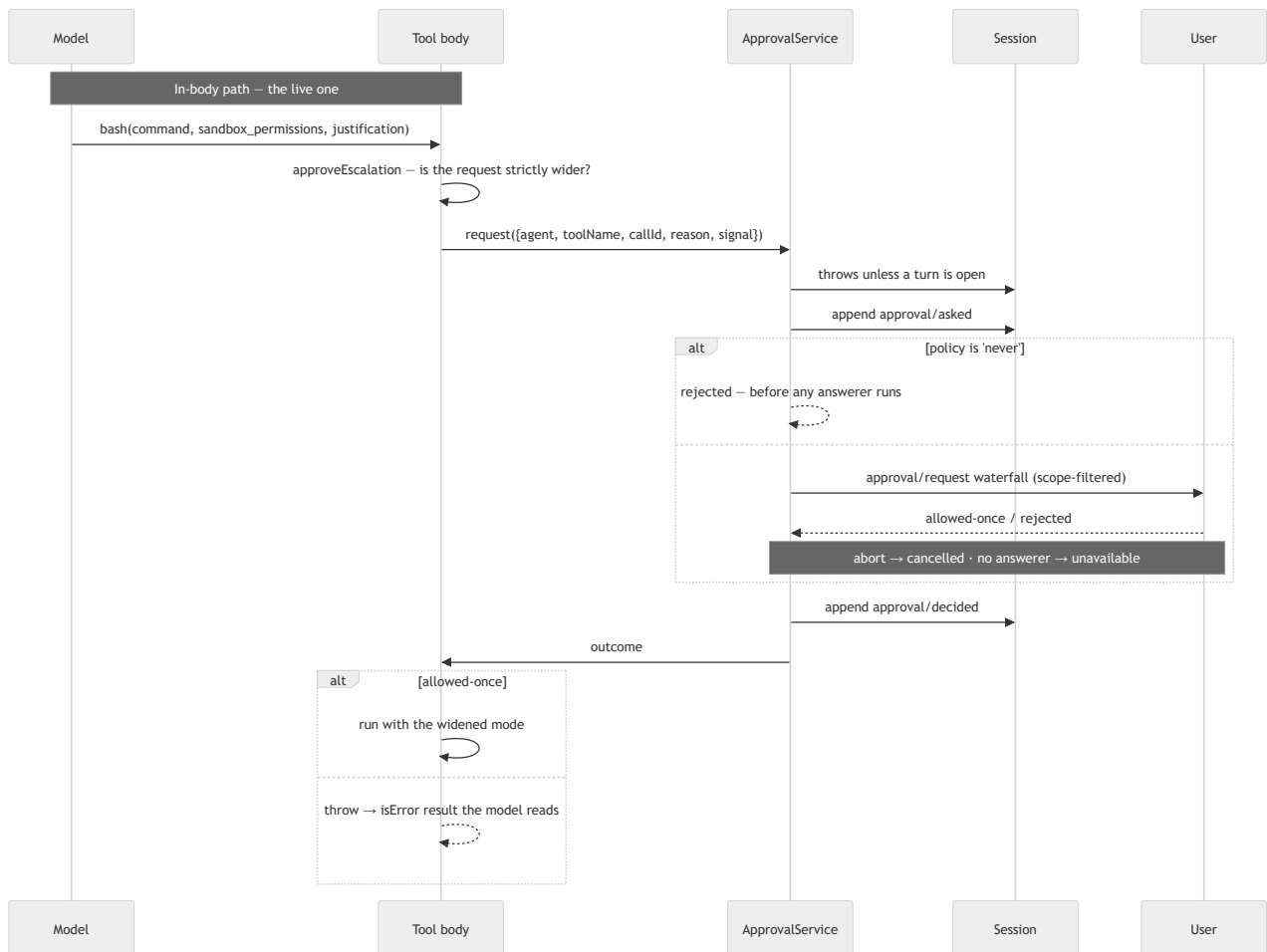
2. Mental model

There are **two separate mechanisms**, and conflating them is the main hazard in this area.

	Pipeline seam	In-body escalation
Where	<code>tools/pre-execute</code> → <code>ask</code> → <code>ApprovalService</code>	inside a tool's own <code>execute</code>
Who triggers	a policy listener	the <i>model</i> , via tool arguments
Asks about	"may this call run at all?"	"may this call run with wider permissions?"
Denial becomes	a synthetic <code>post-result</code>	a thrown error → ordinary <code>isError</code> result
Live in the shipped profile?	No producer	Yes

They share one thing: the underlying `ApprovalService`. Everything else differs.

3. Lifecycle



4. The pipeline seam

A `tools/pre-execute` listener returns one of three decisions:

```
type PreToolDecision =
  | { kind: 'allow' }
  | { kind: 'deny'; reason: string }
  | { kind: 'ask'; reason?: string }
```

— `packages/core/tools/src/index.ts:576–584`

An `ask` is resolved by `serviceAsk` (:1680–1720), called from inside `prepare` (:1470–1472) — so **approval happens within `prepare`**, not as a separate stage and not inside `dispatch`. That placement matters for the scheduler: `prepare` is the ordered stage (Ch 17), so approvals for a parallel group are requested in model order rather than in a race.

Two degradations are deterministic:

```
const approval = this.ctx.get('approval')
if (approval === undefined) return { decision: { kind: 'deny', reason: `... requires approval (not yet supported)` }, ... }
if (exec.agent === undefined) return { decision: { kind: 'deny', reason: `... no agent to route it through` }, ... }
```

— :1680–1720

No approval service composed → `ask` becomes `deny`. No agent on the execution → `deny`, because there is no session to route the prompt through. **An unanswerable question is refused, never silently allowed.**

The four outcomes map as: `allowed-once` → `allow`; `rejected`, `cancelled`, `unavailable` → `deny` with distinct reasons, and `cancelled` additionally sets `approvalCancelled`, which is what makes the call a `post-result` rather than a `final-result` (Ch 16).

It has no producer

✓ **Verified.** Grepping the whole tree for a non-test `{ kind: 'ask' }` `PreToolDecision` finds exactly one source: `packages/hooks/hooks-claude-code/src/index.ts:243`, a compatibility bridge translating an external hook protocol's `decision: 'ask'` into this seam. The hook bridges are mounted in no composition layer (Ch 2).

So the seam is complete, tested, and inert in the shipped product. It is what a deployment would use to add a policy layer; it is not how the shipped tools gate themselves.

5. The approval service

`ApprovalService.request()` (`packages/interaction/user-approval/src/index.ts:222–241`):

It refuses outside a turn. `hasOpenTurn` (:92–99 , checked at :224–230) throws if the session has no open turn. Approval audit events must live inside a turn boundary — otherwise a reader folding the log finds an `approval/asked` belonging to nothing.

It logs both halves. `approval/asked` before deciding (:232–237), `approval/decided` with the same id after (:239). Both are durable and **log-only** — they never enter the model transcript (Ch 6: they are not surface-eligible types). The model does not see that a human was asked; it sees only the result.

never is not delegatable:

```
if (policy === 'never') return 'rejected'
```

— :277 , inside `decide()` (:269–309)

This is checked **inline, before the approval/request waterfall runs at all**. The source explains the choice: a `prepend: true` listener could otherwise claim the request ahead of a policy check registered as an ordinary listener. Making it a plain early return means no registration order can leak a grant. That is a security-relevant design decision, and the right one — the same reasoning as the invariant's `prepend` in [Chapter 11](#), applied in the opposite direction.

Otherwise the request goes to the `approval/request` waterfall, scope-filtered to the agent, with `'unavailable'` as the default when nothing answers, raced against the request signal so an abort yields `'cancelled'`.

6. In-body escalation — what actually ships

The live path is inside tools. Both sandbox-enforcing families call one shared helper:

```
// approveEscalation(request, approval)
// 1. is the requested mode strictly wider than the call's effective mode?
//   WIDER_MODES: read-only → workspace-write | danger-full-access
//               workspace-write → danger-full-access
//   if not, THROW — never ask
// 2. throw if no approver, or no agent
// 3. approval.approver.request({ agent, toolName, callId,
//   reason: 'escalate sandbox to <mode>: <justification>', signal })
// 4. 'allowed-once' → return the granted mode; every other outcome throws
```

— `packages/sandbox/sandbox/src/escalation.ts:157-189`

Step 1 is the interesting one: a request that is not *strictly widening* throws rather than prompting. Asking a human to approve a no-op, or a narrowing, would be noise that trains people to click yes.

How the model requests it

Not through any registry mechanism — through **arguments on the same tool call**:

```
async execute(args: BashToolArgs, exec) {
  validateBashArgs(args)
  const standingPolicy = resolveSandboxPolicy(exec)
  const approvedMode = args.sandbox_permissions !== undefined && args.justification !== undefined
    ? await approveBashEscalation(args.sandbox_permissions, args.justification, exec,
    standingPolicy)
    : undefined
  const policy = approvedMode === undefined ? standingPolicy : { ...standingPolicy, mode:
  approvedMode }
  ...
}
```

— `packages/shell/tool-bash/src/index.ts:329-338`

The `sandbox_permissions` and `justification` parameters are **only advertised when a confining executor is mounted** (`escalationModes.length > 0`, :192; schema at :258–268). If nothing confines the shell, the model is never told the parameters exist — the tool description does not mention a capability that would be meaningless.

This is a neat piece of design: the escalation request is *part of the call*, so it is logged in the `tool/call` event's arguments like everything else, and the justification the model gave is durable.

Denial becomes an ordinary result

`approveEscalation` throws a plain `Error` — `'the user rejected escalating this command to "${mode}'` — which propagates out of `execute`, is caught by `dispatchToolBody` (`packages/core/tools/src/index.ts:1545–1546`), and becomes `toolErrorResult` (:1861–1869):

```
{ content: [{ type: 'text', text: 'Error: the user rejected escalating this command to "danger-
full-access" }],
  isError: true,
  error: { message: ..., info: ... } }
```

So from the model's point of view, a refused escalation is just a failed tool call with a clear explanation — and it can react within the same turn.

7. What the model is told up front

The approval and sandbox policies are not only enforced; they are **stated in the prompt**, as runtime-context sections ([Ch 21](#)). From the recorded session (`snapshots/session/text-turn/session.jsonl:10`):

```
Current DSH file policy: danger-full-access. The DSH file sandbox does not restrict
file modifications by available operations.

Approval prompts are disabled in this session: actions that require approval are
rejected automatically – do not request sandbox escalation (do not set 'sandbox_permissions').
```

Two sections, `sandbox:policy` (order 110) and `approval:policy` (order 115), registered by the `sandbox-policy` and `user-approval` plugins respectively (`packages/sandbox/sandbox-policy/src/index.ts:140–151`, `packages/interaction/user-approval/src/index.ts:169–181`).

Note what the second one does under `policy: 'never'`: it tells the model **not to ask**, because asking would be auto-rejected and would waste a turn. The enforcement and the instruction are kept consistent, which is why both live in the same plugin as the policy itself.

8. Control decisions

Decision	Condition	Location
<code>ask</code> → <code>deny</code>	no approval service composed	<code>tools/index.ts:1680-1720</code>
<code>ask</code> → <code>deny</code>	no agent on the execution	same
Throw before asking	the session has no open turn	<code>user-approval:224-230</code>
<code>rejected</code> without consulting anyone	policy is <code>never</code>	<code>user-approval:277</code>
<code>unavailable</code>	the waterfall's default — nothing answered	<code>user-approval:269-309</code>
<code>cancelled</code>	the request signal aborted	same
Throw, don't ask	requested mode is not strictly wider	<code>escalation.ts:157-189</code>
Return the widened mode	outcome is <code>allowed-once</code>	<code>escalation.ts:157-189</code>

9. Configuration knobs

Setting	Default	Effect
<code>approval policy</code>	<code>ask</code>	Becomes <code>never</code> iff <code>DSH_PERMISSION_MODE === 'danger-full-access'</code> — a <code>!!js</code> expression at <code>packages/bundle/base/cordis.patch.yml:233</code>
<code>sandbox mode</code>	<code>workspace-write</code>	From <code>DSH_PERMISSION_MODE (base:217)</code>
<code>workspaceRoot</code>	<code>process.cwd()</code>	<code>base:218</code>
<code>presets</code>	<code>read-only / workspace-write / danger-full-access</code>	Named sandbox+approval pairs (<code>base:238-247</code>)

The presets are the user-facing form: picking one sets both halves coherently, so a session cannot end up with `danger-full-access` file access and `ask` prompts that never fire, or the reverse.

10. Interactions

- **Ch 16** — `ask` is resolved inside `prepare`; `approvalCancelled` decides `post-result` vs `final-result`.
- **Ch 17** — because approval is in `prepare`, prompts for a parallel group are ordered.
- **Ch 21** — the two policy sections reach the model as a runtime-context snapshot.
- **Ch 5** — `approval/asked` and `approval/decided` are durable and log-only.

11. Build it yourself

Minimal version:

```
if (needsApproval(call)) {
  const ok = await ui.confirm(`Allow ${call.name}?`)
  if (!ok) return toolErrorResult(new Error('denied'))
}
```

What the real one adds:

Addition	Why it exists
Both halves logged with a shared id	"Did someone approve this?" is asked later, about a resumed session
Refusal outside an open turn	An audit event belonging to no turn is unreadable
<code>never</code> checked inline, not as a listener	No registration order may leak a grant
Deterministic degradation to <code>deny</code>	An unanswerable question must never become an allow
Strictly-wider check before prompting	Prompting for a no-op trains people to click yes
Escalation as tool arguments	The request and its justification become durable log content
Conditional schema advertisement	Don't tell the model about a capability that cannot apply
Policy stated in the prompt	Enforcement and instruction must agree, or the model wastes turns

Key takeaways

- Two independent mechanisms share one service; only the in-body one is live in the shipped profile.
- The generic `tools/pre-execute` → `ask` seam is complete and has no always-on producer.
- Approval resolves inside `prepare`, so prompts stay in model order across a parallel group.
- Both the ask and the decision are logged, and never shown to the model.
- `policy: 'never'` is enforced inline so no listener ordering can turn it into a grant.
- The model requests elevation through arguments on the same call, and a refusal comes back as an ordinary readable tool error.
- Those parameters are only advertised when something actually confines the tool.

Exercises

1. The `never` check is an early return rather than a prepended listener. Write the listener that would defeat the alternative design, and say whether it would look malicious.
2. `approveEscalation` throws when the requested mode is not strictly wider. Give a concrete argument pair that hits this, and say what the model sees.

3. `approval/asked` and `approval/decided` are log-only. Design a UI feature that needs both, and say why putting them on the surface instead would corrupt the next request.

Next: [Chapter 19 · Prompt assembly](#)

Chapter 19 • Prompt assembly

What you'll learn: how the system prompt and tool list are composed from contributions scattered across dozens of plugins, and how one agent's prompt differs from another's without either knowing about the other.

Prerequisites: [Chapter 15](#) for scope layering.

1. The problem

The system prompt is not written anywhere. It is assembled, on every step, from fragments contributed by whichever plugins happen to be mounted: a harness identity line, a deployment persona, a paragraph per tool explaining when to use it, a plan-mode block when plan mode is on.

Four requirements pull against each other.

Order must be stable and meaningful. The identity line goes first; tool guidance goes in the middle; structured-output instructions go last. Contributors do not know about each other, so they cannot agree on order by convention.

Different agents need different prompts. Two sessions in the same process may run different presets with different tools and different personas. One global prompt cannot serve both.

The result must be deterministic. Same contributions, same prompt — byte for byte, on every machine. Otherwise the provider's prefix cache misses and every request costs more ([Ch 10](#)).

Some values are only known at assembly time. The model name, the working directory. A fragment cannot hardcode them.

2. Mental model

New term — `PromptAssembly`. The resolved, still-uninterpolated result:

```
export interface PromptAssembly {
  sections: AssembledSection[] // { name, text } - become the system prompt
  contexts: AssembledContext[] // { name, text } - become the runtime-context snapshot
  tools: ToolSchema[]          // the tool list for this exact assembly
  variables: Record<string, string | undefined>
}
```

— `packages/core/system-prompt/src/index.ts:114-119`

Two things to notice immediately.

sections and contexts are different destinations. Sections become the system prompt. Contexts become a *message* — the runtime-context snapshot (Ch 21). A contributor chooses which by calling `section()` or `context()`, and the choice is about volatility: the system prompt should stay byte-stable for caching, so anything that changes during a session goes in a context instead.

Text is uninterpolated here. `{{model}}` is still literally `{{model}}` in `assembly.sections`; substitution happens at render time (Ch 20).

3. The four registries

Method	Registers	Collision behavior
<code>section(s)</code>	<code>{ name, order, text, complete? }</code>	shadowed by name — nearest scope wins
<code>context(c)</code>	<code>{ name, order, text }</code>	shadowed by name
<code>tools(provider)</code>	<code>(ctx) => ToolProviderResult</code>	anonymous — all contribute
<code>variable(name, fn)</code>	<code>(ctx) => string undefined</code>	overwritten by name

— `index.ts:432-441`, `:467-476`, `:499-505`, `:515-524`

All four return the Cordis effect disposer, and registering or disposing fires `system-prompt/change` (`:37`, `:398-401`).

The tools asymmetry is deliberate: sections shadow because two plugins claiming `deployment:persona` mean one should win, while tool providers *accumulate* because every mounted tool package contributes its own schemas. Uniqueness for tools is enforced on the tool *names*, later, at ordering time.

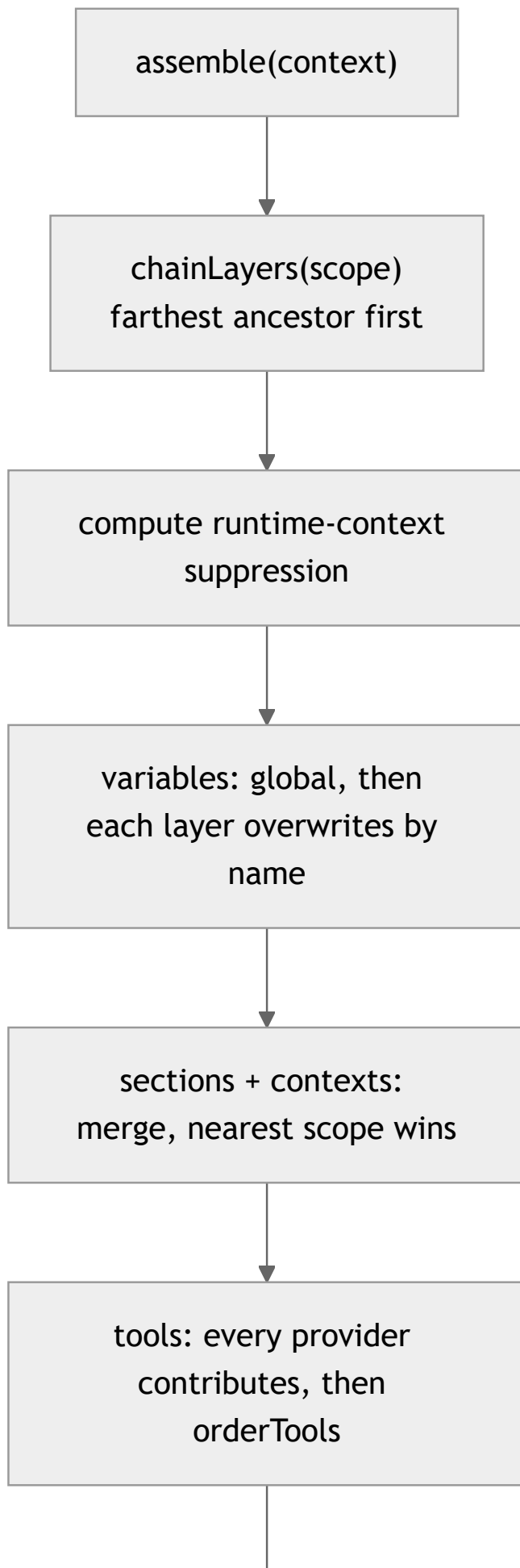
Order is centrally assigned

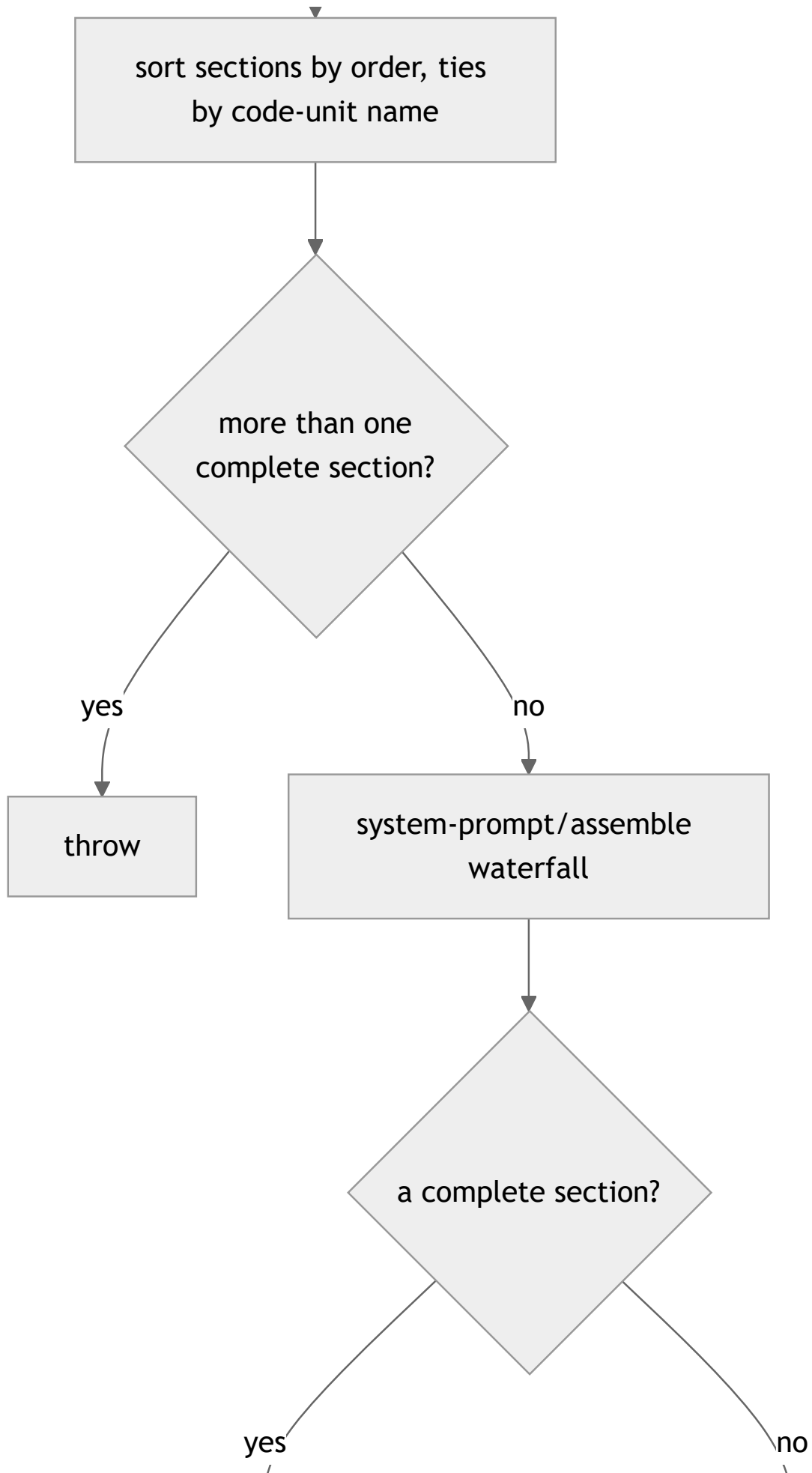
```
SECTION_ORDERS = { HARNESS_IDENTITY: -1000, ..., TOOL_BASH: 1000, ..., STRUCTURED_OUTPUT: 9900 }
CONTEXT_ORDERS = { SANDBOX_POLICY: 110, APPROVAL_POLICY: 115, SUBAGENT_DELEGATION: 120 }
```

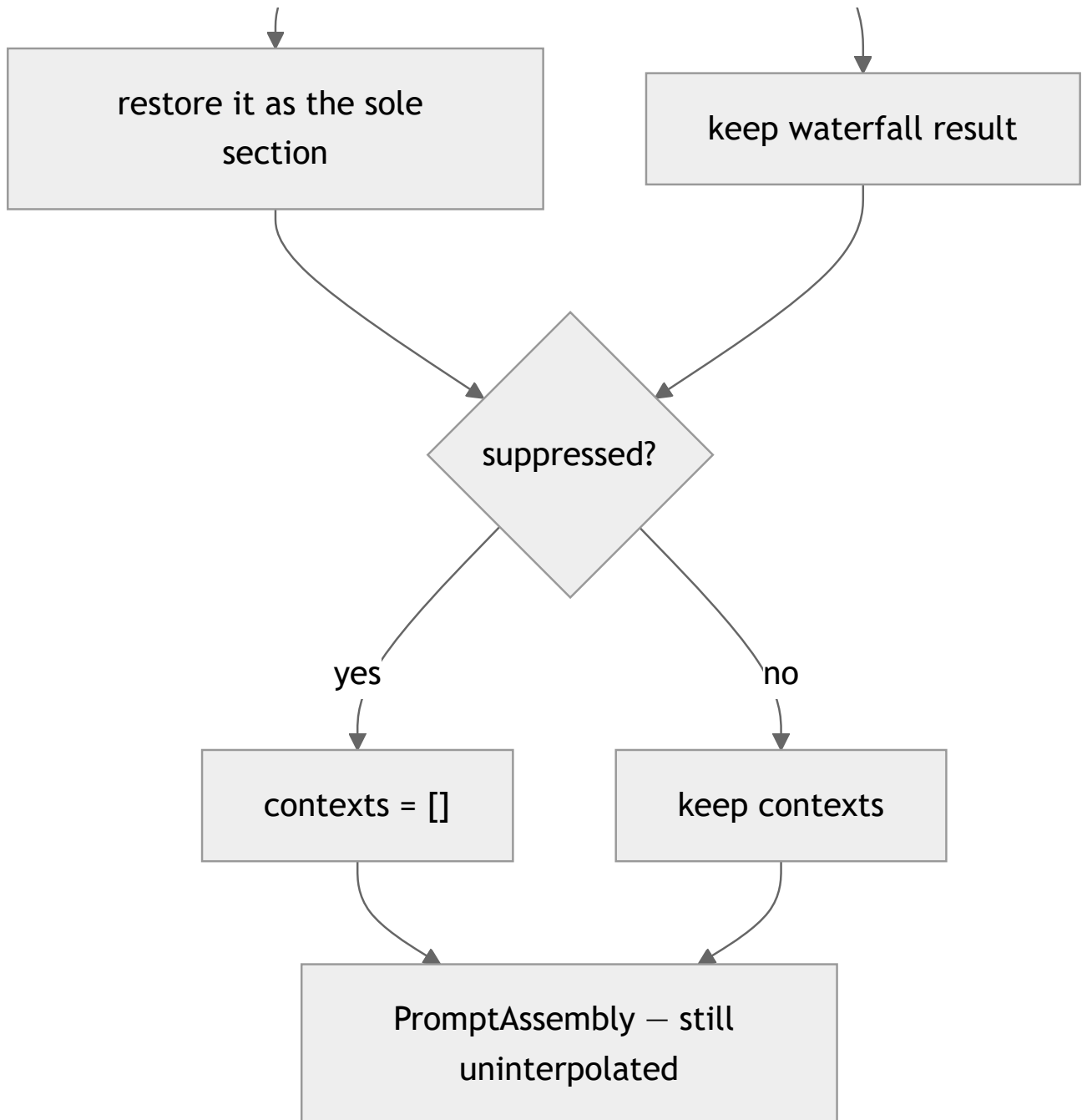
— `index.ts:121-152`, `:157-161`

Contributors do not pick numbers freely; they look theirs up (`getSectionOrder`, `:448-459`). A test asserts every declared order is unique and at least ten apart (`tests/system-prompt.spec.ts:33-49`) — the gap leaves room to insert something between two existing sections without renumbering.

Ties break by code-unit name comparison (`compareNames`, `:222-224`), which is locale-independent — so the same contributions produce the same prompt on a machine in Istanbul as in Seattle. That is a real bug class avoided: locale-aware comparison would reorder sections and silently break prefix caching.







4. Assembly, step by step

`SystemPrompt.assemble(context)` — `index.ts:536-611`.

① **Resolve the scope chain.** `chainLayers(scope)` returns overlays **farthest ancestor first, exact scope last** (`packages/core/scope/src/store.ts:192-199`).

② **Compute suppression.** Whether any layer suppresses runtime context (`:539-540`).

③ **Variables.** Global providers evaluate first, then each scope layer's providers overwrite by name (`:542-551`) — "farthest first, so the nearest scope wins a name."

④ **Sections and contexts.** `merge(scope, layer => layer.sections)` builds a map the same way (:553–554 , `store.ts:208–217`). This is **literal replacement by name**, not textual merging: a scoped `deployment:persona` *replaces* the global one for that scope (`tests/scoped.spec.ts:33–44` asserts exactly this).

⑤ **Tools.** Every global provider plus every scope-chain provider is called and their schemas concatenated (:556–572). Then `orderTools` (:205–219) applies a configured order with a `TOOL_ORDER_REST` marker:

```
with toolOrder: ['todo_write', TOOL_ORDER_REST, 'bash'] and registered tools
['bash', 'echo_b', 'todo_write', 'echo_a'], the result is ['todo_write', 'echo_a', 'echo_b', 'bash']
— tests/tool-order.spec.ts:45–49
```

Named tools go to exact positions; everything else is inserted lexicographically at the marker.

⑥ **Sort sections** by `order` , ties by name (:227–229).

⑦ **The complete rule.** At most one section may set `complete: true` , meaning "treat this as the entire system prompt"; more than one throws (:574–577). The waterfall still runs afterwards so tools, contexts, and variables resolve — then the original complete text is forcibly restored as the sole section (:605–609).

This is not a hypothetical. The `minimal` agent preset uses it:

```
- id: persona
  name: '@deepseek-ai/dsh-persona'
  config:
    text: You are a helpful software engineer assistant.
    complete: true
    includeRuntimeContext: false
```

— `packages/preset/agent-presets/presets/minimal/agent.cordis.yml:9–14`

with the preset's own header explaining the effect: "The persona is the complete system prompt, so global identity, Web orientation, tool guidance, and later assembly listeners cannot add prompt text."

The plugin passes it through conditionally (`packages/preset/persona/src/index.ts:61`):

```
ctx.effect(() => ctx.systemPrompt.section({
  name: PERSONA_SECTION,
  order: ctx.systemPrompt.getSectionOrder('DEPLOYMENT_PERSONA'),
  text: config.text,
  ...(config.complete ? { complete: true } : {}),
}), 'persona.section()')
if (!(config.includeRuntimeContext ?? true)) ctx.systemPrompt.suppressRuntimeContext()
```

So `complete: true` turns an agent's prompt into exactly one fixed string — every tool paragraph, the harness identity line, and anything a `system-prompt/assemble` listener adds are all discarded. The `minimal` preset pairs it with `includeRuntimeContext: false` , which is **the producer of the suppression** in step ② and

step ⑨ above.

Both are per-preset. The shipped default is `standard`, which sets neither — but `minimal` is a selectable preset, not dead code.

⑧ **The `system-prompt/assemble waterfall` (:601–604)** — one expert extension point for rewriting a whole assembly.

⑨ **Suppression wins last.** If runtime context was suppressed, `contexts` is forced to `[]` regardless of what the waterfall did (:605–610).

An installed invariant validates the **post-waterfall** result — non-empty and non-duplicate section and context names, string text, non-empty tool names, valid variable names (`src/invariant.ts:16–52`). Like [Chapter 11](#)'s, it is prepended and global; like it, it only runs where the registry is mounted.

5. Rendering

```
export function renderPrompt(assembly: PromptAssembly): string {
  return assembly.sections
    .map(section => interpolate(section, assembly.variables, 'section'))
    .filter(text => text.length > 0)
    .join('\n\n')
}
```

— `index.ts:263–268`

Interpolate, drop anything that renders empty, join with a blank line. A section whose text function returns `''` for this agent simply disappears — which is how conditional guidance works without a conditional mechanism.

6. A real assembly

`snapshots/session/text-turn/system-prompt.expected.md` is a real rendered prompt. Its first lines:

```
You are an AI agent powered by deepseek Harness.

You are a coding assistant powered by the deepseek-v4-flash model. Your working
directory is {{cwd}}. Your bash tool runs under a file sandbox - ...

Verify your work by running the code or tests. Keep answers brief and factual.

Check the [exit code: N] marker on every bash result; investigate failures before moving on.

Use the read tool - not shell commands like cat - to inspect text files. ...
```

Reading it against the mechanism:

- **Line 1** is `harness:identity`, order `-1000`. First, as designed.

- **Line 3** is the persona, with `{{model}}` already substituted to `deepseek-v4-flash`.
- **Everything after** is one paragraph per tool, in `SECTION_ORDERS` order — `bash`, `read`, `write`, `edit`, `glob`, `grep`, `jobs`, `web_search`, `goal`, `workflow`, `ralph`, `subagent`. Each is contributed by the plugin that registers that tool.

Two cautions about this fixture:

`{{cwd}}` **is not an uninterpolated variable**. It *was* interpolated, to a real absolute path; the snapshot normalizer then replaced that path with the literal `{{cwd}}` so the fixture is machine-independent. `renderPrompt` would have thrown on a genuinely unresolved variable (Ch 20).

This is the snapshot's composition, not the web profile's. Each snapshot directory carries its own `cordis.yml`, and this persona text differs from the web-app bundle's. Read it as a real example of *the mechanism*, not as the exact prompt a web session receives.

7. Control decisions

Decision	Condition	Location
Nearest scope wins	a name exists in several layers	<code>store.ts:208-217</code>
All providers contribute	tools, always	<code>:556-572</code>
Throw	more than one <code>complete</code> section	<code>:574-577</code>
Force <code>contexts = []</code>	suppression active in any layer	<code>:605-610</code>
Restore complete text	a complete section survived the waterfall	<code>:605-609</code>
Drop a section	it rendered empty	<code>:263-268</code>
Throw	a tool provider returns the reserved rest marker as a name	<code>:205-219</code>

8. Configuration knobs

Setting	Default	Effect
<code>persona</code>	<code>''</code> in the base bundle; a real string in web-app and in each preset	The <code>deployment:persona</code> section
<code>toolOrder</code>	unset → lexicographic	Exact positions with a <code>TOOL_ORDER_REST</code> marker

In the web profile the persona is set twice — once at the host layer (`packages/bundle/web-app/cordis.patch.yml:16-19`) and once inside the standard preset (`presets/standard/agent.cordis.yml:24-28`). The preset's is scoped to its agents, so by §4 ④ it shadows the host one for every session on that preset.

9. Interactions

- **Ch 20** — renders what this produces.
- **Ch 21** — consumes `contexts`, not `sections`.
- **Ch 10** — `renderPrompt(assembly)` becomes `system`; `assembly.tools` becomes `tools`; both enter the request header.
- **Ch 15** — the tools registry registers a provider here, which is how `assembly.tools` is populated.
- **Ch 34** — scope layering is what makes per-preset personas work.

10. Build it yourself

Minimal version:

```
function assemble(sections: PromptSection[]): string {
  return sections.sort((a, b) => a.order - b.order).map(s => s.text).join('\n\n')
}
```

What the real one adds:

Addition	Why it exists
Scope-chain shadowing	Two agents in one process need different prompts
Centrally-assigned orders, ≥ 10 apart	Contributors cannot negotiate; gaps allow later insertion
Code-unit tie-breaking	Locale-aware sorting would reorder prompts per machine and break caching
Sections vs contexts	Volatile content must not sit in the cache-sensitive prefix
Accumulating tool providers	Every tool package contributes; none should shadow another
<code>TOOL_ORDER_REST</code>	Pin a few tools, order the rest deterministically
Empty-render dropping	Conditional guidance without a conditional mechanism
Post-waterfall validation	One expert hook can rewrite everything; the result still has to be well-formed

Key takeaways

- Sections become the system prompt; contexts become a message. The split is about volatility and prefix-cache stability.
- Sections, contexts, and variables shadow by name with the nearest scope winning; tool providers all contribute.
- Orders are centrally assigned, spaced ≥ 10 apart, and tie-broken by code-unit comparison so the result is machine-independent.
- A section that renders empty disappears — the conditional mechanism.

- The assembly is uninterpolated; variables are substituted at render time.

Exercises

1. A preset and the host both register `deployment:persona`. Which wins, and what would happen if the merge concatenated instead of replacing?
2. Tool providers accumulate rather than shadow. Construct the failure that happens if two providers return a tool with the same name, and say where it would be caught.
3. A plugin wants to tell the model the current git branch. Section or context? Give the reasoning in terms of prefix caching, then check your answer against [Chapter 21](#).

Next: [Chapter 20 · Variable interpolation](#)

Chapter 20 • Variable interpolation

What you'll learn: the hand-rolled scanner that substitutes `{{name}}` in prompt text, and the three distinct ways it can refuse.

Prerequisites: [Chapter 19](#).

1. The problem

A prompt fragment needs values it cannot know: which model is answering, what the working directory is. The obvious implementation is `text.replace(/\{\{(\w+)\}\}/g, (_, n) => vars[n])`, and it has three defects that matter here.

It silently accepts unknown names. `vars[n]` is `undefined`, which stringifies to `"undefined"` and ships to the model as prompt text. A typo becomes a quiet degradation rather than a loud failure.

It rescans substituted values. A value containing `{{` — a code sample, a file path, user-supplied text — gets interpolated again. That is a template-injection bug: content becomes instructions.

It cannot distinguish "unregistered" from "registered but has no value this time". Those need different diagnostics: the first is a typo, the second is a provider legitimately returning nothing.

2. Mental model

A single forward pass. Walk the text, find each `{{...}}` group, replace it, and **continue after the replacement** — never re-examining what was just inserted. Substituted values are data, permanently.

New term — prompt variable. A named provider `(context) => string | undefined`, registered on the prompt service and evaluated once per assembly.

```
systemPrompt.variable('model', context => context.agent?.options.model)
```

Names must match `/^[a-z][a-z0-9_]*$/` (`packages/core/system-prompt/src/index.ts:175`), checked at registration (`:516-518`).

3. The engine's own variables

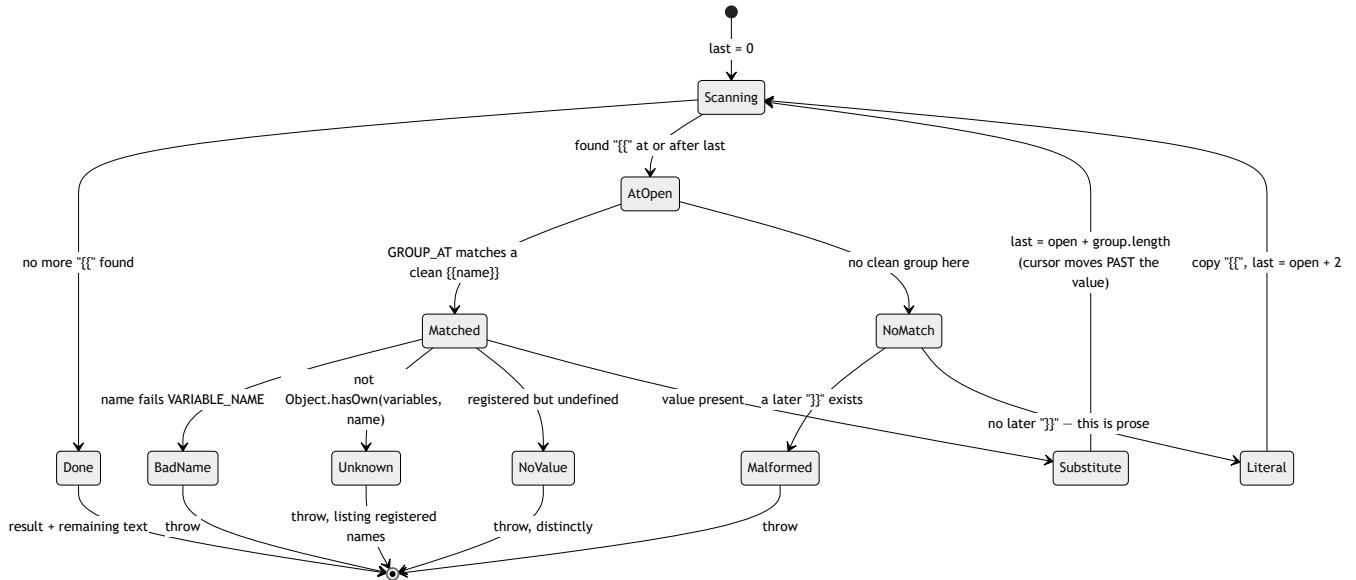
The agent loop registers exactly three, in its constructor:

```
ctx.systemPrompt.variable('provider', context => context.agent?.options.provider)
ctx.systemPrompt.variable('model', context => context.agent?.options.model)
ctx.systemPrompt.variable('cwd', context => context.agent?.session.header.cwd)
```

— `packages/core/agent-loop/src/index.ts:414-416`

All three read from the agent in the assembly context, and all three can return `undefined` — for a bare `assemble()` with no agent, as tests and diagnostics do. That is legal: the name stays registered, and only a section that *actually references* it fails.

These are the variables the personas use. From [Chapter 19](#)'s example: "You are a coding assistant powered by the `{{model}}` model. Your working directory is `{{cwd}}`."



The transition out of `Substitute` is the security property: `last` moves past the inserted value, so a `{{` inside it is never scanned.

4. The scanner

```
function interpolate(input, variables, kind: 'section' | 'context'): string {
  const text = input.text
  let result = ''
  let last = 0
  for (let open = text.indexOf('{{'); open >= 0; open = text.indexOf('{{', last)) {
    const group = GROUP_AT.exec(text.slice(open))    // /^\\{\\{([\\^\\{\\}*\\)\\}\\}\\}/
    if (group === null) {
      if (text.indexOf('{{', open + 2) >= 0) throw ... // malformed
      result += text.slice(last, open + 2)           // lone "{" is literal prose
      last = open + 2
      continue
    }
    const name = group[0].slice(2, -2)
    if (!VARIABLE_NAME.test(name)) throw ...
    if (!Object.hasOwn(variables, name)) throw ...
    const value = variables[name]
    if (value === undefined) throw ...
    result += text.slice(last, open) + value
    last = open + group[0].length
  }
  return result + text.slice(last)
}
```

— index.ts:309–346

The loop's third clause is the whole security property: `text.indexOf('{{', last)` searches from `last`, which was advanced **past** the inserted value. A `{{` inside a substituted value is never seen.

`GROUP_AT` is anchored (`^`) and its character class excludes braces (`[^\\{\\}*\\)`), so it matches only a well-formed group starting exactly at the found position.

Three refusals and one non-refusal

Input	Result	Why
<code>{{model}}</code> , registered with a value	substituted	—
<code>{{ model }}</code> or <code>{{mo del}}</code>	throws — malformed reference	<code>GROUP_AT</code> matches but the name fails <code>VARIABLE_NAME</code>
<code>{{a{b}}</code>	throws — malformed	inner brace defeats <code>GROUP_AT</code> , but a later <code>}}</code> exists
<code>{{typo}}</code>	throws , listing every registered name	<code>Object.hasOwn</code> fails (:335–336)
<code>{{cwd}}</code> registered but provider returned <code>undefined</code>	throws , distinctly	value check (:339–341)
use <code>{{</code> as an opener (no later <code>}}</code>)	literal prose	:320–326

That last row is the one that surprises people. A lone `{{` with no closing `}}` anywhere later is *not* an error — it is text. Prompt fragments discuss templating, regexes, and code; a brace pair that was never meant as a reference should not break the session. But a `{{` that *does* have a later `}}` is almost certainly a botched reference, so that throws.

`Object.hasOwn` rather than `name in variables` matters too: it never falls through to `Object.prototype`, so `{{constructor}}` is an unknown name rather than a function stringified into your prompt.

5. When errors surface

`assemble()` does **not** interpolate. Errors appear only when `renderPrompt`, `renderContextSections`, or `renderContextSnapshot` is called on the assembly — in practice, inside the loop's `preStep` and `step` (Ch 9).

So a malformed template registered by a plugin fails at the **first step of the first agent that uses that scope**, not at mount time. That is later than ideal for diagnosis, though the error text is precise: it names the kind (`section` or `context`), the name, and for unknown names the full registered list.

6. Control decisions

Decision	Condition	Location
Reject registration	name fails <code>VARIABLE_NAME</code>	:516–518
Treat as literal	<code>{{</code> with no later <code>}}</code>	:320–326
Throw — malformed	<code>{{</code> with a later <code>}}</code> but no clean group	same
Throw — bad name	group content fails <code>VARIABLE_NAME</code>	:333–334
Throw — unknown	<code>!Object.hasOwn(variables, name)</code>	:335–336
Throw — no value	registered, but <code>undefined</code> this assembly	:339–341
Advance past the value	always	:341–342

7. Edge cases

Every variable is evaluated once per assembly, even ones nothing references (:542–551). A provider doing expensive work costs that on every step of every agent in scope.

Nearest scope wins a name (Ch 19 ③), so a preset can redefine `model` for its agents without touching the global provider.

Contexts and sections use the same scanner, differing only in the `kind` label used in error messages — so a runtime-context section can interpolate too.

8. Configuration knobs

None.

9. Interactions

- **Ch 19** — produces `variables` and the uninterpolated text.
- **Ch 21** — `renderContextSections` runs the same scanner.
- **Ch 10** — the rendered prompt enters the request header, so a change here changes the header.

10. Build it yourself

Minimal version:

```
text.replace(/\{\{([a-z][a-z0-9_]*)\}\}/g, (_, name) => {
  if (!Object.hasOwn(vars, name)) throw new Error(`unknown prompt variable "${name}"`)
  const value = vars[name]
  if (value === undefined) throw new Error(`prompt variable "${name}" has no value`)
  return value
})
```

Close, and it fixes two of the three defects in §1. What the real one adds:

Addition	Why it exists
Manual scan instead of <code>replace</code>	<code>replace</code> with a global regex is fine, but a manual <code>last</code> cursor makes non-rescanning explicit and auditable
Lone <code>{{</code> tolerated	Prompt text legitimately discusses braces
Malformed detected by "a later <code>}}</code> exists"	Distinguishes prose from a botched reference
Unknown vs valueless as separate errors	A typo and an absent provider need different fixes
Error lists registered names	The fix for a typo is usually visible in the list

Key takeaways

- One forward pass; the cursor advances past each substituted value, so inserted content is never re-scanned.
- A lone `{{` with no later `}}` is literal text; one with a later `}}` is a hard error.
- Unknown name and registered-but-valueless are separate, distinctly-reported failures.
- `Object.hasOwn` keeps prototype names from resolving.
- Nothing interpolates at assembly time — errors surface at the first render, inside the loop.

- The engine registers exactly three variables: `provider` , `model` , `cwd` .

Exercises

1. A plugin's section text is `Match files with {{glob}} patterns` and no `glob` variable exists. What happens, and when? Now change the text to `Match files with {{ glob patterns` and answer again.
2. A variable `provider` returns text containing `{{cwd}}` . Show why the result is not re-interpolated, citing the specific expression.
3. Why is a registered-but- `undefined` variable an error only when referenced, rather than at assembly time? Give the case this supports.

Next: [Chapter 21 · Runtime context injection](#)

Chapter 21 • Runtime context injection

What you'll learn: how volatile facts — the sandbox mode, the approval policy — reach the model without destroying prefix caching, and the small state machine that keeps them from being repeated on every step.

Prerequisites: [Chapter 19](#), [Chapter 6](#).

1. The problem

Some things the model needs to know change during a session. The file sandbox may widen after an approval. A subagent's delegation scope is fixed differently from its parent's.

Putting them in the system prompt is the obvious move and the wrong one. The system prompt is the **cache prefix**: providers cache the tokenized prefix of a request, and a prefix that changes invalidates that cache for the entire remaining conversation. A sandbox mode that flips once would make every subsequent request cost full price.

So volatile facts have to arrive as a *message*. But then a second problem appears: a message repeated on every step accumulates. A hundred-step turn would carry a hundred near-identical snapshots, each one crowding out real conversation.

And a third: when a snapshot is superseded — either by a newer one or because compaction shadowed it — the model must not act on the stale one.

2. Mental model

Inject a snapshot message **only when its text differs from the last one that is still visible**.

That requires tracking one thing: the most recent snapshot message this mechanism wrote, and whether it is still on the surface. Three states:

```
private retained: { seq: number; text: string | undefined } | null | undefined
```

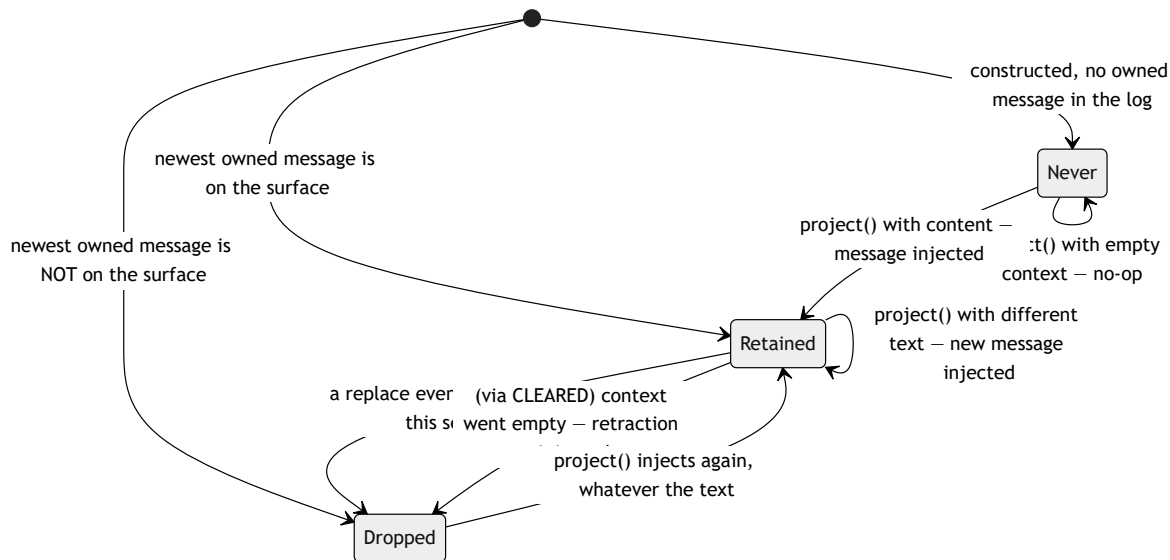
— `packages/core/agent-loop/src/runtime-context.ts:27`

State	Meaning
<code>undefined</code>	no snapshot has ever existed in this session
<code>null</code>	one existed but is no longer retained — compacted away
<code>{ seq, text }</code>	this exact durable event is the currently-visible snapshot

New term — snapshot message. A synthetic user/message carrying the joined runtime-context sections, tagged with `source: { kind: 'plugin', plugin: '@deepseek-ai/dsh-system-prompt', form: 'snapshot', sections }`.

That tag is how the mechanism recognizes its own past messages on replay — it owns nothing but what it wrote.

3. Lifecycle



4. Reconstructing state on construction

```
// walk session.events BACKWARDS, looking for the newest owned user/message
// if it is on the current surface → retained = { seq, text }, stop
// if it is not → retained ??= null
```

— runtime-context.ts:34-45

Backwards, because only the *newest* matters. `isOwned ( :15-17 )` checks the message's `source.kind === 'plugin'` and `source.plugin === SOURCE`.

So a resumed agent knows whether the model has already been told the current policy, without storing anything outside the log. Like the turn number (Ch 8) and the inbox (Ch 12), this is recomputed, never persisted separately.

A live listener keeps it current (:46–55): a new owned message updates `retained` ; a **replacement surface event whose `sourceEventSeqs` includes the retained seq** clears it to `null` . That is the compaction hook — [Chapter 6](#)'s coverage rule guarantees a replacement cites every node it shadows, so checking the citation list is a complete test for "was my snapshot compacted away."

This is a good example of one mechanism's invariant being load-bearing for another. Without mandatory coverage, this check would have to scan ranges and could miss.

5. The decision

```
project(current: string, sections: readonly ContextSnapshotSection[]): UserMessage | undefined {
  if (this.retained === undefined && current.length === 0) return
  const snapshot = current.length === 0 ? CLEARED : current
  if (this.retained?.text === snapshot) return
  return createUserMessage({
    content: [{ type: 'text', text: snapshot }],
    source: sections.length === 0
      ? { kind: 'plugin', plugin: SOURCE }
      : { kind: 'plugin', plugin: SOURCE, form: 'snapshot', sections },
  })
}
```

— `runtime-context.ts:64–75`

No-op 1: nothing has ever been injected *and* there is nothing to say. Most sessions never get a snapshot message at all, because none of the three context producers apply.

The `CLEARED` sentinel: when context goes empty but a snapshot *is* retained, the engine cannot simply say nothing — the model is still holding the old snapshot. So it injects an explicit retraction:

```
Current runtime context: none. Earlier runtime-context snapshots no longer apply. — runtime-
context.ts:13
```

No-op 2: the retained text already equals the computed snapshot. This is the dedup that solves §1's second problem — and it covers the `CLEARED` case too, so a retraction is not repeated either.

Note the source shape differs for a retraction: the bare two-field form, because "the cleared marker has no contributions left to attribute" (:70–71).

6. Where it enters the step

```
const assembly = await this.loopCtx.systemPrompt.assemble(assembleContextFor(this, signal))
signal.throwIfAborted()
const sections = renderContextSections(assembly)
const context = this.runtimeContext.project(joinContextSections(sections), sections)
const decision = await this.dispatch.waterfall(
  'agent/pre-step', { messages: claimed, ...position, signal },
  (): Promise<PreStepDecision> => Promise.resolve<PreStepDecision>({
    kind: 'enter',
    messages: context === undefined ? claimed : [...claimed, context],
  }),
)
```

— `packages/core/agent-loop/src/agent.ts:239-249`

The snapshot is appended to the **innermost default** of the `agent/pre-step` waterfall — the `next()` that runs when every listener has delegated. Consequences:

- a listener calling `next()` inherits the snapshot;
- a listener returning its own decision without delegating **omits it**, which is how a `reject` or a hand-built message list suppresses runtime context for that step ([Ch 22](#)).

`joinContextSections` supplies the fixed preamble:

```
return `Current runtime context. This snapshot supersedes earlier runtime-context
snapshots.\n\n${body}`
```

— `packages/core/system-prompt/src/index.ts:287-291`

"Supersedes earlier snapshots" is doing real work: because old snapshots stay in history, the model needs to know which one is authoritative.

Then `turn()` appends every message in the decision as an ordinary `user/message` with `surfaceOp: 'append'` (`agent.ts:291-293`). **The snapshot becomes indistinguishable from any other logged message** apart from its `source.plugin` tag — no side channel, fully replayable.

7. A real snapshot

From `snapshots/session/text-turn/session.jsonl:10`:

```
{"content":[{"type":"text","text":"Current runtime context. This snapshot supersedes earlier
runtime-context snapshots.\n\nCurrent DSH file policy: danger-full-access. ...\n\nApproval prompts
are disabled in this session: ..."}],
"source":{"kind":"plugin","plugin":"@deepseek-ai/dsh-system-prompt","form":"snapshot",
"sections":[{"name":"sandbox:policy","text":"..."},
{"name":"approval:policy","text":"..."}]},
"role":"user","id":{"message:2}}"
```

The `sections` array is preserved on the message. So a UI can attribute each paragraph to the plugin that contributed it, and a reader can tell *why* the snapshot changed between two versions — not just that it did.

8. Who actually contributes contexts

Exactly three producers exist in the entire repository, matching the three declared `CONTEXT_ORDERS` :

Name	Order	Source
sandbox:policy	110	packages/sandbox/sandbox-policy/src/index.ts:140-151
approval:policy	115	packages/interaction/user-approval/src/index.ts:169-181
subagent:delegation	120	packages/subagent/subagent/src/child-agent.ts:205-209

The first two return `''` for an assembly with no agent, so a bare `assemble()` in a test produces no snapshot.

 **A naming trap worth flagging.** None of the packages under `packages/context/**` — `agent-instructions`, `time-context`, `tmux-context`, `session-reference`, `file-reference-local` — use `systemPrompt.context()` at all. Despite the directory name, they inject through the `agent/pre-step waterfall` directly, splicing their own `UserMessage`s into `decision.messages` (Ch 22). One of them, `file-reference-local`, registers an ordinary prompt **section** instead (`packages/context/file-reference-local/src/index.ts:66-76`).

So "context" in `packages/context/` names a product category — things that inject situational information — not this mechanism. Reading that directory expecting `PromptContext` producers will mislead you.

9. Control decisions

Decision	Condition	Location
Return <code>undefined</code>	never injected and nothing to say	:65
Use <code>CLEARED</code>	context is empty but a snapshot is retained	:66
Return <code>undefined</code>	retained text equals the computed snapshot	:67
Bare source shape	<code>sections</code> is empty	:68-71
Clear to <code>null</code>	a replacement cites the retained seq	:46-55
Omit from the step	a pre-step listener did not delegate	agent.ts:243-249

10. Edge cases

A snapshot can be injected mid-turn. `preStep` runs for *every* step, not only the first. If a tool widens the sandbox at step 3, step 4 carries a fresh snapshot — which is exactly the point.

Old snapshots stay in history. Nothing retracts them from the log; the preamble tells the model which is authoritative. Compaction may eventually shadow them ([Ch 27](#)), which clears `retained` and causes the next step to re-inject.

A per-session projection, not a global one. The test confirms a message committed to a *different* session never affects this projection's view (`packages/core/agent-loop/tests/runtime-context.spec.ts:16-45`).

11. Configuration knobs

None directly. What appears is determined by which context producers are mounted and what they return — in practice, by the sandbox mode and approval policy ([Ch 18](#)).

12. Build it yourself

Minimal version:

```
let lastText: string | undefined
function project(current: string): UserMessage | undefined {
  if (current === lastText) return
  lastText = current
  return createUserMessage({ content: [{ type: 'text', text: current }], source: {...} })
}
```

What the real one adds:

Addition	Why it exists
Reconstruct from the log on construction	A resumed agent must not re-announce what the model already knows
Three states, not two	"never said" and "said but compacted away" need different handling
Watch replacement events	Compaction can silently remove the snapshot the model is relying on
The <code>CLEARED</code> sentinel	Going quiet leaves the model holding a stale snapshot
<code>sections</code> on the message source	Attribution for UIs, and a diff showing <i>why</i> it changed
Injection as the waterfall's innermost default	Listeners inherit it by delegating and suppress it by not

Key takeaways

- Volatile facts arrive as a message, not in the system prompt, to protect the provider's prefix cache.
- A snapshot is injected only when its text differs from the last still-visible one.
- Three states distinguish never-said, currently-said, and compacted-away.
- Going empty produces an explicit retraction rather than silence.
- The snapshot enters as the innermost default of `agent/pre-step`, so listeners inherit it by delegating.
- Once appended it is an ordinary `user/message` — no side channel.
- `packages/context/**` does *not* use this mechanism, despite the name.

Exercises

1. A session runs 50 steps with an unchanged sandbox mode. How many snapshot messages are in the log, and which line is responsible?
2. Compaction shadows the retained snapshot at step 20. Trace what the model sees at step 21, naming the two events involved.
3. The mechanism detects compaction by checking whether a replacement's `sourceEventSeqs` includes its seq. Which rule from [Chapter 6](#) makes that check complete rather than best-effort?

Next: [Chapter 22](#) · [Extension points](#)

Chapter 22 · Extension points

What you'll learn: the three dispatch semantics every plugin in this system uses, what `next()` actually is, and a census of who listens to what — including one extension point with no live listeners at all.

Prerequisites: [Chapter 9](#). Most preceding chapters have referred here.

1. The problem

The engine is 1,756 lines and does not know that compaction, retry, subagents, skills, plan mode, or approval exist. Every one of those reaches in through a named point in the loop.

That demands more than an event bus. Different interventions need different powers:

- *"tell me when the status changed"* — a notification, and a listener failing must not break the agent;
- *"let me decide whether this turn really ends"* — participation, awaited, possibly by several listeners;
- *"let me replace the config for this request"* — transformation, where one listener's output is the next one's input, and any of them may decline to delegate.

Those are three different shapes, and conflating them is how event systems become unreliable.

2. Mental model

Three semantics, one wrapper.

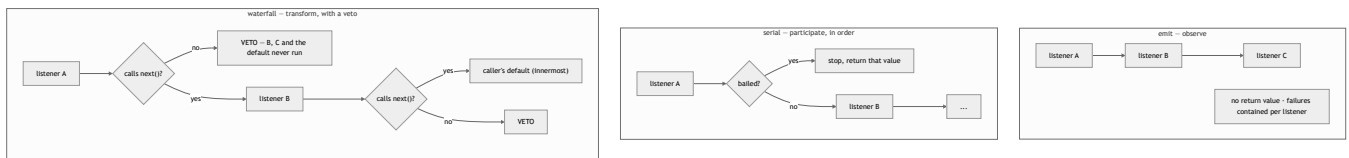
Mode	Awaited	Returns	A listener can
<code>emit</code>	no	nothing	observe
<code>serial</code>	yes, in order	first "bailed" value	act, and stop later listeners
<code>waterfall</code>	yes, nested	the outermost listener's value	transform, delegate, or veto

New term — `AgentEventDispatch`. A fused, agent-scoped wrapper built once per agent in its constructor (`packages/core/agent-loop/src/agent.ts:94`) so hot-path dispatches allocate nothing.

It does two things on every call (`packages/core/agent/src/dispatch.ts:107-149`):

1. **Injects the `agent` into the payload.** The caller passes everything *except* `agent` (`PayloadRest` , `:47`), so a caller literally cannot dispatch an event claiming to be about a different agent.
2. **Dispatches through the agent's scope carrier** (`:94-96`), so delivery is restricted to listeners whose scope is this agent or an ancestor.

The coupling is deliberate: "the agent's scope carrier so the scope key and the payload's `agent` cannot diverge" (`dispatch.ts:2-6`). Identity and routing come from the same source.



3. The three semantics, from the implementation

emit

```
emit(...args: any[]) {
  this.dispatch('emit', args).map(cb => cb(...args))
}
```

— `vendor/cordis/src/events.ts:194-196`

Synchronous, return values ignored, promises not awaited. **Note what is absent: any error isolation.** A listener that throws propagates out through `.map()` and aborts the remaining calls.

So the agent dispatcher does not use it directly. It resolves the callback list itself and wraps each one (`dispatch.ts:120-137`), catching synchronous throws and attaching `.catch()` to returned promises, logging a warning per failure. The rationale is in the doc: "a notification cannot veto lifecycle progress or starve a later observer" (`:56-59`).

That difference is worth carrying: `ctx.emit()` **does not protect later listeners from an earlier one's throw**. Code wanting that guarantee must iterate `dispatch()` itself — which is exactly what `AgentLoop.reportConfiguredStartupFailure` does when reporting a config startup failure (`packages/core/agent-loop/src/index.ts:457`).

serial

```
async serial(...args: any[]) {
  for (const cb of this.dispatch('serial', args)) {
    const result = await cb(...args)
    if (isBailed(result)) return result
  }
}
```

— `events.ts:204-209`

Awaited, in registration order, stopping at the first "bailed" result — anything other than `null`, `false`, or `undefined` (`isBailed`, `:13-15`).

Used for `agent/turn-stopping`, whose listeners return `void`. They participate by **side effect** — calling `agent.steer(...)` to queue a message that the re-test in [Chapter 9](#) ⑨ then sees. The documentation gives the reason: "Data decides, so listener order cannot change the outcome" (`runtime-types.ts:275-278`). Two listeners both wanting continuation both queue, and the result is the same regardless of order.

waterfall

```
waterfall(...args: any[]) {
  const cbs = this.dispatch('waterfall', args)
  const inner = args.pop()           // the caller's default
  const next = () => {
    const cb = cbs.shift() ?? inner
    return cb(...args)
  }
  args.push(next)
  return next()
}
```

— `events.ts:234–243`

Ten lines, and worth reading closely because the whole plugin architecture rests on them.

Listeners are collected in registration order. The caller's **default value factory becomes the innermost link**. `next` is a closure that shifts the next listener off the front — or, when exhausted, calls the default. Each listener receives the payload plus its own `next`.

So the chain is nested continuations, outermost-first. A listener may:

- **call** `next()` and return its result — pure delegation;
- **call** `next()`, **then modify the result** — transformation;
- **return without calling** `next()` — which skips every remaining listener *and the caller's default behavior*.

The last one is the crucial semantic:

"a listener that does not call `next()` vetoes the rest of the chain, including the built-in behavior" —
`events.ts:79–81`

There is **no framework enforcement**. `next` is an ordinary closure; not calling it is simply not calling a function. That is why the repo states it as a rule for humans: "Waterfall listeners **MUST** call `next()` to delegate; returning without it short-circuits the chain."

Two consequences seen earlier in this book:

- [Chapter 21](#)'s snapshot lives in the *innermost default*, so a listener that does not delegate suppresses it;
- [Chapter 11](#)'s check uses `prepend: true` precisely because a short-circuiting listener registered ahead of it would silence it.

4. The census

Every `agent/*` point, with real non-test listeners and whether they are mounted in the shipped web profile.

Event	Mode	Live listeners in web	Also implemented by (not mounted)
<code>agent/created</code>	emit	file-reference-local, goal-round-driver, agent-presets, tool-subagent, schedule*	agent-team
<code>agent/disposed</code>	emit	file-reference-local, goal-round-driver, subagent continuation, tool-subagent	agent-team
<code>agent/status</code>	emit	compaction-basic , session-controller, goal-round-driver	agent-team
<code>agent/session-start</code>	emit	goal, goal-round-driver	hooks-claude-code, hooks-codex, agent-team
<code>agent/inbox/inserted</code>	emit	goal-round-driver	—
<code>agent/inbox/claimed</code>	emit	tool-jobs, goal-round-driver, subagent continuation	—
<code>agent/inbox/discarded</code>	emit	goal-round-driver, subagent continuation	—
<code>agent/pre-step</code>	waterfall	compaction-basic , tool-skill (×2), session-checkpoint-policy, plan-mode, repeat-tool-reminder, goal-round-driver, agent-instructions, time-context, tmux-context, session-reference, subagent driver, tool-cordis	hooks bridges
<code>agent/request</code>	waterfall	model-selection (via session-controller)	webhook
<code>agent/request-error</code>	waterfall	compaction-basic , llm-retry	—
<code>agent/turn-stopping</code>	serial	— none —	hooks-claude-code, hooks-codex
<code>agent/error</code>	emit	session-controller, session-telemetry, goal-round-driver	—

* `schedule` is itself an opt-in overlay ([Ch 2](#)).

Three readings worth taking from this table.

agent/pre-step is the dominant surface. Twelve live listeners. Anything wanting to see or shape a step attaches here — which is why [Chapter 21](#) had to explain that `packages/context/**` uses this rather than the prompt-context mechanism.

agent/request is sparse but live. One listener: `installModelSelection` (`packages/core/agent/src/model-selection.ts:54–70`), which overrides provider, model, and reasoning effort from a per-agent selection ref, wired by the session controller. That is the `/model` command's implementation ([Ch 10](#)).

`agent/turn-stopping` **has zero live listeners**. Its only implementations are the two hook bridges, and neither is mounted. The point is defined, typed, exercised by tests, and dispatched on every qualifying turn — to nobody. It is a designed-in seam awaiting a consumer, and the book states that rather than describing a feature nothing uses.

5. Control decisions

Decision	Condition	Location
Deliver to a listener	its scope is the agent or an ancestor	<code>scope/src/index.ts:170-185</code>
Contain a failure	always, in agent-scoped <code>emit</code>	<code>dispatch.ts:120-137</code>
Stop a serial chain	a listener returns a bailed value	<code>events.ts:204-209</code>
Continue a waterfall	the listener calls <code>next()</code>	<code>events.ts:234-243</code>
Veto a waterfall	the listener returns without calling <code>next()</code>	same
Use the default	every listener delegated	same

6. Edge cases

`agent/created` **can veto publication**. Documented as: a **synchronous throw** rolls the agent back, while a returned-promise rejection is only logged (`packages/core/agent/src/index.ts:551-567`). That asymmetry is why [Chapter 14](#)'s `publish` re-checks liveness after every announcement.

A claimed message may never be delivered. `agent/inbox/claimed` fires when a message leaves the inbox into an open turn — but if that step is then rejected, the message never becomes a `user/message` (`runtime-types.ts:196-197`). Consumers must treat `claimed` as "consumed", not "delivered" ([Ch 12](#)).

Scope filtering means a subagent's events do not reach a sibling's listeners. Delivery follows the scope parent chain, so a listener registered on one agent's scope sees that agent and its descendants only ([Ch 34](#)).

Registration order matters for waterfalls and not for serial. A deliberate asymmetry: transformation chains are inherently ordered, so `prepend` is meaningful; `serial` participation is data-driven precisely so ordering cannot change outcomes.

7. Configuration knobs

None. Which listeners exist is a composition question ([Ch 34](#)).

8. Interactions

Every mechanism chapter in this book attaches here. The load-bearing ones:

- **Ch 27** — `agent/pre-step` (pressure) and `agent/request-error` (overflow recovery), plus `agent/status` to reset counters.
- **Ch 25** — `agent/request-error` is the only way a request is ever retried.
- **Ch 21** — lives in `agent/pre-step`'s innermost default.
- **Ch 11** — `prepend: true` on `llm/stream` for exactly these semantics.

9. Build it yourself

Minimal waterfall:

```
async function waterfall<T>(listeners: Listener<T>[], payload: P, fallback: () => Promise<T>):
Promise<T> {
  let i = 0
  const next = (): Promise<T> => (i < listeners.length ? listeners[i++]!(payload, next) :
fallback())
  return next()
}
```

Six lines, and it captures the semantics exactly. What the real system adds:

Addition	Why it exists
Three distinct modes	Observation, participation, and transformation need different guarantees
Scope-filtered delivery	One process runs many agents; listeners must not cross-talk
Agent fused into the payload	A caller must not be able to dispatch about another agent
Per-listener failure containment in <code>emit</code>	Raw <code>ctx.emit</code> lets one throw abort the rest
<code>prepend</code>	Some checks must run before anything that could short-circuit
Dispatcher built once per agent	The hot path dispatches several times per step

Key takeaways

- Three semantics: `emit` observes, `serial` participates in order, `waterfall` transforms with a veto.
- `next()` is an ordinary closure; not calling it skips every later listener *and* the built-in default, with no framework enforcement.
- The agent dispatcher fuses identity and scope routing so they cannot diverge, and contains `emit` failures that raw Cordis would not.
- `agent/pre-step` carries twelve live listeners and is the system's main integration surface.
- `agent/turn-stopping` has **zero** live listeners in the shipped profile.
- `agent/created` can veto publication, but only by throwing synchronously.

Exercises

1. Write a `agent/pre-step` listener that adds a message to every step. Now write one that *replaces* the step's messages entirely, and say what it accidentally suppresses ([Ch 21](#)).
2. `agent/turn-stopping` is `serial` with `void` listeners, so it cannot bail. Why choose `serial` over `emit`? What would change if it were `emit`?
3. The invariant uses `prepend: true`. Name one other listener in this book that would be safer prepended, and one where prepending would be actively wrong.

Next: [Chapter 23 · Adapters and `prepareCall`](#)

Chapter 23 · Adapters and `prepareCall`

What you'll learn: how a provider name becomes a live connection, what a "prepared call" pins down, and why the engine resolves a request in two stages instead of one.

Prerequisites: [Chapter 10](#).

1. The problem

`{ provider: 'deepseek-official', model: 'deepseek-v4-flash' }` is not enough to make a request. Something must find the code that speaks to that provider, discover what that exact model supports, fill in defaults the caller omitted, and produce a dispatchable call.

Doing all of that at dispatch time creates a subtle hazard. Between logging a request header and actually sending the request, configuration can change — a user edits provider settings, a hot reload swaps an adapter. If resolution happens twice, or late, the request that gets logged and the request that gets sent can differ. That is precisely the drift [Chapter 11](#) exists to catch.

So resolution happens **once**, up front, and the result is *bound* to the dispatch that follows.

2. Mental model

New term — adapter. An object that knows how to talk to one or more providers. Only one method is required:

```
abstract stream(options: GenerateOptions): AsyncIterable<StreamChunk>
```

— `packages/llm/llm/src/index.ts:274`

Everything else has a default (:193–275): `providerInfo`, `listModels`, `resolveModel`, `providerRetryPolicy`, and a `prepareCall` that simply binds `resolveModel` + `stream` together (:262–267).

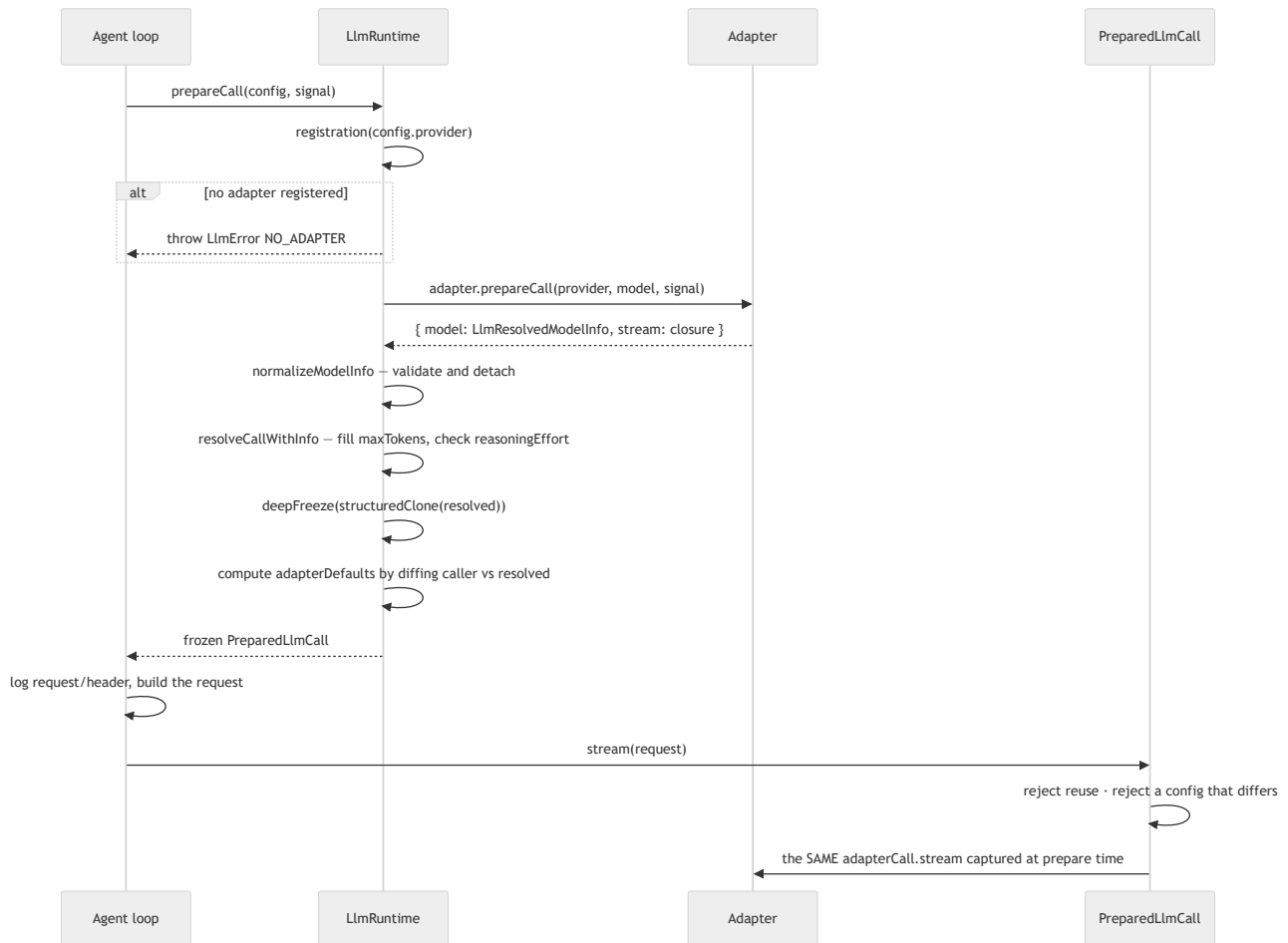
New term — prepared call. A frozen, single-use object pinning everything about one request:

```
export interface PreparedLlmCall {
  readonly config: LlmCallConfig
  readonly retryPolicy: ResolvedRetryPolicy
  readonly context?: LlmModelContext // { contextWindow }
  readonly inputModalities?: readonly ModelModality[]
  readonly adapterDefaults: LlmCallConfigAdapterDefaults
  stream(options: GenerateOptions): AsyncIterable<StreamChunk>
}
```

— :157–177

Two stages, then: **resolve** (`prepareCall`) and **dispatch** (`prepared.stream`). The engine logs its header between them (Ch 10), which is exactly why the split exists.

3. Lifecycle



4. Registration

```
ctx.llm.registerAdapter(providers: string[], adapter: LlmAdapter): AdapterRegistrationHandle
```

— : 380–409

Wrapped in `ctx.effect(...)` (: 387), so an adapter disappears with its plugin. Validation is **all-or-nothing** (`prepareRoutes` , : 416–438): every provider name must be non-empty, a name already held by another registration throws `DUPLICATE_ADAPTER`, and the adapter's own `providerInfo(provider).id` must equal the name it is registering for, else `INVALID_ADAPTER`. There is no partial registration.

One detail with consequences later:

```
// providerRetryPolicy(provider) is captured HERE, at registration time
```


The retry policy is resolved once when the route is registered, not per request.

`PreparedLlmCall.retryPolicy` reads back that captured value. So changing a provider's retry configuration requires **re-registering** — which is why the handle exposes `.replace(providers)` (:281–299) for an atomic same-instance swap.

Selection is a flat string lookup. `provider` picks the registration (:937–941); `model` is just a string handed to that adapter. The harness does **not** validate the model against any catalog — `listModels` is explicitly advisory (`types.ts:284`), and DeepSeek's adapter treats an unrecognized model as text-only rather than rejecting it (`llm-deepseek/src/adapter.ts:398–407`).

5. `prepareCall`, step by step

:890–935 :

1. **Find the registration** (:891), else `LlmError('no adapter registered for provider "...", 'NO_ADAPTER')`.
2. **Ask the adapter to prepare** (:892) — returns model info plus a `stream` closure.
3. **Validate the model info** (`normalizeModelInfo`, :730–820): `provider` and `id` must match, `name` non-empty, `contextWindow` a positive integer, `defaultMaxTokens` a safe integer, `reasoning.efforts` well-formed. Failures are coded `INVALID_MODEL_*`. The result is detached from the adapter's own object.
4. **Resolve against the model** (`resolveCallWithInfo`, :846–880):
 - `maxTokens` — filled from `info.defaultMaxTokens` when omitted;
 - `reasoningEffort` — if the model declares no reasoning capability but one was requested, throw `UNSUPPORTED_REASONING_EFFORT`; otherwise the effective effort is `requested ?? reasoning.defaultEffort`, and if that is not in `reasoning.efforts`, throw.
5. **Freeze** (:895–898): `deepFreeze(structuredClone(resolved))`.
6. **Compute adapterDefaults** (:899–906) by diffing the caller's original config against the resolved one: `{ reasoningEffort?: true, maxTokens?: true }`, set exactly where the caller omitted a field and the adapter supplied it.
7. **Return a frozen object** (:908–934).

Step 6 is the one that matters most upstream. It is the record [Chapter 10](#) uses to strip adapter-supplied fields on the next request so they get re-resolved rather than inherited. Confirmed by test: a bare `{ provider, model }` call yields `adapterDefaults: { maxTokens: true }`, while an explicit `maxTokens` yields `{}` (`tests/service.spec.ts:684–689`).

6. What the prepared call enforces

```
// stream(options):  
//   throws INVALID_PREPARED_CALL if already dispatched (:917-919)  
//   throws INVALID_PREPARED_CALL if options are not field-wise  
//     equal to the config resolved at prepare time (:920-925)  
//   otherwise dispatches through the SAME registration,  
//     model info, and adapter closure captured at prepare (:927-932)
```

Three guarantees in about fifteen lines.

Single use. A prepared call cannot be dispatched twice — so a retry cannot silently reuse a stale resolution. [Chapter 9](#)'s `retry continue` rebuilds the request from scratch, which is required, not merely tidy.

Config equality. The options handed to `stream` must match what was resolved. You cannot prepare with one config and dispatch with another — which is what keeps the logged header honest.

Generation pinning. Dispatch goes through the adapter closure captured at prepare time. A settings change or hot reload between prepare and dispatch cannot mix generations.

Concrete adapters lean on this. DeepSeek overrides `prepareCall` to snapshot its connection facts once:

```
prepareCall(provider, model, signal) {  
  const connection = this.config.options()  
  return { model: this.modelInfoFor(connection, provider, model),  
          stream: options => this.streamWithConnection(options, connection) }  
}
```

— `packages/llm/llm-deepseek/src/adapter.ts:432-438`

Endpoint, credential reference, and model catalog are all fixed for the duration of that one call.

7. Control decisions

Decision	Condition	Location
<code>NO_ADAPTER</code>	provider not registered	:937-941
<code>DUPLICATE_ADAPTER</code>	name held by another registration	:422
<code>INVALID_ADAPTER</code>	<code>providerInfo.id</code> ≠ the registered name	:388, 420, 426
<code>INVALID_MODEL_*</code>	model metadata fails validation	:747-808
<code>UNSUPPORTED_REASONING_EFFORT</code>	effort requested on a non-reasoning model, or not in <code>efforts</code>	:859, 868
<code>INVALID_PREPARED_CALL</code>	reuse, or a config mismatch at dispatch	:918, 921
Set <code>adapterDefaults.X</code>	caller omitted X and the resolved config has it	:899-906

8. Edge cases

Only `NO_ADAPTER` is tolerated upstream. [Chapter 10](#) catches that one code and proceeds with the unresolved config; every other error from this function fails the step. A model with an unsupported reasoning effort is a real error, not something to paper over.

No `preparedCall` means no retry policy. When the fallback path is taken, `preparedCall` is `undefined`, so the `agent/request-error` payload carries `retryPolicy: undefined` — and `llm-retry` delegates immediately when the policy is absent ([Ch 25](#)). The failure is terminal.

`replayState` is stripped across adapter instances. `forAdapter()` (:944–957) removes adapter-private replay state from historical assistant messages when the current target's adapter instance differs from the one that produced them, reverting the source to a plain `{ kind: 'model', provider, model }`. Opaque provider state is only trusted back to the same live adapter.

Two adapters exist; in the web profile, neither registers a route by default. `llm-deepseek` provides `deepseek-official` but is `disabled: true` in web; `llm-pi-ai` is mounted and dormant with an empty `providers` dict. [Chapter 32](#) traces what that means for a fresh install.

9. Configuration knobs

Setting	Default	Where
default route	<code>deepseek-official / deepseek-v4-flash</code>	<code>packages/bundle/base/cordis.patch.yml:73–79</code>
<code>defaultContextWindow</code> (DeepSeek)	<code>1_000_000</code>	<code>llm-deepseek/src/adapter.ts:140</code>
<code>DEFAULT_MAX_TOKENS</code> (DeepSeek)	<code>256_000</code>	<code>llm-deepseek/src/adapter.ts:142</code>
<code>reasoningEffort</code> (DeepSeek)	<code>high</code> , or <code>off</code> only when <code>thinking: 'disabled'</code>	<code>llm-deepseek/src/adapter.ts:187–193</code>
<code>llm-pi-ai providers</code>	<code>{}</code> — dormant	<code>llm-pi-ai/src/config.ts:340–342</code>

10. Interactions

- [Ch 10](#) — the sole caller; consumes `config`, `adapterDefaults`, and `context.contextWindow`.
- [Ch 24](#) — consumes the returned `stream`.
- [Ch 25](#) — reads the registration-captured `retryPolicy`.
- [Ch 27](#) — uses `resolveModelInfo` to get the real context window for its threshold.

11. Build it yourself

Minimal version:

```
const adapters = new Map<string, LlmAdapter>()
function stream(options: GenerateOptions) {
  const adapter = adapters.get(options.provider)
  if (!adapter) throw new LlmError(`no adapter for "${options.provider}"`, 'NO_ADAPTER')
  return adapter.stream(options)
}
```

That works — and is exactly the fallback path the engine keeps for middleware. What the two-stage version adds:

Addition	Why it exists
Resolve before dispatch	The header must be logged from the same resolution that is sent
Frozen, single-use prepared call	A retry must not silently reuse a stale resolution
Config equality at dispatch	Prepare-with-one, dispatch-with-another would make the log lie
Closure captured at prepare	A settings change mid-request must not mix generations
<code>adapterDefaults</code>	Otherwise one model's defaults follow you to the next
Validated model metadata	A bad <code>contextWindow</code> would silently break compaction's threshold
Registration-time retry capture	Policy is a property of the route, not of a request
All-or-nothing route registration	A partially-registered adapter leaves unusable routes

Key takeaways

- Resolution and dispatch are separate stages so the logged header and the sent request come from one resolution.
- A prepared call is frozen, single-use, config-checked, and pinned to the adapter generation that produced it.
- `adapterDefaults` records which fields the adapter supplied — the input to the next request's strip-and-re-resolve.
- Provider selection is a flat string lookup; the model string is not validated against any catalog.
- Retry policy is captured when a route registers, not per request; changing it means re-registering.
- Only `NO_ADAPTER` is tolerated by the caller — and taking that path also means no retry policy.

Exercises

1. A user changes provider settings between `prepareCall` and `stream`. Name the two mechanisms that stop the in-flight request being affected.
2. Retry policy is captured at registration. Give a scenario where that is surprising, and say how `.replace()` resolves it.
3. The harness does not validate model names. Name one benefit and one failure mode, and say where the failure would first surface.

Next: [Chapter 24 · Streaming and block assembly](#)

Chapter 24 · Streaming and block assembly

What you'll learn: how a stream of deltas becomes content blocks, what happens to partial output when a stream is cancelled, and the one rule that silently drops tool calls.

Prerequisites: [Chapter 3](#) for `StreamChunk`, [Chapter 9](#) for where this runs.

1. The problem

A provider sends tokens. The engine needs blocks — a reasoning block, a text block, a tool call with complete JSON arguments. Reassembly is straightforward until you ask what happens when the stream does not finish cleanly.

Three failure shapes, each needing a different answer:

- **The user cancels mid-sentence.** The model said something useful. Discarding it loses work the user watched appear on screen.
- **The model hits its output ceiling.** Its last tool call may be truncated — a half-written JSON argument. Executing it would be acting on a request the model never finished making.
- **The provider errors halfway.** Some blocks completed, then the connection died.

A reassembler that just concatenates deltas gets all three wrong.

2. Mental model

New term — `BlockAssembler`. A stateful accumulator fed one chunk at a time, exposing the assembled result through several views depending on how the stream ended.

```
partials: Map<number, PartialBlock> // keyed by stream block index
order: number[] // first-seen index order
_usage, _finish, _replayState
```

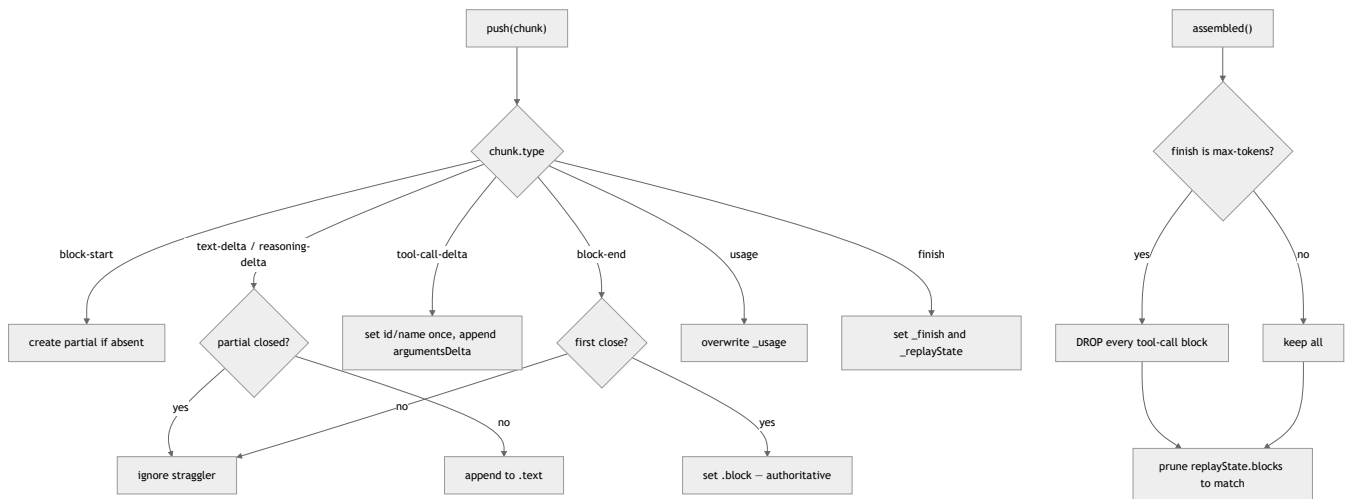
— `packages/llm/llm/src/assembler.ts:39-43`

Each partial holds `{ blockType, text, toolCallId?, toolCallName?, toolCallArguments, block? }` (:16-24). The `block` field is the important one: it is set **only** by a `block-end` chunk, and once set the partial is closed — later deltas for that index are ignored as "stragglers" (:65-66, :71-72).

Three exit views:

View	Used when	Includes
<code>blocks()</code>	normal completion	everything, minus tool calls if <code>max-tokens</code>
<code>interruptedBlocks()</code>	cancellation	text and reasoning only, non-empty after trim
<code>finish</code>	always	the terminal reason, defaulting to <code>stop</code>

3. Lifecycle



4. Accumulation

`push(chunk)` (:49-96) is a switch:

- `block-start` — lazily creates a partial if one does not exist for that index.
- `text-delta` / `reasoning-delta` — appends to `.text`, ignored once `.block` is set.
- `tool-call-delta` — sets `toolCallId` and `toolCallName` once known, appends `argumentsDelta` to `toolCallArguments`. **Arguments stay a raw string** through the whole pipeline (Ch 17 parses them).
- `block-end` — sets `.block` on **first** close only, so a duplicate close chunk is idempotent.
- `usage` — overwrites.
- `finish` — records the reason and any replay state.

It ends with `assertNever(chunk, 'BlockAssembler.push')` (:94) for exhaustiveness against the known union.

Assembling one block

`assemble(partial, index)` (:108-121): if `block-end` provided a block, **return it verbatim** — the provider's own structured close is authoritative over anything reconstructed from deltas. Otherwise synthesize from the accumulated text, with fallbacks for a tool call missing its id or name.

Any other still-open block type **throws**: cannot assemble incomplete block of type "... " (:119). A plugin-added block type that never closed is an error, not something to guess at.

5. The `max-tokens` rule

```
// assembled(): if this.finish.kind === 'max-tokens',  
// every tool-call block is dropped from the emitted set
```

— :134–150

This is the answer to §1's second failure. When the model runs out of output budget, its final tool call is likely truncated — arguments cut off mid-JSON. The comment gives the reasoning: max-token truncation "drops tool calls that cannot be executed safely."

Text and reasoning blocks survive; only tool calls are removed. So the model's partial explanation reaches the user, and nothing is executed on its behalf that it did not finish asking for.

Two consequences worth connecting:

- [Chapter 9](#) returns `{ kind: 'max-tokens' }` from the step, and that ending is **sticky** — so a later clean step cannot mask the truncation.
- The resulting assistant message may have **zero content blocks** if the model produced nothing but a truncated tool call. That message is still logged, to carry the usage numbers — and [Chapter 7](#)'s derivation skips empty-content assistant messages so it never enters the transcript.

`replayState.blocks` is pruned to the same positions, or the whole envelope discarded if its length no longer matches (:142–143).

6. Cancellation

```
// interruptedBlocks(): only text/reasoning blocks (open or closed)  
// with non-whitespace .trim() content, in stream order.  
// Tool-call blocks and any other open unknown-type block are omitted.
```

— :169–179

The reason is given in the source: "interruption precedes dispatch; retaining one would require a fabricated result" (:165–167). A tool call that was never dispatched has no result — and [Chapter 6](#)'s pairing rules mean an unbalanced call would make the conversation invalid. Simpler to drop it.

The engine calls this only when the signal actually aborted:


```

catch (error: unknown) {
  if (signal.aborted) {
    const content = assembler.interruptedBlocks()
    if (content.length > 0) {
      this.session.append('assistant/message', {
        turn, step,
        message: createAssistantMessage({ content, source: { provider, model } }),
        interrupted: true,
        ...assembler.usage === undefined ? {} : { usage: assembler.usage },
      }, { surfaceOp: 'append', sourceEventSeqs: chunkSeqs })
    }
  }
  throw error
}

```

— `packages/core/agent-loop/src/agent.ts:372-389`

`interrupted: true` marks it in the log. Usage is carried if it arrived. The error rethrows regardless — salvaging is not recovering.

7. A real stream

From `snapshots/session/text-turn/session.jsonl`, the chunks in order:

Line	Chunk	Assembler state after
14	<code>block-start</code> index 0, <code>reasoning</code>	partial 0 open
15	<code>20 × reasoning-delta</code> (stored packed)	partial 0 text grows
16	<code>block-start</code> index 1, <code>text</code>	partial 1 open
17-18	<code>text-delta</code> "P", "ONG"	partial 1 text = "PONG"
19	<code>block-end</code> index 0 with the full reasoning block	partial 0 closed, authoritative
20	<code>block-end</code> index 1 with the full text block	partial 1 closed
21	<code>usage</code>	<code>_usage</code> set
22	<code>finish</code> { kind: 'stop' }	<code>_finish</code> set

Note the interleaving: block 1 starts and receives deltas *before* block 0 closes. Indices, not arrival order, determine identity — which is why `partials` is a `Map` keyed by index and `order` tracks first-seen position separately.

Line 15 is a packed storage row, not a single event ([Ch 30](#)); it decodes back to 20 individual `assistant/chunk` events.

8. Control decisions

Decision	Condition	Location
Ignore a delta	the partial is already closed by <code>block-end</code>	:65-66, 71-72
Set <code>.block</code>	first <code>block-end</code> for that index	:49-96
Prefer the closed block	<code>.block</code> is set	:108-121
Throw	an open block of an unknown type at assembly	:119
Drop tool calls	<code>finish.kind === 'max-tokens'</code>	:134-150
Discard <code>replayState</code>	its <code>blocks</code> length no longer matches	:142-143
Keep a block on interrupt	text/reasoning with non-whitespace content	:169-179
Default to <code>stop</code>	no <code>finish</code> chunk ever arrived	:187-189

9. Edge cases

A stream that ends without `finish` is treated as a clean stop. `finish` getter returns `this._finish ?? { kind: 'stop' }` (:187-189). A provider that closes the connection after its last block produces a normal completion rather than an error.

Every chunk is logged before assembly. Chapter 9 ④ appends an `assistant/chunk` event *and* pushes into the assembler. So the raw stream is durable independently of the assembler's interpretation — if reassembly were ever wrong, the evidence is still in the log. That is also why the packing codec exists: hundreds of tiny events per response.

`assembler.message(source)` has an unused default. It defaults `source` to `{ kind: 'plugin', plugin: 'dsh-llm/assembler' }` (:204-207), but the loop always builds its own message via `createAssistantMessage` with real provider/model (`agent.ts:410-417`).

Verified unused. A repo-wide search for a non-test caller of `assembler.message()` across `packages/llm`, `packages/core`, and `packages/compaction` returns nothing. The method and its default source are dead in the shipped tree — harmless, but worth knowing if you are tempted to rely on that default.

10. Configuration knobs

None.

11. Interactions

- **Ch 9** — drives `push` per chunk and reads every exit view.
- **Ch 23** — supplies the stream.

- **Ch 25** — `finish.kind` of `error` / `aborted` routes to the retry waterfall.
- **Ch 30** — packs the logged chunk runs.
- **Ch 7** — skips the empty-content message a truncated response can produce.

12. Build it yourself

Minimal version:

```
const blocks: ContentBlock[] = []
let text = ''
for await (const chunk of stream) {
  if (chunk.type === 'text-delta') text += chunk.text
  if (chunk.type === 'block-end') blocks.push(chunk.block)
}
```

What the real one adds:

Addition	Why it exists
Index-keyed partials	Blocks interleave; arrival order is not identity
<code>block-end</code> authoritative	The provider's structured close beats reconstructed deltas
Straggler suppression	A delta after close would corrupt a completed block
Idempotent close	Duplicate <code>block-end</code> chunks happen
Dropping tool calls on <code>max-tokens</code>	A truncated call's arguments are incomplete JSON
<code>interruptedBlocks</code>	Cancelling should not discard what the user already watched
Omitting tool calls on interrupt	Retaining one would require fabricating a result
Throwing on an open unknown block	Guessing at an unfinished plugin block would fabricate content
<code>finish</code> defaulting to <code>stop</code>	A clean close without an explicit finish is not an error

Key takeaways

- Partials are keyed by stream index, because blocks interleave.
- A `block-end` block is authoritative; deltas after it are ignored.
- On `max-tokens`, every tool call is dropped — truncated arguments are unsafe to execute.
- On cancellation, only text and reasoning with real content survive, and the message is logged `interrupted: true`.
- A stream with no `finish` chunk counts as a clean stop.
- Every chunk is logged before assembly, so the raw stream outlives the interpretation.

Exercises

1. A model emits a text block, then a tool call, then hits `max-tokens`. What does the assistant message contain, what does the step return, and what does the turn record?
2. A provider sends `block-end` twice for index 0 with different blocks. Which wins, and which line decides?
3. `interruptedBlocks` filters on `.trim()` being non-empty. Construct the case this prevents, and say what would be logged without it.

Next: [Chapter 25 · Failures and retry](#)

Chapter 25 · Failures and retry

What you'll learn: how any thrown value becomes a structured failure, and why retrying is a plugin rather than a feature of the loop.

Prerequisites: [Chapter 24](#), [Chapter 22](#).

1. The problem

Requests fail in many ways: a rate limit, an expired key, a socket reset, a context window exceeded, an empty response, a provider SDK throwing something that is not even an `Error`. Downstream, several parties need to make decisions about that failure — should it be retried, should history be compacted, should the turn end, what should the user be shown.

Each needs the same thing: a **serializable, comparable description** of what went wrong, with a code that can be trusted. A raw thrown value provides none of that. It might have a `code` property that means something in another library's taxonomy, or a getter that throws when read, or come from another realm where `instanceof` lies.

And then the policy question. Retrying with backoff is standard — but whose policy? A deployment on a flaky network wants different behavior from one that would rather fail fast. Baking a retry loop into the engine means every deployment gets one answer.

2. Mental model

Two separations.

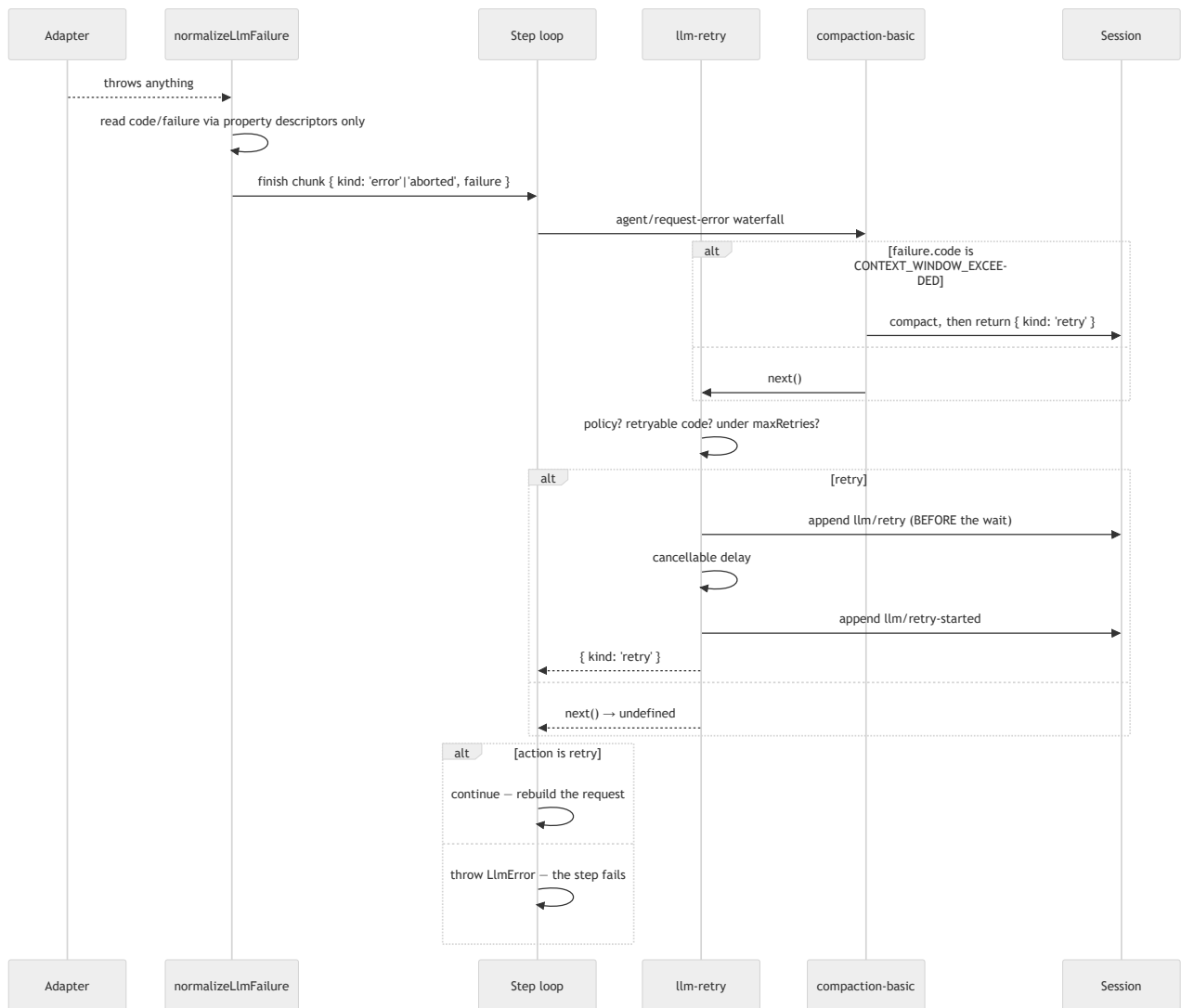
Failure description is separated from failure handling. The adapter boundary normalizes anything thrown into an `LlmFailure`; nothing downstream sees a raw error.

```
export interface LlmFailure {
  readonly message: string
  readonly code: string
  readonly status?: number
  readonly providerRetryAfterMs?: number
  readonly requestId?: ProviderRequestId
}
```

— `packages/llm/llm/src/types.ts:40–51`

Policy is separated from mechanism. A `ResolvedRetryPolicy` is pure data. The thing that *acts* on it lives in a separate package, attached to an extension point. The engine provides the seam and the terminal default.

3. Lifecycle



4. Normalization

`normalizeLlmFailure` (`packages/llm/llm/src/adapter-failure.ts`) is the single point where any thrown value from an adapter becomes an `LlmFailure` . It is called from `adapterFailureChunk` (`packages/llm/llm/src/index.ts:1069-1077`), the only place a caught adapter exception becomes a terminal `StreamChunk` .

It is defensive in a way worth noticing:

- it reads `error.code` and `error.failure` **only via** `Object.getOwnPropertyDescriptor` , never by property access — so a getter cannot throw or lie during error handling (:40-88);
- it trusts a carried `failure` snapshot only when its `code` agrees with the error's own `code` ;
- anything that is not an `Error` , or is a foreign `Error` whose code cannot be trusted, becomes code `UNKNOWN` — "Trust only Harness-owned codes; third-party SDK codes are not our taxonomy" (:101-104).

The resulting failure rides on the terminal chunk as `{ type: 'finish', reason: { kind: 'error' | 'aborted', failure } }` — `aborted` when the signal aborted or the code is `ABORTED`, `error` otherwise.

The code taxonomy

Group	Codes
Registry	<code>NO_ADAPTER</code> , <code>DUPLICATE_ADAPTER</code> , <code>INVALID_ADAPTER</code> , <code>INVALID_PREPARED_CALL</code> , <code>INVALID_MODEL_*</code> , <code>UNSUPPORTED_REASONING_EFFORT</code>
Credentials	<code>INVALID_CREDENTIAL</code> , <code>MISSING_CREDENTIAL</code>
Transport / provider	<code>AUTH</code> , <code>INVALID_REQUEST</code> , <code>RATE_LIMIT</code> , <code>SERVER</code> , <code>HTTP_<status></code> , <code>QUOTA</code> , <code>CONTEXT_WINDOW_EXCEEDED</code>
Adapter-local	<code>TRANSPORT</code> , <code>ABORTED</code> , <code>TIMEOUT</code> , <code>MALFORMED_RESPONSE</code> , <code>STREAM_CLOSED</code> , <code>UNSUPPORTED_CONTENT</code> , <code>EMPTY_RESPONSE</code>
Fallback	<code>UNKNOWN</code>

Two are load-bearing elsewhere. `CONTEXT_WINDOW_EXCEEDED` is what triggers compaction's recovery path (Ch 27). `EMPTY_RESPONSE` is emitted by DeepSeek's translation when a `stop` completion produced zero blocks (`llm-deepseek/src/translate.ts:117-126`) and is explicitly noted as "safe to repeat," which is why it is in the default retryable set.

Provider errors are classified by **pattern-matching the provider's error text** — `isContextWindowExceededError` and `isQuotaExceededError` (`llm/src/error.ts:80-100`) sniff the `code`, `type`, and `message` fields with regexes. That is fragile by nature: a provider rewording a message could silently stop triggering compaction (Ch 35).

5. Policy is data

```
interface NormalRetryPolicyConfig { mode: 'normal'; maxRetries?: number; retryableCodes?: string[];
backoff?: BackoffConfig }
interface AlwaysRetryPolicyConfig { mode: 'always'; backoff?: BackoffConfig }
```

— `packages/llm/llm/src/retry-policy.ts`

Defaults (`:14-24`): `maxRetries = 5`, `initialDelayMs = 500`, `maxDelayMs = 10_000`, `jitterRatio = 0.1`, and

```
retryableCodes = [EMPTY_RESPONSE, RATE_LIMIT, SERVER, TIMEOUT, TRANSPORT]
```

Note what is absent: `AUTH`, `INVALID_REQUEST`, `QUOTA`, `CONTEXT_WINDOW_EXCEEDED`. Retrying those unchanged would fail identically. Context-window overflow is retryable *only after something changes the history*, which is why compaction — not retry — owns that path.

The policy is captured at adapter registration (Ch 23) and read back through `PreparedLlmCall.retryPolicy`. **It performs no I/O and schedules nothing.**

6. Who actually retries

The engine's own contribution is the seam and a terminal default:

```
const action = await this.dispatch.waterfall('agent/request-error', {...},
  () => Promise.resolve<RequestErrorAction>(undefined))
if (action?.kind !== 'retry') {
  throw new LlmError(finish.failure.message, finish.failure.code, finish.failure)
}
continue
```

— `packages/core/agent-loop/src/agent.ts:390-408`

`RequestErrorAction` is `{ kind: 'retry' } | undefined`, and the documented default is that `undefined` "leaves the failure terminal" (`runtime-types.ts:255-256`).

The listener that supplies retries is `packages/llm/llm-retry/src/index.ts:243-252`, mounted host-plane in the base bundle (`packages/bundle/base/cordis.patch.yml:84-85`). Its `recover()` (`:194-241`):

No policy → delegate immediately. `if (policy === undefined) return next() (:198)`. A request whose `preparedCall` was `undefined` — the `NO_ADAPTER` fallback path — is never retried. This is exactly the fresh-install case (Ch 32).

normal mode gates on `policy.retryableCodes.includes(failure.code)`, else delegates.

always mode awaits `next()` first — giving other listeners a chance to claim the retry — and otherwise falls through to the same backoff logic. It has no `retryableCodes` gate and **no maxRetries enforcement**, so it retries indefinitely until abort or disposal (`:223`).

The `always` branch is easy to misread: there is no early `return` between the mode check and the shared backoff section for the case where downstream did not retry. Quote the lines rather than paraphrasing if you depend on this.

Retry counting is durable

```
ctx.sessionProjections.stateOf(agent.session, 'llmRetry')
```

— `:220`

The count comes from a **projection** (Ch 8) folded from `llm/retry` session events, keyed by `[provider, policyKey]`, cleared on `step/start` and `turn/end` (`:125-138`).

And the ordering is deliberate:

```
// append llm/retry → then the cancellable wait → then append llm/retry-started
```

— `backoff()` , :149–192 ; comment at :3 : "Each scheduled retry is durable before its cancellable wait"

So a crash during the backoff delay leaves evidence that a retry was scheduled. A retry budget survives a process restart, because it was never in memory.

Delay honors `providerRetryAfterMs` when present and within `maxDelayMs` ; otherwise bounded exponential backoff (`initialDelayMs * 2^min(retry-1, 1024)` , capped) with symmetric jitter (:59–64).

Disposal is clean: the plugin's `ctx.effect` disposer aborts an internal controller and awaits all in-flight `recover()` promises before finishing (:254–258).

7. Control decisions

Decision	Condition	Location
Code becomes <code>UNKNOWN</code>	not an <code>Error</code> , or a foreign code	<code>adapter-failure.ts:101–104</code>
<code>aborted</code> vs <code>error</code> finish	signal aborted or code <code>ABORTED</code>	<code>llm/index.ts:1069–1077</code>
Step fails	waterfall result is not <code>{kind: 'retry'}</code>	<code>agent.ts:404–406</code>
Rebuild and re-send	result is <code>{kind: 'retry'}</code>	<code>agent.ts:407</code>
Delegate, no retry	<code>policy === undefined</code>	<code>llm-retry:198</code>
Delegate, no retry	<code>normal</code> mode and code not retryable	<code>llm-retry:243</code>
Delegate, no retry	<code>normal</code> mode and <code>previousRetry >= maxRetries</code>	<code>llm-retry:223</code>
Retry indefinitely	<code>always</code> mode	<code>llm-retry:199–213</code>

8. Edge cases

Two listeners share this point. `compaction-basic` (:180) and `llm-retry` (:243). Compaction checks for `CONTEXT_WINDOW_EXCEEDED` and, if it successfully shrank history, returns `{kind: 'retry'}` without delegating — a legitimate veto (Ch 22). Otherwise it calls `next()` and retry gets its turn.

A retry rebuilds the request completely. `continue` re-enters the step loop at `buildRequest` (Ch 9), so it picks up whatever compaction just did to history, re-resolves the adapter, and re-checks the header. That is what makes compact-then-retry work at all.

The engine has no retry cap of its own. The step loop's `while (true)` is bounded only by the listener. An `always` policy will retry forever; the stopping mechanisms are cancellation and disposal ([Ch 13](#)).

LlmError validates its own construction. Non-empty message and code, `status` an integer in 100–599, `providerRetryAfterMs` positive and finite — throwing a plain `Error` if violated (`llm/index.ts:96–109`). A malformed failure description fails loudly rather than propagating.

9. Configuration knobs

Setting	Default	Effect
<code>mode</code>	<code>normal</code>	<code>always</code> removes both the code gate and the retry cap
<code>maxRetries</code>	<code>5</code>	<code>normal</code> only
<code>initialDelayMs</code> / <code>maxDelayMs</code>	<code>500</code> / <code>10_000</code>	Exponential backoff bounds
<code>jitterRatio</code>	<code>0.1</code>	Symmetric jitter
<code>retryableCodes</code>	<code>EMPTY_RESPONSE</code> , <code>RATE_LIMIT</code> , <code>SERVER</code> , <code>TIMEOUT</code> , <code>TRANSPORT</code>	<code>normal</code> only

— `packages/llm/llm/src/retry-policy.ts:14–24` ; the plugin is mounted at `packages/bundle/base/cordis.patch.yml:84–85`

Set per provider, captured at registration ([Ch 23](#)).

10. Interactions

- [Ch 9](#) — owns the seam and the terminal default.
- [Ch 23](#) — supplies the policy; its absence disables retry entirely.
- [Ch 27](#) — the other listener, and the only handler for `CONTEXT_WINDOW_EXCEEDED` .
- [Ch 8](#) — the durable retry counter.
- [Ch 32](#) — traces a failure that nothing retries.

11. Build it yourself

Minimal version:

```

for (let attempt = 0; attempt <= maxRetries; attempt++) {
  try { return await stream(request) }
  catch (error) {
    if (!RETRYABLE.has(codeOf(error))) throw error
    await delay(initialDelay * 2 ** attempt)
  }
}

```

What the real one adds:

Addition	Why it exists
Normalization at the adapter boundary	Downstream must never see a raw thrown value
Property-descriptor reads	An error object's getter must not throw during error handling
<code>UNKNOWN</code> for foreign codes	Another library's taxonomy is not this one's
Retry as a waterfall listener	Deployments differ; the engine should not pick
Durable retry counting	A budget in memory resets on restart
Append-before-wait	A crash during backoff must leave evidence
<code>providerRetryAfterMs</code> honored	The provider knows better than local backoff
Two listeners on one point	Overflow needs compaction, not repetition

Key takeaways

- Every thrown value is normalized to a serializable `LlmFailure` with a trusted code before anything downstream sees it.
- The engine never retries; it offers a waterfall whose default is terminal.
- `llm-retry` supplies retries, counts them in a session projection, and appends the retry event *before* its cancellable wait.
- `CONTEXT_WINDOW_EXCEEDED` is deliberately not retryable — it needs compaction first.
- No prepared call means no policy means no retry, which is exactly the unconfigured-install case.
- `always` mode has no retry cap; only cancellation stops it.

Exercises

1. A provider returns HTTP 429 with `Retry-After: 30`. Trace the delay chosen, and say what happens if the header said 300 instead.
2. Retry counts live in a projection cleared on `step/start`. What does that mean for a turn with five steps that each fail twice?
3. Both compaction and retry listen to `agent/request-error`. Registration order is not specified. Construct the two orderings and say whether the outcome differs — then explain why.

Next: [Chapter 26 · Measuring and spilling](#)

Chapter 26 · Measuring and spilling

What you'll learn: how the engine estimates how full the context window is — without a tokenizer — and the first line of defense that keeps a huge tool result from entering history at all.

Prerequisites: [Chapter 16](#), [Chapter 23](#).

1. The problem

Two separate questions, often confused.

"How much room is left?" Compaction needs a number to compare against the model's context window. But the harness cannot tokenize: tokenization is provider- and model-specific, the vocabularies are large, and shipping one per provider would be a maintenance burden for an answer that only needs to be approximately right.

"Should this result be in history at all?" A `grep` across a large repository can return hundreds of kilobytes. Once that lands in the log it is on the surface, it counts against the window, and it will keep counting for the rest of the session. Compacting it later is possible but wasteful — better not to admit it.

These get different mechanisms, at different times, and it is worth keeping them apart.

2. Mental model

New term — token meter. A per-session measurement combining two sources: real provider-reported usage when available, and a character-count heuristic otherwise.

New term — spill. Writing an oversized tool result out of line at execution time, leaving the model a bounded preview plus a pointer to retrieve the rest.

	Spill	Compaction
When	tool execution	step boundary or overflow
Granularity	one tool result	a region of history
Uses a model?	no	yes (Ch 27)
Acts on	the result before it is logged	the surface after messages exist

They never call each other. They compose only through the shared session log and the shared meter.

3. Measuring

```
// measure(session):  
// baseline = usage !== undefined && usageTokens(usage) >= estimatedAnchorTokens  
//   ? { kind: 'usage', ... }  
//   : { kind: 'estimated', ... }
```

— `packages/llm/token-meter/src/index.ts:133–177`, the decision at `:154–156`

Real usage is preferred, but only when it is at least as large as a full heuristic re-price of the same anchor point. That guard matters: provider usage counts the request *as sent*, which may differ from what the meter would estimate. Taking the larger keeps the meter from under-reporting and compacting too late.

When no usage is available — before the first successful call, or from a provider that does not report it — the fallback is a fixed-density heuristic:

```
CHARS_PER_TOKEN = 4  
BLOCK_OVERHEAD  = 4  
ROLE_OVERHEAD   = 4
```

— `packages/llm/token-meter/src/estimate.ts:13–19`

A text block costs `Math.ceil(text.length / 4) + 4`. Structural and image blocks are priced by their JSON-stringified length over the same divisor (`:28–30`).

This is not a tokenizer. There is no BPE table, no vocabulary lookup, nothing provider-specific. Four characters per token is a rough average for English prose and is wrong in both directions for code, non-Latin scripts, and long identifiers.

That is a deliberate trade, and the design compensates in two ways: real usage supersedes the estimate as soon as one request succeeds, and the threshold it feeds is 80% rather than 100% ([Ch 27](#)) — leaving headroom for the estimate being wrong. It remains a real limitation ([Ch 35](#)).

The capacity side of the comparison comes from the adapter: `LlmModelContext.contextWindow`, validated as a positive integer at `prepareCall` ([Ch 23](#)) and flowed into a `request/context` event for presentation.

4. Spilling

Three packages, cleanly separated:

`spill` defines the seam — `ctx.spillStore`, one method:

```
saveText(input): Promise<SpillRef>
```

— `packages/spill/spill/src/index.ts:45–58`

Deliberately minimal: no retention policy, no replacement logic, no retrieval API of its own (:8-13).

`spill-local` is the host-filesystem implementation (`packages/spill/spill-local/src/index.ts:65-162`). It writes to a private, session-scoped path — `0700` directory, `0600` file — under a configured root or a lazily-created OS-temp default, and returns a `SpillRef` carrying a path locator and a retrieval hint:

```
Use read with offset/limit, or grep this path to search within it. — :159
```

The hint is addressed to the model. Rather than inventing a bespoke retrieval tool, spill leaves the content where the tools the model already has can reach it.

It also runs one best-effort startup sweep for files older than `cleanupPeriodDays` (default `30`, :68).

`spill-policy` is where the decision lives (`packages/spill/spill-policy/src/index.ts:110-232`). It registers a **prepended** listener on `tools/post-execute` (:190-209) — a `dsh-tools` extension point (Ch 16), not one of the agent-loop's.

The algorithm:

1. **No-op unless configured.** Without `maxInlineBytes` the whole plugin does nothing (:113).
2. **Plain text only.** `flattenPlainText` (:80-87) extracts text; non-text and multimodal results are left untouched. Spilling an image would break it.
3. **Under the cap → untouched.**
4. **Over the cap** → `ctx.spillStore.saveText(...)` (:155), then the model-facing content is replaced with a bounded head/tail preview (via `TextRetainer`, :98-101) plus a notice naming the locator and the retrieval hint (`spillNotice`, :104-108).
5. **Best-effort.** A save failure logs and keeps the original inline content (:156-161). Spill degrades to doing nothing rather than losing the result.

The `read` exclusion

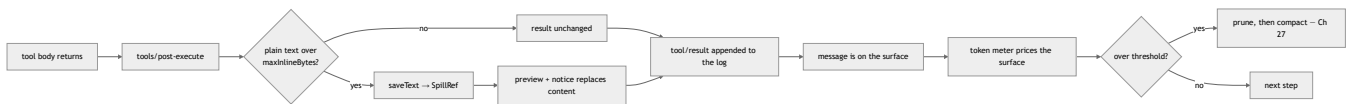
```
// skips the `read` tool
```

— :197

Without it: the model reads a large file, the result spills, the notice says "use read on this path", the model reads *that*, and the result spills again. The exclusion breaks the cycle at its source.

This is the kind of detail that only shows up once a system is running, and it is worth noting as an example of a mechanism having to know one thing about its neighbours.

5. Where each runs



Spill acts **before** the result becomes a message. Compaction acts on messages that already exist. A result small enough to pass the spill cap but which accumulates with dozens of others is exactly what compaction is for.

6. Control decisions

Decision	Condition	Location
Use real usage	present and $\geq$ the estimated anchor	<code>token-meter:154-156</code>
Use the heuristic	otherwise	same
Do nothing	<code>maxInlineBytes</code> unconfigured	<code>spill-policy:113</code>
Do nothing	result is not plain text	<code>spill-policy:80-87</code>
Do nothing	tool is <code>read</code>	<code>spill-policy:197</code>
Spill	plain text over the cap	<code>spill-policy:155</code>
Keep original	the save failed	<code>spill-policy:156-161</code>

7. Configuration knobs

Setting	Default	Where
<code>maxInlineBytes</code>	<code>50000</code>	<code>packages/bundle/base/cordis.patch.yml:393-396</code>
<code>cleanupPeriodDays</code>	<code>30</code>	<code>spill-local/src/index.ts:68</code>
<code>root</code>	lazily-created OS temp	<code>spill-local</code>

`spill-policy` and `spill-local` are **host-plane** rows in the base bundle and are **not** touched by the web-app patch — verified absent from `packages/bundle/web-app/cordis.patch.yml`. So the 50 KB cap applies to every web session, and unlike most tool-adjacent rows it is not re-declared per preset.

The token meter is also deliberately host-plane. The standard preset's comment explains why: it "keys every fold by Session, and owns the context-meter projection units the browser reads for every session — behind a realm those units would come and go with whichever presets happen to be mounted" (`presets/standard/agent.cordis.yml:131-136`).

8. Edge cases

The meter's estimate can be badly wrong for code. Four characters per token under-counts dense identifiers and over-counts repetitive whitespace. The 80% threshold is the safety margin.

A spilled result is still in the log. Spill replaces the *model-facing content* before the `tool/result` event is appended — so what is logged is the preview. The full text lives on disk outside the log, under a path that only that session's files sit in. It is not part of the session's durable record, and a `cleanupPeriodDays` sweep will eventually remove it.

Confirmed: it cannot. The sweep deletes "files whose `mtime` is strictly older than the cutoff" (`spill-local/src/index.ts:40-46`), computed as `cleanupPeriodDays` × one day. It is purely age-based and knows nothing about which sessions are live, resumable, or referenced.

So a session resumed after 30 days holds spill notices pointing at paths that no longer exist. The model will be told to `read` a file that is gone, and will get an ordinary file-not-found tool error.

Two mitigations exist in config: `cleanupPeriodDays: 0` **disables cleanup entirely** (`:40`), and the sweep runs **once at startup** — best-effort, owned by the plugin fiber, launched without delaying service availability but awaited at disposal so no sweep I/O outlives the fiber (`:60-94`).

 One safety property is worth naming, because the test suite names it: the sweep **does not follow symlinks**. `spill-local`'s own tests include "does NOT follow a symlinked session directory (no deletion in the target)" and "returns only real `dsh-spill-*` directories, excluding symlinks and non-matches". A cleanup routine that deleted through a symlink would let anything able to plant one in the spill root direct deletions at an arbitrary target.

(Those two cases are also the only failures observed when running this repository's suite on Windows — not because the property is broken, but because Windows refuses to *create* the symlink the test needs: `EPERM: operation not permitted, symlink.`)

This is a genuine, bounded limitation rather than a bug: the spilled text was never part of the durable record ([Ch 5](#)), and the preview that *is* in the log remains intact.

`ptc-dispatch-log` **gets the same treatment**. The policy also listens there (`:217-231`), so `run_code` sub-dispatch output is capped the same way.

9. Interactions

- [Ch 16](#) — `tools/post-execute` is where spill attaches.
- [Ch 27](#) — consumes the meter for its threshold, and is the next defense when spill was not enough.
- [Ch 23](#) — supplies `contextWindow`.
- [Ch 8](#) — the meter owns context-meter projection units the browser reads.

10. Build it yourself

Minimal version:

```
const estimate = messages
  .flatMap(m => m.content)
  .reduce((n, b) => n + Math.ceil(JSON.stringify(b).length / 4) + 4, 0)

if (resultText.length > MAX_INLINE) {
  const path = await writeToTemp(resultText)
  return { ...result, content: [{ type: 'text', text: `${head(resultText)}\n[full output at
${path}]]` }] }
}
```

What the real one adds:

Addition	Why it exists
Prefer real usage over the estimate	The provider's count is authoritative once one call succeeds
The $\geq$ -anchor guard	A smaller reported usage would under-report and compact too late
Spill as a separate seam + backend + policy	Storage location, retention, and the decision are independent concerns
Retrieval hint naming existing tools	Avoids a bespoke retrieval tool the model must learn
Plain-text-only	Spilling an image reference would break it
The <code>read</code> exclusion	Otherwise read $\rightarrow$ spill $\rightarrow$ read loops
Best-effort degradation	A failed save must not lose the tool's output
Private, per-session, mode-restricted paths	Tool output can contain anything the tool could read

Key takeaways

- The meter prefers real provider usage and falls back to a fixed 4-characters-per-token heuristic; it is not a tokenizer.
- Real usage is only preferred when it is at least as large as a heuristic re-price of the same anchor.
- Spill acts at tool-execution time, before an oversized result ever becomes a message.
- Only plain text spills; the `read` tool is excluded to prevent a retrieval loop.
- Spill is best-effort — a failure keeps the original content.
- Both the meter and the spill policy are host-plane, applying uniformly to every session.

Exercises

1. A tool returns 200 KB of JSON as a text block. Trace it through spill and say exactly what the model sees. Now make it an image block and answer again.

2. The meter prefers usage only when it is $\geq$ the estimate. Construct the case this guards against, and say what would go wrong without the guard.
3. `maxInlineBytes` is 50 KB and the compaction threshold is 80% of a 1M-token window. Roughly how many maximum-size spilled results fit before compaction triggers? What does that tell you about which mechanism does the real work?

Next: [Chapter 27 · Pruning and compaction](#)

Chapter 27 · Pruning and compaction

What you'll learn: how a conversation that outgrows the context window is shrunk without losing anything — the payoff for everything Chapter 6 set up.

Prerequisites: [Chapter 6](#), [Chapter 26](#), [Chapter 22](#).

1. The problem

Every message is permanent ([Ch 5](#)) and every request carries the whole visible history ([Ch 7](#)). Those two facts guarantee that a long enough session will exceed any context window.

The naive fixes all fail. Dropping the oldest messages loses decisions the model needs. Dropping the middle breaks tool-call/result pairing and a provider will reject the request. Truncating messages produces incoherent history. And doing any of it destructively contradicts the permanence the rest of the design depends on.

There is also a timing question. Compacting too early wastes a model call and discards useful detail. Too late and the request fails. And when it *does* fail, something must recover — the user should not lose a turn because the history grew.

2. Mental model

Two mechanisms, cheapest first.

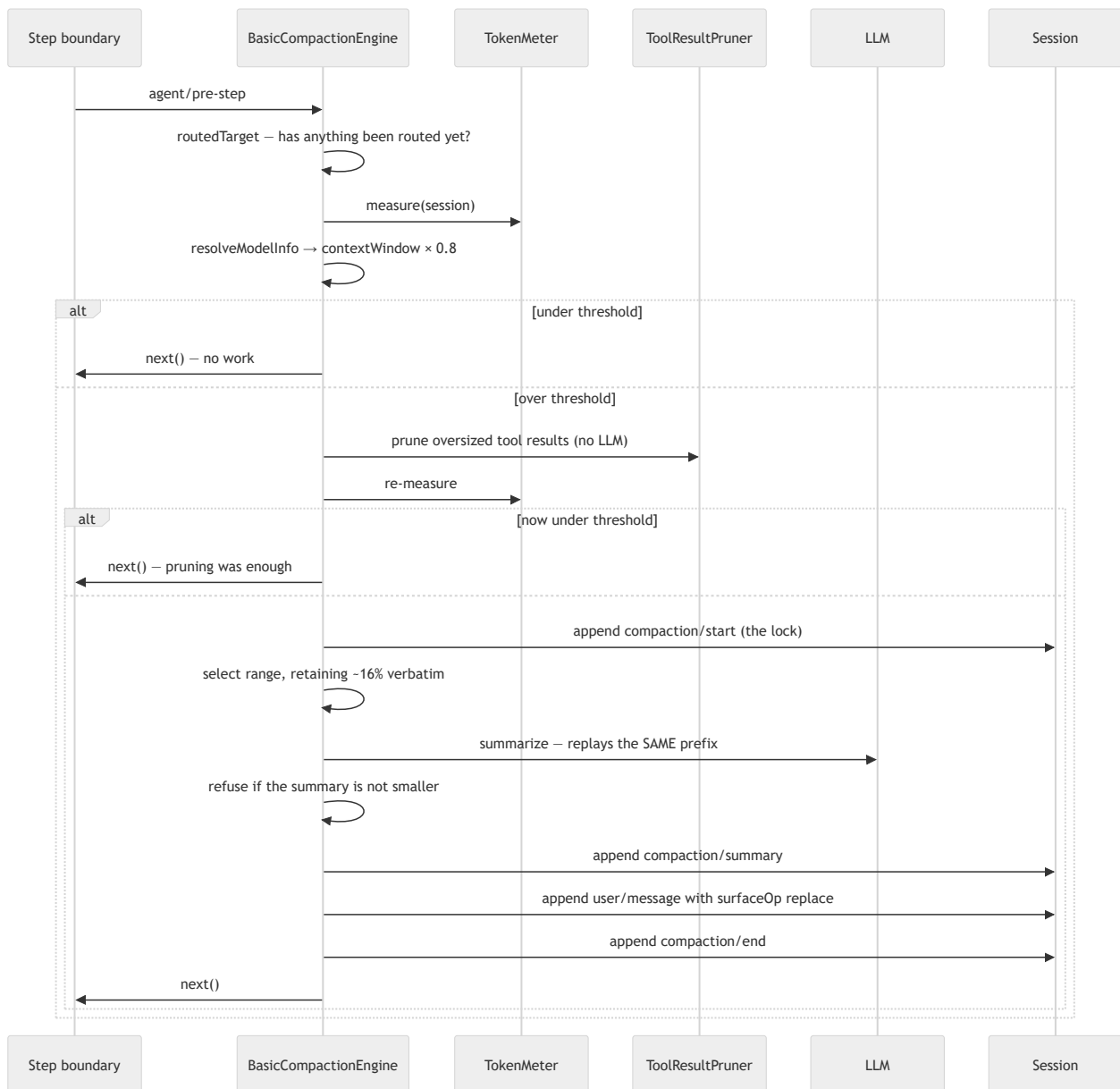
Pruning is model-free: find individual tool results whose text is enormous and replace each with a head/tail excerpt. Deterministic, fast, no LLM call.

Compaction is the heavy option: take a *region* of old history, ask the model to summarize it, and replace the whole region with one checkpoint message.

Both use the same primitive — [Chapter 6](#)'s `replace` surface op. Neither deletes anything. The original events stay in the log at their original positions; the surface simply stops pointing at them.

New term — compaction transaction. A `compaction/start` event, some work, and a `compaction/end` event. The start event is the lock: a second compaction cannot begin while one is open.

3. Lifecycle



4. Two triggers

Both registered in `_registerAutomaticCompaction()` (`packages/compaction/compaction-basic/src/index.ts:138-225`), gated by `config.auto` (default `true` , `config.ts:95`).

Pressure, at every step

A listener on `agent/pre-step` (`:148-166`) calls `compactIfNeeded(agent, 'pressure', signal)` and **always** calls `next()` (`:165`). It never rejects a step; it mutates history as a side effect and lets the step proceed.

Checking on every step rather than on a timer means the decision is made with the exact history the next request will use.

Overflow, as recovery

A listener on `agent/request-error` (:180–224) fires only when `failure.code === CONTEXT_WINDOW_EXCEEDED` — the provider itself said the window was exceeded. It calls `compactIfNeeded(agent, 'context-overflow', signal)` and then:

```
// if the surface's replaceGeneration actually advanced, return { kind: 'retry' }
```

— :219–223

So compaction shrinks the history and asks the step loop to re-issue **the same request** (Ch 9 ⑥). The `replaceGeneration` check is the honest part: it only claims a retry if it genuinely changed something. Otherwise it delegates and retry gets its chance (Ch 25).

This path is capped by `maxOverflowRetries` (default 1, `config.ts:93`), tracked per agent in a `WeakMap` and reset when the next `assistant/message` lands (:174–178) or when the agent goes idle (:168–170). One rescue attempt per failure, not a loop.

5. Deciding whether to compact

`compactIfNeeded` (:259–333):

① **Is there a route yet?** `routedTarget()` (:53–61) reads `session.requestHeader()?.config`. Before any request has been routed there is no model to measure against, and it returns `null`.

② **Resolve a per-target policy.** `resolveTargetPolicy` (`config.ts:105–125`) lets a deployment override thresholds for an exact `provider/model` pair via `modelPolicies`. None are configured in the web composition, so the defaults apply everywhere.

③ **Measure.** `ctx.tokenMeter.measure(agent.session)` (Ch 26).

④ **Compare — for pressure only.** `ctx.llm.resolveModelInfo(provider, model, signal)` (:294) gives the real context window; the threshold is `floor(contextWindow × thresholdRatio)` with `thresholdRatio = 0.8` (`config.ts:20`, via `resolveCompactSpec`, `config.ts:133–167`). Proceed only if `measurement.totalTokens >= spec.thresholdTokens` (:305, :313).

For overflow, the threshold check is skipped entirely (:284–291). The provider already said the window was exceeded; arguing with it via an estimate would be absurd.

⑤ **Try the cheap thing first.** `ctx.get('toolResultPruner')` (:282) — optional — then re-measure (:309–313). If pruning brought the session under threshold, `compactIfNeeded` returns **without any LLM call**.

That ordering is the chapter's best structural idea: the expensive, lossy, model-dependent operation runs only after the cheap, deterministic, nearly-lossless one has failed to be enough.

6. Pruning

`ToolResultPruner` (`packages/compaction/compaction-tool-result-pruner/src/index.ts`) injects only `tokenMeter` (:47) — it never touches an LLM.

`pruneSession` (:136–184) walks every current `tool/result` surface node, and for each one over budget appends a `compaction/prune` shadow-price event followed by a replacement `tool/result` with `surfaceOp: { op: 'replace', start: seq, end: seq }` (:162–173).

Single-node replacement, content-only — which is exactly what [Chapter 6](#)'s `assertToolResultRewrite` permits and nothing more. The narrow rewrite rule exists for this caller: it is structurally impossible for pruning to change whether a tool failed, or which call it belonged to. Only the text shrinks.

Budget: `thresholdChars: 8192`, keeping `headChars: 4096` and `tailChars: 1024`.

7. Compaction

`compactSurfaceRegion` (`packages/compaction/compaction-basic/src/region.ts:154–256`).

Selecting the range

`selectCompactableRange` (:100–136) walks the surface **from the end backwards**, accumulating token counts until at least `retainTokens` is preserved verbatim — `retainRatio = 0.16`, about 16% of the window (`config.ts:23`). Then it walks **forward** from that cut point to the nearest boundary that does not split a tool-call/result pair (`toolPairingBalancedBefore`).

Two properties fall out: the most recent conversation always survives intact, and the boundary never orphans a tool call from its result — which would make the next request invalid.

Locking

```
session.append('compaction/start', { compactionId, turn })
```

— :191

Written **synchronously, before any async work**. `assertCompactionInactive` (:288–300) rejects a second compaction while a `compaction/start` has no matching `compaction/end`, and re-checks after every `await` (:307–314).

The lock is a log event, not a mutex — so it survives a crash, and a reader can see that a compaction was in progress when the process died.

Summarizing

`buildSummarizationInput` (:508-524) replays **the conversation's own last routed system prompt, tools, and the shadowed messages verbatim**. Deliberately: reusing the exact prefix the real requests use keeps the provider's KV/prefix cache warm, so the summarization call is much cheaper than a cold request of the same size.

Then `summarizeWithLlm` (`packages/compaction/compaction-basic/src/summarizer.ts:121-182`) appends one synthetic user message carrying a fixed instruction (:31-66) asking for a structured Markdown checkpoint with these sections (:36-58):

```
Primary Request and Intent · Key Technical Concepts · Files and Code · Errors and Fixes · Pending Jobs · Current Work · Next Step · Critical Context
```

and calls `ctx.llm.stream(options)` with `purpose: 'compaction'` (:161).

That call is **not** a loop-built request — it is exactly the one-shot case [Chapter 11](#) excludes from the reconstruction invariant. It carries a session id and is frozen, but it is not meant to equal the derivation.

Refusing a useless compaction

```
// if the framed summary's estimated token cost is NOT smaller
// than the shadowed span's, the whole compaction throws - nothing commits
```

— :383-388

A compaction that would not shrink anything is refused outright. Without this, a session near the threshold with already-terse history could compact repeatedly, each time spending a model call to replace short messages with a summary of similar size.

Committing

`commitCompactionBody` (:437-488) appends, in order:

1. `compaction/summary` — the summary text, provenance, shadowed range, seqs, token count (:457-471);
2. a `user/message` carrying the framed checkpoint (wrapped in `<compacted-summary>` tags by `frameSummary`, `summarizer.ts:189-195`) with

```
surfaceOp: { op: 'replace', start, end },
sourceEventSeqs: [startEvent.seq, summaryEvent.seq, ...shadowedSeqs],
```

— `region.ts:472-475`

3. `compaction/end` (:217).

The provenance cites more than [Chapter 6](#)'s coverage rule requires — the transaction's own start and summary events as well as every shadowed node — so the whole operation is reconstructable, not just its effect.

Nothing is deleted. The original messages remain in the JSONL log forever, consistent with the repository's own rule that released session logs never move, overwrite, or delete committed generations.

Failure

If summarization or commit throws after `compaction/start` landed, the code still appends a `compaction/end` carrying an `error` field (:224–230). The durable lock is always released — `assertCompactionInactive`'s job is to reject a concurrent attempt, not to wedge the session.

8. Manual `/compact`

`compactNow` (:369–421) differs in three ways:

- it requires the agent to be **idle**, running inside `agent.runMaintenance(...)` and throwing `ManualCompactionError('busy', ...)` otherwise (:414–420) — the maintenance phase from [Chapter 13](#), which is why a manual compaction does not make the UI show "running";
- it selects with `retainTokens: 0` (:380–384) — compact as much as possible, rather than preserving 16%;
- it uses `stability: 'selected-span'` rather than `'whole-surface'` (:392), a looser check requiring only the selected span to be unchanged during summarization, since a human-invoked compaction can tolerate racing the agent's own activity less strictly.

9. Control decisions

Decision	Condition	Location
Skip entirely	<code>config.auto</code> is false	:138–225
Skip	nothing routed yet	:53–61
Skip	measured tokens < 80% of the window (pressure only)	:305, 313
Skip threshold	trigger is <code>context-overflow</code>	:284–291
Prune first	<code>toolResultPruner</code> is composed	:282
Stop after pruning	re-measure is under threshold	:309–313
Reject concurrent	an open <code>compaction/start</code> exists	<code>region.ts</code> :288–300
Refuse commit	the summary is not smaller	<code>region.ts</code> :383–388
Return <code>{kind:'retry'}</code>	<code>replaceGeneration</code> advanced	:219–223
Stop retrying	<code>maxOverflowRetries</code> reached	<code>config.ts</code> :93
Require idle	manual <code>/compact</code>	:414–420

10. Edge cases

Compaction invalidates the derived-message cache. The `replace` bumps `replaceGeneration`, so the next `deriveMessages()` rebuilds from scratch (Ch 7) — the one non-incremental case.

It also starts a new request series. The next header logs `reason: 'series'` even if its bytes are identical (Ch 10).

And it can clear the runtime-context snapshot. If the shadowed range contained one, Chapter 21's projection sees its seq in `sourceEventSeqs` and re-injects on the next step.

The human transcript is unaffected. A UI reads append-origin events (Ch 6), so a user scrolling back still sees everything. Only the model's view shrank.

11. Configuration knobs

Setting	Default	Effect
<code>auto</code>	<code>true</code>	Enables both triggers
<code>thresholdRatio</code>	<code>0.8</code>	Compact at 80% of the context window
<code>retainRatio</code>	<code>0.16</code>	Verbatim recent history to preserve
<code>maxOverflowRetries</code>	<code>1</code>	Compact-and-retry attempts per failure
<code>modelPolicies</code>	none configured	Per- <code>provider/model</code> overrides
<code>pruner thresholdChars / headChars / tailChars</code>	<code>8192 / 4096 / 1024</code>	Per-result truncation budget

Mount status, which took three layers to establish ([Ch 34](#)): `compaction-basic`, `command-compact`, and `tool-result-pruner` are mounted in the base bundle, **disabled** in the web-app patch, and **re-mounted by the standard preset** inside an `isolate: { compaction: true, toolResultPruner: true }` group (`presets/standard/agent.cordis.yml:137-155`). They are live for a default web session, per-agent. Reading only the first two layers would wrongly conclude compaction is off.

12. Interactions

- [Ch 6](#) — the `replace` primitive and the narrow tool-result rewrite rule.
- [Ch 26](#) — supplies the measurement.
- [Ch 25](#) — shares `agent/request-error`; compaction vetoes, retry handles the rest.
- [Ch 13](#) — manual compaction runs in the maintenance phase.
- [Ch 11](#) — the summarization call is a one-shot, deliberately out of scope.

13. Build it yourself

Minimal version:

```
if (estimateTokens(messages) > contextWindow * 0.8) {
  const keep = messages.slice(-10)
  const summary = await summarize(messages.slice(0, -10))
  messages = [summaryMessage(summary), ...keep]
}
```

What the real one adds:

Addition	Why it exists
<code>replace</code> instead of reassignment	The log is permanent; the surface is the view
Prune before summarizing	The cheap deterministic fix often suffices
Tool-pair-balanced boundary	Splitting a call from its result invalidates the next request
Durable <code>compaction/start</code> lock	A mutex does not survive a crash or show up in a log
Prefix reuse for the summarization call	Keeps the provider's KV cache warm
Refusing a non-shrinking compaction	Otherwise a terse session compacts repeatedly for nothing
Overflow trigger + <code>{kind: 'retry'}</code>	The user should not lose a turn to a window they cannot see
<code>replaceGeneration</code> check before claiming retry	Only claim a retry if something actually changed
<code>compaction/end</code> even on failure	A wedged lock would freeze the session
Full provenance in <code>sourceEventSeqs</code>	The whole transaction stays reconstructable

Key takeaways

- Two triggers: pressure at every step boundary, and overflow recovery driven by the provider's own error.
- Cheap first — model-free pruning runs before any summarization, and often ends the matter.
- The range always retains recent history and never splits a tool call from its result.
- `compaction/start` is a durable lock that survives a crash and is always released.
- The summarization call deliberately replays the conversation's own prefix to keep the provider cache warm.
- A compaction that would not shrink anything is refused.
- Nothing is deleted — `replace` shadows, and the human transcript is unaffected.

Exercises

1. Pruning brings a session from 85% to 78% of the window. Trace what happens next, and say how many LLM calls were made.
2. Compaction commits, and the next step builds a request. Name the four things that change as a consequence — one each from Chapters 7, 10, 21, and 26.
3. The overflow path is capped at one retry per failure. Construct the loop that would happen without the cap, and say which reset would break it.

Next: [Chapter 28 · Subagents](#)

Chapter 28 • Subagents

What you'll learn: how the engine delegates to a copy of itself, the two ways a child can start, and how results come back.

Prerequisites: [Chapter 14](#), [Chapter 17](#).

1. The problem

Some work is better done in a separate conversation. Exploring four possible causes of a bug pollutes the main history with three dead ends. A long mechanical task fills the context window with detail nobody needs afterwards.

The obvious approach — a "sub-conversation" abstraction with its own simplified loop — creates a second engine to maintain, and the child immediately wants everything the parent has: tools, a sandbox, cancellation, persistence, compaction. You end up reimplementing the loop badly.

The alternative is recursion: a child is a *real* agent, built by the same factory, running the same loop. Then the questions become interesting: what does it inherit, how deep can it go, and how does its answer get back?

2. Mental model

New term — subagent. A genuine nested `Agent` created through the ordinary agent factory.

```
const handle = await parent.ctx.agents.create({
  sessionId: childId, meta: ..., agentOptions: ..., signal, setup,
})
```

— `packages/subagent/subagent-in-process-driver/src/index.ts:133-140`

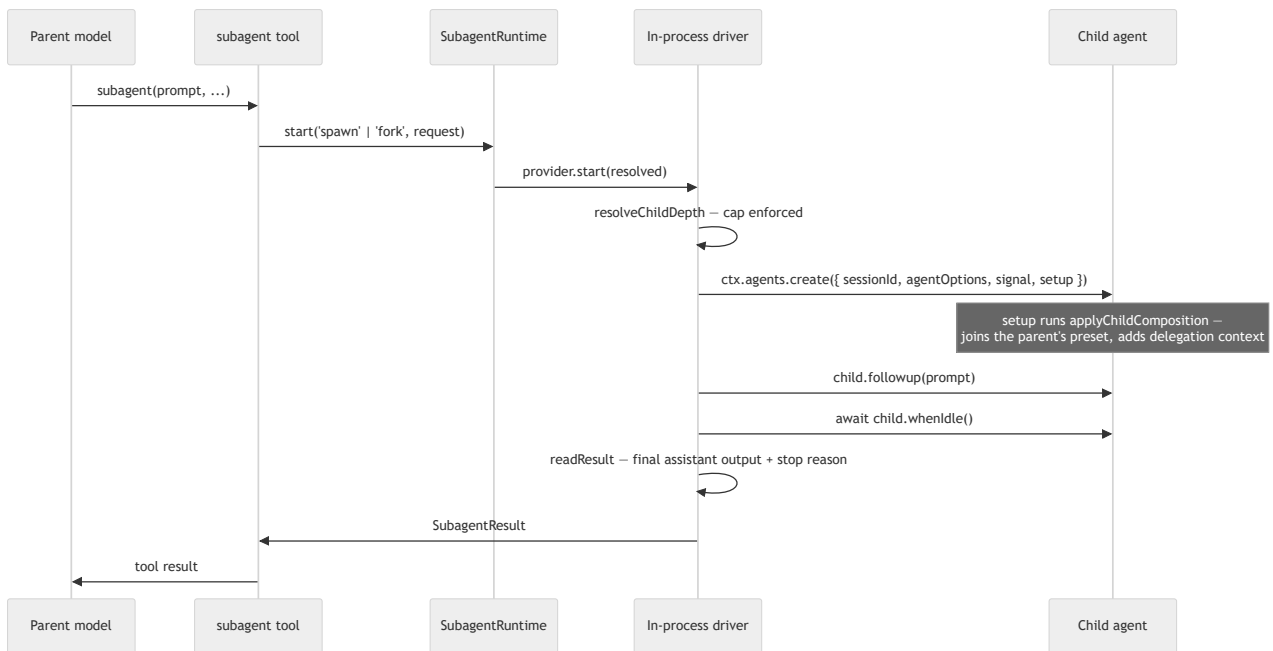
That is `createAgent` from [Chapter 14](#). **There is no separate lightweight execution path.** A child has its own session, its own log, its own inbox, its own compaction, and can itself delegate.

Two providers, differing in exactly one thing:

Provider	Child's session starts	For
spawn	empty	independent work; nothing to mislead it
fork	seeded with the parent's completed turns	work needing context; the shared prefix stays KV-cache eligible

`spawn` declares `inheritsParentContext = false` (`packages/subagent/subagent-spawn-in-process/src/index.ts:50`); `fork` supplies `seed?: SessionEvent[]` to the same driver (`:69-72`).

3. Lifecycle



4. Creating a child

`SubagentRuntime` (`ctx.subagents`, `packages/subagent/subagent/src/index.ts:197-651`) is a named-provider registry: `start(name, request)` (`:565-577`) resolves a provider and delegates.

Depth capping

`resolveChildDepth` (`packages/subagent/subagent/src/child-agent.ts:49-58`) throws `SubagentDepthError` past the limit — and does so using **the persisted parent depth as a floor**. So a resumed parent cannot escape the cap by forgetting how deep it already was. The depth lives in the session header (`delegationDepth`, [Ch 5](#)), which is why it survives.

Composition

`applyChildComposition` (`child-agent.ts:199-218`) runs inside the child's creation window — the `setup` hook from [Chapter 14](#), before publication, so the child is never visible without its tools.

The critical line:

```
childCtx.get('agentPresets')?.composeFrom(childCtx, parent.ctx)
```

— `child-agent.ts:204`

The child **joins the parent's mounted preset** rather than getting an empty plane. A subagent spawned under the `standard` preset has the same tool roster its parent has ([Ch 34](#)).

It also injects a fixed `SUBAGENT_DELEGATION_CONTEXT` (:171-176) as a runtime-context section (Ch 21, order 120) telling the model its permission scope is fixed and cannot be widened. The child is told, in the prompt, not to attempt escalation — consistent with Chapter 18's pattern of keeping enforcement and instruction together.

5. Running and returning

`drivePublishedRun` (`subagent-in-process-driver/src/index.ts:155-206`):

1. `child.followup(prompt)` — the prompt becomes one turn's worth of input (Ch 12);
2. `await child.whenIdle()` — genuine quiescence (Ch 13);
3. `readResult()` (:209-234) extracts the final assistant output from the child's **own session events** after its activation boundary (`finalAssistantOutput()`), and maps the turn-end reason to a `SubagentStopReason` via `toStopReason()` (:49-66): `completed` | `max-tokens` | `aborted` | `refusal` | `error`.

The result is read *from the child's log*, not from a return value threaded through the loop. Consistent with everything else: the log is the interface.

That `SubagentResult` becomes the parent's tool result — so from the parent model's perspective, delegation is just a tool call that took a while.

Continuable children

For background mode, results flow instead through `SubagentContinuationManager` (`packages/subagent/subagent/src/continuation.ts`) and an explicit `reportFrom()` (:296-302) the child can call to push content into its parent's inbox **at any time**, not only at turn end. That is the `tool-subagent-report` package, which registers a "continuable setup" on the runtime.

`tool-subagent-report` is host-plane, and the reason is precise: it registers a setup contribution on a process singleton rather than a tool the agent calls, and the setup list is not scope-aware — "one copy per mounted preset means every child gets `report` registered once per live session, which throws on the second" (`packages/bundle/web-app/cordis.patch.yml:420-424`).

Structured output

The in-process driver's structured-output tool calls `exec.concludeTurn()` immediately after staging the model's answer (`packages/subagent/subagent-in-process-driver/src/structured.ts:94`). That is the production user of `concludesTurn` (Ch 17): once the child has produced its structured answer, no further steps run.

6. What is mounted

Row	Plane	Note
<code>subagent</code> (registry)	host	process singleton with a cross-session query surface the browser reads
<code>subagent-spawn-in-process</code> , <code>subagent-fork-in-process</code>	host	provider names must be globally unique
<code>tool-subagent-report</code>	host	registers a continuable setup, not a tool
<code>tool-subagent</code> (<code>subagent</code> , provider <code>spawn</code>)	preset	<code>backgroundMode: continuable</code> , <code>modelSelectionSettings: true</code>
<code>tool-subagent-fork</code> (<code>subagent_fork</code> , provider <code>fork</code>)	preset	<code>backgroundMode: continuable</code>
<code>tool-subagent-control</code> , <code>list-agents</code>	preset	the control surface

— `presets/standard/agent.cordis.yml:174-217`

The registry stays host-plane because it is a singleton whose `listChildren` / `followup` surface the API serves to the browser, and because a provider name may only be registered once. What a preset chooses is **which delegation tools its agent sees**.

Note a config difference the three-layer composition makes easy to get wrong: the base bundle declares `tool-subagent-fork` with `backgroundMode: one-shot` , but the base row is disabled in web and **the preset re-declares it as** `continuable` (`preset:198-203`). The preset's value is what runs. The preset's own comment acknowledges the tradeoff — a continuable fork's `report` tool and prompt section precede the inherited history and invalidate the same prefix it was forked to preserve, with an issue tracking cache-preserving continuable fork.

7. Control decisions

Decision	Condition	Location
Throw <code>SubagentDepthError</code>	depth cap exceeded, floored by persisted depth	<code>child-agent.ts:49-58</code>
Seed the child's log	provider is <code>fork</code>	<code>driver :69-72</code>
Join the parent's preset	<code>agentPresets</code> is composed	<code>child-agent.ts:204</code>
Conclude the child's turn	structured output staged	<code>structured.ts:94</code>
Report at turn end	one-shot mode	<code>driver:155-206</code>
Report at any time	continuable mode	<code>continuation.ts:296-302</code>

8. Edge cases

A child is a full agent, so everything recurses. It compacts, retries, gets approval prompts, and persists — independently. A long-running child can compact its own history while the parent waits.

Cancellation propagates through the signal. The child is created with the parent's signal, so cancelling the parent's turn cancels the child's lifecycle via the fused abort of [Chapter 14](#).

Fork's cache benefit is fragile. The point of seeding is that the shared prefix remains KV-cache eligible. Anything prepended to the child's history — a `report` tool, an extra prompt section — invalidates it. The shipped preset accepts that cost.

The cap is 3 by default, and it is the delegation tool's config, not the runtime's:

```
maxDepth: z.union([z.natural().max(Number.MAX_SAFE_INTEGER), z.const('provider-managed' as const)]).default(3)
```

— `packages/subagent/tool-subagent/src/index.ts:129`

So a root agent may delegate, its child may delegate, and that grandchild may delegate — the fourth level throws `SubagentDepthError`. `resolveChildDepth` treats the cap as optional (`number | undefined`, `child-agent.ts:49`); when nothing supplies one there is no limit beyond the safe-integer range. The tool always supplies one unless configured to `'provider-managed'`, which hands the budget to the provider — and the tool **refuses to mount** a numeric cap against a provider lacking the `depthLimit` capability, with an actionable message (`:323–326`).

The persisted floor is the interesting part:

```
return Math.max(agent.session.header.delegationDepth ?? 0, runtime ?? 0)
```

— `packages/subagent/subagent/src/depth.ts:36`

The header value and the runtime option, whichever is larger. The comment states why: "runtime `AgentOptions.subagentDepth` may DEEPEN the count but can never lower it — a resumed child arrives with fresh options, and counting it from zero would let it delegate as if it were top-level."

9. Interactions

- [Ch 14](#) — a child is created through the same factory, `setup` and all.
- [Ch 12](#) — `followup` delivers the prompt; `reportFrom` pushes into the parent's inbox.
- [Ch 13](#) — `whenIdle()` is how the driver knows the child finished.
- [Ch 17](#) — `concludesTurn` ends the child's turn.
- [Ch 21](#) — the delegation context section.
- [Ch 34](#) — how a child joins its parent's preset.

10. Build it yourself

Minimal version:

```
async function subagent(parentCtx: Context, prompt: UserMessage): Promise<string> {
  const { agent, dispose } = await parentCtx.agents.create({ sessionId: freshId() })
  try {
    agent.followup(prompt)
    await agent.whenIdle()
    return finalAssistantText(agent.session)
  } finally { await dispose() }
}
```

That is genuinely most of it — which is the point. What the real one adds:

Addition	Why it exists
Named providers	<code>spawn</code> and <code>fork</code> differ only in seeding; more backends can register
Depth cap floored by persisted depth	A resumed parent must not forget how deep it is
Preset composition via <code>composeFrom</code>	A child with no tools is useless
Delegation context section	The child must know its scope is fixed
Stop-reason mapping	"It stopped" is not enough; the parent needs to know why
Continuation channel	A background child should report before it finishes
Host-plane registry	Cross-session queries and unique provider names

Key takeaways

- A subagent is a real `Agent` from the same factory — no second engine.
- `spawn` starts empty; `fork` seeds the parent's completed turns for cache reuse.
- Depth is capped using the persisted parent depth as a floor, so resume cannot escape it.
- The child joins the parent's preset during `setup`, before publication.
- Results are read from the child's own log, not returned through the loop.
- The registry and providers are host-plane singletons; only the delegation *tools* are per-preset.

Exercises

1. A forked child inherits history for KV-cache reuse, then the preset adds a `report` tool ahead of that history. Explain precisely what is lost and why the shipped preset accepts it.
2. The driver awaits `whenIdle()` rather than a promise from the loop. Name two things this gets right that a returned promise would not ([Ch 13](#)).

3. The child's result is read from its session log. Design the alternative where the loop returns it directly, and name two capabilities you would lose.

Next: [Chapter 29](#) · Persistence

Chapter 29 • Persistence

What you'll learn: where sessions actually live on disk, why the durable format is one append-only file, and the careful dance that stops two processes clobbering each other.

Prerequisites: [Chapter 5](#), [Chapter 14](#).

1. The problem

An append-only log is an unusually good fit for storage: the write pattern is "add a line," which every filesystem does well. But the gap between that observation and a correct implementation contains most of the hard parts.

A process can die **mid-write**, leaving a half-written line. Two processes can open the same session. A crash leaves a turn that started and never ended — the log is syntactically fine and semantically broken. And a long session must be resumable without re-reading a hundred megabytes.

2. Mental model

Two layers, deliberately split.

A **backend** knows about bytes. The contract is small (`packages/session/session-persistence/src/coordinator.ts:128-219`): `loadStored` , `readStoredRevision` , `appendBatch` , `commitRepair` , `list` , plus optional `loadStoredFrom` (seek-capable suffix read), `materializeHeader` , `locate` , and `close` .

The **coordinator** knows about correctness. Batching, per-id serialization, crash-repair sequencing, an LRU of unpublished prepared sessions, and dispose-time draining all live in `PersistenceCoordinator` , which every first-party backend composes rather than reimplements:

```
new PersistenceCoordinator(ctx, this, options)
```

— `packages/session/session-persistence-jsonl/src/index.ts:162-165`

So "how do I store bytes" and "how do I not corrupt a session" are separate questions with separate answers.

3. What actually runs

JSONL, one file per session. Verified two ways: the plugin is mounted in the base bundle —

```
- id: session-persistence-jsonl
  name: '@deepseek-ai/dsh-session-persistence-jsonl'
  config:
    root: !!js dshHomePath('sessions')
```

— `packages/bundle/base/cordis.patch.yml:110-113`

— and it is not disabled or overridden by the web-app patch.

A file is a header line plus one line per event, grouped into human-readable project directories. Two encoding options: Zstandard compression (default on) and lossless chunk-run packing (default on, [Ch 30](#)).

SQLite exists but is not the log. `packages/session-query/session-query-sqlite` is a *search index* — "Concrete `ctx.sessionQuery` backend with SQLite FTS5 search" — and in the shipped profile it is mounted with:

```
path: ':memory:'
openAt: never
```

— `packages/bundle/base/cordis.patch.yml:129-133`, restated in web-app

`openAt: never` keeps `ctx.sessionQuery` mounted so exact reads, titles, and lineage traces stay available, while **SQLite is never opened at all** and content search fails with `SESSION_QUERY_SEARCH_DISABLED`. The web sidebar's search matches titles and workspace names only.

That is a good example of why this book checks composition rather than package names: "the sessions are in SQLite" would be wrong in two ways at once.

4. Writing safely

Two mechanisms in the JSONL backend, both worth knowing.

Materialization uses `link()` + `unlink()`, not `rename()` (`packages/session/session-persistence-jsonl/src/index.ts:560-565`). `rename()` silently replaces an existing file; `link()` fails with `EEXIST`. If two processes race to materialize the same session, one wins and the other gets a detectable error instead of silently destroying the winner's file.

Appends are `fsync'd` with rollback. `appendLines` (`:670-698`) records the pre-write size and, on failure, truncates back to it. A partial write is removed rather than left for the next reader to trip over.

Sequence contiguity is re-checked at the storage boundary. `appendCore` (`coordinator.ts:722-726`) verifies every event in a batch against `state.cursor + i` before writing — the fourth of the four checks from [Chapter 5](#).

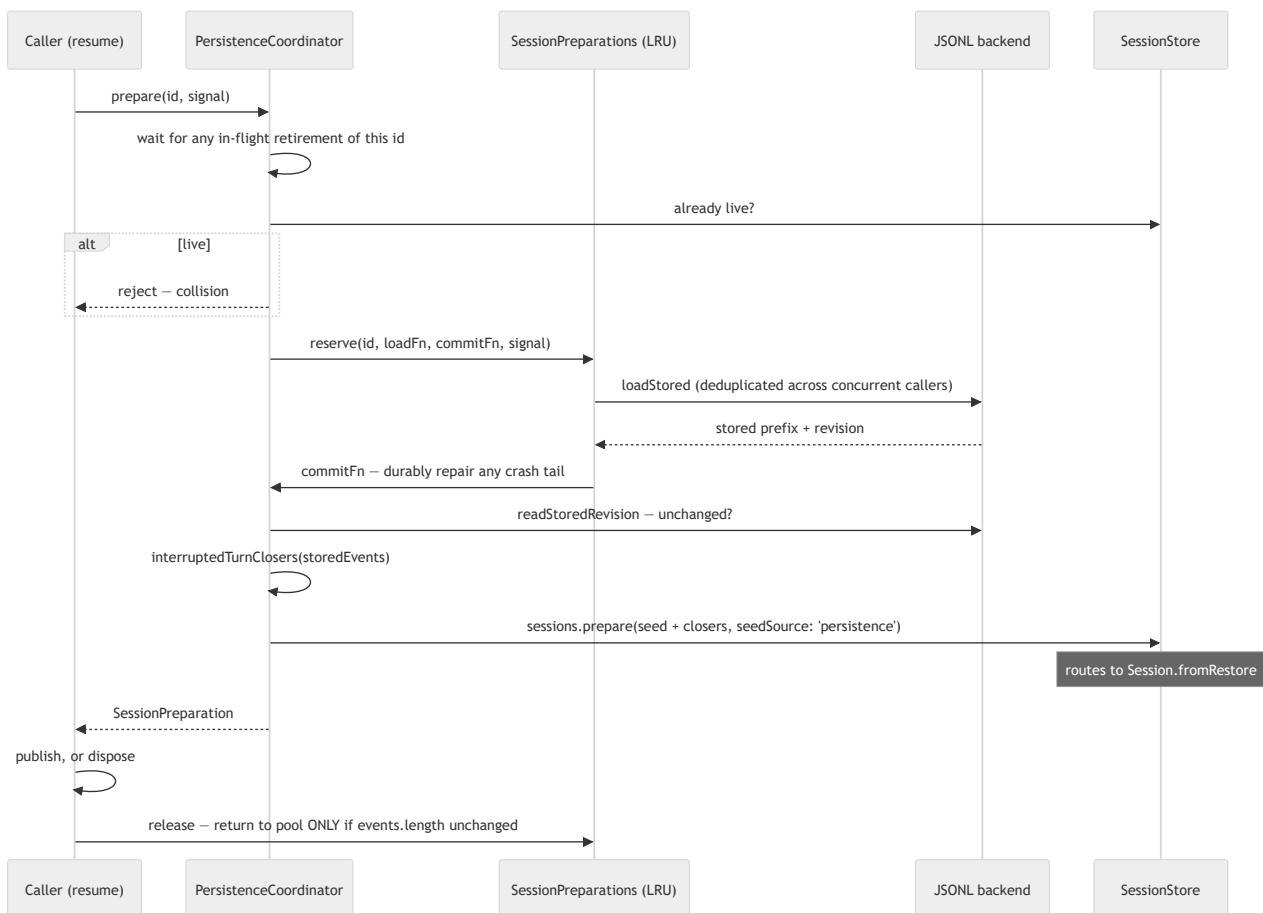
5. Reading safely

Two gates before any stored log is interpreted.

Version (`assertVersion` , `coordinator.ts:1128-1131`): a header whose `version` is not `SESSION_FORMAT_VERSION` throws `SessionFormatUnsupportedError` , with directional messages — "written by a newer harness — upgrade the harness" versus an older one with no upgrade path (`:78-82`).

Vocabulary (`assertEventsSupported` , `:1143-1148`): every event type must be in the generated known set, or be marked `ignorable` (Ch 5). An unrecognized required event refuses the whole session.

Both fail loud. Neither tries to guess.



6. Preparation and reservation

New term — `SessionPreparation`. A `Disposable` wrapper around one *unpublished* session (`packages/core/session/src/preparation.ts:20-49`).

```
export class SessionPreparation implements Disposable {
  readonly session: Session
  static create(session: Session, options?: SessionPreparationOptions): SessionPreparation
  [Symbol.dispose](): void { /* calls options.release?().() once */ }
}
```

It exists so a caller can build a fully validated session, decide whether to publish it, and cleanly release backend state if not — the ownership boundary [Chapter 14](#) relies on.

The generic `SessionPersistence.prepare` (`packages/session/session-persistence/src/index.ts:186-199`) just loads and wraps. The JSONL backend **overrides** it (`:190-192`) to use the coordinator's real path.

`PersistenceCoordinator.prepare` (`coordinator.ts:744-771`) loops until it has a race-free reservation:

1. **Wait** for any in-flight retirement of the same id.
2. **Reject** if the id is already live in `ctx.sessions` — two live sessions with one id is a collision, not something to resolve silently.
3. **Reserve** via `preparations.reserve(id, loadFn, commitFn, signal)` — a bounded structure that de-duplicates concurrent cold reads of the same id and hands one caller exclusive use. `commitFn` durably repairs any crash tail (`commitPrepared` , `:1016-1045`) and only returns once the durable revision is confirmed unchanged (`isPreparedSourceCurrent` , `:1048-1053`).
4. **Wrap** in a `SessionPreparation` whose `release` returns the reservation to the reusable pool **only if the caller never mutated it**:

```
reservation.source.session.events.length === reservation.source.sessionLength
```

— `:764-767`

An untouched preparation can serve the next caller; a touched one is discarded. That is the cheap half of making repeated open-attempts fast without ever serving stale state.

`prepareCore` (`:974-1013`) is the cold path: load the stored prefix, upgrade legacy shapes, compute crash-tail closers ([Ch 30](#)), append them to the seed, and construct via `ctx.sessions.prepare(id, { seed, meta, seedSource: 'persistence' })` — which routes to `Session.fromRestore` ([Ch 5](#)), the exclusive-ownership constructor that freezes restored graphs in place rather than re-validating them.

7. Adopting a live session

`onCreated` (`coordinator.ts:1318-1375`) handles a session that is already running in-process when the backend meets it — most commonly after a dev-server hot reload. Four documented cases (`:1305-1317`):

Case	Action
Already tracked	no-op, or claim ownerless state
On-disk prefix matches the live log	adopt and persist the live suffix
On-disk artifact mismatches	reject as a collision
Genuinely new	register and persist the seed

The second case routes through `adoptLivePrefix` (:1383–1405) and explicitly **does not** use `coldPrepareCore` — because that would crash-repair a turn the live session is still actively extending. Synthesizing "this turn was interrupted" closers into a turn that is currently running would corrupt it.

8. Control decisions

Decision	Condition	Location
Refuse the session	header version $\neq$ <code>SESSION_FORMAT_VERSION</code>	:1128–1131
Refuse the session	unknown required event type	:1143–1148
Refuse the write	batch seq $\neq$ <code>cursor + i</code>	:722–726
Roll back	append failed mid-write	jsonl :670–698
Fail materialization	the file already exists (<code>EEXIST</code>)	jsonl :560–565
Reject preparation	id is already live	:744–771
Reuse a reservation	the caller did not mutate it	:764–767
Adopt rather than repair	the live prefix matches disk	:1383–1405

9. Edge cases

The store is in-memory; persistence is a subscriber. `SessionStore` says so itself: "Persistence is intentionally not implemented here — persistence plugins subscribe to `session/event` and flush on `session/flush / dispose`" (`packages/core/session/src/index.ts:786–789`). A composition with no persistence backend is entirely valid; sessions simply do not survive the process.

Forking is a prefix copy. `SessionStore.fork()` (:1079–1093) creates a child from a prefix of a parent's log and **rejects a boundary inside an open turn** (`SessionForkError` , code `OPEN_TURN` , :1126–1133). Same reasoning as compaction's tool-pair balancing (Ch 27): a structurally incomplete prefix is not a valid conversation.

The throttled projection cache is separate. `session-projection-cache` writes checkpoints every 200 events or 5 seconds (`packages/bundle/base/cordis.patch.yml:162–166`) so a cold open need not re-fold a long log (Ch 8). It is an optimization over the log, never a substitute for it.

10. Configuration knobs

Setting	Default	Where
<code>root</code>	<code>dshHomePath('sessions')</code>	base:110-113
<code>compression</code>	<code>zstd</code>	jsonl backend
<code>packChunks</code>	<code>true</code>	jsonl backend (Ch 30)
<code>session-query-sqlite</code> <code>openAt</code>	<code>never</code>	base:129-133
projection cache cadence	200 events / 5000 ms	base:162-166

11. Interactions

- [Ch 5](#) — the format being stored; the fourth contiguity check lives here.
- [Ch 14](#) — `prepare()` and `SessionPreparation` ownership, including `releaseAbandoned`.
- [Ch 30](#) — repair and the packing codec.
- [Ch 8](#) — the checkpoint cache.

12. Build it yourself

Minimal version:

```
function append(sessionId: string, event: SessionEvent): void {
  appendFileSync(`${root}/${sessionId}.jsonl`, JSON.stringify(event) + '\n')
}
function load(sessionId: string): SessionEvent[] {
  return readFileSync(`${root}/${sessionId}.jsonl`, 'utf8')
    .split('\n').filter(Boolean).map(line => JSON.parse(line))
}
```

Genuinely usable. What the real one adds, and the failure each prevents:

Addition	Failure it prevents
Backend / coordinator split	Every backend reimplementing crash handling, differently
<code>link()</code> over <code>rename()</code>	A second process silently destroying the first's file
<code>fsync</code> + truncate rollback	A half-written line breaking every future load
Seq contiguity at the write boundary	A gap reaching disk and failing much later
Version and vocabulary gates	Silently misinterpreting a log from another build
Reservation with mutation check	Serving a stale prepared session to the next caller
Rejecting an already-live id	Two live sessions diverging under one identity
Live-prefix adoption	Crash-repairing a turn that is still running
Fork rejecting an open turn	Producing a child whose first request is invalid

Key takeaways

- The durable format is JSONL, one append-only file per session; SQLite is a search index and is never even opened in the shipped profile.
- Backends handle bytes; the shared coordinator handles correctness.
- `link()` + `unlink()` makes a materialization race detectable instead of destructive.
- Version and vocabulary are both gated on read, and both fail loud.
- A preparation is reusable only if untouched, and a live id can never be prepared over.
- Adopting a live session deliberately skips crash repair — the turn is not crashed, it is running.

Exercises

1. `rename()` would be simpler and atomic. Give the exact two-process interleaving where it loses data and `link()` does not.
2. A load finds a stored log whose last event is `step/start`. What happens before the session becomes usable, and which chapter covers it?
3. `openAt: never` keeps `ctx.sessionQuery` mounted but never opens SQLite. Why mount it at all rather than omitting the row?

Next: [Chapter 30 · Crash repair and chunk packing](#)

Chapter 30 • Crash repair and chunk packing

What you'll learn: how a session that died mid-turn becomes loadable again, and the lossless codec that stops a streamed response from costing 56× its own size.

Prerequisites: [Chapter 29](#), [Chapter 24](#).

1. Two problems

A turn that never ended. The process died between `turn/start` and `turn/end`. The log is syntactically perfect and semantically broken: a tool call with no result, a step with no end. Loading it as-is produces a conversation a provider will reject, and the model has no idea which of its requests actually ran.

A response costs more to store than it contains. Every streamed chunk is logged as its own event ([Chapter 9](#) ④) so the raw stream stays durable. But a chunk carrying three characters of text sits inside an envelope carrying its type, seq, time, turn, step, and block index. The module doc reports the measured ratio: ~56× on a real DeepSeek session.

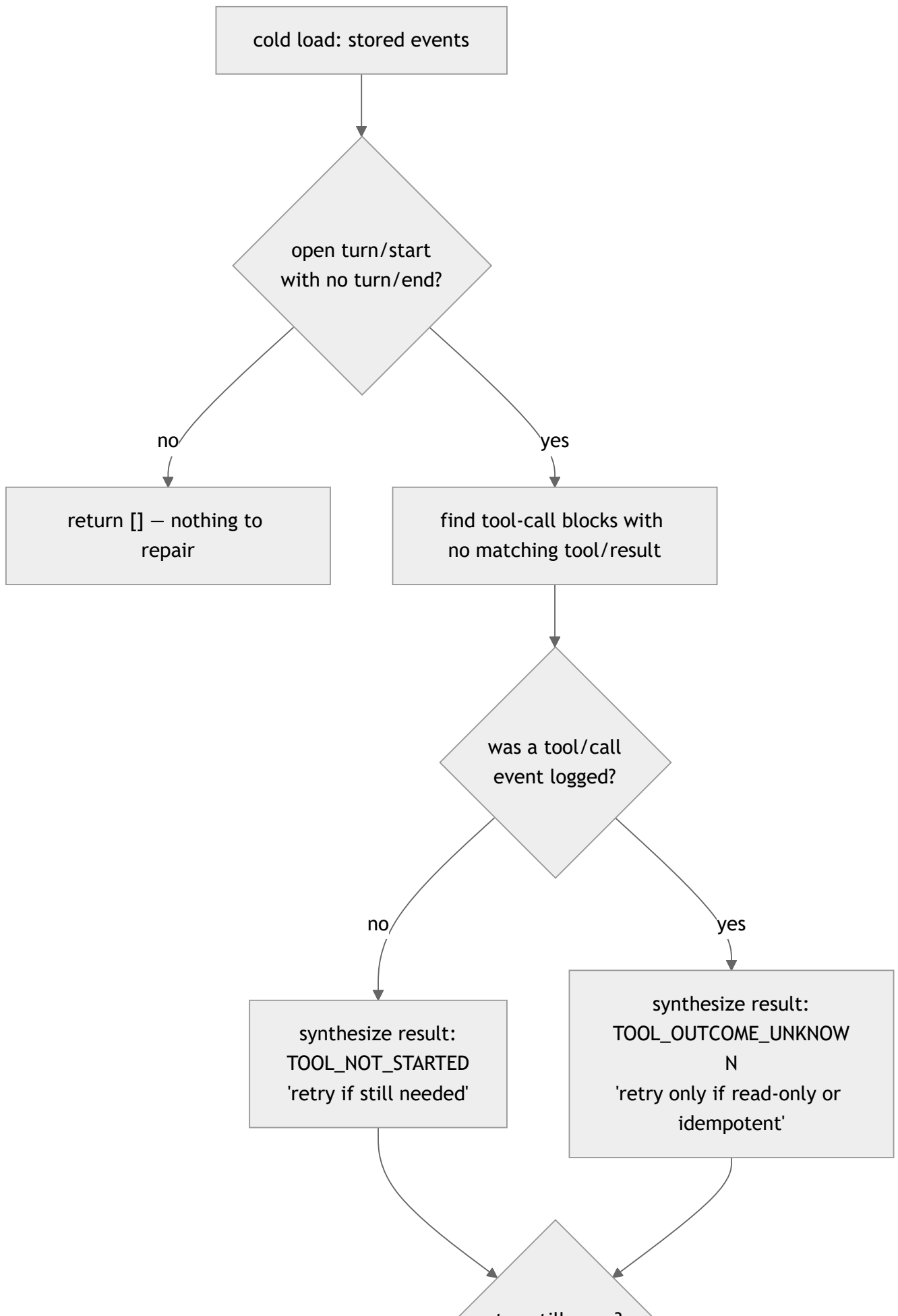
Different problems; they share a chapter because both are about the gap between "what the log means" and "what is on disk."

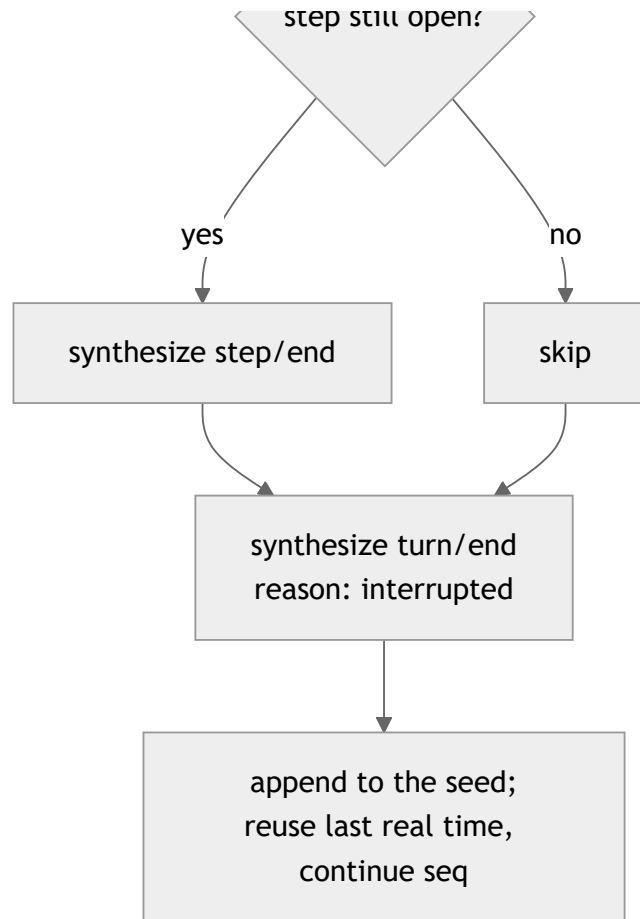
2. Crash repair

Mental model

Do not guess what happened. Instead, **close the turn honestly**, recording for each interrupted tool call exactly how much is known — and in particular, whether it might have run.

That distinction is the whole design. A tool the model asked for but which never started is safe to retry. A tool that started and whose outcome is unknown might have already deleted a file.





Note that the branch at **E** is decided purely by whether a `tool/call` event exists — which is why [Chapter 17](#) appends it *before* `prepare` runs.

The mechanism

`interruptedTurnClosers(events)` (`packages/core/session/src/repair.ts:28-134`) is a pure function: it scans a stored log for an unterminated tail and returns the events needed to balance it, or `[]` for an already-balanced log.

When it finds an open `turn/start` with no matching `turn/end`, it synthesizes:

① **One `tool/result` per still-pending call** — an `assistant/message` registered a `tool-call` block that never got a matching result — with one of two codes:

Code	Meaning	Guidance given to the model
<code>TOOL_NOT_STARTED</code> (<code>repair.ts:14</code>)	No <code>tool/call</code> event was ever logged	"Retry it if it is still needed."
<code>TOOL_OUTCOME_UNKNOWN</code> (<code>repair.ts:17</code>)	A <code>tool/call</code> was logged, so the tool may have run	"Decide whether to retry from the tool semantics: retry only if the operation is read-only or idempotent... Do not retry blindly."

That second message is the most careful piece of prose in the engine. The system genuinely does not know whether the call took effect, and rather than choosing for the model it hands over the one fact that matters and the criterion to reason with.

This is also why [Chapter 17](#) appends `tool/call before prepare` runs: the presence of that event is precisely what distinguishes the two cases here.

② A **synthetic** `step/end` if a step was open.

③ A **synthetic** `turn/end` with `reason: { kind: 'interrupted' }` — the **only** writer of that variant. The type documentation says so: "A persistence backend closed a crash-orphaned turn on reload. The loop never emits this marker" (`types.ts:165-168`). So `interrupted` in a log always means "this turn was cut off by a process death," never anything else.

Times and sequence

All synthetic events reuse the last real event's `time` — the code "never invents a 'future' time" (`repair.ts:85`) — and continue its `seq` sequence, preserving the contiguity of [Chapter 5](#).

Where it runs

Only on the **cold** path: `prepareCore` (`coordinator.ts:974-1013`) computes closers and appends them to the seed before constructing the session. A *live* session being adopted after a hot reload deliberately skips this ([Ch 29](#)) — its open turn is not crashed, it is running.

A torn *physical* tail — a half-written JSONL line — is a separate concern handled by the backend's own `commitRepair` /torn-marker mechanism (`packages/session/session-persistence-jsonl/src/index.ts:451-460`). Byte-level damage and semantic incompleteness get different fixes.

3. Chunk packing

Mental model

Runs of consecutive same-block delta chunks are packed into **one storage row**, and expanded back to the exact original events on read.

The module is unusually explicit about what these rows are not:

"Packed rows are an encoding vocabulary, NOT session events: they never enter `Session.events` , have no `SessionEventMap` entry, and use bare (slash-less) type tags so a reader cannot confuse them with the event taxonomy." — `packages/core/session/src/chunk-rows.ts:9-12`

That is why the three row tags are `text-chunks`, `reasoning-chunks`, and `tool-call-chunks` — no slash, unlike every real event type. A reader that sees `reasoning-chunks` and reaches for `SessionEventMap` finds nothing, which is the intended signal.

The format

```
export type ChunkRow =  
  | { type: 'text-chunks';      seq0: number; time0: number; data: TextRunData }  
  | { type: 'reasoning-chunks'; seq0: number; time0: number; data: TextRunData }  
  | { type: 'tool-call-chunks'; seq0: number; time0: number; data: ToolCallRunData }
```

— `chunk-rows.ts:66-69`

`seq0` and `time0` anchor the first member. `dt` holds epoch-ms **gaps**, one shorter than the member count. Member k reconstructs as seq `seq0 + k` and time `time0` plus the first k gaps (:39-46). Gaps may be negative — the wall clock can step backwards between events, and the format says so rather than assuming monotonicity.

Text runs carry `texts: string[]`, **one entry per member, never joined**: token boundaries are data, and joining would lose them.

`MIN_RUN = 3` (:99) — below three, a row's envelope rivals the lines it replaces. It is documented as a format constant rather than a tunable, because both layouts decode identically: changing it never invalidates stored logs.

Round-trip safety

This is where the module spends most of its code, and the care is instructive.

The encoder whitelists exact shapes. `classify` (:118-145) requires the event to have exactly the keys `{type, seq, time, data}`, `seq / time` to be safe integers, `data` to have exactly `{turn, step, chunk}`, and the chunk to have an exact key set per kind. Anything unrecognized falls through to verbatim storage — "unknown fields or future chunk variants lose compression, never data" (:14-15).

Runs only extend under strict conditions. `continues` (:158-173) requires consecutive seq, same turn/step, same block index, and:

```
if (!Number.isSafeInteger(next.time - prev.time)) return false
```

— :165

Two safe integers can sit further apart than a double can subtract exactly. A rounded gap would decode to a different timestamp, so such a pair simply does not pack. For tool-call runs, `name` must match in **presence and value** — a mixed run is not representable, so it is not attempted.

The decoder validates before expanding. `validateRow` (:270–312) checks the exact envelope keys, the payload arity against `dt.length` , and then walks the reconstruction bounds: every member seq and every accumulated time must stay a safe integer (:303–310). A malformed row **throws** rather than being treated as an event, because "treating it as an event would silently drop a whole run" (:356–358).

Every one of those checks exists to make one guarantee: **`decode(encode(x)) === x`**, exactly, or it refuses.

Where it runs

Persistence and bounded history transport both use it (:13). `decodeStorageRecord` (:363–370) expands rows back into `assistant/chunk` events before anything above sees them — so nothing outside storage ever knows packing happened.

4. A real packed row

`snapshots/session/text-turn/session.jsonl:15:`

```
{"type":"reasoning-chunks","data":{"turn":1,"step":1,"index":0,
  "dt":[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0],
  "texts":["The"," user"," wants"," me"," to"," reply"," with"," exactly",
    " the"," word"," \"","P","ONG","\\"," and"," not"," use"," any"," tools","." ]}}
```

Twenty reasoning deltas in one line. Nineteen gaps, all zero but one — the whole run arrived within two milliseconds. Decoded, this becomes twenty separate `assistant/chunk` events, which is what [Chapter 24](#)'s assembler consumes.

Note the tokens: `" \""` , `"P"` , `"ONG"` , `"\""` — the model's tokenizer split `"PONG"` across three chunks. Joining the strings would lose that, which is why the format keeps them separate.

5. Control decisions

Decision	Condition	Location
Return []	the stored log is already balanced	repair.ts:28-134
TOOL_NOT_STARTED	no tool/call was logged for the pending call	repair.ts:14
TOOL_OUTCOME_UNKNOWN	a tool/call was logged, no result	repair.ts:17
Skip repair	the session is live, not cold-loaded	coordinator.ts:1383-1405
Store verbatim	the event's shape is not whitelisted	chunk-rows.ts:118-145
Break a run	gap not a safe integer, differing index/turn/step, mixed name	chunk-rows.ts:158-173
Pack	run length ≥ 3	chunk-rows.ts:99, 214-243
Throw	a row-tagged value fails validation	chunk-rows.ts:270-312

6. Edge cases

Repair is idempotent in practice. It runs on cold load and its output is appended to the seed, so the restored session's log is already balanced — a second load finds nothing to do.

A complete final turn is untouched. Only a genuinely unterminated tail produces closers.

Packing is invisible above storage. `Session.events` never contains a row. Anything reading a raw JSONL file directly — including the snapshot fixtures — will see them, which is why [Chapter 3](#) warns that a fixture line is not always an event.

 **One unverified figure.** The "~56× measured on a real DeepSeek session" ratio is a claim in the module's own doc comment, not something this book measured. Cite it as the authors' figure.

7. Configuration knobs

Setting	Default	Effect
packChunks	true	Enables the codec in the JSONL backend
MIN_RUN	3	A format constant , not configurable

8. Interactions

- **Ch 29** — the cold path that invokes repair; the backend that applies packing.
- **Ch 24** — produces the chunk runs that get packed.
- **Ch 17** — appending `tool/call` before `prepare` is what makes the two repair codes distinguishable.
- **Ch 5** — synthetic events preserve seq contiguity.
- **Ch 3** — `turn/end interrupted` is written only here.

9. Build it yourself

Minimal repair:

```
function closers(events: SessionEvent[]): SessionEvent[] {
  const lastStart = findLast(events, e => e.type === 'turn/start')
  if (!lastStart || hasMatchingEnd(events, lastStart)) return []
  return [{ type: 'turn/end', data: { turn: lastStart.data.turn, reason: { kind: 'interrupted' } } },
    ... {}]
}
```

Minimal packing:

```
if (run.length >= 3) out.push({ type: 'text-chunks', seq0: run[0].seq, time0: run[0].time,
  data: { ..., texts: run.map(e => e.data.chunk.text) } })
```

What the real ones add:

Addition	Why it exists
Per-call result synthesis	A dangling tool call makes the next request invalid
Two distinct codes	Retrying a call that may have run is dangerous; the model must be told which case it is
Reusing the last real time	Inventing a future timestamp corrupts ordering
Skipping repair for live sessions	Repairing a running turn would corrupt it
Exact-shape whitelisting	An unrecognized variant must lose compression, never data
Safe-integer gap check	A rounded gap decodes to a different timestamp
Reconstruction-bounds validation	Catches the first departure from exactness rather than the tenth
Throwing on a malformed row	Treating it as an event would silently drop a whole run
Slash-less tags	A reader must not mistake a row for an event

Key takeaways

- Crash repair closes an orphaned turn by synthesizing results, a step end, and a `turn/end` marked `interrupted` — the only writer of that variant.
- It distinguishes "never started" from "outcome unknown" and hands the model the criterion rather than deciding for it.
- Repair runs only on cold load; a live session's open turn is running, not crashed.
- Chunk packing compresses runs of ≥ 3 same-block deltas into one storage row, losslessly.
- The codec whitelists exact shapes and refuses to pack anything whose round-trip it cannot prove exact.
- Packed rows are an encoding vocabulary, not events, and are expanded before anything above storage sees them.

Exercises

1. A crash happens after `tool/call` is appended but before the tool body runs. Which code does repair use, and is that the *safe* choice? Give a tool where the answer costs something.
2. `MIN_RUN` is documented as a format constant rather than a tunable. Justify that, using the fact that both layouts decode identically.
3. Packing requires `Number.isSafeInteger(next.time - prev.time)`. Construct a pair of timestamps that fails this, and say what would decode wrongly without the check.

Next: [Chapter 31 · One real turn, annotated](#)

Chapter 31 • One real turn, annotated

What you'll learn: every mechanism in this book, in motion, across one captured session — 25 lines of real log, start to finish.

Prerequisites: Part III. This chapter assumes the mechanisms and shows them cooperating.

The session

`snapshots/session/text-turn/` is a recorded session held in the repository as a replay fixture. The prompt was:

Reply with exactly the word: PONG. Do not use any tools.

It is about the smallest possible complete turn: no tools, one model call, one answer. That is why it is a good specimen — every event present is one the engine cannot skip.

The fixture is normalized. Session ids, paths, message ids, and the two largest payloads appear as placeholders: `{{session:1}}`, `{{cwd}}`, `{{message:N}}`, `{{system}}`, `{{tools}}`. Event order, chunk shapes, and token counts are as captured. The real system prompt and tool schemas sit beside it in `system-prompt.expected.md` and `tool-schemas.expected.json`.

The numbering matters more than it looks. `identity.ts` performs **relationship-preserving** redaction: each distinct id gets a token numbered by first appearance (`tokenByValue.set(value, \`
`{{{kind}}:${next}}}\`) , packages/test-support/session-snapshot/src/identity.ts:64). So {{message:1}} appearing in two different events is proof they reference the same message — the redaction scrubs the values while preserving the graph. {{cwd}}, {{system}}, and {{tools}} are flat constants from normalize.ts:20-22.`

That property is what makes the next section work.

Line numbers are not sequence numbers

One thing to establish before reading. The file has 25 lines, but the session has **43 events**, because line 15 is a packed chunk row holding twenty of them ([Ch 30](#)).

We can verify the mapping from the data itself, without trusting any assumption. Two independent checks:

- **Line 11** (`session/title`) carries `messageSeqs: [7]`, citing the user's message. Counting events from the header line, the user's message is at **seq 7**. ✓

- **Line 23** (`assistant/message`) carries `sourceEventSeqs` spanning **12 through 39** — 28 consecutive values. Counting the streaming events, the first `block-start` is at seq 12 and the `finish` at seq 39: $39 - 12 + 1 = 28$. ✓

Two citations, computed by different mechanisms at different times, agreeing with one arithmetic model. That is the provenance system of [Chapter 6](#) doing exactly what it was built for — and it is how this chapter can annotate seq numbers the file never prints.

The walkthrough

Line 1 — the header, which is not an event

```
{ "type": "session", "version": 0, "id": "{{session:1}}", "createdAt": 1783600629539, "cwd": "{{cwd}}", "delegationDepth": 0 }
```

Immutable creation metadata, deliberately kept **out** of the event log because it is "a storage concern, not replayable conversation state" ([Ch 5](#)). `delegationDepth: 0` means a root agent, not a subagent ([Ch 28](#)). The bare `session` tag is slash-less, like the packed chunk rows — the convention for "this line is storage, not an event."

Lines 2–4 (seq 0–2) — session policy

```
{ "type": "permission/preset", "data": { "preset": "danger-full-access" } }
{ "type": "sandbox/mode", "data": { "mode": "danger-full-access" } }
{ "type": "approval/policy", "data": { "policy": "never" } }
```

Three events from three different plugins, none of which the engine knows about ([Ch 2](#)). None is surface-eligible, so none reaches the model as a message — but their *effects* will, at line 10.

This snapshot runs with approval disabled, which [Chapter 18](#) showed is resolved **inline** rather than by any listener.

Line 5 (seq 3) — input arrives

```
{ "type": "agent/inbox/spliced", "data": { "target": "next-turn", "start": 0, "inserted": [ { ... } ] } }
```

`followup()` → `send(message, 'next-turn', true)` ([Ch 12](#)). The queue is durable: this event *is* the queue. The agent was idle, so the wake starts a driver rather than being latched ([Ch 13](#)).

Line 6 (seq 4) — the turn opens

```
{ "type": "turn/start", "data": { "turn": 1 } }
```

`turn()` entry (Ch 9). Turn 1, because the `turnBoundary` projection folded an empty log to `lastTurn: 0` (Ch 8).

Line 7 (seq 5) — the claim

```
{"type": "agent/inbox/spliced", "data": {"target": "next-turn", "start": 0, "removedCount": 1, "inserted": []}}
```

`claim('next-turn', 1)` popping **exactly one** item (Ch 12). Note the ordering: the claim is logged *before* `step/start`, because `preStep` runs before the step opens.

`removedCount: 1` with an empty `inserted` — a removal recorded as durably as an insertion. The inbox's whole history is in the log.

Line 8 (seq 6) — the step opens

```
{"type": "step/start", "data": {"turn": 1, "step": 1}}
```

Line 9 (seq 7) — the human's message

```
{"type": "user/message", "data": {"content": [{"type": "text", "text": "Reply with exactly the word: PONG. Do not use any tools."}], "source": {"kind": "user"}, ...}, "surfaceOp": "append"}
```

The claimed message, now durable and on the surface. `source: { kind: 'user' }` distinguishes it from what comes next.

Line 10 (seq 8) — the runtime-context snapshot

```
{"type": "user/message", "data": {"content": [{"type": "text", "text": "Current runtime context. This snapshot supersedes earlier runtime-context snapshots.\n\nCurrent DSH file policy: danger-full-access...\n\nApproval prompts are disabled in this session..."}], "source": {"kind": "plugin", "plugin": "@deepseek-ai/dsh-system-prompt", "form": "snapshot", "sections": [{"name": "sandbox:policy", ...}, {"name": "approval:policy", ...}], ...}, "surfaceOp": "append"}}
```

The chapter's best single illustration. This message:

- was produced by `RuntimeContextProjection.project()` because `retained` was `undefined` and the rendered context was non-empty (Ch 21);
- entered as the **innermost default** of the `agent/pre-step` waterfall, so every listener inherited it by delegating (Ch 22);
- carries the two sections that lines 3 and 4 caused, each attributed to its plugin;
- was appended by `turn()` as an ordinary `user/message` — **indistinguishable from line 9 except by its source tag**. No side channel.

Read the second section's text again: *"do not request sandbox escalation (do not set `sandbox_permissions`)"*. The policy from line 4 is being enforced *and* explained, so the model does not waste a turn attempting something that would be auto-rejected (Ch 18).

Line 11 (seq 9) — a title appears

```
{ "type": "session/title", "data": { "title": "Reply with exactly the word:", "messageSeqs": [7], "source": { "kind": "fallback" } } }
```

A plugin, not the engine. `source: { kind: 'fallback' }` means the LLM titler did not run; the fallback truncated the first message to five words. `messageSeqs: [7]` cites its source — the same provenance discipline, in a plugin the engine never heard of.

Lines 12–13 (seq 10–11) — the request is pinned

```
{ "type": "request/header", "data": { "header": { "config": { "provider": "deepseek-official", "model": "deepseek-v4-flash", "system": "{{system}}", "tools": "{{tools}}" }, "reason": "initial" } }, { "type": "request/context", "data": { "provider": "deepseek-official", "model": "deepseek-v4-flash" } } }
```

`reason: 'initial'` — first request of this loop instance over a log with no prior header (Ch 10). `system` and `tools` are the normalized placeholders; the real ones are 4.4 KB and 35.5 KB respectively.

At this moment the request's `messages` array is exactly **two entries** — seq 7 and seq 8 — derived by walking the surface (Ch 7). Everything else logged so far is not surface-eligible.

Had the invariant been mounted, this is where it would have compared those two messages against a fresh derivation (Ch 11).

Lines 14–22 (seq 12–39) — the stream

Line	Seq	Chunk
14	12	block-start index 0, reasoning
15	13–32	20 packed reasoning-delta
16	33	block-start index 1, text
17	34	text-delta "P"
18	35	text-delta "ONG"
19	36	block-end index 0
20	37	block-end index 1
21	38	usage
22	39	finish { kind: 'stop' }

Three things worth pausing on.

The blocks interleave. Block 1 starts (seq 33) and receives both its deltas (34, 35) *before* block 0 closes (36). Identity is the index, not arrival order — which is why the assembler keys partials by index (Ch 24).

"PONG" arrived as two deltas, and inside the packed reasoning row the model's tokenizer split the quoted word into `" \\" , "P" , "ONG" , "\\"`. The codec stores each member separately precisely so those boundaries survive (Ch 30).

Line 15's dt array is `[0,0,...,1,0,0,0]` — nineteen gaps, all zero but one. The whole reasoning block arrived within two milliseconds.

Line 23 (seq 40) — the assistant message

```
{ "type": "assistant/message", "data": { "turn": 1, "step": 1, "message": { "role": "assistant", "content":
[ { "type": "reasoning", "text": "The user wants me to reply with exactly the word \"PONG\" and not use
any tools. \" }, { "type": \"text\", \"text\": \"PONG\" } ], \"source\": { \"kind\": \"model\", \"provider\": \"deepseek-
official\", \"model\": \"deepseek-v4-flash\" } }, \"usage\":
{ \"inputTokens\": 3091, \"outputTokens\": 23, \"cacheReadTokens\": 0, \"reasoningTokens\": 20 } }, \"sourceEventSeqs\":
[ 12, 13, ..., 39 ], \"surfaceOp\": \"append\" }
```

The assembled result, citing **all 28 chunk events**. Both blocks survive because `finish.kind` was `stop`, not `max-tokens` — had it been the latter, any tool call would have been dropped (Ch 24).

The usage numbers are worth sitting with: **3,091 input tokens to produce 23 output tokens**. The input is almost entirely system prompt and tool schemas — 35.5 KB of tool JSON for a request that was told not to use tools. That is the standing cost of a stable tool catalog, and the reason Chapter 15 noted plan mode keeps tools listed rather than removing them: changing the catalog would change the header and invalidate the provider's prefix cache, which is what makes the *next* request cheap.

Lines 24–25 (seq 41–42) — closing

```
{ "type": "step/end", "data": { "turn": 1, "step": 1 } }
{ "type": "turn/end", "data": { "turn": 1, "reason": { "kind": "completed" } } }
```

The step returned `{ kind: 'completed' }` because the assistant message contained no tool-call blocks (Ch 9 ⑨). `turnEnds` was set, `inbox.nextStep` was empty, `agent/turn-stopping` dispatched to nobody (Ch 22), the re-test still found the queue empty, and the loop broke. `turn()` returned `false` — no pending input — and the driver went idle.

What the turn cost

Log lines	25
Events	43
Surface nodes at request time	2
Messages sent to the model	2
Model calls	1
Input tokens	3,091
Output tokens	23 (20 of them reasoning)

Forty-three events for one word. That ratio is not waste — it is the price of the property in [Chapter 1](#). Every one of those events is something a resumed, forked, compacted, or replayed session would need.

Which mechanisms were active

Mechanism	Where
Inbox	5, 7
Phase machine	5 (wake), 25 (idle)
Turn loop	6, 24, 25
Step loop	8, 24
Prompt assembly	before 12
Runtime context	10
Extension points	10 (pre-step default), 25 (turn-stopping, no listeners)
Request construction	12, 13
Adapters	before 14
Streaming + assembly	14–23
Surface	9, 10, 23
Derivation	before 12
Chunk packing	15
Projections	6 (turn number)
Token meter	21 (real usage recorded)

Not exercised: tools, approval, compaction, retry, subagents, crash repair. The next chapter covers the paths where those appear — by looking at what happens when things go wrong.

Key takeaways

- A minimal turn is 43 events; two of them reach the model.
- Two independent provenance citations confirm the seq mapping without assuming it.
- The runtime-context snapshot is an ordinary logged `user/message`, distinguished only by its source tag.
- Streaming blocks interleave; index is identity.
- 3,091 input tokens for a 23-token answer, almost all of it the stable tool catalog that keeps later requests cheap.
- Boundary events carry no model-visible content and exist to make the log parseable.

Exercises

1. Recompute the seq mapping yourself from line 1, then check it against both citations. Which line would shift if `MIN_RUN` were 2?
2. Suppose the model had emitted one tool call. List every additional event, in order, with its seq.
3. The request carried two messages but the log has 43 events. Which chapters explain each of the 41 that did not reach the model?

Next: [Chapter 32 · When things go wrong](#)

Chapter 32 · When things go wrong

What you'll learn: three failure paths traced end to end — one the system recovers from, one it unwinds cleanly, and one it cannot fix at all.

Prerequisites: Part III, and [Chapter 31](#) for the happy path.

Failures are where a design's real structure shows. A system that only works when nothing goes wrong has not been designed; it has been demonstrated.

A · Context overflow — recovery

The situation. A long session. The step loop builds a request, dispatches it, and the provider rejects it: the conversation no longer fits in the context window.

The trace

① **The failure is normalized.** The adapter throws. `normalizeLlmFailure` reads its properties via descriptors, recognizes the provider's error text through `isContextWindowExceededError`, and produces `{ code: 'CONTEXT_WINDOW_EXCEEDED', ... }` ([Ch 25](#)). It arrives as a terminal chunk: `{ type: 'finish', reason: { kind: 'error', failure } }`.

② **The step loop asks.** `finish.kind === 'error'`, so it dispatches the `agent/request-error` waterfall with a default of `undefined` — terminal ([Ch 9](#) ⑥).

③ **Compaction claims it.** Its listener checks the code and matches (`compaction-basic/src/index.ts:180-224`). It calls `compactIfNeeded(agent, 'context-overflow', signal)` — and because the trigger is overflow, **the 80% threshold check is skipped entirely**. The provider has already given a definitive answer; arguing with it via a character-count estimate would be absurd ([Ch 26](#)).

④ **Cheap first.** The tool-result pruner runs: every oversized `tool/result` on the surface is replaced in place with a head/tail excerpt, one `replace` op each, content-only. No model call. Re-measure ([Ch 27](#)).

⑤ **Summarize, if needed.** A durable `compaction/start` lock is appended. A range is selected retaining ~16% of the window verbatim, ending at a boundary that does not split a tool call from its result. The summarization request **replays the conversation's own system prompt and tools**, keeping the provider's prefix cache warm. The summary is size-checked — a compaction that would not shrink is refused — then committed as `compaction/summary` plus a `user/message` with `surfaceOp: { op: 'replace', start, end }`. `compaction/end` closes the transaction.

⑥ **The claim is honest.** Compaction returns `{ kind: 'retry' }` **only if** `replaceGeneration` **actually advanced**. If nothing changed, it calls `next()` and `llm-retry` gets its turn — which, since `CONTEXT_WINDOW_EXCEEDED` is not in the default retryable set, means delegating to the terminal default.

⑦ **The request is rebuilt.** `continue` re-enters the step loop at `buildRequest` (Ch 9). Four things differ:

- `deriveMessages()` rebuilds from scratch, because the generation changed (Ch 7);
- the header logs `reason: 'series'` even though its bytes are identical, because a surface rewrite starts a new series (Ch 10);
- the runtime-context snapshot may re-inject, if the shadowed range contained one (Ch 21);
- the prepared call is new — the old one was single-use (Ch 23).

⑧ **Bounded.** `maxOverflowRetries` defaults to `1`, tracked per agent and reset on the next `assistant/message` or when the agent goes idle. One rescue per failure, not a loop.

What the user saw

A pause. The turn completed.

What the log shows

```
turn/start · step/start · user/message · request/header(initial) · assistant/chunk...
compaction/prune · tool/result(replace) · compaction/start · compaction/summary
user/message(replace) · compaction/end
request/header(series) · assistant/chunk... · assistant/message · step/end · turn/end(completed)
```

Every step of the recovery is in the record, in order, and the superseded messages are still at their original positions.

B · Cancellation mid-stream — clean unwinding

The situation. The model is streaming. Three tool calls from the previous step are still running. The user presses stop.

The trace

① **The signal fires.** `cancel(cause)` clears the inbox, clears the wake latch, then aborts the current phase's controller (Ch 13). The cause travels with it.

② **Partial output is salvaged.** The streaming loop's abort check throws. The `catch` runs `assembler.interruptedBlocks()` — text and reasoning with real content, **tool calls omitted** because "interruption precedes dispatch; retaining one would require a fabricated result." If anything survives, it is appended as an `assistant/message` with `interrupted: true`, citing every chunk seq so far (Ch 24). Then the error rethrows: salvaging is not recovering.

- ③ **In-flight tools drain.** The scheduler does not abandon them. It awaits everything already dispatched, commits their results in **model order**, and accepts their `additionalContexts` (Ch 17).
- ④ **Unstarted calls get synthetic results.** Every call the scheduler never started receives a `tool/call` + `tool/result` pair with code `ABORTED_BEFORE_DISPATCH`. The module doc gives the reason: "so replay stays valid." A `tool/call` with no matching result would make the *next* request invalid — a cancelled turn must still be a well-formed conversation.
- ⑤ **The turn records why.** `turnEnds = { kind: 'aborted', reason: signal.reason }` — the cause, not just the fact. `step/end` and `turn/end` are appended by their `finally` blocks, so the log stays balanced (Ch 9).
- ⑥ **The driver returns to idle.** `kick()`'s empty `catch` contains the error — already reported via `agent/error` — and its `finally` sets the phase back to idle, replaying a latched wake only if the inbox still has something (Ch 13).
- ⑦ **A late message is not lost.** A user message arriving during the aborting window is retargeted to `next-turn` by `send`'s `wakingAfterAbort` check, computed *before* the splice so a reentrant cancel cannot reclassify it (Ch 12).

The property this preserves

A cancelled session is resumable. Balanced turn and step boundaries, every tool call paired with a result, partial output preserved and marked. Reopening it produces a valid request on the first try.

Where it can still stall

Convergence is **cooperative**. There is no watchdog. A tool ignoring its abort signal delays `whenIdle()` — and therefore disposal — indefinitely (Ch 13). The timeout-policy plugin bounds individual calls; the engine itself waits.

C • No provider configured — the failure it cannot fix

The situation. A fresh install. The user opens the web UI, types a message, and sends it.

This is the most surprising path in the system, and establishing it took reading three composition layers plus two plugins.

Why there is no provider

- ① **The native adapter is off.** The base bundle mounts `llm-deepseek`, but the web-app patch disables it:

```
- id: llm-deepseek
  disabled: true
```

— `packages/bundle/web-app/cordis.patch.yml:41-42`

with the comment: "The Web surface exposes only providers declared through the Models settings document."

② **The generic adapter is dormant.** `llm-pi-ai` is mounted, but its `providers` dict defaults to `{}` (`llm-pi-ai/src/config.ts:340-342`). Its `ensureRegistrationFacts()` runs once at mount, sees `routes.length === 0`, records the facts and **returns without registering anything** (`llm-pi-ai/src/index.ts:284-287`). The source calls this "the dormant posture."

So with no user configuration, **zero provider routes are registered.**

The trace

③ **prepareCall fails.** `registration(config.provider)` finds nothing → `LlmError('no adapter registered for provider "deepseek-official"', 'NO_ADAPTER')` ([Ch 23](#)).

④ **The loop tolerates it — narrowly.** `buildRequest` catches **only** `NO_ADAPTER` and proceeds with the unresolved config. `preparedCall` stays undefined ([Ch 10](#)).

⑤ **The fallback path fails identically.** `preparedCall?.stream(request) ?? this.loopCtx.llm.stream(request)` takes the bare call, which does its own registration lookup and throws `NO_ADAPTER` again — caught by the adapter-failure wrapper and turned into a terminal `error` finish chunk.

⑥ **Nothing retries.** The `agent/request-error` waterfall runs. Compaction checks the code — not `CONTEXT_WINDOW_EXCEEDED` — and delegates. `llm-retry` receives `retryPolicy: undefined`, because there was no prepared call to carry one, and its first line is `if (policy === undefined) return next()` ([Ch 25](#)). Nothing else listens.

⑦ **The turn fails.** The default `undefined` is terminal, so the step throws `LlmError`. `turn()` records `{ kind: 'error', error: failure }`, `throwError` emits `agent/error`, and the driver contains it.

The fix, and why it is not "re-enable the row"

A disabled Cordis row stays disabled. What makes a route live is a **settings write**:

`llm-pi-ai` installs a settings section named `llm-pi-ai` (`index.ts:296-330`). When the Models page writes it to `$DSH_HOME/settings.yaml` — hot-reloaded by `dsh-settings-file` — the section's `onChange` re-runs `ensureRegistrationFacts()`, which now sees a non-empty route set and calls `ctx.llm.registerAdapter(routes, adapter)` (`:288`). Later changes use `registration.replace(routes)` (`:290`), an atomic same-instance swap.

No restart, no composition change. And the failure handling is careful: `registeredFacts` only advances once the registry actually holds the new set, so a rejected update leaves the previous routes serving and returning to a working configuration re-applies (`:274-292`).

Why this is worth documenting

A reader who knows the default model is `deepseek-v4-flash` (Ch 23) would reasonably assume it works out of the box. It does not. The default names a *route*; whether that route is *registered* is a separate question answered by a different layer.

It is also a good demonstration of the composition hazard this book keeps returning to: **the base bundle mounts it, the surface bundle disables it, and a settings document revives it**. No single file tells you the answer.

What the three have in common

	A • Overflow	B • Cancellation	C • No provider
Recoverable?	yes, automatically	n/a — user-initiated	only by configuration
Who handles it	a plugin on an extension point	the loop's own <code>finally</code> blocks	nobody
Log stays valid	yes	yes	yes
Turn ending	<code>completed</code> after retry	<code>aborted</code> with cause	<code>error</code> with a code

All three leave a **resumable session**. That is the strongest claim the design makes, and each failure path tests a different part of it: A tests that history can be rewritten safely, B tests that boundaries close under abort, C tests that an unhandleable failure is still structured.

Key takeaways

- Overflow recovery is compaction claiming the retry seam — and it only claims it if the surface actually changed.
- A rebuilt request differs in four ways from the one that failed, each traceable to a different mechanism.
- Cancellation drains started tools and synthesizes results for unstarted ones, so a cancelled turn is still a valid conversation.
- Salvaged partial output is marked `interrupted: true` and the error still propagates.
- A fresh install has **no registered provider**; the first turn fails `NO_ADAPTER` and nothing retries it.
- The fix is a settings write that registers routes live, not a change to the disabled row.

Exercises

1. In path A, compaction runs but the summary is not smaller, so the commit is refused. Trace what happens next and give the turn's ending.
2. In path B, a tool ignores its signal and runs for ten minutes. What exactly is blocked, what still proceeds, and which config bounds it?

3. In path C, suppose `llm-retry` had *not* checked for an undefined policy. Describe the resulting behavior and say why the guard is load-bearing rather than defensive.

Next: [Chapter 33 · Adding a tool](#)

Chapter 33 · Adding a tool

What you'll learn: how to build a tool that behaves correctly in this engine, grounded in how the two most complex shipped tools are actually built.

Prerequisites: [Chapter 15](#), [Chapter 16](#), [Chapter 17](#).

1. The decisions

Writing the function is the easy part. Six decisions determine whether the tool behaves correctly under replay, cancellation, concurrency, and compaction. Get them wrong and the failures appear weeks later, in sessions nobody can reopen.

Decision	Wrong answer costs you
What is the canonical value ?	replay cannot re-render; the UI has nothing structured
Is it concurrency-safe ?	data races, or needless serialization
Does it need approval ?	either a security hole or an unusable tool
Does it belong on the host plane or in a preset ?	a service collision, or an invisible registry
What does it present ?	a wall of text where a diff belongs
Should it conclude the turn ?	almost always no

2. The skeleton

```
import { defineTool } from '@deepseek-ai/dsh-tools'

export function apply(ctx: Context, config: Config): void {
  ctx.tools.register(defineTool({
    name: 'my_tool',
    description: buildDescription(config),
    parameters: { /* schema DSL */ },
    output: {
      schema: { /* what execute returns */ },
      render: (args, value) => [{ type: 'text', text: format(value) }],
    },
    async execute(args, exec) {
      return doTheWork(args, exec)
    },
    isConcurrencySafe: () => true,
  }))
}
```

`register()` returns the disposer and runs inside `ctx.effect` — unmounting the plugin removes the tool with no bookkeeping (Ch 15). The `apply()` function is already inside the plugin's effect scope, so both levels unwind together.

3. Value, not prose

The single most consequential choice. `execute` returns a **canonical value**; `output.render` projects it into what the model reads.

Compare the two shipped extremes.

bash returns structure. Its `output.schema` is a `oneOf` of two shapes (`packages/shell/tool-bash/src/index.ts:270-321`):

```
{ kind: 'background', jobId }
{ kind: 'foreground', exitCode, signal, timedOut, aborted, timeoutMs, stdout, stderr, sandbox? }
```

`execute` (:329-389) returns one of those; `render` turns it into text. Because the exit code is a *field*, the UI presenter can parse it into a status pill, and replay re-renders identically without re-running anything.

str_replace_editor returns a string. `output.schema: { type: 'string' }`, and `render` wraps it in one text block (`packages/fs/tool-str-replace-editor/src/index.ts:469`). Simpler, and it gives up the structured affordances.

The rule: if any consumer — a UI, a policy, a future presenter — would want to branch on part of the result, that part belongs in the value, not in formatted prose.

And remember from [Chapter 16](#): `value` **never reaches the durable log**. Only the rendered content, `error.info`, and `meta` are persisted. If a presenter needs data at replay time, put it in `meta`.

4. Concurrency

```
isConcurrencySafe: () => true
```

Declaring it opts into the parallel pool. Omitting it means exclusive — and classification is **fail-closed**: no declaration, a throwing classifier, or any non- `true` return all yield exclusive (Ch 17).

Shipped choices:

Tool	Declares	Reasoning
<code>read (tool-fs)</code>	<code>() => true</code>	"Observation races fail closed because guarded mutations re-check the version in-lock"
<code>web_search</code>	<code>() => true</code>	"Provider reads do not mutate parent-agent state"
<code>bash</code>	—	arbitrary commands; unsafe by default
<code>str_replace_editor</code>	—	even for its read-only <code>view</code> command

That last row is a real decision worth weighing. `str_replace_editor` could classify per-argument (`args.command === 'view'`), and the mechanism supports it — but it does not. Two first-party file-reading tools therefore make opposite choices.

The conservative default is right: a wrong "safe" is a data race; a wrong "unsafe" is only slower.

If you do classify per-argument, note that `defineTool` wraps the classifier **softly** — invalid arguments yield `false` rather than throwing (Ch 15), because it runs against historical arguments on replay.

5. Approval

Two options, and Chapter 18 established which one ships.

The pipeline seam — a `tools/pre-execute` listener returning `{ kind: 'ask' }` — is fully implemented and has **no always-on producer**. Use it for a deployment-wide policy layer, not for one tool.

In-body escalation is what shipped tools do:

```
const approvedMode = args.sandbox_permissions !== undefined && args.justification !== undefined
  ? await approveBashEscalation(args.sandbox_permissions, args.justification, exec, standingPolicy)
  : undefined
```

— `tool-bash/src/index.ts:329-338`

Three properties worth copying:

1. **The request is part of the call.** `sandbox_permissions` and `justification` are ordinary parameters, so both are logged in the `tool/call` event. The model's stated reason is durable.
2. **The parameters are conditionally advertised.** They appear in the schema only when something actually confines the tool (`escalationModes.length > 0`, :192). Never tell the model about a capability that cannot apply.
3. **A denial is just an error.** `approveEscalation` throws; `dispatchToolBody` catches it; the model reads `Error: the user rejected...` and can react in the same turn.

6. Host plane or preset

A composition question with a concrete test, stated in the standard preset's own comments:

A row that **publishes a service** must sit inside an `isolate` realm, or it becomes process-global and collides with another preset's copy.

A **registry that something outside the preset reads** belongs to the host plane.

Worked examples from the shipped composition:

Row	Plane	Why
<code>tools</code> , <code>skill</code> registries	host	preset rows register <i>into</i> them
<code>jobs</code> registry	host	<code>tool-bash</code> (a preset row) resolves it with <code>ctx.get</code> ; a realm would make it invisible
<code>subagents</code> registry	host	process singleton; the API serves cross-session queries from it
<code>token-meter</code>	host	owns projection units the browser reads for every session
<code>tool-bash</code> , <code>tool-fs</code> , <code>tool-web</code>	preset	register into the host registry, publish nothing → need no realm
<code>compaction-basic</code> + <code>tool-result-pruner</code>	preset, shared realm	the engine reads the pruner via <code>ctx.get</code> , so they must share one
<code>plan-mode</code>	preset, own realm	plan state is per-agent by nature

The default for a new tool is a preset row that registers into the host registry and publishes nothing. That needs no realm at all.

7. Presentation

Two optional pure functions, both run against possibly-historical arguments:

```
presentCall?(args): ToolCallView | undefined
presentResult?(args, result): ToolResultView | undefined
```

`bash` renders a terminal view, parsing its `[exit code: N]` marker into a pill. `str_replace_editor` returns a `DiffCallView` with `oldText` / `newText` for `create` and `str_replace`, and a generic view for `view` and `insert` (:376-423) — so the UI shows a diff for edits without the tool knowing anything about rendering.

They must be **pure functions of their inputs**. A presenter reading external state produces different output on replay than it did live.

8. A worked example

A tool reporting the current git branch.

```
ctx.tools.register(defineTool({
  name: 'git_branch',
  description: 'Report the current git branch and whether the working tree is clean.',
  parameters: { type: 'object', properties: {}, additionalProperties: false },

  // Structured: a consumer may want to branch on `clean`.
  output: {
    schema: {
      type: 'object',
      properties: { branch: { type: 'string' }, clean: { type: 'boolean' }, ahead: { type: 'number' } },
    },
    required: ['branch', 'clean', 'ahead'],
  },
  render: (_args, value) =>
    [{ type: 'text', text: `${value.branch}${value.clean ? '' : ' (dirty)'}, ${value.ahead} ahead` }],
  },

  // A read. No parent-agent state mutated.
  isConcurrencySafe: () => true,

  async execute(_args, exec) {
    const result = await ctx.shell.run({ command: 'git status --porcelain -b', signal: exec.signal })
    return parseStatus(result.stdout)
  },
}))
```

Checked against §1:

- **Value** — structured, so a UI can style `clean` and replay re-renders without running git.
- **Concurrency** — safe; it reads.
- **Approval** — none; it is non-mutating and inside the existing sandbox.
- **Plane** — a preset row registering into the host `tools` registry, publishing nothing. No realm.
- **Presentation** — the default render is adequate; a `presentResult` could add a branch chip later without touching `execute`.
- `concludesTurn` — no. Almost nothing should (Ch 17).

Note `exec.signal` is passed through. A tool that ignores it delays cancellation with no watchdog to save it (Ch 13).

9. What else you can reach

Need	Mechanism
Inject a message into the next step	<code>exec.deferContext(message)</code> → <code>additionalContexts</code> (Ch 17)
End the turn after this result	<code>exec.concludeTurn()</code> — only for a genuine terminal answer
Persist replay-time presentation data	<code>meta</code> on the result
Content even when the pipeline bypasses post-execute	<code>finalizeContent</code> — snapshotted at call creation
Bound the call's duration	<code>timeoutMs</code> — cooperative, enforced by a policy plugin, never shown to the model

10. The checklist

Before registering:

- ☐ Does `execute` return a **value**, with `render` doing the formatting?
- ☐ Would any consumer want to branch on part of the result? Then it is a field, not prose.
- ☐ Is `isConcurrencySafe` declared only if genuinely true for **every** argument shape?
- ☐ Is `exec.signal` threaded into everything that can block?
- ☐ Are presenters pure functions of their arguments?
- ☐ Does the description tell the model when *not* to use it? (Every shipped description does.)
- ☐ Does it publish a service? If so, it needs an `isolate` realm.
- ☐ Does anything outside a preset need to read it? If so, it belongs on the host plane.
- ☐ Have you resisted `concludeTurn()` ?

Key takeaways

- `execute` returns a canonical value; `render` produces what the model reads. That split is what makes replay and UI presentation possible.
- `value` is never persisted — use `meta` for replay-time data.
- Concurrency classification is fail-closed; declare safe only if it is true for every argument shape.
- Shipped tools gate themselves in-body, with the escalation request as ordinary logged parameters.
- The default placement is a preset row that registers into a host registry and publishes nothing.
- Presenters must be pure; they run against historical arguments.

Exercises

- Rewrite the `git_branch` example to return a formatted string instead of a value. List everything that breaks, citing the chapter for each.

2. `str_replace_editor` is exclusive even for `view`. Write the per-argument classifier that would fix it, then give two reasons the authors might still decline it.
3. Design a tool that must ask the user before running. Write it both ways — pipeline seam and in-body — and say which you would ship here, and why.

Next: [Chapter 34 · Composition in full](#)

Chapter 34 • Composition in full

What you'll learn: how ninety-odd plugins become a running system, why no file lists them, and the two unrelated isolation mechanisms that keep one agent's plugins out of another's.

Prerequisites: [Chapter 2](#). Deferred until now deliberately — this is far more interesting once you know what is being composed.

1. The problem

The engine is one plugin among many, and which others are present changes what the system is. A composition mechanism has to answer four questions at once:

- How does a deployment change one plugin's config without forking the whole list?
- How do two agents in one process have different tools?
- How does a per-agent plugin publish a service without colliding with another agent's copy?
- How does config that depends on runtime facts — the platform, an environment variable, a bound port — get evaluated at the right moment?

2. The boot chain

Grounded end to end:

```
apps/server/src/index.ts      - hardcodes profile: 'web' (:16)
  → args.ts:22 parseServerArgs - rejects --profile (:33-35)
  → profile-boot.ts:208 runProfile
    → composeProfile (:156)      - resolves bundles, collects patch layers
    → prepareProfile (:80-84)    - WRITES cordis.yml = the literal "[]"
  → app-boot/src/index.ts:772 boot
    → new Context()              (:779)
    → ctx.plugin(Loader)         (:786)
    → mountRootInclude (:789)    - one root entry: cordis:include + all patches
      → Include[Service.init]    - reads the file, applies patches
      → applyEntryPatches        - the patch algebra (§3)
      → EntryGroup.update        - creates every row CONCURRENTLY
        → Entry._start → registry.plugin → new Fiber
          → new AgentLoop(ctx, config)
```

Two things stand out.

The root config is empty. `prepareProfile` writes the literal string `[]` on every boot. The entire plugin graph arrives as *patches* over nothing.

A profile is not a file. `PROFILE_TEMPLATES.web` names two bundles:

```
web: { bundles: ['@deepseek-ai/dsh-base', '@deepseek-ai/dsh-web-app'], patchReload: 'live' }
```

— packages/boot/app-boot/src/profile.ts:137-142

and the patch stack is assembled in a fixed order (profile-boot.ts:136-143):

- 1. each bundle's cordis.patch.yml , in bundles order;
- 2. the profile's own cordis.patch.yml (empty by default, user-editable, hot-reloaded);
- 3. \$DSH_HOME/cordis.patch.yml (machine-wide);
- 4. --patch overlays.

No file in the repository lists the web profile's composition. It exists only as the result of applying that stack.

3. The patch algebra

applyEntryPatches (vendor/include/src/index.ts:58-128) over an entry array:

Patch shape	Effect
{ insert: [...] } with no id	append rows to the top level (:94)
{ insert: [...] } with an id	append into that id's group config array (:91-92)
{ id, ...overrides }	copy each override key onto the matching row (:121-124)

The third is the one with consequences:

```
for (const [key, value] of Object.entries(overrides)) target[key] = value
```

config is replaced wholesale, not merged. A later layer restating config: discards every key the earlier layer set. Both bundle files say so in their headers, and the code confirms it. This is why the standard preset restates thresholdChars , headChars , and tailChars identically rather than relying on the base's values (Ch 27).

Inserted rows are indexed **immediately** (:101), so a later patch in the same list can target a row an earlier insert just added — a documented local modification to the vendored code.

Row order is not load order. Rows appear in array order, but a fiber activates only when its inject dependencies resolve, and EntryGroup.update creates every row concurrently (vendor/loader/src/config/group.ts:71). Startup sequencing is a dependency graph, not a list.

4. !!js

Config that depends on runtime facts is written as a tagged scalar:

```
disabled: !!js process.platform === 'win32'
mode: !!js process.env.DSH_PERMISSION_MODE ?? 'workspace-write'
root: !!js dshHomePath('sessions')
```

The dialect is defined once:

```
const JsExpr = new yaml.Type('tag:yaml.org,2002:js', {
  kind: 'scalar',
  resolve: (data) => typeof data === 'string',
  construct: (data) => ({ __jsExpr: data }),
  ...
})
export const entryListSchema = yaml.JSON_SCHEMA.extend(JsExpr)
```

— vendor/include/src/index.ts:9-23

Why `!!js` and never `!js`. `!!foo` is YAML's shorthand for the resolved tag `tag:yaml.org,2002:foo`; a single `!foo` is an *application-local* tag that never resolves to that URI. A single-bang `!js` would not match this `Type` at all. The repo's rule is mechanical, not stylistic.

A parsed expression becomes `{ __jsExpr: "<source>" }` — inert data until something evaluates it.

Evaluation is lazy and per row. The Loader's `internal/config` listener interpolates each entry's config (`vendor/loader/src/index.ts:92-101`) — but **skips tree-carrier plugins** (`Include`, `Group`, marked by `static readonly [EntryGroup.key] = true`, `include:182`), whose config is entry and patch lists that must stay literal.

So an expression referencing a service is resolved only after that row's declared injections are active. That is what lets the web bundle write:

```
- id: webserver
  inject: [webStartup]
  config:
    host: !!js ctx.webStartup.host ?? '127.0.0.1'
```

`disabled` gets the same treatment separately (`Entry.disabledOf`, `vendor/loader/src/config/entry.ts:100-108`), which is how platform gating works.

The evaluator itself is four lines:

```
export const evaluate = new Function('ctx', 'expr', `
  with (ctx) {
    ...
```

— vendor/loader/src/config/utils.ts:5-6

A `Function` constructor wrapping a `with (ctx)` block. So an expression's free identifiers resolve against the row's own context — which is exactly why `ctx.webStartup.host` works in a row that injects `webStartup`, and why an identifier the scope cannot resolve throws rather than silently yielding `undefined`.

It also means `!!js` is **arbitrary code execution at config-load time**. That is a deliberate property of a composition format meant to be edited by the deployment operator, and it puts `cordis.patch.yml` in the same trust class as the code it composes — the same reasoning the preset system applies to `$DSH_HOME/.agent-presets`, which "carries the same trust as shell access because a preset IS a composition."

5. Isolate realms

The first of two isolation mechanisms, and it is pure Cordis.

Normally a service registers under a process-wide symbol: `provide()` looks up `ctx.root[symbols.isolate][name] ??= Symbol(name)`, then stores the implementation at `this.store[ctx[symbols.isolate][name]]` (`vendor/cordis/src/reflect.ts:286-287`). Every reader resolves the same symbol, so the service is global.

An `isolate:` declaration changes which symbol that row's context maps a name to. The `isolate` Loader plugin (`vendor/loader/src/config/isolate.ts`) builds a new map for the declared names and **replaces the entry context's own isolate map** via `Object.setPrototypeOf + swap` (`:123-126`) before reloading the fiber.

Two realm kinds:

Declaration	Realm	Suffix
<code>isolate: { compaction: true }</code>	<code>LocalRealm</code> — this entry's private instance	<code>'#' + entry.options.id</code>
<code>isolate: { compaction: 'shared-label' }</code>	<code>GlobalRealm</code> — shared by every entry naming that label	<code>'@' + label</code>

A shared label does **not** pool instances: `provide()` throws on a second registration under the same realm symbol. Labels *join* realms — they let several entries agree to resolve one instance.

The key property, stated precisely: a service published with no `isolate:` is visible process-wide because its context inherits the root's symbol table unchanged. **Visibility is the default; a realm is an opt-in narrowing, never a widening.**

That is exactly why the standard preset's comments read as they do:

- "Both register into the host `tools` registry and provide nothing, so they need no realm" — a row that *publishes* nothing cannot collide.
- `compaction-basic` and `tool-result-pruner` share one realm because the former reads the latter via `ctx.get`.
- `plan-mode` gets its own realm because "plan state is per-agent by nature — an entry-local realm is not a workaround here, it is the correct lifetime."

And `mountPreset` **enforces** it: `leakedServices()` (`mount.ts:210-224`) walks the reflect store and flags any implementation whose isolate key equals the root's symbol for that name — a row that forgot its realm. A misconfigured preset fails session creation loudly rather than silently sharing one agent's service with every other.

6. Scope parentage — the second mechanism

Completely separate, and application-level. `packages/core/scope` is built from ordinary `ctx.extend()`, a private `WeakMap`, and Cordis's listener-filter hook. It touches `Context.isolate` not at all.

It provides two things:

Event routing. `scopeTarget()` (`scope/src/index.ts:170-185`) builds the dispatch filter: a listener registered under scope `T` receives events dispatched to scope `K` iff `T` is `K` or one of `K`'s ancestors ([Ch 22](#)).

Registry layering. `ScopedLayers` (`store.ts:159-267`) holds a global layer plus one per scope key. `merge()` composes `global` + `chainLayers(scope)` — farthest ancestor first, nearest last so it wins name collisions (`:192-217`). This is what makes a preset's prompt sections and tools per-agent ([Ch 19](#), [Ch 15](#)).

One relation, `scopeParents`, with two consumers: event delivery and registry layering.

7. Agent presets

Now the pieces combine.

The web bundle mounts the roster:

```
- id: agent-presets
  name: '@deepseek-ai/dsh-agent-presets'
  config:
    default: standard
```

— `packages/bundle/web-app/cordis.patch.yml:451-456`

and disables ~24 agent-plane rows, moving them behind presets.

Mounting, verified

The preset file's own comment claims it is "mounted once per process under a standing scope; every session naming it joins by scope parentage." The code says something **more precise**.

`ensureStanding(preset)` (`packages/preset/agent-presets/src/index.ts:747-795`) keys a `Map<string, Promise<StandingMount>>` by preset id:

1. A cached promise is **reused** — the "once" guarantee, implemented as an in-memory single-flight cache, not a Cordis-level singleton.

2. It also checks the composition **file's mtime/size stamp** on every call (`compositionStamp` , :806–816). If the file changed, the cached entry is evicted and a **new generation** created (:767). Sessions already joined keep running on their original generation's plugin instances (:759–763).
3. On a miss: mint a `ScopeKey` (a fresh plain object used purely for identity),
`createScope(this.selfCtx, key)`, then `await mountPreset(scope.ctx, preset)`.

So the accurate statement is "**once per (preset id, on-disk file generation), single-flighted and created lazily on first use.**" A live-edited preset forks a generation for new sessions.

`mountPreset` (`mount.ts:378–433`) does `agentCtx.plugin(PresetTree, config)` where `PresetTree` extends `Include` — the preset's entry tree plugged directly as one Cordis plugin, not routed through a `Loader` entry. It then verifies **every row reached a usable state** (`inactiveRows` , :304–322) and **no row leaked a service** (`leakedServices` , §5) before recording the mount.

Joining

`AgentPresets.mount(agentCtx, id?)` (:414–427) is called from the agent factory's `setup` hook (Ch 14) — the module names it "the one supported call site." It resolves the standing mount, then:

```
this.bindings.set(agentKey, bindScopeParent(agentKey, standing.key))
```

— :425

`bindScopeParent` (`scope/src/index.ts:72–82`) writes **one** `WeakMap` **entry** from the agent's scope key to the standing mount's, with a cycle check.

No new fiber. No new plugin instance. No module re-import. The second, third, and hundredth agent on a preset each cost one `WeakMap` entry. And because that same parent link drives both event routing and `ScopedLayers`, the agent immediately sees the preset's tools, prompt sections, and scoped services.

The deliberate backdoor

Host code sometimes must read *inside* a specific agent's realm-private service — a browser RPC about a session. `serviceForAgent` (`mount.ts:259–293`) does it by walking the reflect store's own symbols and testing **fiber-tree membership** (`withinFiber` , :189–197) rather than using the isolate map at all.

It deliberately bypasses the invisibility realms enforce, by using a different addressing scheme than dependents use. Worth knowing that the escape hatch exists and is narrow.

`preset.yml`

Display metadata only — three lines of name, description, and sort order. Discovery treats unreadable metadata as non-fatal: "a preset with unreadable metadata still mounts, it just shows its id" (`discovery.ts:309–310`).

8. Reading a composition correctly

The practical procedure this book has used throughout:

1. **Start at** `packages/bundle/base/cordis.patch.yml` — what exists at all.
2. **Apply** `packages/bundle/web-app/cordis.patch.yml` — `disabled: true` rows, and note that any `config:` replaces the base's wholesale.
3. **Check the preset** — `presets/standard/agent.cordis.yml` re-mounts most of what layer 2 disabled, sometimes with *different* config.
4. **Check for** `!!js` — the value may depend on the platform or environment.
5. **Check openAt -style dormancy** — a mounted row can still do nothing (`session-query-sqlite` , `llm-pi-ai`).

Skipping step 3 produces confident wrong answers: compaction, subagents, skills, plan mode, and most tools all look disabled after step 2.

9. The vendored framework

`vendor/cordis` is `cordis 4.0.0-rc.7` from `cordiverse/cordis` at commit `56b3d4f7...` ; the companions (`include` , `group` , `timer` , `hmr` , `logger-console`) come from a **fork**, `deepseek-harness/cordis` , at a different commit. `vendor/README.md` lists 19 local modifications.

Spot-checking those against the source (rather than trusting the changelog) confirmed four: fiber lifecycle hardening, `applyEntryPatches` ' export and insert-then-index behavior, lazy `!!js` resolution with the tree-carrier exemption, and `disabled: !!js` interpolation.

All 19 were subsequently verified by cloning `cordiverse/cordis` at the pinned commit `56b3d4f7...` and diffing. The result is worth stating plainly: **the log is accurate — no contradictions, no omissions found.**

Thirteen were checked against the exact pinned baseline:

Item	Verification
2, 3	<code>package.json</code> gains <code>private</code> , <code>precise files</code> , <code>lib/types/**</code> declarations; <code>tsconfig</code> extends <code>../../tsconfig.base.json</code> rather than upstream's own base
4	<code>'./events' → './events.ts'</code> throughout
5	Both <code>tsdown.config.ts</code> files exist here and nowhere upstream — "ours, not upstream files" is exactly right
6	Effect wrapper registered before setup (<code>fiber.ts:520</code> before <code>:522</code>); <code>update()</code> returns its waterfall result
7	"Comment-only; no code changes" — confirmed. Stripping comments from <code>service.ts</code> , <code>index.ts</code> , <code>context.ts</code> , <code>reflect.ts</code> , <code>registry.ts</code> , <code>logger.ts</code> , and <code>utils.ts</code> leaves zero functional differences ; every residual is either an import rewrite items 4/10/17 already declare, or whitespace from the repo's own linter (<code>{ } → { }</code> , <code>;C → ; ()</code>)
10	<code>import { Dict } → import type { Dict }</code> ; <code>InjectKey → type InjectKey</code>
11	<code>applyEntryPatches</code> exported; inserted rows indexed inside the patch loop
15	Lazy <code>internal/config</code> resolution; <code>Include</code> carries the tree-carrier marker
16	Upstream <code>files: ["lib", "bin.js"]</code> ; vendored adds <code>"src"</code>
17	<code>cordis → @deepseek-ai/cordis</code> , <code>cosmokit → @deepseek-ai/cosmokit</code>
18	<code>disabledOf()</code> interpolates a <code>!!js node</code>
19	Upstream branches on <code>major >= 24</code> ; vendored probes for <code>getOrCreateModuleJob / getModuleJobForImport</code> . A genuine bug fix — the v2 interface landed in 24.12.0, so Node 24.0–24.11.1 are mistagged by a major-version test

Five more (items 1, 9, 12, 13, 14) target packages sourced from `deepseek-harness/cordis`, which is **not public** — so they were checked for *presence of the described change* rather than diffed against their own baseline. All five are present and match their descriptions: the `hmr` locales directory and its `.i18n()` call are gone (with an inline marker comment left behind), `registerConfig` exists, the main watcher takes `ignoreInitial: true` while `registerConfig`'s own takes `false`, `writeTask` is widened to `| undefined`, and the `EACCES / EBUSY / EPERM` retry is there.

Item 8's loader half shows real divergence in exactly the named files (`loader/src/index.ts` 166 → 202 lines, `config/group.ts` 88 → 129); its `include / group` half shares item 1's baseline limitation.

⚠ **One caveat on attribution.** For the five fork-sourced items, "present here and absent from `cordiverse`" cannot distinguish a change the harness made from one the fork made. The changelog attributes them to local modification; nothing contradicts that, but it is not independently separable without access to the fork.

10. Key takeaways

- A profile is an empty root config plus an ordered patch stack; no file lists the composition.

- A patch **replaces** a row's whole `config` rather than merging.
- Row order is cosmetic; activation is dependency-driven and concurrent.
- `!!js` is a tagged scalar evaluated lazily per row, after that row's injections resolve — `!js` would not match at all.
- Isolate realms remap which symbol a name resolves to; visibility is the default and a realm is a narrowing.
- `dsh-scope` is a separate, application-level mechanism: one `WeakMap` parent link driving both event routing and registry layering.
- A preset is mounted once per (id, file generation) and joined by writing one `WeakMap` entry per agent.

Exercises

1. A deployment wants `maxParallelToolCalls: 4`. Write the patch, say which layer it belongs in, and explain what happens if it is placed in a layer the standard preset later restates.
2. A new per-agent plugin publishes `ctx.myService`. Where does it go, and what does `mountPreset` do if you forget the realm?
3. `serviceForAgent` bypasses realm invisibility using fiber membership. Name one legitimate use and one way it could be abused.

Next: [Chapter 35 · Limits and fragile areas](#)

Chapter 35 · Limits and fragile areas

What you'll learn: where this engine's guarantees stop, which mechanisms are approximate, and which parts the code itself flags as unfinished.

Prerequisites: Part III.

Every limit here is grounded in code or in the codebase's own acknowledgements. Nothing is speculation about what *might* go wrong.

1. The token meter is not a tokenizer

```
CHARS_PER_TOKEN = 4
BLOCK_OVERHEAD  = 4
ROLE_OVERHEAD    = 4
```

— `packages/llm/token-meter/src/estimate.ts:13-19`

There is no BPE table and nothing provider-specific. Four characters per token is a rough average for English prose and is wrong in both directions for code, non-Latin scripts, long identifiers, and repetitive whitespace.

What depends on it. The compaction threshold ([Ch 27](#)). An under-estimate delays compaction until the provider rejects the request; the overflow path then recovers, but at the cost of a wasted round trip.

What compensates. Real provider usage supersedes the estimate as soon as one request succeeds ([Ch 26](#)), and the threshold is 80% rather than 100%, leaving deliberate headroom.

When it would bite. A session whose first request is already near the window — a large seeded fork, or a long first prompt — has no usage to anchor on, and CJK text or dense code can be badly mis-priced in exactly that window.

2. `maxParallelToolCalls` is deployment-wide

One value, read per group from a settings namespace, applying to every agent in the process ([Ch 17](#)).

There is no per-agent or per-tool cap. A session running ten parallel file reads and one running ten parallel network fetches get the same budget, though their costs differ by orders of magnitude. A subagent inherits it too, so N concurrent subagents can each run up to the cap.

3. Sibling effects tear down concurrently

The nuance most likely to surprise someone reading the Cordis docs:

- disposers **nested inside one** `ctx.effect()` run in strict LIFO;
- but when the **whole fiber** unloads, `_unload()` clears the entire disposables list and runs them all with `Promise.all` (`vendor/cordis/src/fiber.ts:675-696`).

So two top-level `ctx.effect()` calls on the same fiber tear down **in parallel**, with no ordering guarantee between them. `AgentLoop` 's constructor registers exactly two — `ownership.dispose()` and `agents.setFactory()` — and their relative order on unload is undefined.

Any code assuming "my second effect tears down before my first" is relying on something the framework does not promise.

4. `ctx.emit()` does not contain listener failures

```
emit(...args: any[]) {  
  this.dispatch('emit', args).map(cb => cb(...args))  
}
```

— `vendor/cordis/src/events.ts:194-196`

A listener that throws synchronously propagates out and **aborts the remaining calls**. Later listeners silently do not run.

The agent dispatcher works around this by resolving the callback list and wrapping each call itself ([Ch 22](#)), and `AgentLoop.reportConfiguredStartupFailure` does the same manually. But **any code calling `ctx.emit()` directly inherits the hazard**, and the fix is per-call-site rather than central.

5. Provider errors are classified by text matching

`isContextWindowExceededError` and `isQuotaExceededError` (`packages/llm/llm/src/error.ts:80-100`) pattern-match the provider's `code`, `type`, and `message` fields with regexes.

What depends on it. Compaction's entire overflow-recovery path triggers on `CONTEXT_WINDOW_EXCEEDED` ([Ch 32](#)). A provider rewording its error message could silently stop producing that code — and the symptom would be *turns failing where they used to recover*, with no error in the harness and nothing obviously changed.

This is the most fragile coupling in the system: a behavior-critical branch depending on another vendor's prose.

6. Cancellation has no watchdog

Convergence is entirely cooperative ([Ch 13](#)). The loop checks its signal at six points per turn, the scheduler drains rather than abandons, and `whenIdle()` waits for genuine quiescence.

A tool that ignores its abort signal delays teardown indefinitely. There is no timeout, no grace period, no forced kill. The timeout-policy plugin bounds individual calls, but the engine itself will wait as long as a tool takes.

Consequence worth knowing: a badly-behaved tool can block agent disposal, which in turn blocks factory teardown, which `Promise.all`s every live agent's disposal ([Ch 14](#)).

7. `always` retry mode is unbounded

`maxRetries` is enforced only for `mode: 'normal'`. An `always` policy has no code gate and no retry cap — it retries indefinitely until abort or disposal ([Ch 25](#)).

The engine's step loop is `while (true)` with no iteration cap of its own; bounding is entirely the listener's job.

Also worth flagging: the `always` branch reads as if it returns early when downstream declines, and does not — it falls through into the shared backoff path. If you depend on that behavior, quote the lines rather than paraphrasing.

8. Runtime invariants ship switched off

The registry that runs package-owned invariant companions is mounted **nowhere** ([Ch 11](#)). The engine's reconstruction check — the executable form of this book's central claim — does not run in the shipped product.

Two further limits stated by the package itself:

- **Filters are fixed for the service lifetime.** `enabled`, `package_allowlist`, and `package_blocklist` compile once at startup; changing them needs a plugin reload.
- **Live-only companions miss pre-reload operations.** A companion that observes only live operations cannot reconstruct ones that began before its own reload.

And a documentation defect: the package's README contradicts itself, claiming in its summary that "the standard agent composition already mounts it" while its usage section says `dsh-base` "deliberately omits runtime diagnostics." The composition files settle it.

9. `agent/turn-stopping` has no live listeners

Defined, typed, dispatched on every qualifying turn — to nobody ([Ch 22](#)). Its only implementations are the two hook bridges, which are mounted in no composition layer.

Not a bug. But a reader should know that the turn-stopping seam is untested in production conditions, and the first plugin to use it will be exercising a path nothing currently exercises.

10. There is no working install out of the box

The native adapter is disabled in the web layer and the generic one mounts dormant, so a fresh install has **zero registered provider routes**. The first turn fails `NO_ADAPTER`, nothing retries it, and the fix is a settings write ([Ch 32](#)).

Defensible as a design — the Web surface exposes only what the Models page declares — but it means the failure a new user sees first is one the engine cannot handle.

11. The session format has no migration path

`SESSION_FORMAT_VERSION = 0`, and the doc comment is blunt: while unreleased, "no compatibility is implied, incompatible logs are rejected, and no migration is provided" (`packages/core/session/src/types.ts:28-50`).

Version mismatches are refused in both directions. The `ignorable` flag plus the generated known-type set lets *vocabulary* grow safely ([Ch 5](#)), but a structural change would strand every existing session.

12. Acknowledged in-source unfinished work

Three the code flags itself:

`TODO(call-config-shape)` — `LlmCallConfig`'s shape is explicitly unsettled: "Revisit which fields are epoch-level for cache reuse and where provider-specific request options belong" (`packages/llm/llm/src/call-config.ts:15-16`). Since this type is the request header's `config`, changing it touches log compatibility.

`DSH_TOOLS_MODE` **is a temporary seam**. The web bundle calls it a "TEMPORARY workaround... while per-session tool-presentation selection is being designed. Remove the env seam once the web UI owns the choice per session" (`packages/bundle/web-app/cordis.patch.yml:33-37`). A process-wide env var stands in for what should be per-session.

Continuable fork invalidates the cache it exists to preserve. Fork seeds a child with the parent's history so the shared prefix stays KV-cache eligible — but a continuable child's `report` tool and prompt section precede that history and invalidate the same prefix. The standard preset accepts the cost, with an issue tracking cache-preserving continuable fork ([Ch 28](#)).

13. Smaller sharp edges

Edge	Consequence
<code>str_replace_editor</code> is exclusive even for view	Read-only views serialize with mutations; <code>tool-fs</code> 's <code>read</code> does not
A live-edited preset forks a generation	Existing sessions keep the old plugin instances; only new ones get the edit
<code>JSON.stringify</code> comparison in the invariant	Key-order sensitive — stricter than deep equality, and the safe direction
The first <code>deriveMessages()</code> after compaction	The one non-incremental case; rebuilds the whole cache
Raw <code>ToolDefinition</code> gets no argument validation	Only <code>defineTool</code> validates; a hand-built tool receives whatever <code>parseArguments</code> produced
Spill files are swept by age	A resumed session may reference a locator whose file was cleaned up — see below

Confirmed: a session resumed after `cleanupPeriodDays` (default 30) **cannot** resolve spill locators in its history. The sweep deletes any file whose `mtime` is strictly older than the cutoff and knows nothing about which sessions are live or referenced (`spill-local/src/index.ts:40-46`). The model is told to `read` a path that is gone and gets a file-not-found error. Setting `cleanupPeriodDays: 0` disables cleanup entirely. → [Ch 26](#)

Key takeaways

- The token meter is a character heuristic; the 80% threshold and real-usage preference are what make it workable.
- `maxParallelToolCalls` is process-wide, with no per-agent or per-tool refinement.
- Sibling top-level effects tear down concurrently; only nested disposers are ordered.
- Raw `ctx.emit()` lets one listener's throw suppress the rest.
- Compaction's overflow trigger depends on regex-matching another vendor's error text — the most fragile coupling here.
- Cancellation is cooperative with no watchdog; an unresponsive tool blocks teardown.
- Runtime invariants, the turn-stopping seam, and a working default provider are all absent from the shipped configuration.

Exercises

1. Pick the limit you consider most likely to cause a production incident. Justify it against the others, then design the smallest change that would detect it early.

2. §3 says sibling effects tear down concurrently. Write the `AgentLoop` teardown bug that would require an ordering guarantee, and say whether the current code has it.
3. §5 depends on matching provider error text. Design a fallback that degrades safely when the pattern stops matching, without a false positive whenever a request fails.

Next: [Chapter 36 · Design decisions](#)

Chapter 36 · Design decisions

What you'll learn: the reasoning behind six choices that shaped everything else, each as Context → Decision → Alternatives → Tradeoffs.

Prerequisites: Part III.

A note on evidence. These are **reconstructions**, assembled from the code, its comments, and the repository's own design notes. Where the codebase states a rationale I quote it; where I am inferring from structure I say so. A reader should treat the Context and Decision sections as verified and the Alternatives sections as my analysis.

ADR-1 · The log is the single source of truth

Context. An agent needs the conversation to build each request. It also needs to resume after a crash, fork at a known-good point, shrink history that outgrew the window, and replay a session deterministically for tests.

Decision. Store an append-only event log. Recompute the message list from it immediately before every dispatch. Never hold a separate transcript.

The rule is repo-wide: *"Model-visible $\Leftrightarrow$ logged: anything that reaches a model request must be reconstructable from the session log; a new model-visible input requires a session event"* (`AGENTS.md`). It is stated as an executable assertion in `agent-loop/src/invariant.ts` (Ch 11).

Alternatives.

- *In-memory array, persisted periodically.* Simplest. Resume needs a separate serialization; fork needs a deep copy; replay needs a fixture format that can drift from the live shape.
- *Log plus a cached transcript, synchronized.* Faster. Two sources that can disagree — and the disagreement appears as a session that replays differently than it ran.

Tradeoffs.

- **Cost:** ~43 events for a single one-word turn (Ch 31). Every streamed chunk is an event, which is why a compression codec had to be written (Ch 30).
 - **Cost:** derivation runs on every request, needing an incremental cache with a generation guard (Ch 7).
 - **Benefit:** resume, fork, compaction, and replay all become operations on one data structure.
 - **Benefit:** the correctness property is *checkable*. Most designs cannot state their central invariant as an assertion.
-

ADR-2 · Rewrite history by superseding, never by mutating

Context. ADR-1 makes the log permanent. Context windows are finite. Something has to shrink.

Decision. Add a **surface** — an ordered list of visible log positions — and a `replace` operation that splices a range out of that list and substitutes one new event. The raw events stay where they are.

Alternatives.

- *Delete old events.* Contradicts ADR-1 outright; the user's scrollbar loses content they saw.
- *Rewrite messages in place.* Same problem, plus it breaks provenance: nothing can say what a summary stands for.
- *Keep a separate "visible" copy.* Two sources again, and compaction becomes a synchronization problem.

Tradeoffs.

- **Cost:** an indirection every reader must understand. The log and the surface are different things, and a UI reading the wrong one shows a user their own history disappearing ([Ch 6](#)).
 - **Cost:** provenance rules — complete shadowed-node coverage, the narrow `tool/result` rewrite — are real complexity.
 - **Benefit:** compaction is a normal append. No special write path, no locking of history, no risk of losing what was summarized.
 - **Benefit:** `replaceGeneration` gives four other mechanisms a precise "history changed" signal ([Ch 7](#), [Ch 10](#), [Ch 21](#)).
-

ADR-3 · Registrations are effects

Context. Plugins contribute tools, prompt sections, projections, adapters, and listeners. Plugins unload — on hot reload, on preset teardown, when a dependency disappears. Every contribution must be removable, exactly once, without a bookkeeping registry of what to undo.

Decision. Every contribution goes through `ctx.effect()`, and every `register()` returns the disposer that undoes it. Stated as a repo rule: *"Registrations are effects: every contribution goes through `ctx.effect()` / `ctx.on()`; a registry's `register()` returns the disposer."*

Alternatives.

- *Explicit `unregister(name)`.* Requires every caller to remember names and call sites, and double-unregister becomes a real bug class.
- *Garbage-collect on plugin unload.* Needs the framework to know every registry — inverting the dependency the plugin system exists to avoid.

Tradeoffs.

- **Cost:** a nuance that surprises people — nested disposers are ordered LIFO, but sibling top-level effects tear down concurrently ([Ch 35](#)).
 - **Benefit:** teardown is structurally correct by default. An agent's whole lifecycle, including its running driver, is disposed by one effect on the owner fiber ([Ch 14](#)).
 - **Benefit:** the pattern generalizes — `provide()` is an effect, so *un-providing* a service is a synchronization point that awaits every dependent's re-settlement.
-

ADR-4 · The agent plane moves behind presets

Context. One process serves many browser sessions. Different sessions may want different tools, personas, and delegation backends. But registries (`tools` , `skill` , `jobs` , `subagents`) are process singletons that host code and the API layer must reach.

Decision. Split into two planes. **Host plane:** registries, sandbox and approval stack, persistence, model routes — mounted once, visible everywhere. **Agent plane:** the tools, prompt sections, and per-agent services a session gets — mounted once per preset under a standing scope, joined by scope parentage.

The test is stated in the composition's own comments: a row that **publishes** a service must sit in an `isolate` realm; a registry something **outside** the preset reads belongs to the host plane.

Alternatives.

- *One global composition.* Simple, and every session gets identical tools. Rules out per-session presets entirely.
- *A full plugin tree per session.* Maximum isolation, and a fresh import and fiber graph per session — plus every process singleton would be duplicated and collide.

Tradeoffs.

- **Cost:** the hardest thing in this codebase to read correctly. Three layers, last-write-wins, `config` replaced rather than merged. A reader checking two layers concludes compaction and most tools are disabled ([Ch 34](#)).
 - **Cost:** two unrelated isolation mechanisms coexist — Cordis realms and `dsh-scope` layering — with similar-sounding names and different jobs.
 - **Benefit:** the marginal cost of a session joining a preset is **one** `WeakMap` **entry**. No fiber, no instance, no import.
 - **Benefit:** `mountPreset` mechanically rejects a row that leaked a service, so the hazard fails loudly at mount rather than silently at runtime.
-

ADR-5 • Retry is an extension point, not a loop feature

Context. Requests fail transiently. Retrying is standard. But the right policy is deployment-specific: a flaky network wants aggressive retries, an interactive product may prefer failing fast, and *some* failures need a different response entirely — an oversized context needs compaction, not repetition.

Decision. The loop offers `agent/request-error` with a terminal default. It contains no retry logic. A listener returning `{ kind: 'retry' }` causes the request to be rebuilt and re-sent; anything else ends the step.

Alternatives.

- *Retry in the loop, configured.* Every deployment gets one policy shape. Compaction-on-overflow would need a separate hook anyway, or would be special-cased into the loop.
- *Retry in the adapter.* Invisible to the session log, so a retry budget could not be durable, and compaction could not participate at all.

Tradeoffs.

- **Cost:** no retries unless something is mounted. A request with no prepared call carries no policy and is never retried — which is exactly the unconfigured-install path ([Ch 32](#)).
- **Cost:** the loop cannot bound retries. An `always` policy retries forever ([Ch 35](#)).
- **Benefit:** compaction and retry compose on one seam without knowing about each other. Compaction claims overflow; retry handles the rest.
- **Benefit:** because a retry rebuilds from the log, a listener can *change history* and have the rebuilt request pick it up. That is what makes compact-then-retry work at all, and it falls out of ADR-1 rather than being designed separately.

ADR-6 • `policy: 'never'` is not delegatable

Context. A deployment can set approval to `never`, meaning actions requiring approval are auto-rejected. Approval requests are answered through a waterfall, and waterfall listeners are ordered, with `prepend` available ([Ch 22](#)).

Decision. Check the policy **inline, before the waterfall runs at all:**

```
if (policy === 'never') return 'rejected'
```

— `packages/interaction/user-approval/src/index.ts:277`

Alternatives.

- *A prepended listener enforcing the policy.* Reads more uniformly — every decision through one mechanism. But "prepended" is only a position in a list, and another listener can also prepend. Correctness would depend on registration order, which depends on mount order, which depends on composition.

Tradeoffs.

- **Cost:** an asymmetry. One decision is special-cased while the rest go through the extension point, which a reader must notice.
 - **Benefit:** the guarantee is structural rather than configurational. No plugin, in any composition, in any order, can turn `never` into a grant.
 - **Benefit:** it pairs with the opposite use of the same insight — the reconstruction invariant uses `prepend: true` precisely *because* a short-circuiting listener could otherwise silence it ([Ch 11](#)). The codebase knows ordering is unreliable and reasons about it in both directions.
-

What the six have in common

Two principles recur.

Make the correctness property checkable. ADR-1 produces an assertion. ADR-4 produces `leakedServices()`. ADR-6 produces a guarantee no configuration can defeat. Where a rule could be stated as code that fails, it was.

Prefer a mechanism that fails loudly over one that degrades quietly. Unknown event types refuse a session rather than being skipped. A malformed chunk row throws rather than dropping a run. A preset that leaks a service fails session creation. An approval that cannot be answered is denied, never allowed.

The exceptions are deliberate and narrow: `spill` degrades to keeping the original content, presenters swallow invalid historical arguments, and `emit` listener failures are contained. Each is a case where failing loudly would cost more than the failure itself.

Key takeaways

- The log-as-truth decision is the root; ADRs 2 and 5 are consequences of making it work.
- Superseding instead of mutating buys safe compaction at the cost of an indirection every reader must learn.
- Registrations-as-effects makes teardown correct by default, with one concurrency nuance.
- The preset split costs real readability and buys near-zero marginal cost per session.
- Retry lives outside the loop so compaction can share the same seam.
- One security decision is deliberately *not* delegatable, because listener order is not a guarantee.

Exercises

1. Pick an ADR and argue the alternative. What would this codebase look like if it had gone the other way, and which chapters would disappear?
2. ADR-5's benefit — a listener can change history and have the retry pick it up — is a consequence of ADR-1, not a designed feature. Find another such consequence elsewhere in the book.
3. §"What the six have in common" claims failures are loud except in three narrow cases. Find a fourth, and decide whether it belongs with the exceptions or is an oversight.

Next: [Chapter 37 · Rebuild it](#)

Chapter 37 • Rebuild it

What you'll learn: a milestone path from an empty folder to a working engine, each step adding one mechanism, with the real files to compare against and a way to tell it works.

Prerequisites: all of Part III, at least skimmed.

How to use this

Ten milestones. Each adds **one** mechanism, in dependency order. Each names the real files that implement it, a test you can write without a provider, and the trap that milestone exists to avoid.

You do not need this repository's framework. Milestones 1–7 are plain TypeScript with no plugin system at all; the composition machinery only becomes necessary at milestone 9, and only if you want more than one agent configuration.

A deliberate ordering note: **the plugin framework comes last, not first**. Building the loop against concrete dependencies and extracting seams once you know where they belong is a better path than starting with a plugin system and discovering which seams you needed.

Milestone 1 • The log

Build. A `Session` class holding `SessionEvent[]`, with one method:

```
append(type, data): SessionEvent    // seq = log.length, frozen, deep-snapshotted
get events(): readonly SessionEvent[]
```

Event types: `turn/start`, `turn/end`, `step/start`, `step/end`, `user/message`, `assistant/message`, `tool/call`, `tool/result`.

Real files. `packages/core/session/src/index.ts:602–653`, types at `src/types.ts:216–320`.

Test it. Append ten events; assert `seq` is `0..9` with no gaps and every event is frozen. Assert that mutating the object you passed in does not change the logged copy.

The trap. Storing a reference instead of a snapshot. Callers reuse objects; without a deep copy your history mutates under you, and the bug surfaces as a session that replays differently than it ran.

Milestone 2 • Derivation

Build. `deriveMessages(): Message[]` — walk the log, project `user/message`, `assistant/message`, and `tool/result` into messages, and skip everything else.

Real files. `packages/core/session/src/surface.ts:83-114` (the per-event rule), `index.ts:724-745` (the walker).

Test it. Log a turn's worth of events including boundaries; assert `deriveMessages()` returns only the message-producing ones, in order.

The trap. Adding formatting here. Wrap a plugin message in tags at *append* time, never at derive time — otherwise changing the formatting changes the meaning of every session recorded under the old rule ([Ch 7](#)).

Milestone 3 • The 100-line loop

Build. [Chapter 4](#)'s `MinimalAgent`: a driver over turns, a turn over steps, and a step that derives messages, calls a model, and appends the result. Use a mock adapter returning a fixed response.

Real files. `packages/core/agent-loop/src/agent.ts:219-438`.

Test it. A mock adapter returning one text block. Assert the log contains `turn/start`, `step/start`, `user/message`, `assistant/message`, `step/end`, `turn/end` in that order, and that the mock received exactly the messages `deriveMessages()` returns.

The trap. Keeping `this.messages` alongside the log. The moment those two can disagree, you have lost the property the whole design is for.

Milestone 4 • Tool calls

Build. After appending the assistant message, filter for `tool-call` blocks. For each: append `tool/call`, execute, append `tool/result` citing the call's seq. Return "no ending yet" so the turn runs another step.

Real files. `packages/core/agent-loop/src/tool-calls.ts` (start with the sequential path), `packages/core/tools/src/index.ts:1028-1053`.

Test it. A mock returning one tool call, then a text answer. Assert the turn has two steps and the second request's messages include the tool result.

The trap. Forgetting that `null` from a step means *continue*. If a step that ran tools returns an ending, the model never sees its results.

Also: parse arguments defensively. Invalid JSON should reach the tool as a raw string and become a readable error, never a crash ([Ch 17](#)).

Milestone 5 • The surface

Build. Add `nodes: number[]` and `replaceGeneration`. Require `surfaceOp` on the three eligible event types. Implement `replace` as a splice. Make `deriveMessages()` walk `nodes` rather than the log.

Real files. `packages/core/session/src/surface.ts` in full.

Test it. Log five messages; replace the middle three with one; assert the surface has three nodes, `replaceGeneration` incremented, and **all five original events are still in the log at their original positions**.

The trap. Letting a replacement cite less than it shadows. Require the coverage rule from day one ([Ch 6](#)) — retrofitting provenance later means every replacement written before it is unattributable.

Milestone 6 • Streaming

Build. A `BlockAssembler` keyed by block index. Log every chunk as its own event. Handle `block-end` as authoritative. Drop tool calls when the finish reason is `max-tokens`.

Real files. `packages/llm/llm/src/assembler.ts`, types at `src/types.ts:364-376`.

Test it. Feed interleaved chunks — block 1's deltas before block 0's `block-end` — and assert both blocks assemble correctly. Feed a truncated tool call with `max-tokens` and assert it is dropped.

The trap. Keying by arrival order. Blocks interleave; the index is the identity ([Ch 24](#)).

Milestone 7 • Cancellation

Build. An `AbortController` per turn. Check it before each boundary append. On abort: salvage text/reasoning blocks into an `interrupted: true` message, drain started tools, and synthesize results for unstarted ones.

Real files. `packages/core/agent-loop/src/agent.ts:143-232`, `tool-calls.ts:238-260`.

Test it. Abort mid-stream. Assert the log is **balanced** — every `step/start` has a `step/end`, every `tool/call` has a `tool/result` — and that a fresh request built from the cancelled session is valid.

The trap. Abandoning unstarted tool calls. A `tool/call` with no result makes the *next* request invalid, so a cancelled session cannot be resumed. This is the milestone where "resumable" starts being true or false.

Milestone 8 • The inbox and step boundaries

Build. Two queues, `next-turn` and `next-step`, as a replay of `agent/inbox/spliced` events. `claim(target)` drains all of `next-step` and pops **one** from `next-turn`. Wire `followup` / `steer` / `inject`.

Real files. `packages/core/agent/src/inbox.ts`, `agent-loop/src/agent.ts:122-141`.

Test it. Call `inject` on an idle agent; assert **no turn starts** and only the splice event is logged. Then `steer` and assert the message is claimed at the next step boundary.

The trap. Making the queue a plain array. Then a crashed agent loses pending input, and the reason the queue is durable — it is the log — is lost.

Milestone 9 • Prompt assembly and extension points

Build. A registry of named prompt sections with numeric orders, rendered by order and joined. A `waterfall(listeners, payload, default)` where each listener gets a `next`. Convert your hardcoded step logic into an `agent/pre-step` waterfall whose innermost default is the current behavior.

Real files. `packages/core/system-prompt/src/index.ts:536-611`, `vendor/cordis/src/events.ts:234-243`, `packages/core/agent/src/dispatch.ts`.

Test it. Register two sections with different orders; assert deterministic output. Register a waterfall listener that returns without calling `next()`; assert the default behavior did **not** run.

The trap. Assuming listeners always delegate. The veto is the point — and it means anything living in the innermost default can be suppressed ([Ch 21](#), [Ch 22](#)).

Milestone 10 • Compaction

Build. A token estimate over the surface. At a threshold, select a range that retains recent history and does not split a tool call from its result. Summarize it with a model call. Commit as a `replace` citing every shadowed node. Refuse to commit if the summary is not smaller.

Real files. `packages/compaction/compaction-basic/src/region.ts:154-256`, `src/index.ts:259-333`.

Test it. Build a long session; compact; assert the surface shrank, the log did not, `replaceGeneration` incremented, and `deriveMessages()` returns the summary in place of the range. Then assert a second compaction of an already-terse session is **refused**.

The trap. Splitting a tool-call/result pair. The next request will be rejected by the provider, and the cause will look like a model problem rather than a boundary-selection bug.

What you will have

An engine that resumes, forks, cancels cleanly, and compacts — around 1,500–2,000 lines.

What the shipped engine adds beyond this, and where to read it:

Not in the ten milestones	Chapter
The four-stage tool pipeline and approval	16, 18
Parallel scheduling with ordered commits	17
Adapter resolution and prepared calls	23
Retry as a listener	25
Spill and pruning	26, 27
Subagents	28
Persistence, crash repair, chunk packing	29, 30
Presets, realms, and scope layering	34

Each is genuinely optional. None is needed to demonstrate the central property — and that is the strongest evidence that the property is the design, rather than a consequence of the size.

Key takeaways

- Ten milestones, in dependency order; the first seven need no plugin framework.
- Build the loop first and extract seams once you know where they belong.
- Milestone 5 (the surface) is the point where compaction becomes possible without loss.
- Milestone 7 (cancellation) is where "resumable" becomes true or false.
- Every milestone has a trap that produces a bug you will find much later than you introduced it.

Exercises

1. Do milestones 1–3. Then write the test from milestone 3 that asserts the mock received exactly `deriveMessages()` . Keep it green for the rest.
2. At milestone 5, retrofit the surface onto milestone 4's tool results. What breaks, and what does that tell you about introducing it earlier?
3. Skip milestone 7 and go straight to 8. Build a session, cancel it mid-tool, resume. Describe the exact failure — and then say which of the three pieces of milestone 7 would have prevented it.

Next: [Appendix A · Glossary](#)

Appendix A • Glossary

Every domain and project-specific term the book introduces, with where it is defined in the text.

adapter — An object that knows how to talk to one or more model providers. Only `stream()` is required; everything else has a default. Registered against provider names, with its retry policy captured at registration. → [Ch 23](#)

adapterDefaults — A record of which config fields the *caller* omitted and the *adapter* supplied (`{ reasoningEffort?: true, maxTokens?: true }`). Used to strip those fields from the next request so they get re-resolved rather than inherited across a model change. → [Ch 10](#), [Ch 23](#)

agent plane — The set of plugins composing what *one agent* contributes: its tools, prompt sections, and delegation backends. Mounted per preset, in contrast to the host plane. → [Ch 34](#)

AgentEventDispatch — The fused, agent-scoped wrapper over Cordis's three dispatch modes. Injects the agent into every payload and routes through the agent's scope carrier so identity and routing cannot diverge. → [Ch 22](#)

barrier — An exclusive tool call that runs alone: everything before it has committed, nothing after it starts until it finishes. → [Ch 17](#)

BlockAssembler — Accumulates stream chunks into content blocks, keyed by block index. Exposes different views depending on how the stream ended. → [Ch 24](#)

canonical result — A tool result already validated and rendered through its tool's output contract; re-normalizing one is a no-op. → [Ch 16](#)

compaction transaction — A `compaction/start` event, some work, and a `compaction/end` event. The start event is the lock, so it survives a crash and is visible in the log. → [Ch 27](#)

Cordis — The vendored dependency-injection and plugin framework everything is built on. Four concepts matter: context, service, fiber, effect. → [Ch 2](#), [Ch 34](#)

context (Cordis) — The object every plugin receives. Reading an unknown property is a *service lookup* that walks the fiber chain. → [Ch 2](#)

context sections — Prompt contributions that become a runtime-context *message* rather than part of the system prompt, because they change during a session. Only three producers exist. → [Ch 19](#), [Ch 21](#)

durable-first — Committing a change to the session log *before* mutating memory, so synchronous observers see the pre-change state and the change survives a crash. The inbox works this way. → [Ch 12](#)

effect — `ctx.effect(fn, label)` runs `fn` immediately and collects its return as a teardown handle. Every registration in the codebase is one. → [Ch 2](#), [Ch 36](#)

fiber — One running instance of one plugin, with a lifecycle. Stays `PENDING` until every injected dependency resolves; unloads automatically when one disappears. → [Ch 2](#)

host plane — Plugins mounted once, visible everywhere: registries, sandbox and approval, persistence, model routes. → [Ch 34](#)

inbox — Two ordered queues of pending user messages (`next-turn` , `next-step`), drained at step boundaries, reconstructed by replaying `agent/inbox/spliced` events. → [Ch 12](#)

invariant companion — A small plugin shipped by a package that checks a durable relationship the package owns, at runtime. About thirty exist; the registry that runs them is mounted nowhere. → [Ch 11](#)

isolate realm — A Cordis mechanism remapping which symbol a service name resolves to for one entry's subtree, so a per-preset service does not collide process-wide. Visibility is the default; a realm is a narrowing. → [Ch 34](#)

maintenance — A phase for work needing the agent to hold still (manual compaction) during which no turn may start, but which reports externally as `idle` . → [Ch 13](#)

`markAgentLoopRequest` — A process-local `WeakSet` tag marking a request object as loop-built (frozen, derived from the log) versus a hand-built one-shot call. Never serialized. → [Ch 10](#)

phase — The agent's internal state: `idle` , `maintenance` , or `running` . Collapses to two external statuses. → [Ch 13](#)

prepared call — A frozen, single-use object pinning one request's resolved config, retry policy, adapter defaults, and bound dispatch closure. → [Ch 23](#)

preset — A named agent-plane composition. Mounted once per (preset id, file generation) under a standing scope; each agent joins by one `WeakMap` parent link. → [Ch 34](#)

profile — Not a file. An empty root config plus an ordered stack of patch layers. The shipped one is `web` . → [Ch 2](#), [Ch 34](#)

projection — A registered, versioned, pure fold over the session log. The general form of what `deriveMessages` does for messages. → [Ch 8](#)

`PromptAssembly` — The resolved but still-uninterpolated result of prompt assembly: sections, contexts, tools, and variables. → [Ch 19](#)

prompt variable — A named `(context) => string | undefined` provider, evaluated once per assembly and substituted at render time. The engine registers `provider` , `model` , and `cwd` . → [Ch 20](#)

PTC / `run_code` — A tool-presentation mode where the model writes a program that calls tools, rather than calling them directly. Behind `DSH_TOOLS_MODE` ; unset keeps the `native` default. → [Ch 15](#), [Ch 16](#)

request header — The non-message half of a request: config, system prompt, tool schemas, and `adapterDefaults` . Logged only when it changes, with four possible reasons. → [Ch 10](#)

replaceGeneration — A counter incremented once per committed surface replace. The system's "history was rewritten" signal, read by four separate mechanisms. → [Ch 6](#)

seq — An event's monotonic sequence number, always equal to its zero-based index in the log. Assigned in one place, verified in four. → [Ch 5](#)

Session — A plain class (not a service) holding one conversation's append-only log, its surface, and its immutable header. → [Ch 5](#)

SessionPreparation — A `Disposable` wrapper around one *unpublished* session, letting a caller decide whether to publish and cleanly release backend state if not. → [Ch 29](#)

snapshot message — The synthetic `user/message` carrying runtime-context sections, injected only when its text differs from the last still-visible one. → [Ch 21](#)

spill — Writing an oversized plain-text tool result out of line at execution time, leaving a bounded preview plus a retrieval pointer. → [Ch 26](#)

step — One model request within a turn. A turn with three tool round-trips has four steps. → [Ch 9](#)

surface — The ordered list of log positions the model currently sees. The model's view; the log is the record. → [Ch 6](#)

surfaceOp — The mandatory declaration on a surface-eligible event: `'append'` or `{ op: 'replace', start, end }`. → [Ch 6](#)

subagent — A genuine nested `Agent` created through the same factory. `spawn` starts empty; `fork` seeds the parent's completed turns. → [Ch 28](#)

token meter — A per-session measurement preferring real provider usage and falling back to a fixed 4-characters-per-token heuristic. Not a tokenizer. → [Ch 26](#)

turn — One unit of conversation: roughly one user prompt and everything the model does in response. → [Ch 9](#)

turn ending — A `TurnEndReason` recording why a turn stopped: `completed`, `max-tokens`, `blocked`, `aborted`, `error`, or `interrupted`. Written once, in a `finally`. → [Ch 9](#)

waterfall — A dispatch mode where listeners form nested continuations. A listener returning without calling `next()` vetoes everything after it, including the caller's default. → [Ch 22](#)

wake latch — A deferred wake recorded when the agent cannot act on it now, replayed at convergence — but never during disposal. → [Ch 13](#)

Appendix B • Data-structure reference

Every core type, its fields, and where it is defined. Paths are repo-relative.

Session layer — `packages/core/session/src/`

`SessionEvent<T>` • `types.ts:391–398`

Field	Type	Notes
<code>seq</code>	<code>number</code>	Zero-based log index; assigned only at <code>index.ts:627</code>
<code>time</code>	<code>number</code>	Epoch ms
<code>type</code>	<code>T</code>	Key of <code>SessionEventMap</code>
<code>data</code>	<code>SessionEventMap[T]</code>	Deep-snapshotted, frozen
<code>ignorable?</code>	<code>true</code>	Reader may skip if the type is unrecognized (<code>types.ts:399–409</code>)
<code>surfaceOp?</code>	<code>SurfaceOp</code>	Surface-eligible types only
<code>sourceEventSeqs?</code>	<code>number[]</code>	Surface-eligible types only (<code>types.ts:410–422</code>)

`SessionEventMap` (core entries) • `types.ts:216–320`

Type	Payload
<code>turn/start</code>	<code>{ turn }</code>
<code>turn/end</code>	<code>{ turn, reason: TurnEndReason }</code>
<code>step/start / step/end</code>	<code>{ turn, step }</code>
<code>user/message</code>	<code>UserMessage</code> — the data is the message
<code>assistant/chunk</code>	<code>{ turn, step, chunk: StreamChunk }</code>
<code>assistant/message</code>	<code>{ turn, step, message, usage?, interrupted? }</code>
<code>tool/call</code>	<code>{ turn, step, callId, name, arguments }</code>
<code>tool/result</code>	<code>{ turn, step, message, error?, meta? }</code>
<code>request/header</code>	<code>{ header: EpochHeader, reason, startsSeries? }</code>
<code>request/context</code>	<code>RequestContext</code>
<code>session/end-seed</code>	<code>Record<string, never></code>

Merge-extensible; the complete build-wide set is generated into `known-event-types.ts:22–74` (~50 types).

SessionHeader • types.ts:56-94

version • id • createdAt • cwd? • parentSession? • seedLength? • origin? • delegationDepth? • agentPreset?

Kept **out** of the event log. Deep-frozen at construction.

TurnEndReason • types.ts:150-169

completed • max-tokens • blocked • aborted{reason} • error{error: LlmFailure} • interrupted
(written only by crash repair)

SurfaceOp • types.ts:359-361

```
'append' | { op: 'replace'; start: number; end: number }
```

start / end name **surface nodes by seq**, not log positions.

SessionSurface • surface.ts:137-142

nodes: readonly number[] • replaceGeneration: number

EpochHeader • types.ts:179-188

config: LlmCallConfig • adapterDefaults? • system? • tools?: ToolSchema[]

ChunkRow • chunk-rows.ts:66-69

```
{ type: 'text-chunks' | 'reasoning-chunks' | 'tool-call-chunks'  
  seq0: number; time0: number; data: { turn, step, index, dt: number[], texts|args: string[], id?,  
  name? } }
```

Not a session event. A storage encoding, expanded before anything above persistence sees it.

Constants

Name	Value	Where
SESSION_FORMAT_VERSION	0	types.ts:51
MIN_RUN	3	chunk-rows.ts:99
TOOL_NOT_STARTED / TOOL_OUTCOME_UNKNOWN	—	repair.ts:14,17

LLM layer — packages/llm/llm/src/

Message • message.ts:131-140

id: MessageId · role: 'system'|'user'|'assistant' · content: ContentBlock[] · source: MessageSource

Specializations: UserMessage, AssistantMessage (source: ModelMessageSource), ToolResultMessage (role: 'user', one ToolResultBlock).

MessageSource • message.ts:102-107

{kind:'user'} · {kind:'plugin', plugin} & ContextFormed · {kind:'model', provider, model, replayState?} · {kind:'tool', callId}

ContentBlock • types.ts:54-110

Type	Shape
text	{ type, text }
reasoning	{ type, text }
image	{ type, attachment }
tool-call	{ type, id, name, arguments } — arguments are a raw JSON string
tool-result	{ type, toolCallId, content, isError? }

StreamChunk • types.ts:364-376

block-start{index, blockType} · text-delta{index, text} · reasoning-delta{index, text} · tool-call-delta{index, id, name?, argumentsDelta} · block-end{index, block} · usage{usage} · finish{reason, replayState?}

FinishReason • types.ts:116-125

stop · tool-calls · max-tokens · aborted{failure} · error{failure}

GenerateOptions • types.ts:393-429

provider · model · reasoningEffort? · messages · system? · tools? · temperature? · maxTokens? · stop? · signal? · sessionId? · purpose?

LlmCallConfig • call-config.ts:23-30

provider · model · reasoningEffort? · temperature? · maxTokens? · stop?

Carries a file-level `TODO(call-config-shape)` — the shape is explicitly unsettled ([Ch 35](#)).

LlmCallConfigAdapterDefaults • `call-config.ts:36-39`

`reasoningEffort?: true` • `maxTokens?: true`

LlmFailure • `types.ts:40-51`

`message` • `code` • `status?` • `providerRetryAfterMs?` • `requestId?`

PreparedLlmCall • `index.ts:157-177`

`config` • `retryPolicy` • `context?: {contextWindow}` • `inputModalities?` • `adapterDefaults` • `stream(options)`

Frozen; `stream` is single-use and config-checked.

ToolSchema • `types.ts:385-390`

`name` • `description` • `parameters` (JSON Schema)

Tools layer — `packages/core/tools/src/`

ToolDefinition • `index.ts:214-280`

Extends `ToolSchema`, plus: `output` • `execute` • `finalizeContent?` • `timeoutMs?` • `isConcurrencySafe?` • `presentCall?` • `presentResult?`

ToolOutputDefinition • `index.ts:204-211`

`schema` • `render(args, value)` • `presentationMeta?(args, value)`

ToolExecutionInput • `index.ts:307-331`

`callId` • `rootCallId?` • `name` • `arguments` • `agent?` • `parent?` • `signal`

ToolRunContext • `index.ts:397-414`

Extends `ToolExecution` (which adds `rootCallId` and an opaque `token`), plus `deferContext(message)` and `concludeTurn()`.

ToolExecutionResult • index.ts:549–573

```
{ isError: false; value: JsonValue; content; meta?; additionalContexts?; concludesTurn? }  
{ isError: true; error: ToolFailure; content; meta?; additionalContexts? }
```

value is never persisted. concludesTurn is typed never on failures.

PreToolDecision / ScheduledToolPreparation • index.ts:576–584, :428–437

```
{kind:'allow'} | {kind:'deny', reason} | {kind:'ask', reason?} {kind:'dispatch', exec} |  
{kind:'post-result', result} | {kind:'final-result', result}
```

Constants

```
TOOL_ABORTED = 'ABORTED' ( :462 ) · TOOL_ABORTED_BEFORE_DISPATCH = 'ABORTED_BEFORE_DISPATCH'  
( :465 ) · TOOL_RUNTIME_SCHEDULER ( :459 ) · DEFAULT_MAX_PARALLEL_TOOL_CALLS = 10 ( agent-  
loop/src/constants.ts:6 )
```

Prompt layer — packages/core/system-prompt/src/

PromptAssembly • index.ts:114–119

```
sections: AssembledSection[] · contexts: AssembledContext[] · tools: ToolSchema[] · variables:  
Record<string, string|undefined>
```

Text is **uninterpolated** at this stage.

PromptSection / PromptContext • index.ts:53–74, :77–84

```
{ name, order, text, complete? } / { name, order, text }
```

Order constants • index.ts:121–161

```
SECTION_ORDERS — HARNESS_IDENTITY: -1000 ... STRUCTURED_OUTPUT: 9900 CONTEXT_ORDERS —  
SANDBOX_POLICY: 110 , APPROVAL_POLICY: 115 , SUBAGENT_DELEGATION: 120
```

Agent layer — `packages/core/agent/src/` and `agent-loop/src/`

Phase • `agent-loop/src/agent.ts:39-47`

```
{ kind: 'idle'; lastTurn }  
{ kind: 'maintenance'; abort; lastTurn; wakeRequested }  
{ kind: 'running'; abort; turn; step; wakeRequested }
```

InboxTarget • `agent/src/types.ts:29`

`'next-turn' | 'next-step'`

PreStepDecision / RequestErrorAction • `runtime-types.ts`

`{kind:'reject'} | {kind:'enter', messages, startsRequestSeries?} · {kind:'retry'} | undefined`

ProjectionDefinition • `session-projection/src/index.ts:42-86`

`key · stateSchema · init(header) · apply(state, event) · wire? · stateVersion`

TurnBoundaryProjection • `agent-loop/src/index.ts:55-93`

`openTurnStartSeq · lastStepStartSeq · lastStepBoundary · lastTurn — stateVersion: 2`

Appendix C · Configuration reference

Every value whose effect on a mechanism this book traced, with the default **actually in force in the shipped web profile**.

How to read a default

A value reaches a running session through up to three layers, last write winning **per row id**:

- 1. `packages/bundle/base/cordis.patch.yml` — the shared core
- 2. `packages/bundle/web-app/cordis.patch.yml` — the web surface; `disabled: true` on most agent-plane rows
- 3. `packages/preset/agent-presets/presets/standard/agent.cordis.yml` — re-mounts the agent plane per agent

A patch **replaces a row's whole config** rather than merging (`vendor/include/src/index.ts:121-124`). So for any per-agent tool, **the preset's value is the live one** and the base's never reaches a web session.

Two rows where they genuinely differ:

Row	base	standard preset	Live
<code>tool-web</code>	<code>fetch: false</code>	<code>fetch: true</code>	<code>fetch: true</code>
<code>tool-subagent-fork</code>	<code>backgroundMode: one-shot</code>	<code>backgroundMode: continuable</code>	<code>continuable</code>

→ [Ch 34](#)

Engine core

Setting	Default	Where	Effect
<code>maxParallelToolCalls</code>	<code>10</code>	<code>agent-loop/src/constants.ts:6</code>	Max in-flight parallel-safe calls per group. Integer ≥ 1 or throw. Read through a getter, destructured once per group. Deployment-wide. → Ch 17
<code>agents</code>	<code>[]</code>	<code>base:486-489</code>	Agents created at startup. Deliberately not user-settable. → Ch 14

Model routing

Setting	Default	Where	Effect
default route	deepseek-official / deepseek-v4-flash	base:73-79	What new agents start from
llm-deepseek	disabled: true	web-app:41-42	The native adapter never registers in web
llm-pi-ai providers	{}	llm-pi-ai/src/config.ts:340-342	Dormant — zero routes until settings supply profiles → Ch 32
defaultContextWindow	1_000_000	llm-deepseek/src/adapter.ts:140	Capacity fact; feeds the compaction threshold
DEFAULT_MAX_TOKENS	256_000	llm-deepseek/src/adapter.ts:142	Fills <code>maxTokens</code> when omitted → stamps <code>adapterDefaults</code>
reasoningEffort	high (off only when thinking: 'disabled')	llm-deepseek/src/adapter.ts:187-193	

Retry

Setting	Default	Effect
mode	normal	<code>always</code> removes both the code gate and the retry cap
maxRetries	5	<code>normal</code> only
initialDelayMs / maxDelayMs	500 / 10_000	Exponential backoff bounds
jitterRatio	0.1	Symmetric jitter
retryableCodes	EMPTY_RESPONSE, RATE_LIMIT, SERVER, TIMEOUT, TRANSPORT	<code>normal</code> only

— `packages/llm/llm/src/retry-policy.ts:14-24` ; plugin mounted at `base:84-85`. Captured **at adapter registration**, not per request. → [Ch 25](#)

Context-window management

Setting	Default	Where	Effect
<code>auto</code>	<code>true</code>	<code>compaction-basic/src/config.ts:95</code>	Enables both triggers
<code>thresholdRatio</code>	<code>0.8</code>	<code>config.ts:20</code>	Compact at 80% of the context window
<code>retainRatio</code>	<code>0.16</code>	<code>config.ts:23</code>	Verbatim recent history preserved
<code>maxOverflowRetries</code>	<code>1</code>	<code>config.ts:93</code>	Compact-and-retry attempts per failure
<code>modelPolicies</code>	none configured	<code>config.ts:105-125</code>	Per- <code>provider/model</code> overrides
<code>pruner thresholdChars</code>	<code>8192</code>	<code>base:404-409, preset:150-155</code>	Truncate results above this
<code>pruner headChars / tailChars</code>	<code>4096 / 1024</code>	same	What survives truncation
<code>maxInlineBytes (spill)</code>	<code>50000</code>	<code>base:393-396</code>	Host-plane; not touched by web-app
<code>cleanupPeriodDays (spill)</code>	<code>30</code>	<code>spill-local/src/index.ts:68</code>	Startup sweep of old spill files
token estimate	<code>CHARS_PER_TOKEN=4 , BLOCK_OVERHEAD=4 , ROLE_OVERHEAD=4</code>	<code>token-meter/src/estimate.ts:13-19</code>	Not a tokenizer → Ch 26

Mount status: `compaction-basic`, `command-compact`, and `tool-result-pruner` are base-mounted, web-app-disabled, and **re-mounted by the standard preset** inside an `isolate: {compaction, toolResultPruner}` group. Live per agent. → [Ch 27](#)

Permissions and sandbox

Setting	Default	Where
sandbox mode	workspace-write	base:217, from DSH_PERMISSION_MODE
workspaceRoot	process.cwd()	base:218
approval policy	ask	base:233 — a <code>!!js</code> expression; becomes <code>never</code> iff <code>DSH_PERMISSION_MODE === 'danger-full-access'</code>
presets	read-only / workspace-write / danger-full-access	base:238-247

policy: `'never'` is enforced **inline**, before any answerer runs — not waterfall-overridable. → [Ch 18](#)

Persistence

Setting	Default	Where
root	dshHomePath('sessions')	base:110-113
compression	zstd	jsonl backend
packChunks	true	jsonl backend → Ch 30
session-query-sqlite	path: <code>':memory:'</code> , openAt: <code>never</code>	base:129-133, web-app:26-29 — SQLite is never opened
projection cache	writeEveryEvents: 200, writeIntervalMs: 5000	base:162-166

Prompt and tools

Setting	Default	Where
persona	' ' base; set in web-app and again in the preset (which shadows)	base:481-482, web-app:16-19, preset:24-28
tools.mode	native (from DSH_TOOLS_MODE)	web-app:31-37 — documented as a temporary seam
agent-instructions.maxBytes	65536	base:274-277, preset:30-33
tool-todo.allowParallelInProgress	true	base:411-414, preset:224-227
tool-ralph.maxRounds	64	base:422-427, preset:213-217
repeat-tool-reminder	thresholds: [3,5,8] , argumentsPreviewChars: 500	base:434-438
tool-web.searchTimeoutMs	60000	base:464-468, preset:231-235
agent-presets.default	standard	web-app:451-456
webserver	127.0.0.1:3080 , gzip level 1 above 1024 bytes	web-app:121-129

Environment variables

Var	Effect
DSH_PERMISSION_MODE	Sets sandbox mode and derives the approval policy
DSH_TOOLS_MODE	native ptc both ; unset = native
DSH_TELEMETRY_MODE	FEEDBACK_ONLY (default) FULL DISABLED
DSH_TELEMETRY_DISABLED	Any non-empty value — including '0' / 'false' — opts out
DSH_TELEMETRY_OTLP_URL	Overrides the exporter endpoint
DEEPSEEK_API_KEY	Credential for chat and DeepSeek web search
DSH_HOME	Root for sessions, storages, credentials, settings, user presets

Platform-conditional rows

Evaluated lazily per row via Entry.disabledOf() (vendor/loader/src/config/entry.ts:100-108):

Row	Disabled when
<code>tool-bash</code> , <code>bash-sandbox</code>	<code>process.platform === 'win32'</code>
<code>tool-pwsh</code> , <code>pwsh-sandbox</code>	<code>process.platform !== 'win32'</code>

Mounted but inert

Worth knowing when reading a composition — a row can be present and do nothing:

Row	Why inert
<code>session-query-sqlite</code>	<code>openAt: never</code> — SQLite never opened; search fails <code>SESSION_QUERY_SEARCH_DISABLED</code>
<code>llm-pi-ai</code>	empty <code>providers</code> dict — registers zero routes
<code>hmr</code>	<code>disabled: true</code> in base
<code>skill-badge</code>	<code>disabled: true</code> in base
<code>ui-schedule</code>	<code>disabled: true</code> in web-app
<code>dsh-invariants</code>	mounted nowhere → Ch 11

Appendix D • Mechanism index

Mechanism → chapter → key files. Use this to jump from a file you are reading to the chapter that explains it.

By mechanism

Mechanism	Chapter	Key files
Append-only log	5	<code>packages/core/session/src/index.ts:602-653</code>
Surface (append / replace)	6	<code>packages/core/session/src/surface.ts</code>
Message derivation	7	<code>surface.ts:83-114</code> , <code>index.ts:724-745</code>
Projections	8	<code>packages/session/session-projection/src/index.ts</code>
Turn loop	9	<code>packages/core/agent-loop/src/agent.ts:255-339</code>
Step loop	9	<code>agent-loop/src/agent.ts:341-438</code>
Request construction	10	<code>agent-loop/src/agent.ts:444-544</code>
Reconstruction invariant	11	<code>agent-loop/src/invariant.ts</code>
Inbox	12	<code>packages/core/agent/src/inbox.ts</code>
Phase machine / cancellation	13	<code>agent-loop/src/agent.ts:39-232</code>
Agent lifecycle	14	<code>agent-loop/src/index.ts:522-773</code>
Tool registry	15	<code>packages/core/tools/src/index.ts:1028-1053</code>
Execution pipeline	16	<code>core/tools/src/index.ts:1450-1667</code>
Tool-call scheduling	17	<code>agent-loop/src/tool-calls.ts</code>
Approval	18	<code>packages/interaction/user-approval/src/index.ts:222-309</code>
Sandbox escalation	18	<code>packages/sandbox/sandbox/src/escalation.ts:157-189</code>
Prompt assembly	19	<code>packages/core/system-prompt/src/index.ts:536-611</code>
Variable interpolation	20	<code>system-prompt/src/index.ts:309-346</code>
Runtime context	21	<code>agent-loop/src/runtime-context.ts</code>
Extension points	22	<code>packages/core/agent/src/dispatch.ts</code> , <code>vendor/cordis/src/events.ts:194-243</code>
Adapters / <code>prepareCall</code>	23	<code>packages/llm/llm/src/index.ts:890-935</code>
Block assembly	24	<code>packages/llm/llm/src/assembler.ts</code>
Failure normalization	25	<code>packages/llm/llm/src/adapter-failure.ts</code>
Retry	25	<code>packages/llm/llm-retry/src/index.ts:194-258</code>
Token metering	26	<code>packages/llm/token-meter/src/{index,estimate}.ts</code>

Mechanism	Chapter	Key files
Spill	26	packages/spill/spill-policy/src/index.ts:110-232
Tool-result pruning	27	compaction-tool-result-pruner/src/index.ts:136-184
Compaction	27	compaction-basic/src/{index,region,summarizer}.ts
Subagents	28	subagent/subagent-in-process-driver/src/index.ts:103-234
Persistence	29	session-persistence/src/coordinator.ts , session-persistence-jsonl/src/index.ts
Crash repair	30	packages/core/session/src/repair.ts:28-134
Chunk packing	30	packages/core/session/src/chunk-rows.ts
Boot and profiles	34	apps/server/src/ , packages/boot/app-boot/src/
Patch algebra	34	vendor/include/src/index.ts:58-128
Isolate realms	34	vendor/loader/src/config/isolate.ts
Scope layering	34	packages/core/scope/src/{index,store}.ts
Agent presets	34	packages/preset/agent-presets/src/{index,mount}.ts

By file (reverse lookup)

File	Chapter
<code>packages/core/agent-loop/src/agent.ts</code>	9, 10, 13
<code>packages/core/agent-loop/src/index.ts</code>	14, 8
<code>packages/core/agent-loop/src/tool-calls.ts</code>	17
<code>packages/core/agent-loop/src/runtime-context.ts</code>	21
<code>packages/core/agent-loop/src/invariant.ts</code>	11
<code>packages/core/session/src/index.ts</code>	5, 7
<code>packages/core/session/src/surface.ts</code>	6, 7
<code>packages/core/session/src/chunk-rows.ts</code>	30
<code>packages/core/session/src/repair.ts</code>	30
<code>packages/core/agent/src/inbox.ts</code>	12
<code>packages/core/agent/src/dispatch.ts</code>	22
<code>packages/core/tools/src/index.ts</code>	15, 16
<code>packages/core/system-prompt/src/index.ts</code>	19, 20
<code>packages/core/scope/src/store.ts</code>	19, 34
<code>packages/llm/llm/src/index.ts</code>	23
<code>packages/llm/llm/src/assembler.ts</code>	24
<code>packages/llm/llm-retry/src/index.ts</code>	25
<code>packages/llm/token-meter/src/</code>	26
<code>packages/compaction/compaction-basic/src/</code>	27
<code>packages/spill/spill-policy/src/index.ts</code>	26
<code>packages/session/session-persistence/src/coordinator.ts</code>	29
<code>packages/session/session-projection/src/index.ts</code>	8
<code>packages/interaction/user-approval/src/index.ts</code>	18
<code>packages/sandbox/sandbox/src/escalation.ts</code>	18
<code>packages/preset/agent-presets/src/mount.ts</code>	34
<code>packages/bundle/*/cordis.patch.yml</code>	34, App C
<code>vendor/cordis/src/events.ts</code>	22
<code>vendor/cordis/src/fiber.ts</code>	2, 34

File	Chapter
vendor/include/src/index.ts	34
vendor/loader/src/config/isolate.ts	34

By extension point

Point	Mode	Chapter	Live listeners in web
agent/pre-step	waterfall	22	12 — the dominant surface
agent/request	waterfall	10, 22	1 (model selection)
agent/request-error	waterfall	25, 27	2 (compaction, retry)
agent/turn-stopping	serial	22	0
agent/status	emit	13	3
agent/created / agent/disposed	emit	14	5 / 4
agent/inbox/*	emit	12	1–3 each
agent/error	emit	9	3
agent/session-start	emit	14	2
tools/pre-execute	waterfall	16, 18	0 producing ask
tools/execute	waterfall	16	timeout policy
tools/post-execute	waterfall	16, 26	spill policy
tools/result	emit	16	UI bridges
llm/stream	waterfall	11	(invariant — not mounted)
system-prompt/assemble	waterfall	19	(invariant — not mounted)
approval/request	waterfall	18	UI approval
session/event	emit	5	persistence, projections, telemetry